

**LAPORAN AKHIR PRAKTIKUM
APLIKASI WEB DAN MOBILE
PERTEMUAN 1 – 15**



DISUSUN OLEH :

Radhitya Adha Fadhlur Rohman

23051430005

**PROGRAM STUDI TEKNIK INDUSTRI
FAKULTAS TEKNIK
UNIVERSITAS NEGERI YOGYAKARTA
2026**

LEMBAR PENGESAHAN
LAPORAN AKHIR PRAKTIKUM APLIKASI WEB DAN MOBILE

LAPORAN AKHIR PRAKTIKUM APLIKASI WEB DAN MOBILE
PERTEMUAN 1 – 15

Radhitya Adha Fadhlur Rohman

NIM. 23051430005

Laporan ini Disusun Guna Memenuhi Salah Satu Persyaratan

Menempuh Mata Kuliah Aplikasi Web dan Mobile

Program Studi Teknik Industri

Fakultas Teknik UNY

Menyetujui/Mengesahkan :

Dosen Pengampu,

Mahasiswa,



Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT
NIP. 198103172005011001

Radhitya Adha Fadhlur Rohman
NIM. 23051430005

DAFTAR ISI

LEMBAR PENGESAHAN	2
Modul 1 Pengenalan Pemrograman Web & HTML Dasar	4
Modul 2 CSS Dasar & Styling	18
Modul 3 Layout CSS Lanjutan (Flexbox & Grid) + Bootstrap	36
Modul 4 JavaScript Dasar (Variabel, Tipe Data, Operator, Control Flow)	65
Modul 5 Function, Array, Object di JavaScript	77
Modul 6 DOM Manipulation & Event Handling	96
Modul 7 Form Validation & Local Storage	119
Modul 8 Asynchronous JavaScript (Fetch API, JSON, REST API Sederhana) ..	149
Modul 9 Pengenalan React.js (Component, JSX, Props)	180
Modul 10 State, Hooks (useState, useEffect), dan Conditional Rendering	203
Modul 11 React Routing & Multi-Page App + Integrasi API	231
Modul 12 Proyek Web: Dashboard Monitoring Produksi (React + Chart.js)	258
Modul 14 Pengenalan React Native & Setup Expo	293
Modul 15 Component, Navigation, dan Styling di React Native + Proyek Mobile	322

Modul 1 Pengenalan Pemrograman Web & HTML Dasar

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul Aplikasi Web dan Mobile Manual Praktikum (Pertemuan 1)
Topik Utama	Pengenalan Pemrograman Web & HTML Dasar
Sub-Topik	Struktur halaman web, elemen HTML semantik, dan formulir dasar.
Durasi Praktikum	340 Menit
Tanggal Praktikum	14 February 2026
Nama Mahasiswa	Radhitya Adha Fadhlur Rohman
NIM	23051430005
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami peran dan fungsi aplikasi web dalam mendukung sistem informasi di lingkungan industri.
2	Mampu menjelaskan struktur dasar HTML serta mekanisme kerja HTML dalam menampilkan halaman web melalui browser.
3	Mampu membuat halaman web statis yang memuat elemen teks, gambar, dan tabel sesuai dengan kebutuhan informasi.
4	Mampu menyusun formulir input data sederhana, seperti form pencatatan atau data produksi, menggunakan elemen HTML yang tepat.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor

2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	Live Server Extension	64-bit	Lingkungan pengembangan
5	Git & GitHub	Versi terbaru	Manajemen versi & repository

D. DASAR TEORI

Perkembangan teknologi digital menuntut mahasiswa untuk memiliki pemahaman dasar mengenai bagaimana sebuah website dibangun dan dijalankan. Salah satu fondasi utama dalam pengembangan web adalah HTML dan CSS. Menurut Mardiansyah dkk. (2025), HTML (HyperText Markup Language) berperan sebagai kerangka atau struktur dasar dari sebuah halaman web, sedangkan CSS (Cascading Style Sheets) berfungsi untuk mengatur tampilan visual seperti warna, tata letak, dan tipografi. Artinya, HTML membentuk “rangka” halaman, sementara CSS mempercantik dan menyusun tampilannya agar lebih menarik serta mudah dipahami pengguna. Pemahaman terhadap kedua teknologi ini menjadi langkah awal sebelum masuk ke tahap pengembangan web yang lebih kompleks.

Dalam konteks struktur halaman web, HTML memiliki elemen-elemen dasar yang menyusun sebuah dokumen, seperti `<html>`, `<head>`, dan `<body>`. Elemen-elemen ini membagi halaman menjadi bagian informasi (metadata) dan bagian konten utama yang ditampilkan di browser. Selain itu, HTML juga memiliki elemen semantik seperti `<header>`, `<nav>`, `<section>`, `<article>`, dan `<footer>` yang membantu memberikan makna pada struktur konten. Jeffry dkk. (2025) menjelaskan bahwa pemahaman struktur dan logika dasar pemrograman web sangat penting karena menjadi pondasi sebelum mahasiswa mempelajari teknologi lanjutan seperti framework atau integrasi database. Dengan menggunakan elemen semantik, halaman web tidak hanya tersusun rapi, tetapi juga lebih mudah diakses oleh mesin pencari dan teknologi pembaca layar.

Selain struktur dan elemen semantik, formulir dasar (form) merupakan komponen penting dalam halaman web karena memungkinkan interaksi antara pengguna dan sistem. Elemen seperti `<form>`, `<input>`, `<textarea>`, `<select>`, dan `<button>` digunakan untuk mengumpulkan data dari pengguna, misalnya pada halaman login, pendaftaran, atau pengisian survei. Menurut Mardiansyah dkk. (2025), penguasaan elemen-elemen dasar HTML termasuk form menjadi keterampilan awal yang wajib dimiliki dalam membangun website yang fungsional.

D.1 Konsep Utama

Konsep utama pada praktikum ini adalah memahami dasar pembuatan halaman web menggunakan HTML sebagai bahasa markup untuk menyusun struktur tampilan. HTML berfungsi sebagai kerangka yang mengatur elemen seperti judul, paragraf, tabel, gambar, dan formulir agar informasi tersusun rapi serta dapat dibaca dengan baik oleh browser. Setiap elemen dibentuk menggunakan tag yang memiliki fungsi berbeda, sehingga pemahaman struktur menjadi hal yang sangat penting.

Praktikum ini juga membahas struktur dasar HTML5 yang terdiri dari `<!DOCTYPE html>`, `<html>`, `<head>`, dan `<body>`. Bagian `<head>` berisi informasi pendukung seperti judul dan pengaturan halaman, sedangkan `<body>` menampilkan seluruh konten yang dilihat pengguna. Selain itu, diperkenalkan elemen semantik seperti `<header>`, `<main>`, `<section>`, dan `<footer>` yang membantu membuat struktur kode lebih terorganisir dan mudah dipahami.

Tidak hanya struktur, modul ini juga menekankan penggunaan formulir melalui elemen `<form>` dengan berbagai tipe input seperti text, number, radio, dan file. Formulir berfungsi sebagai media pengumpulan data dari pengguna. Secara keseluruhan, praktikum ini memberikan pemahaman dasar tentang bagaimana halaman web dibangun mulai dari penyusunan struktur hingga proses input data sebagai fondasi pengembangan web selanjutnya.

D.2 Keterkaitan dengan Teknik Industri

Materi pada modul ini memiliki hubungan yang cukup erat dengan bidang Teknik Industri, terutama dalam hal pengelolaan sistem dan data. Dalam dunia industri, hampir setiap aktivitas operasional membutuhkan pencatatan dan pengolahan informasi yang terstruktur, mulai dari data produksi, laporan harian, hingga hasil inspeksi kualitas. Sebagai contoh nyata, dalam proses Quality Control di sebuah perusahaan manufaktur, petugas perlu mencatat data seperti kode produk, jumlah produksi, jenis cacat, hingga dokumentasi pendukung. Formulir berbasis web dapat digunakan untuk memasukkan data tersebut secara langsung ke dalam sistem, sehingga pencatatan menjadi lebih cepat, terorganisir, dan meminimalkan kesalahan input. Selain itu, halaman web juga dapat dimanfaatkan untuk menampilkan informasi penting seperti jadwal maintenance mesin, target produksi, atau rekapitulasi kinerja dalam bentuk tabel yang mudah dibaca oleh manajer maupun operator.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

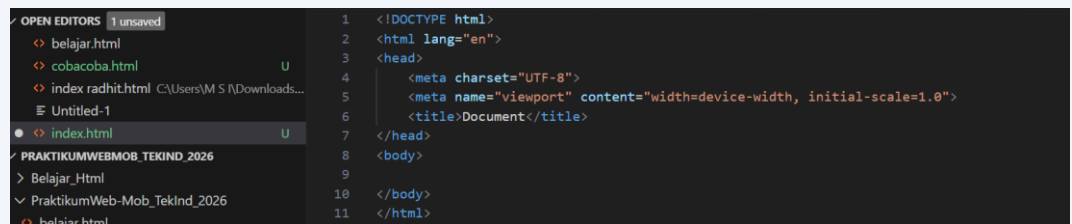
E.1 Langkah 1 – [Kerangka HTML]

Langkah pertama bisa menekan shift+1 atau ketik "!" kemudian tekan Tab atau enter otomatis akan menghasilkan kerangka HTML seperti contoh dibawah ini

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

Screenshot Hasil:



Gambar E.1 – [Kerangka HTML]

E.2 Langkah 2 – [Menyusun Profil Lini Produksi]

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Document</title>
</head>
<body>
  <header>
```

```
<h1>Profil Mesin CNC-01</h1>
<h3>Dibuat oleh : Radhitya Adha Fadhlur Rohman (23051430005)</h3>
<hr>

<hr>
<h2>Spesifikasi Teknis</h2>
<ul>
  <li><strong>Merk:</strong> Yamazaki Mazak</li>
  <li><strong>Kapasitas:</strong> 500 unit/hari</li>
  <li><strong>Status:</strong> <span style="color:
green;">Operasional</span></li>
</ul>
</header>

<section>
<h2>Jadwal Pemeliharaan</h2>
<table border="1" cellpadding="10">
  <thead>
    <tr>
      <th>Tanggal</th>
      <th>Jenis Perawatan</th>
      <th>Penanggung Jawab</th>
      <th>Status</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>12 Jan 2024</td>
      <td>Penggantian Oli</td>
      <td>Teknisi A</td>
      <td>Selesai</td>
    </tr>
    <tr>
      <td>15 Feb 2024</td>
      <td>Kalibrasi Presisi</td>
      <td>Teknisi B</td>
      <td>Pending</td>
    </tr>
  </tbody>
</table>
</form>
</body>
</html>
```

Screenshot Hasil:

Profil Mesin CNC-01

Dibuat oleh : Radhitya Adha Fadhlur Rohman (23051430005)

Spesifikasi Teknis

- Merk: Yamazaki Mazak
- Kapasitas: 500 unit/hari
- Status: Operasional

Jadwal Pemeliharaan

Tanggal	Jenis Perawatan	Penanggung Jawab	Status
12 Jan 2024	Penggantian Oli	Teknisi A	Selesai
15 Feb 2024	Kalibrasi Presisi	Teknisi B	Pending

Gambar E.2 – [Hasil Penyusunan Profil Lini Produksi]

E.3 Langkah 3 – [Membuat Form Input Data Produksi]

```

/* ===== KODE / OUTPUT LANGKAH 3 ===== */
<h2>Input Data Harian</h2>
<form action="">
  <label for="tanggal">Tanggal:</label><br>
  <input type="date" id="tanggal" name="tanggal" required><br><br>

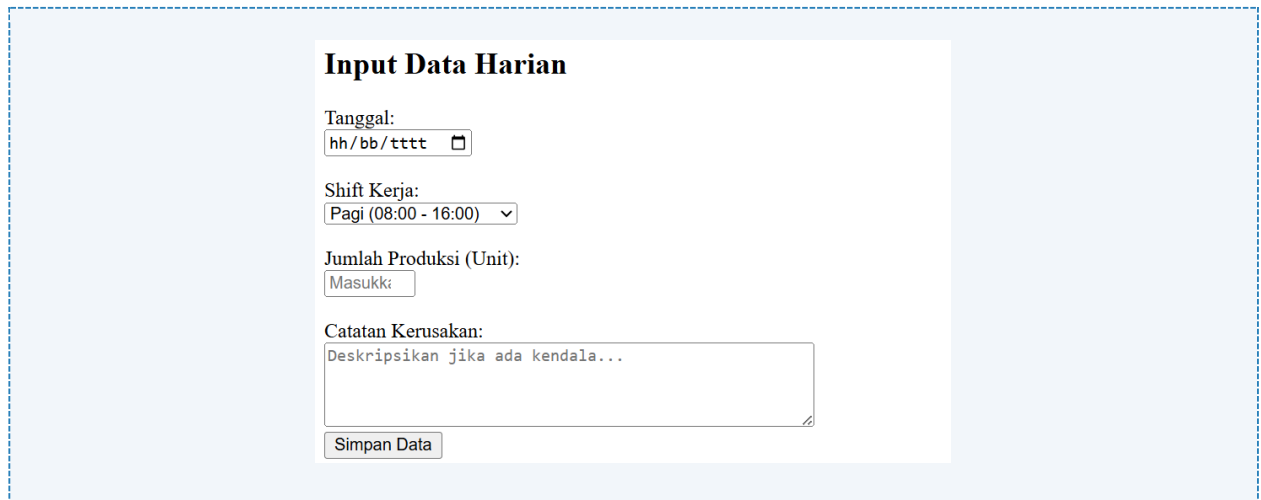
  <label for="shift">Shift Kerja:</label><br>
  <select id="shift" name="shift">
    <option value="Pagi">Pagi (08:00 - 16:00)</option>
    <option value="Siang">Siang (16:00 - 24:00)</option>
    <option value="Malam">Malam (00:00 - 08:00)</option>
  </select><br><br>

  <label for="jumlah">Jumlah Produksi (Unit):</label><br>
  <input type="number" id="jumlah" name="jumlah" min="0" max="600"
  placeholder="Masukkan jumlah..."><br><br>

  <label for="catatan">Catatan Kerusakan:</label><br>
  <textarea id="catatan" name="catatan" rows="4" cols="50"
  placeholder="Deskripsikan jika ada kendala..."></textarea><br>

  <button type="submit">Simpan Data</button>
</form>
</section>
</body>
</html>

```

Screenshot Hasil:

The screenshot shows a web form titled "Input Data Harian". It contains the following fields and controls:

- Tanggal:** A date input field with a placeholder "hh/bb/tttt" and a calendar icon.
- Shift Kerja:** A dropdown menu with "Pagi (08:00 - 16:00)" selected.
- Jumlah Produksi (Unit):** A text input field with the placeholder "Masukkan".
- Catatan Kerusakan:** A text area with the placeholder "Deskripsikan jika ada kendala...".
- Simpan Data:** A button at the bottom of the form.

Gambar E.3 – [Hasil Pembuatan Form Input Harian]

F. LATIHAN DAN TUGAS MANDIRI**F.1 Latihan 1 – [Pengayaan Tabel]**

Pertanyaan / Instruksi Latihan:

Tambahkan satu kolom baru di tabel "Jadwal Pemeliharaan" bernama "Status" dan isi datanya secara manual (Selesai/Pending).

Jawaban / Kode Solusi:

```
<section>
<h2>Jadwal Pemeliharaan</h2>
<table border="1" cellpadding="10">
  <thead>
    <tr>
      <th>Tanggal</th>
      <th>Jenis Perawatan</th>
      <th>Penanggung Jawab</th>
      <th>Status</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>12 Jan 2024</td>
      <td>Penggantian Oli</td>
      <td>Teknisi A</td>
      <td>Selesai</td>
    </tr>
    <tr>
```

```
<td>15 Feb 2024</td>
<td>Kalibrasi Presisi</td>
<td>Teknisi B</td>
<td>Pending</td>
</tr>
</tbody>
</table>
</section>
```

Profil Mesin CNC-01

Dibuat oleh : Radhitya Adha Fadhlur Rohman (23051430005)

Spesifikasi Teknis

- Merk: Yamazaki Mazak
- Kapasitas: 500 unit/hari
- Status: Operasional

Jadwal Pemeliharaan

Tanggal	Jenis Perawatan	Penanggung Jawab	Status
12 Jan 2024	Penggantian Oli	Teknisi A	Selesai
15 Feb 2024	Kalibrasi Presisi	Teknisi B	Pending

Gambar F.1 – [Hasil Latihan 1]

F.2 Latihan 2 – [Validasi Dasar]

Pertanyaan / Instruksi Latihan:

Pada input "Jumlah Produksi", tambahkan atribut `max` dengan nilai 600 (maksimal kapasitas mesin).

Jawaban / Kode Solusi:

```
<label for="jumlah">Jumlah Produksi (Unit):</label><br>
<input type="number" id="jumlah" name="jumlah" min="0" max="600"
placeholder="Masukkan jumlah..."><br><br>

<label for="catatan">Catatan Kerusakan:</label><br>
<textarea id="catatan" name="catatan" rows="4" cols="50"
placeholder="Deskripsikan jika ada kendala..."></textarea><br>

<button type="submit">Simpan Data</button>
</form>
</section>
</body>
</html>
```

Profil Mesin CNC-01

Dibuat oleh : Radhitya Adha Fadhlur Rohman (23051430005)

Spesifikasi Teknis

- Merk: Yamazaki Mazak
- Kapasitas: 600 unit/hari
- Status: Operasional

Jadwal Pemeliharaan

Tanggal	Jenis Perawatan	Penanggung Jawab	Status
12 Jan 2024	Penggantian Oli	Teknisi A	Selesai
15 Feb 2024	Kalibrasi Presisi	Teknisi B	Pending

Input Data Harian

Tanggal:
hh/bb/yyyy

Shift Kerja:
Pagi (08:00 - 16:00)

Jumlah Produksi (Unit):

Catatan Kerusakan:
Deskripsikan jika ada kendala...

Gambar F.2 – [Hasil Latihan 2]

F.3 Tugas Proyek Mini – [Relevansi Industri]

Deskripsi proyek mini:

formulir "Laporan Inspeksi Kualitas" dengan field:

- ID Produk (Text)
- Nama Inspektur (Text)
- Jenis Cacat (Radio Button: Geometri, Permukaan, Fungsional)
- Jumlah Cacat (Number)
- Foto Produk (Input type file - placeholder saja)

Tombol Submit

Penjelasan pendekatan/solusi yang digunakan:

Supaya dapat masuk dalam satu kotak menggunakan `<form action>`

Kode Utama:

```
[<!DOCTYPE html>
<html lang="id">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Laporan Kualitas Produk</title>
</head>

<body>
  <header>
    <h1>Laporan Kualitas Produk</h1>
    <p>Formulir Laporan Inspeksi Kualitas</p>
    <h5>Dibuat oleh : Radhitya Adha F R (23051430005)</h5>
    <hr>
  </header>

  <section>
    <form action="">

      <!-- ID Produk -->
      <label for="id_produk">ID Produk :</label>
      <input type="text" id="id_produk" name="id_produk"
        placeholder="Masukkan ID Produk" required><br><br>

      <!-- Nama Inspektur -->
      <label for="nama_inspektur">Nama Inspektur :</label>
      <input type="text" id="nama_inspektur" name="nama_inspektur"
        placeholder="Masukkan Nama Inspektur" required><br><br>

      <!-- Jenis Cacat -->
      <label>Jenis Cacat :</label><br>

      <input type="radio" id="geometri" name="jenis_cacat"
        value="Geometri" required>
      <label for="geometri">Geometri</label><br>

      <input type="radio" id="permukaan" name="jenis_cacat"
        value="Permukaan">
      <label for="permukaan">Permukaan</label><br>

      <input type="radio" id="fungsional" name="jenis_cacat"
        value="Fungsional">
      <label for="fungsional">Fungsional</label><br><br>

      <!-- Jumlah Cacat -->
      <label for="jumlah_cacat">Jumlah Cacat :</label>
      <input type="number" id="jumlah_cacat" name="jumlah_cacat"
        placeholder="Masukkan Jumlah Cacat">
```

```
min="0" required><br><br>

<!-- Foto Produk -->
<label for="foto_produk">Upload Foto Produk :</label>
<input type="file" id="foto_produk" name="foto_produk"
      accept="image/*"><br><br>

<button type="submit">Simpan Laporan</button>

</form>
</section>
</body>

</html>
```



Laporan Kualitas Produk

Formulir Laporan Inspeksi Kualitas

Dibuat oleh : Radhitya Adha F R (23051430005)

ID Produk :

Nama Inspektur :

Jenis Cacat :

- ☐ Geometri
- ☐ Permukaan
- ☐ Fungsional

Jumlah Cacat :

Upload Foto Produk : Tidak ada file yang dipilih

Gambar F.3 – [Keterangan Hasil Proyek Mini]

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang saya buat sudah sesuai dengan teori dasar HTML yang dipelajari pada modul. Struktur dokumen telah mengikuti standar HTML5, dimulai dari `<!DOCTYPE html>`, lalu elemen `<html>`, `<head>`, dan `<body>`. Pada bagian `<head>` sudah terdapat pengaturan seperti karakter (UTF-8), viewport untuk tampilan responsif, serta `<title>` yang menampilkan judul pada tab browser. Hal ini menunjukkan bahwa kerangka halaman sudah disusun sesuai konsep dasar HTML.

Pada bagian isi, penggunaan elemen semantik seperti `<header>`, `<main>`, dan `<section>` sudah diterapkan untuk membagi struktur halaman agar lebih terorganisir dan memiliki makna yang jelas. Selain itu, implementasi `<form>` dengan berbagai tipe input seperti text, radio, number, dan file juga sudah sesuai dengan fungsi masing-masing. Beberapa atribut seperti `required`, `min`, serta keterhubungan antara `<label>` dan input melalui `for` dan `id` menunjukkan bahwa konsep validasi dan aksesibilitas dasar telah dipahami.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Tampilan form kurang rapi	Memahami bahwa HTML hanya menyusun struktur, sedangkan perapian tampilan akan menggunakan CSS pada modul berikutnya.
Lupa menutup tag atau salah penempatan elemen	Menyusun kode secara runtut dan memanfaatkan fitur pengecekan error pada code editor.

G.3 Kaitan dengan Penerapan di Industri

Materi pada modul ini sangat relevan dengan praktik di bidang Teknik Industri, terutama dalam perancangan sistem pencatatan data yang lebih modern dan efisien. Form HTML yang dibuat pada praktikum dapat menjadi contoh awal sistem digital untuk mencatat data produksi, inspeksi kualitas, atau laporan operasional. Dibandingkan pencatatan manual, sistem berbasis web membuat data lebih rapi, mudah disimpan, dan siap dianalisis untuk mendukung pengambilan keputusan.

Penggunaan tipe input yang sesuai, seperti radio untuk pilihan tunggal atau number untuk data angka, juga mencerminkan prinsip standarisasi dan pengurangan kesalahan kerja. Dalam konteks nyata, sistem seperti ini bisa diterapkan untuk monitoring produksi, pencatatan hasil inspeksi, atau laporan maintenance mesin melalui perangkat komputer maupun tablet. Meskipun masih sederhana dan belum terhubung ke database, keterampilan dasar ini sudah menjadi fondasi penting dalam pengembangan sistem informasi industri yang lebih terintegrasi.

H. KESIMPULAN

1	Berhasil menyusun struktur HTML5 lengkap dan menjalankannya tanpa error di browser.
2	Mampu menggunakan elemen semantik untuk membagi bagian halaman agar lebih terstruktur.
3	Dapat membuat form dengan tipe input yang sesuai (text, radio, number, file) berdasarkan kebutuhan data.
4	Menerapkan validasi dasar seperti required, min, serta menghubungkan label dan input dengan benar.
5	Memahami bahwa struktur dan form HTML dapat menjadi dasar pembuatan sistem pencatatan data sederhana di industri.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Mardiansyah, A., Kasah, B. N., Zamzami, H. R., Arabu, M. Y., Nasro, M. A., Kristanto, N., ... & Wulandari, Y. (2025). Pengenalan dasar HTML dan CSS: Langkah pertama dalam pengembangan web. <i>Abdi Jurnal Publikasi</i> , 3(3), 165-170.
[3]	Jeffry, J., Aziz, F., Rijal, M., Arafah, M. N., Irmawati, I., Rakes, R., & Katibin, A. (2025). Pelatihan Dasar Pemrograman Web: Membangun Aplikasi Pertamamu dengan HTML, CSS, dan JavaScript. <i>Jurnal Aktivitas Sosial dan Inovasi</i> , 1(1), 18-22.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
 Radhitya Adha Fadhlur Rohman NIM: 23051430005	 Maessa Andrea Vallenia NIM: 23051430044	 Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

Modul 2 CSS Dasar & Styling

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 2 (Pertemuan 2)
Topik Utama	CSS Dasar Styling
Sub-Topik	CSS Dasar & Styling
Durasi Praktikum	340 Menit
Tanggal Praktikum	20 Februari 2026
Nama Mahasiswa	Radhitya Adha Fadhlur Rohman
NIM	23051430005
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami cara kerja CSS untuk mempercantik antarmuka pengguna (UI).
2	Mampu memanipulasi warna, font, dan ukuran elemen.
3	Menguasai konsep Box Model (Margin, Border, Padding) untuk tata letak
4	Menerapkan CSS pada formulir agar terlihat profesional dan mudah digunakan.

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	Laptop	RYZEN 7	Device Utama

5	HTML	-	Struktur
6	CSS	-	Styling

D. DASAR TEORI

CSS (*Cascading Style Sheets*) merupakan bahasa pemrograman yang digunakan untuk mengatur tampilan visual sebuah halaman web. Jika HTML berfungsi sebagai kerangka atau struktur utama halaman, maka CSS berperan dalam mempercantik dan mengatur bagaimana elemen-elemen tersebut ditampilkan kepada pengguna. Dengan CSS, pengembang dapat mengatur warna, jenis huruf, ukuran teks, jarak antar elemen, hingga tata letak halaman secara lebih menarik dan terstruktur. Dalam pengembangan aplikasi web, tampilan yang baik tidak hanya meningkatkan estetika, tetapi juga membantu pengguna memahami informasi dengan lebih mudah dan nyaman.

Salah satu keunggulan CSS adalah kemampuannya untuk memisahkan antara struktur halaman dan desain visual. Penerapan CSS dapat dilakukan melalui beberapa cara, seperti *inline CSS*, *internal CSS*, dan *external CSS*. Pada praktik pengembangan aplikasi modern, penggunaan *external CSS* lebih disarankan karena membuat kode menjadi lebih rapi, mudah dikelola, dan dapat digunakan kembali pada banyak halaman sekaligus. Dengan pendekatan ini, proses pemeliharaan sistem menjadi lebih efisien karena perubahan tampilan cukup dilakukan pada satu file CSS tanpa harus mengubah setiap halaman HTML secara terpisah.

CSS memiliki konsep penting yang dikenal sebagai *Box Model*. Konsep ini menganggap setiap elemen HTML sebagai sebuah kotak yang terdiri atas *content*, *padding*, *border*, dan *margin*. Pemahaman terhadap *Box Model* sangat membantu dalam mengatur posisi dan ukuran elemen agar tampilan halaman terlihat rapi dan proporsional. Melalui penerapan CSS yang tepat, berbagai komponen seperti formulir, tabel, dan tombol dapat ditampilkan secara lebih profesional serta memberikan pengalaman pengguna (*user experience*) yang lebih baik saat berinteraksi dengan aplikasi web.

D.1 Konsep Utama

Konsep utama pada praktikum ini adalah penggunaan CSS (*Cascading Style Sheets*) untuk mengatur tampilan halaman web agar lebih menarik, rapi, dan nyaman digunakan. Jika HTML berfungsi sebagai kerangka halaman, maka CSS berfungsi untuk mengatur aspek visual seperti warna, ukuran teks, jenis huruf, serta tata letak setiap elemen yang ada di dalam halaman web. Dengan CSS, tampilan website menjadi lebih profesional dan mudah dipahami oleh pengguna.

Selain itu, praktikum ini juga membahas penggunaan *selector* dan konsep *Box Model*. *Selector* digunakan untuk menentukan elemen HTML mana yang akan diberikan gaya tertentu, sedangkan *Box Model* membantu mengatur jarak dan ukuran elemen melalui komponen *content*, *padding*, *border*, dan *margin*. Pemahaman kedua konsep tersebut sangat penting karena menjadi dasar dalam membuat antarmuka web yang rapi, terstruktur, dan responsif.

D.2 Keterkaitan dengan Teknik Industri

Materi CSS sangat relevan dengan bidang Teknik Industri karena banyak sistem yang digunakan dalam perusahaan saat ini berbasis web, seperti sistem monitoring produksi, manajemen inventori, pelaporan kualitas, hingga *dashboard* kinerja. Tidak cukup hanya membuat sistem yang berfungsi dengan baik, tetapi tampilan sistem juga harus mudah dipahami agar operator dan manajer dapat mengakses informasi dengan cepat serta mengurangi kemungkinan terjadinya kesalahan dalam pengambilan keputusan. Contoh penerapannya adalah pada *dashboard* monitoring produksi di sebuah pabrik. Dengan menggunakan CSS, data seperti jumlah produksi, tingkat cacat produk, dan status mesin dapat ditampilkan secara lebih rapi dan informatif melalui pengaturan warna, tata letak, serta ukuran elemen yang sesuai. Tampilan yang baik akan membantu pengguna memantau kondisi operasional secara lebih efektif dan meningkatkan efisiensi kerja di lingkungan industri.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat File CSS Eksternal

Membuat file baru dengan nama style.css lalu menghubungkan file tersebut di html dengan menambahkan kode `<link rel="stylesheet" href="style.css">` di dalam `<head>`

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Data Mesin Produksi</title>
  <link rel="stylesheet" href="style.css">
</head>
```

Screenshot Hasil:

E.2 Langkah 2 – Styling Body dan Tipografi

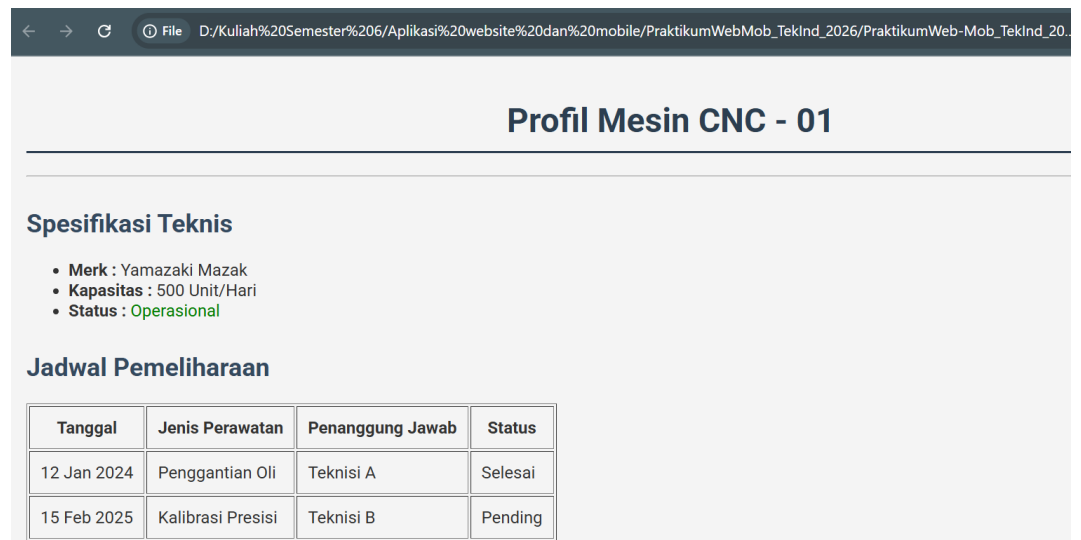
```
/* ===== LANGKAH 2 ===== */
/* Import Font dari Google Fonts */
@import
url(https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap%2
body {
  font-family: 'Roboto', sans-serif;
  background-color: #f4f4f4; /* Warna abu-abu muda untuk background*/
  color: #333;
  margin: 0;
  padding: 20px;
}

h1 {
  color: #2c3e50; /*Biru tua gelap*/
  text-align: center;
```



```
border-bottom: 2px solid #2c3e50;
padding-bottom: 10px;
}

h2 {
color: #34495e;
margin-top: 30px;
```



Gambar E.2 – Output dari Langkah 2

E.3 Langkah 3 – Styling Body dan Tipografi

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
able {
width: 100%;
border-collapse: collapse; /*menggabungkan bordered ganda menjadi
satu*/
margin-top: 20px;
box-shadow: 0 2px 5px rgba((0,0,0,0,.1), green, blue, alpha); /*Efek
bayangan halus*/
}

th, td {
padding: 12px;
text-align: left;
border-bottom: 1px solid #ddd;
}

th {
background-color: #2980b9; /*Biru profesional */
color: white;
}

tr:hover {
background-color: #f1f1f1; /*highlight baris saat mouse diarahkan*/
```

Screenshot Hasil:

Profil Mesin CNC - 01

Spesifikasi Teknis

- Merk : Yamazaki Mazak
- Kapasitas : 500 Unit/Hari
- Status : Operasional

Jadwal Pemeliharaan

Tanggal	Jenis Perawatan	Penanggung Jawab	Status
12 Jan 2024	Penggantian Oli	Teknisi A	Selesai
15 Feb 2025	Kalibrasi Presisi	Teknisi B	Pending

Gambar E.3 – Output dari Langkah 3

E.4 Langkah 4 – Styling Body dan Tipografi

```
/* ===== KODE / OUTPUT LANGKAH 4 ===== */
[form {
  background-color: white;
  padding: 20px;
  border-radius: 8px; /*sudut melengkung*/
}
label {
  display: block;
  margin-bottom: 5px;
  font-weight: bold;
}

input[type="text"],
input[type="number"],
input[type="date"],
select,
textarea {
  width: 100%; /*lebar penuh*/
  padding: 10px;
  margin-bottom: 15px;
  border: 1px solid #ccc;
  border-radius: 4px;
  box-sizing: border-box; /* padding tidak menambah lebar total*/
}

/* Styling khusus tombol */
button {
  background-color: #27ae60; /*hijau*/
  color: white;
  padding: 12px 20px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  width: 100%;
```

```
font-size: 16px;
}

button:hover {
  background-color: #219150; /*hijau lebih gelap saat di hover*/
}
```

Screenshot Hasil:

Profil Mesin CNC - 01

Spesifikasi Teknis

- Merk : Yamazaki Mazak
- Kapasitas : 500 Unit/Hari
- Status : Operasional

Jadwal Pemeliharaan

Tanggal	Jenis Perawatan	Penanggung Jawab	Status
12 Jan 2024	Penggantian Oli	Teknisi A	Selesai
15 Feb 2025	Kalibrasi Presisi	Teknisi B	Pending

Input Data Harian

Tanggal:

Shift Kerja

Pagi (08:00 - 16:00)

Jumlah Produksi (Unit):

Catatan Kerusakan :

Deskripsikan jika anda kendala...

Simpan Data

Gambar E.4 – Output dari Langkah 4

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Class Selector

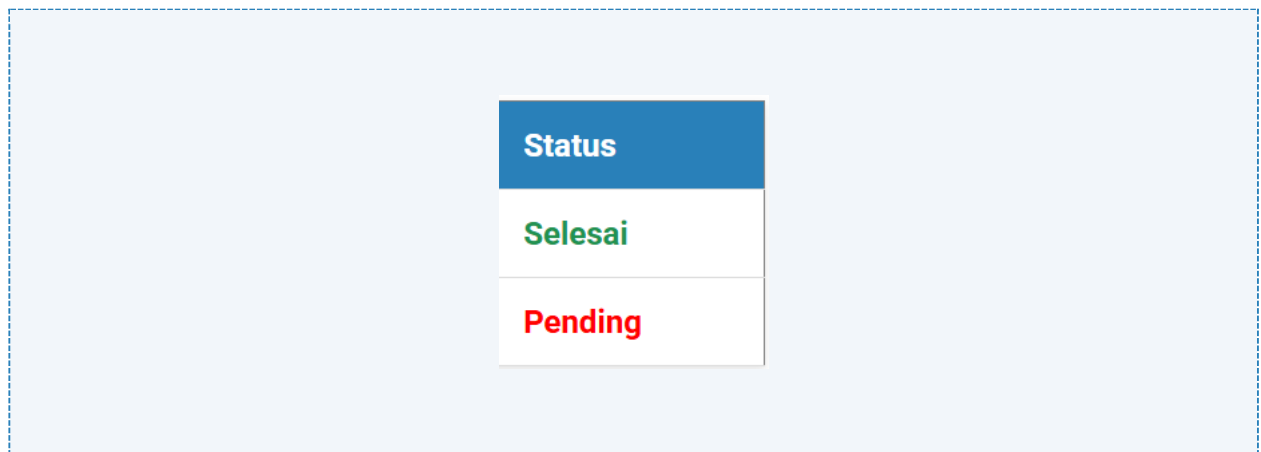
Pertanyaan / Instruksi Latihan:

Berikan kelas `status-ok` pada teks status operasional di HTML, dan berikan warna hijau tebal di CSS. Berikan kelas `status-down` dan berikan warna merah.

Jawaban / Kode Solusi:

```
Pada File HTML Tambahkan ini
<tbody>
    <tr>
        <td>12 Jan 2024</td>
        <td>Penggantian Oli</td>
        <td>Teknisi A</td>
        <td class="status-ok">Selesai</td>
    </tr>
    <tr>
        <td>15 Feb 2024</td>
        <td>Kalibrasi Presisi</td>
        <td>Teknisi B</td>
        <td class="status-down">Pending</td>
    </tr>
</tbody>
</tbody>
Pada File CSS tambahkan ini
/* Penambahan Style status ok dan status down(latihan 1) */
.status-ok {
    color: #219150;
    font-weight: bold;
}

.status-down {
    color: red;
    font-weight: bold;
}
```



Gambar F.1 –Hasil Latihan 1

F.2 Latihan 2 – Layout Container

Pertanyaan / Instruksi Latihan:

Buat sebuah `

` dengan id `container` di HTML yang membungkus seluruh konten. Di CSS, berikan `max-width: 800px`, `margin: 0 auto`, dan `background: white` agar konten terlihat seperti kertas di tengah layar.

Jawaban / Kode Solusi:

```
/* penambahan style container (latihan 2) */
.container {
    max-width: 800px;
    margin: 0 auto;
    background: white;
    padding: 20px;
    border-radius: 15px;
}
```

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="style.css">
    <title>Praktikum 2</title>
</head>
<body>
    <div class="container">
        <h1>Profil Mesin CNC-01</h1>
        <h3>Dibuat oleh : Radhitya Adha Fadhlur Rohman_23051430005</h3>
        <hr>

        <h2>Spesifikasi Teknis</h2>
        <ul>
            <li><strong>Merk:</strong> Yamazaki Mazak</li>
            <li><strong>Kapasitas:</strong> 500 unit/hari</li>
            <!-- Penambahan class status ok di text operasional -->
            <li><strong>Status:</strong> <span class="status-
ok">Operasional</span></li>
        </ul>
        <ul>
            <li><strong>Merk:</strong> Mori Seiki</li>
            <li><strong>Kapasitas:</strong> 300 unit/hari</li>
            <!-- Penambahan class status down di text tidak operasional -->
            <li><strong>Status:</strong> <span class="status-down">Tidak
Operasional</span></li>
        </ul>
        <h2>Jadwal Pemeliharaan</h2>
        <table border="1" cellpadding="10">
            <thead>
                <tr>
                    <th>Tanggal</th>
                    <th>Jenis Perawatan</th>
                    <th>Penanggung Jawab</th>
                    <th>Status</th>
                </tr>
            </thead>
            <tbody>
                <tr>
                    <td>12 Jan 2024</td>
```

```
        <td>Penggantian Oli</td>
        <td>Teknisi A</td>
        <td class="status-ok">Selesai</td>
    </tr>
    <tr>
        <td>15 Feb 2024</td>
        <td>Kalibrasi Presisi</td>
        <td>Teknisi B</td>
        <td class="status-down">Pending</td>
    </tr>
</tbody>
</table>
<hr>
<h2>Input Data Harian</h2>
<form action="">
    <label for="tanggal">Tanggal:</label><br>
    <input type="date" id="tanggal" name="tanggal"
required><br><br>
    <label for="shift">Shift Kerja:</label><br>
    <select id="shift" name="shift">
        <option value="Pagi">Pagi (08:00 - 16:00)</option>
        <option value="Siang">Siang (16:00 - 24:00)</option>
        <option value="Malam">Malam (00:00 - 08:00)</option>
    </select><br><br>
    <label for="jumlah">Jumlah Produksi (Unit):</label><br>
    <input type="number" id="jumlah" name="jumlah" min="0"
placeholder="Masukkan jumlah..."><br><br>
    <label for="catatan">Catatan Kerusakan:</label><br>
    <textarea id="catatan" name="catatan" rows="4" cols="50"
placeholder="Deskripsikan jika ada
kendala..."></textarea><br><br>
    <button type="submit">Simpan Data</button>
</form>
</div>
</body>
</html>
```

Profil Mesin CNC-01

Dibuat oleh : Radhitya Adha Fadhlur Rohman_23051430005

Spesifikasi Teknis

- Merk: Yamazaki Mazak
- Kapasitas: 500 unit/hari
- Status: **Operasional**
- Merk: Mori Seiki
- Kapasitas: 300 unit/hari
- Status: **Tidak Operasional**

Jadwal Pemeliharaan

Tanggal	Jenis Perawatan	Penanggung Jawab	Status
12 Jan 2024	Penggantian Oli	Teknisi A	Selesai
15 Feb 2024	Kalibrasi Presisi	Teknisi B	Pending

Input Data Harian

Tanggal:

Shift Kerja:

Pagi (08:00 - 16:00)

Jumlah Produksi (Unit):

Catatan Kerusakan:

Deskripsikan jika ada kendala...

Simpan Data

Gambar F.2 – Keterangan Hasil Latihan 2

F.3 Tugas Proyek Mini – Industri

Deskripsi proyek mini:

Terapkan styling CSS yang sama ke file `laporan\_kualitas.html` yang Anda buat di pertemuan 1.

- Tambahkan aturan CSS agar tombol "Reset" (jika ada) berwarna merah (`#c0392b`).
- Pastikan form responsif (tetap rapi saat di-zoom).

Penjelasan pendekatan/solusi yang digunakan:

Proyek mini ini diselesaikan dengan membuat sebuah formulir laporan inspeksi kualitas produk menggunakan HTML dan CSS. Langkah pertama yang dilakukan adalah merancang struktur halaman menggunakan HTML yang terdiri dari judul laporan, identitas pembuat, serta beberapa komponen formulir seperti ID produk, nama inspektur, jenis cacat, jumlah produk cacat, dan fitur unggah foto produk. Setiap elemen dipilih sesuai kebutuhan proses inspeksi kualitas sehingga data yang dibutuhkan dapat dikumpulkan secara lengkap dan terstruktur. Setelah struktur halaman selesai dibuat, tahap berikutnya adalah menerapkan CSS untuk memperbaiki tampilan dan meningkatkan kenyamanan pengguna. Styling dilakukan pada bagian latar belakang, judul, formulir, kolom input, serta tombol aksi agar halaman terlihat lebih rapi dan profesional. Selain itu, digunakan konsep *container* untuk memusatkan tampilan formulir, pemberian *border-radius* dan *box-shadow* untuk menambah estetika, serta efek *hover* dan *focus* pada tombol dan kolom input agar interaksi pengguna menjadi lebih mudah. Dengan pendekatan tersebut, dihasilkan halaman formulir inspeksi kualitas yang tidak hanya berfungsi dengan baik tetapi juga memiliki tampilan yang lebih menarik dan mudah digunakan.

Kode Utama:

```
<!DOCTYPE html>
<html lang="id">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Laporan Kualitas Produk</title>
  <link rel="stylesheet" href="stylekualitas.css">
</head>

<body>
  <div class="container">
    <header>
      <h1>Laporan Inspeksi Kualitas</h1>
      <h3>Dibuat oleh : Radhitya Adha Fadhlur
Rohman_23051430005</h3>
    </header>
    <section>
      <form action="">
        <!-- field id produk -->
        <label for="id_produk">Id Produk :</label><br>
        <input type="text" id="id_produk" name="Id Produk"
placeholder="Masukkan id produk" required><br><br>

        <!-- field nama inspektur -->
        <label for="nama_inspektur">Nama Inspektur :</label><br>
        <input type="text" id="nama_inspektur"
name="nama_inspektur" placeholder="Masukkan nama inspektur"
required><br><br>

        <!-- field jenis cacat -->
        <label>Jenis Cacat :</label>
        <!-- Radio opsi geometri -->
```



```

        <input type="radio" id="geometri" name="status"
value="geometri">
        <label class="label-radio"
for="geometri">Geometri</label><br>
        <!-- Radio opsi permukaan -->
        <input type="radio" id="permukaan" name="status"
value="permukaan">
        <label class="label-radio"
for="permukaan">Permukaan</label><br>
        <!-- Radio opsi fungsional -->
        <input type="radio" id="fungsional" name="status"
value="fungsional">
        <label class="label-radio"
for="fungsional">Fungsional</label><br><br>

        <!-- field jumlah cacat -->
        <label for="jumlah_cacat">Jumlah Produk Cacat (Unit)
: </label><br>
        <input type="number" id="jumlah_cacat" name="jumlah_cacat"
min="0"
        placeholder="Masukkan jumlah produk cacat"><br><br>

        <!-- field foto produk -->
        <label for="foto_produk">Upload Foto Produk :</label><br>
        <input type="file" id="foto_produk" name="foto_produk"
        placeholder="Upload foto produk dari perangkat Anda">
        <br><br>
        <button type="submit">Simpan Data</button><br><br>
        <button type="reset">Reset data</button>
    </form>
</section>
</div>
</body>

</html>

```

```

/* Import Font dari Google Fonts */
@import
url('https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap')

body {
    font-family: 'Roboto', sans-serif;
    background-color: #f4f4f4;
    color: #333;
    margin: 0;
    padding: 20px;
}

/* =====
HEADER
===== */
h1 {
    color: #2c3e50;
    text-align: center;
    border-bottom: 2px solid #2c3e50;

```

```
        padding-bottom: 10px;
    }

    h3 {
        color: #34495e;
        text-align: center;
        margin-bottom: 30px;
    }

    /* =====
    CONTAINER (Latihan Layout)
    ===== */
    .container {
        max-width: 800px;
        margin: auto;
        background: white;
        padding: 25px;
        border-radius: 15px;
        box-shadow: 0 5px 15px rgba(0,0,0,0.1);
    }

    /* =====
    FORM STYLE
    ===== */
    form {
        max-width: 600px;
        margin: 0 auto;
    }

    label {
        display: block;
        margin-bottom: 5px;
        font-weight: bold;
    }

    /* Label khusus radio */
    .label-radio {
        display: inline-block;
        font-weight: normal;
        margin-right: 15px;
    }

    /* Input Text & Number */
    input[type="text"],
    input[type="number"],
    input[type="file"],
    select,
    textarea {
        width: 100%;
        padding: 10px;
        margin-bottom: 15px;
        border: 1px solid #ccc;
        border-radius: 6px;
        box-sizing: border-box;
        transition: 0.3s;
    }

    /* Efek saat input diklik */
    input[type="text"]:focus,
```

```
input[type="number"]:focus,
textarea:focus,
select:focus {
    outline: none;
    border-color: #28a745;
    box-shadow: 0 0 5px rgba(40,167,69,0.3);
}

/* Radio tidak dibuat full width */
input[type="radio"] {
    margin-right: 5px;
}

/* =====
   BUTTON STYLE
   ===== */
button {
    padding: 12px;
    border: none;
    border-radius: 6px;
    cursor: pointer;
    width: 100%;
    font-size: 16px;
    margin-bottom: 10px;
    transition: 0.3s;
}

/* Tombol Submit */
button[type="submit"] {
    background-color: #28a745;
    color: white;
}

button[type="submit"]:hover {
    background-color: #218838;
}

/* Tombol Reset */
button[type="reset"] {
    background-color: #dc3545;
    color: white;
}

button[type="reset"]:hover {
    background-color: #ffffff;
}
```

Laporan Inspeksi Kualitas

Dibuat oleh : Radhitya Adha Fadhlur Rohman_23051430005

Id Produk :

Nama Inspektur :

Jenis Cacat :

☐ Geometri

☒ Permukaan

☐ Fungsional

Jumlah Produk Cacat(Unit) :

Upload Foto Produk :

Tidak ada file yang dipilih

Simpan Data

Reset data

Gambar F.3 – Hasil Proyek Mini

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang diperoleh telah sesuai dengan teori dan konsep yang dipelajari pada modul, khususnya mengenai penggunaan HTML sebagai pembentuk struktur halaman web dan CSS sebagai pengatur tampilan visual. Pada hasil pengerjaan, HTML berhasil digunakan untuk membuat formulir inspeksi kualitas yang terdiri dari berbagai elemen input seperti *text field*, *radio button*, *number field*, *file upload*, serta tombol *submit* dan *reset*. Seluruh elemen tersebut dapat ditampilkan dengan baik sesuai fungsi yang telah dirancang, sehingga menunjukkan bahwa konsep dasar pembuatan formulir menggunakan HTML telah diterapkan dengan benar.

Dari sisi CSS, hasil yang diperoleh juga menunjukkan kesesuaian dengan teori mengenai *styling* dan pengaturan tata letak halaman web. Penggunaan *container* dengan lebar maksimum tertentu membuat formulir tampil lebih rapi dan terpusat di halaman. Selain itu, penerapan warna, jenis huruf, *border-radius*, dan *box-shadow* berhasil memberikan tampilan yang lebih menarik dibandingkan tampilan HTML tanpa CSS. Efek *hover* pada tombol serta *focus* pada kolom input juga bekerja sesuai konsep yang dipelajari, yaitu memberikan umpan balik visual kepada pengguna saat berinteraksi dengan halaman.

Output yang dihasilkan membuktikan bahwa teori mengenai pemisahan struktur dan tampilan dalam pengembangan web telah diterapkan dengan baik. HTML digunakan untuk membangun fungsi dan susunan elemen, sedangkan CSS digunakan untuk mengatur aspek visual tanpa mengubah struktur utama halaman. Hasil akhirnya berupa formulir laporan inspeksi kualitas yang tidak hanya dapat digunakan untuk memasukkan data, tetapi juga memiliki tampilan yang lebih profesional, mudah dibaca, dan nyaman digunakan. Hal ini menunjukkan bahwa tujuan praktikum untuk memahami dasar-dasar *web styling* dan penerapan CSS pada halaman web telah berhasil tercapai.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Tampilan formulir pada awalnya kurang rapi karena elemen input dan tombol tidak memiliki pengaturan ukuran serta jarak yang konsisten.	Menambahkan aturan CSS pada elemen <i>input</i> dan <i>button</i> seperti <i>padding</i> , <i>margin</i> , <i>width: 100%</i> , dan <i>box-sizing: border-box</i> sehingga tata letak menjadi lebih rapi dan seragam.
File CSS tidak langsung memberikan perubahan pada halaman web karena terdapat kesalahan pada proses penghubungan file CSS dengan file HTML.	Memeriksa kembali penulisan atribut <i>href</i> pada tag <code><link></code> dan memastikan nama file CSS serta lokasi penyimpanannya sudah sesuai dengan file HTML yang digunakan.

G.3 Kaitan dengan Penerapan di Industri

Keterampilan yang dipelajari pada modul ini memiliki kaitan yang erat dengan dunia Teknik Industri, terutama dalam pengembangan sistem informasi yang digunakan untuk mendukung proses operasional perusahaan. Kemampuan membuat halaman web menggunakan HTML dan mempercantik tampilannya dengan CSS dapat dimanfaatkan untuk membangun berbagai aplikasi berbasis web, seperti sistem monitoring produksi, pelaporan kualitas,

manajemen inventori, hingga dashboard kinerja perusahaan. Tampilan yang rapi dan mudah dipahami sangat penting agar pengguna dapat mengakses informasi dengan cepat dan meminimalkan kesalahan dalam penggunaan sistem.

Salah satu contoh penerapan nyatanya adalah pada proses pengendalian kualitas (*quality control*) di perusahaan manufaktur. Formulir inspeksi kualitas seperti yang dibuat pada proyek mini ini dapat dikembangkan menjadi sistem pelaporan digital yang digunakan oleh petugas inspeksi untuk mencatat data produk cacat secara langsung melalui perangkat komputer atau tablet. Data yang diinput kemudian dapat tersimpan dalam basis data dan ditampilkan dalam bentuk laporan atau dashboard untuk membantu manajemen menganalisis tren cacat produk serta mengambil keputusan perbaikan proses produksi. Dengan demikian, keterampilan pengembangan antarmuka web yang dipelajari pada modul ini dapat mendukung digitalisasi proses kerja dan peningkatan efisiensi di lingkungan industri.

H. KESIMPULAN

1	Praktikum ini memberikan pemahaman mengenai penggunaan HTML untuk membangun struktur halaman web, khususnya dalam pembuatan formulir laporan inspeksi kualitas yang terdiri dari berbagai jenis input sesuai kebutuhan pengguna.
2	CSS berhasil diterapkan untuk meningkatkan tampilan halaman web melalui pengaturan warna, tata letak, ukuran elemen, dan efek interaktif sehingga halaman terlihat lebih rapi, menarik, dan mudah digunakan.
3	Konsep <i>Box Model</i> , <i>selector</i> , serta penggunaan <i>external CSS</i> dapat dipahami dan diterapkan dengan baik dalam proses pengembangan antarmuka web yang terstruktur dan mudah dikelola.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	[Tuliskan referensi ke-2 dalam format: Penulis, A. A. (Tahun). Judul artikel/buku/halaman. Nama Jurnal/Website. URL (jika online)]
[3]	[Tuliskan referensi ke-3 dalam format: Penulis, A. A. (Tahun). Judul artikel/buku/halaman. Nama Jurnal/Website. URL (jika online)]
[4]	[Tuliskan referensi ke-4 dalam format: Penulis, A. A. (Tahun). Judul artikel/buku/halaman. Nama Jurnal/Website. URL (jika online)]

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
 Radhitya Adha Fadhlur Rohman NIM: 23051430005	 Maessa Andrea Vallenia NIM: 23051430044	 Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

Modul 3 Layout CSS Lanjutan (Flexbox & Grid) + Bootstrap**A. IDENTITAS & INFORMASI MODUL**

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 3 (Pertemuan 3)
Topik Utama	Layout CSS Lanjutan (Flexbox & Grid) + Bootstrap
Sub-Topik	<i>CSS Flexbox, CSS Grid, dan Framework CSS (Bootstrap 5)</i>
Durasi Praktikum	340 Menit
Tanggal Praktikum	27 Februari 2026
Nama Mahasiswa	Radhitya Adha Fadhlur Rohman
NIM	23051430005
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami konsep layout modern menggunakan Flexbox dan Grid.
2	Mengenal dan menggunakan Framework CSS (Bootstrap) untuk mempercepat pengembangan UI.
3	Mampu membuat layout dashboard responsif (Sidebar + Main Content).
4	Menerapkan komponen Bootstrap (Card, Navbar, Button) pada konteks industri.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging

3	Bootstrap 5 (CDN)	v5.3.0	Framework CSS untuk membuat layout responsif dan komponen UI
4	Git dan GitHub	Versi Terbaru dan Web-based platform	Manajemen versi & repository
5	Koneksi Internet	Stabil	Untuk mengakses CDN Bootstrap dan upload repository ke GitHub

D. DASAR TEORI

Pada pengembangan web modern, HTML dan CSS menjadi fondasi utama dalam membangun antarmuka pengguna. HTML berfungsi sebagai struktur dasar yang menyusun elemen seperti teks, gambar, tabel, dan form, sedangkan CSS digunakan untuk mengatur tampilan visualnya, mulai dari warna, tipografi, hingga tata letak. Pemisahan antara struktur dan desain ini membuat kode lebih terorganisir dan mudah dikembangkan kembali. Selain itu, konsep responsive web design memungkinkan tampilan halaman menyesuaikan ukuran layar perangkat, sehingga website tetap nyaman digunakan baik di desktop maupun smartphone (Santoso, 2025).

Dalam pengaturan layout yang lebih kompleks, CSS menyediakan pendekatan modern melalui Flexbox dan Grid. Flexbox digunakan untuk mengatur tata letak satu dimensi (baris atau kolom) sehingga memudahkan pengaturan posisi dan distribusi ruang antar elemen dalam satu container. Sementara itu, CSS Grid dirancang untuk tata letak dua dimensi karena mampu mengelola baris dan kolom secara bersamaan dengan lebih sistematis. Kedua metode ini menjadi solusi yang lebih efektif dibanding teknik lama seperti float, karena lebih fleksibel dan mendukung desain yang responsif serta dinamis.

Selain menggunakan CSS murni, pengembangan antarmuka juga dapat dipercepat dengan framework seperti *Bootstrap*. *Bootstrap* menyediakan sistem grid 12 kolom dan berbagai komponen siap pakai seperti navbar, card, button, dan form yang sudah responsif secara bawaan. Menurut Santoso (2025), penggunaan Bootstrap dapat mempercepat proses pembuatan prototipe karena pengembang

tidak perlu menulis seluruh styling dari awal. Dalam konteks praktikum ini, pemanfaatan *Flexbox*, *Grid*, dan *Bootstrap* membantu mahasiswa memahami bagaimana membangun layout dashboard yang rapi, responsif, dan sesuai dengan kebutuhan aplikasi industri.

D.1 Konsep Utama

Konsep utama pada modul ini adalah memahami cara membuat layout web yang rapi dan responsif menggunakan Flexbox, CSS Grid, dan framework Bootstrap. Layout sangat penting karena menentukan bagaimana elemen seperti navbar, card, tabel, dan form disusun di dalam halaman. Flexbox digunakan untuk mengatur elemen dalam satu arah (baris atau kolom), sedangkan Grid membantu menyusun elemen dalam bentuk baris dan kolom sekaligus, sehingga cocok untuk membuat tampilan dashboard.

Selain itu, modul ini juga menekankan pentingnya desain responsif, yaitu tampilan web yang bisa menyesuaikan ukuran layar. Dengan sistem grid 12 kolom pada Bootstrap, kita bisa membuat tampilan yang awalnya tiga kolom di desktop menjadi satu kolom di layar HP secara otomatis. Dari praktikum ini, saya memahami bahwa membuat tampilan web tidak hanya soal desain, tetapi juga soal bagaimana menyusunnya dengan sistem yang efisien dan sesuai praktik industri.

D.2 Keterkaitan dengan Teknik Industri

Materi layout CSS ini cukup relevan dengan Teknik Industri, terutama dalam pembuatan sistem informasi seperti dashboard monitoring produksi, stok gudang, atau performa mesin. Di dunia industri, data harus ditampilkan dengan jelas dan terstruktur supaya mudah dipahami dan tidak menimbulkan salah baca. Dengan bantuan Flexbox, Grid, dan *Bootstrap*, tampilan dashboard bisa dibuat rapi dan responsif sehingga nyaman diakses baik lewat komputer maupun smartphone. Hal ini penting karena pengambilan keputusan sering kali dilakukan berdasarkan data yang ditampilkan secara real-time.

Sebagai contoh, supervisor produksi bisa memantau jumlah output, tingkat reject, atau efisiensi mesin langsung dari dashboard tanpa harus membuka laporan manual. Jika layout-nya tersusun dengan baik, informasi bisa dipahami lebih cepat dan risiko human error juga berkurang. Selain itu, penggunaan framework seperti Bootstrap memungkinkan pembuatan

prototype sistem dilakukan lebih cepat sehingga mempermudah proses perancangan dan evaluasi sistem. Ini sejalan dengan peran Teknik Industri dalam

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Integrasi Bootstrap 5

Langkah pertama adalah membuka kembali file index.html dari pertemuan sebelumnya. Selanjutnya dilakukan integrasi Bootstrap 5 menggunakan CDN dengan menambahkan link CSS pada bagian <head> dan Bootstrap JS Bundle sebelum penutup </body>. Tujuannya agar komponen seperti grid, navbar, card, dan form dapat digunakan dengan mudah tanpa menulis CSS dari awal.

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */  
  
<!DOCTYPE html>  
<html lang="id">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Sistem Monitoring Produksi</title>  
  
  <!-- Bootstrap CSS CDN -->  
  <link  
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"  
  rel="stylesheet">  
  <link rel="stylesheet" href="style.css">  
</head>
```

Screenshot Hasil:

```

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sistem Monitoring Produksi</title>

  <!-- Bootstrap CSS CDN -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
        rel="stylesheet">
  <link rel="stylesheet" href="style.css">
</head>

```

Gambar E.1 – Kode dari Langkah 1

E.2 Langkah 2 – Membuat Navigasi Bar (Navbar)

Langkah berikutnya, menghapus konten lama yang terdapat didalam ‘<body>’, kemudian diganti dengan kode Navbar industri.

```

/* ===== KODE / OUTPUT LANGKAH 2 ===== */

<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container-fluid">
    <a class="navbar-brand" href="#">TekInd
Systems</a>

    <button class="navbar-toggler" type="button"
      data-bs-toggle="collapse"
      data-bs-target="#navbarNav">
      <span class="navbar-toggler-icon"></span>
    </button>

    <div class="collapse navbar-collapse"
id="navbarNav">
      <ul class="navbar-nav">
        <li class="nav-item">
          <a class="nav-link active"
href="#">Dashboard</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="#">Input
Produksi</a>
        </li>
        <li class="nav-item">

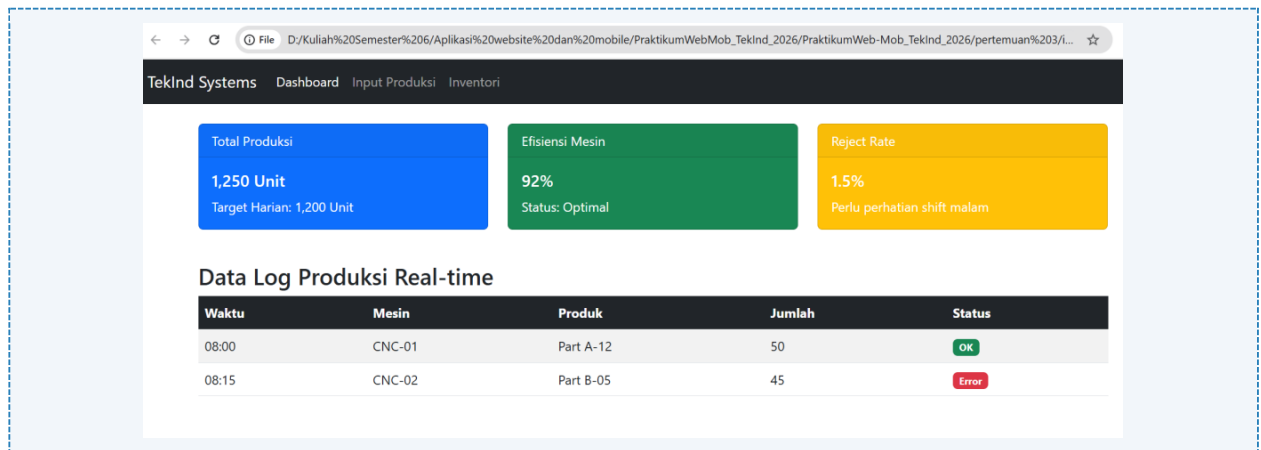
```

```

<a class="nav-link"
href="#">Inventori</a>
</li>
</ul>
</div>
</div>
</nav>

```

Screenshot Hasil:



Gambar E.2 – Output dari Langkah 2

E.3 Langkah 3 – Layout Grid Dashboard (Sistem Grid)

Selanjutnya, di bawah bagian navbar dibuat layout dashboard menggunakan sistem Grid 12 kolom Bootstrap. Baris pertama diisi tiga kartu KPI (Total Produksi, Efisiensi Mesin, dan Reject Rate) masing-masing menggunakan kelas col-md-4, sedangkan baris kedua berisi tabel log produksi real-time. Penggunaan col-md-4 membuat tampilan responsif, sehingga pada layar kecil kartu akan tersusun secara vertikal.

```

<div class="container mt-4">

  <!-- ROW 1: KPI CARDS -->
  <div class="row mb-4">

    <div class="col-md-4">
      <div class="card text-white bg-primary mb-3">
        <div class="card-header">Total
Produksi</div>
        <div class="card-body">
          <h5 class="card-title">1,250 Unit</h5>

```

```

        <p class="card-text">Target Harian:
1,200 Unit</p>
    </div>
</div>
</div>

<div class="col-md-4">
    <div class="card text-white bg-success mb-3">
        <div class="card-header">Efisiensi
Mesin</div>
        <div class="card-body">
            <h5 class="card-title">92%</h5>
            <p class="card-text">Status:
Optimal</p>
        </div>
    </div>
</div>

<div class="col-md-4">
    <div class="card text-white bg-warning mb-3">
        <div class="card-header">Reject Rate</div>
        <div class="card-body">
            <h5 class="card-title">1.5%</h5>
            <p class="card-text">Perlu perhatian
shift malam</p>
        </div>
    </div>
</div>

</div>

<!-- ROW 2: TABEL DATA -->
<div class="row">
    <div class="col-12">
        <h3>Data Log Produksi Real-time</h3>

        <table class="table table-striped table-
hover">
            <thead class="table-dark">
                <tr>
                    <th>Waktu</th>
                    <th>Mesin</th>
                    <th>Produk</th>
                    <th>Jumlah</th>
                    <th>Status</th>

```

```

        </tr>
    </thead>
    <tbody>
        <tr>
            <td>08:00</td>
            <td>CNC-01</td>
            <td>Part A-12</td>
            <td>50</td>
            <td><span class="badge bg-
success">OK</span></td>
        </tr>
        <tr>
            <td>08:15</td>
            <td>CNC-02</td>
            <td>Part B-05</td>
            <td>45</td>
            <td><span class="badge bg-
danger">Error</span></td>
        </tr>
    </tbody>
</table>

</div>
</div>

</div>
}

```

Screenshot Hasil:

The screenshot displays the TekInd Systems Dashboard. At the top, there are navigation links: TekInd Systems, Dashboard, Input Produksi, and Inventori. The dashboard features three main performance indicators (KPIs) in colored boxes:

- Total Produksi:** 1,250 Unit (blue box). Target Harian: 1,200 Unit.
- Efisiensi Mesin:** 92% (green box). Status: Optimal.
- Reject Rate:** 1.5% (yellow box). Perlu perhatian shift malam.

Below the KPIs is a section titled "Data Log Produksi Real-time" containing a table with the following data:

Waktu	Mesin	Produk	Jumlah	Status
08:00	CNC-01	Part A-12	50	OK
08:15	CNC-02	Part B-05	45	Error

Gambar E.3 – Output dari Langkah 3

E.4 Langkah 4 – Styling Form dengan Bootstrap Classes

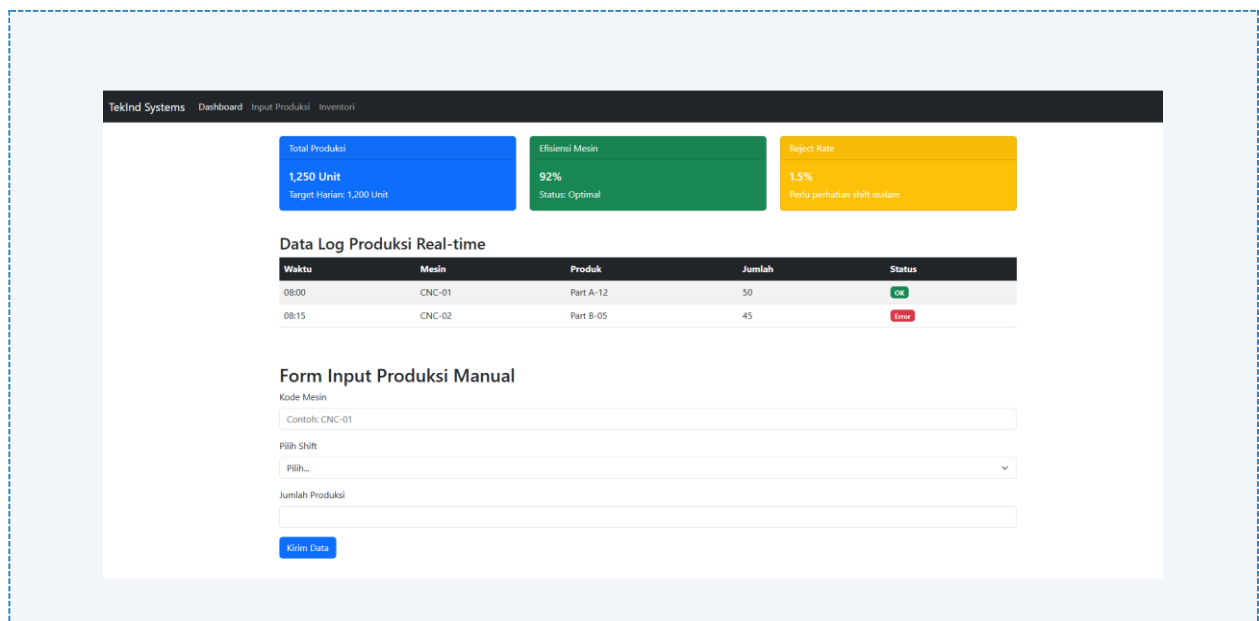
Terakhir, form yang sebelumnya masih ditulis secara manual diperbaiki tampilannya dengan memanfaatkan komponen form dari Bootstrap. Penggunaan class seperti form-control, form-label, dan btn membuat susunan input menjadi lebih rapi, konsisten, dan responsif tanpa perlu menambahkan CSS tambahan.

```
/* ===== KODE / OUTPUT LANGKAH 4 ===== */  
<div class="container mt-5">  
  <h2>Form Input Produksi Manual</h2>  
  
  <form>  
  
    <div class="mb-3">  
      <label for="kodeMesin" class="form-label">Kode Mesin</label>  
      <input type="text"  
        class="form-control"  
        id="kodeMesin"  
        placeholder="Contoh: CNC-01">  
    </div>  
  
    <div class="mb-3">  
      <label for="shiftSelect" class="form-label">Pilih Shift</label>  
      <select class="form-select" id="shiftSelect">  
        <option selected>Pilih...</option>  
        <option value="1">Shift 1 (Pagi)</option>  
        <option value="2">Shift 2 (Siang)</option>  
        <option value="3">Shift 3 (Malam)</option>  
      </select>  
    </div>  
  
    <div class="mb-3">  
      <label for="jumlahProduksi" class="form-label">Jumlah Produksi</label>  
      <input type="number"  
        class="form-control"  
        id="jumlahProduksi">  
    </div>  
  
    <button type="submit" class="btn btn-primary">  
      Kirim Data  
    </button>  
  
  </form>  
</div>
```



```
<!-- Bootstrap JS Bundle -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js">
</body>
</html>
```

Screenshot Hasil:



Gambar E.4 – Output dari Langkah 4

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Grid Responsif

Pertanyaan / Instruksi Latihan:

Coba ubah ukuran layar browser Anda. Perhatikan bagaimana kartu KPI yang tadinya sejajar (3 kolom) akan menumpuk ke bawah (1 kolom) di layar HP. Ini adalah kekuatan Grid System (`col-md-4` artinya 4 kolom di layar medium ke atas, otomatis 12 kolom/1 baris di layar kecil).

Jawaban / Kode Solusi:

```
<!DOCTYPE html>
<html lang="id">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
    <title>Sistem Monitoring Produksi</title>
    <link
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
      rel="stylesheet"
    />
  </head>
  <body>
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
      <div class="container-fluid">
        <a class="navbar-brand" href="#">TekInd Systems</a>
        <button
          class="navbar-toggler"
          type="button"
          data-bs-toggle="collapse"
          data-bs-target="#navbarNav"
        >
          <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="navbarNav">
          <ul class="navbar-nav">
            <li class="nav-item">
              <a class="nav-link active" href="#">Dashboard</a>
            </li>
            <li class="nav-item">
              <a class="nav-link" href="#">Input Produksi</a>
            </li>
            <li class="nav-item">
              <a class="nav-link" href="#">Inventori</a>
            </li>
          </ul>
        </div>
      </div>
    </nav>
    <div class="container mt-4">
      <!-- Row 1: KPI Cards (Menggunakan Grid System) -->
      <div class="row mb-4">
        <!-- Kolom lebar 4 (dari 12) -->
        <div class="col-12 col-md-4">
          <div class="card text-white bg-primary mb-3">
            <div class="card-header">Total Produksi</div>
            <div class="card-body">
```

```

        <h5 class="card-title">1,250 Unit</h5>
        <p class="card-text">Target Harian: 1,200 Unit</p>
    </div>
</div>
<!-- Kolom lebar 4 -->
<div class="col-12 col-md-4">
    <div class="card text-white bg-success mb-3">
        <div class="card-header">Efisiensi Mesin</div>
        <div class="card-body">
            <h5 class="card-title">92%</h5>
            <p class="card-text">Status: Optimal</p>
        </div>
    </div>
</div>
<!-- Kolom lebar 4 -->
<div class="col-12 col-md-4">
    <div class="card text-white bg-warning mb-3">
        <div class="card-header">Reject Rate</div>
        <div class="card-body">
            <h5 class="card-title">1.5%</h5>
            <p class="card-text">Perlu perhatian shift malam</p>
        </div>
    </div>
</div>
</div>
<!-- Row 2: Tabel Data -->
<div class="row">
    <div class="col-12">
        <h3>Data Log Produksi Real-time</h3>
        <table class="table table-striped table-hover">
            <thead class="table-dark">
                <tr>
                    <th>Waktu</th>
                    <th>Mesin</th>
                    <th>Produk</th>
                    <th>Jumlah</th>
                    <th>Status</th>
                </tr>
            </thead>
            <tbody>
                <tr>
                    <td>08:00</td>
                    <td>CNC-01</td>
                    <td>Part A-12</td>
                    <td>50</td>
                </tr>
            </tbody>
        </table>
    </div>
</div>

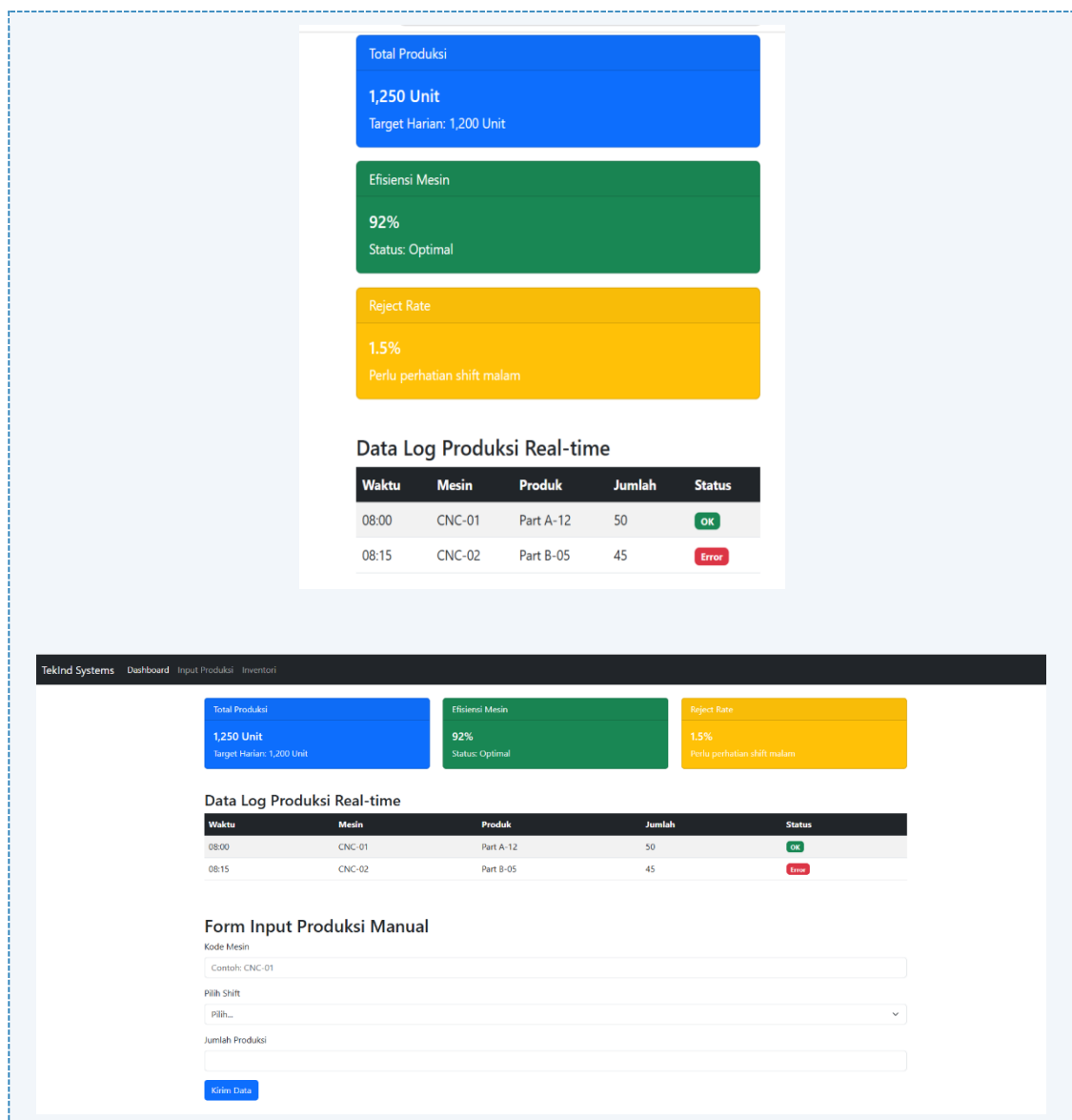
```

```

        <td><span class="badge bg-success">OK</span></td>
      </tr>
      <tr>
        <td>08:15</td>
        <td>CNC-02</td>
        <td>Part B-05</td>
        <td>45</td>
        <td><span class="badge bg-danger">Error</span></td>
      </tr>
    </tbody>
  </table>
</div>
</div>
</div>
<div class="container mt-5">
  <h2>Form Input Produksi Manual</h2>

  <form>
    <div class="mb-3">
      <label for="kodeMesin" class="form-label">Kode Mesin</label>
      <input
        type="text"
        class="form-control"
        id="kodeMesin"
        placeholder="Contoh:
          CNC-01"
      />
    </div>
    <div class="mb-3">
      <label for="shiftSelect" class="form-label">Pilih Shift</label>
      <select class="form-select" id="shiftSelect">
        <option selected>Pilih...</option>
        <option value="1">Shift 1 (Pagi)</option>
        <option value="2">Shift 2 (Siang)</option>
        <option value="3">Shift 3 (Malam)</option>
      </select>
    </div>
    <div class="mb-3">
      <label for="jumlahProduksi" class="form-label">Jumlah Pr<
      <input type="number" class="form-control" id="jumlahProcc
    </div>
    <button type="submit" class="btn btn-primary">Kirim Data</button>
  </form>
</div>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap

```



Gambar F.1 – Output dari Latihan 1

F.2 Latihan 2 – Layout Container

Pertanyaan / Instruksi Latihan:

Ubah kartu "Efisiensi Mesin" menjadi warna merah (`bg-danger`) jika angkanya diubah menjadi di bawah 80%.

Jawaban / Kode Solusi:

```
< !DOCTYPE html>
<html lang="id">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>Sistem Monitoring Produksi</title>
    <link
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
      rel="stylesheet"
    />
  </head>
  <body>
    <!-- NAVBAR -->
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
      <div class="container-fluid">
        <a class="navbar-brand" href="#">TekInd Systems</a>
        <button
          class="navbar-toggler"
          type="button"
          data-bs-toggle="collapse"
          data-bs-target="#navbarNav"
        >
          <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="navbarNav">
          <ul class="navbar-nav">
            <li class="nav-item">
              <a class="nav-link active" href="#">Dashboard</a>
            </li>
            <li class="nav-item">
              <a class="nav-link" href="#">Input Produksi</a>
            </li>
            <li class="nav-item">
              <a class="nav-link" href="#">Inventori</a>
            </li>
          </ul>
        </div>
      </div>
    </nav>
```

```

        </div>
    </div>
</nav>

<!-- DASHBOARD -->
<div class="container mt-4">
    <!-- Row 1: KPI Cards -->
    <div class="row mb-4">

        <!-- Total Produksi -->
        <div class="col-12 col-md-4">
            <div class="card text-white bg-primary mb-3">
                <div class="card-header">Total Produksi</div>
                <div class="card-body">
                    <h5 class="card-title">1,250 Unit</h5>
                    <p class="card-text">Target Harian: 1,200 Unit</p>
                </div>
            </div>
        </div>

        <!-- Efisiensi Mesin -->
        <div class="col-12 col-md-4">
            <div id="cardEfisiensi" class="card text-white bg-success mb-3">
                <div class="card-header">Efisiensi Mesin</div>
                <div class="card-body">
                    <h5 id="nilaiEfisiensi" class="card-title">75%</h5>
                    <p id="statusEfisiensi" class="card-text">Status: Operasional</p>
                </div>
            </div>
        </div>

        <!-- Reject Rate -->
        <div class="col-12 col-md-4">
            <div class="card text-white bg-warning mb-3">
                <div class="card-header">Reject Rate</div>
                <div class="card-body">
                    <h5 class="card-title">1.5%</h5>
                    <p class="card-text">Perlu perhatian shift malam</p>
                </div>
            </div>
        </div>
    </div>

    <!-- Row 2: Tabel -->
    <div class="row">

```

```

<div class="col-12">
  <h3>Data Log Produksi Real-time</h3>
  <table class="table table-striped table-hover">
    <thead class="table-dark">
      <tr>
        <th>Waktu</th>
        <th>Mesin</th>
        <th>Produk</th>
        <th>Jumlah</th>
        <th>Status</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>08:00</td>
        <td>CNC-01</td>
        <td>Part A-12</td>
        <td>50</td>
        <td><span class="badge bg-success">OK</span></td>
      </tr>
      <tr>
        <td>08:15</td>
        <td>CNC-02</td>
        <td>Part B-05</td>
        <td>45</td>
        <td><span class="badge bg-danger">Error</span></td>
      </tr>
    </tbody>
  </table>
</div>
</div>
</div>

<!-- FORM INPUT -->
<div class="container mt-5">
  <h2>Form Input Produksi Manual</h2>
  <form>
    <div class="mb-3">
      <label for="kodeMesin" class="form-label">Kode Mesin</label>
      <input
        type="text"
        class="form-control"
        id="kodeMesin"
        placeholder="Contoh: CNC-01"
      />
    </div>
  </form>
</div>

```



```
<div class="mb-3">
  <label for="shiftSelect" class="form-label">Pilih Shift<
  <select class="form-select" id="shiftSelect">
    <option selected>Pilih...</option>
    <option value="1">Shift 1 (Pagi)</option>
    <option value="2">Shift 2 (Siang)</option>
    <option value="3">Shift 3 (Malam)</option>
  </select>
</div>

<div class="mb-3">
  <label for="jumlahProduksi" class="form-label">Jumlah Pr
  <input type="number" class="form-control" id="jumlahProo
</div>

  <button type="submit" class="btn btn-primary">Kirim Data</
</form>
</div>

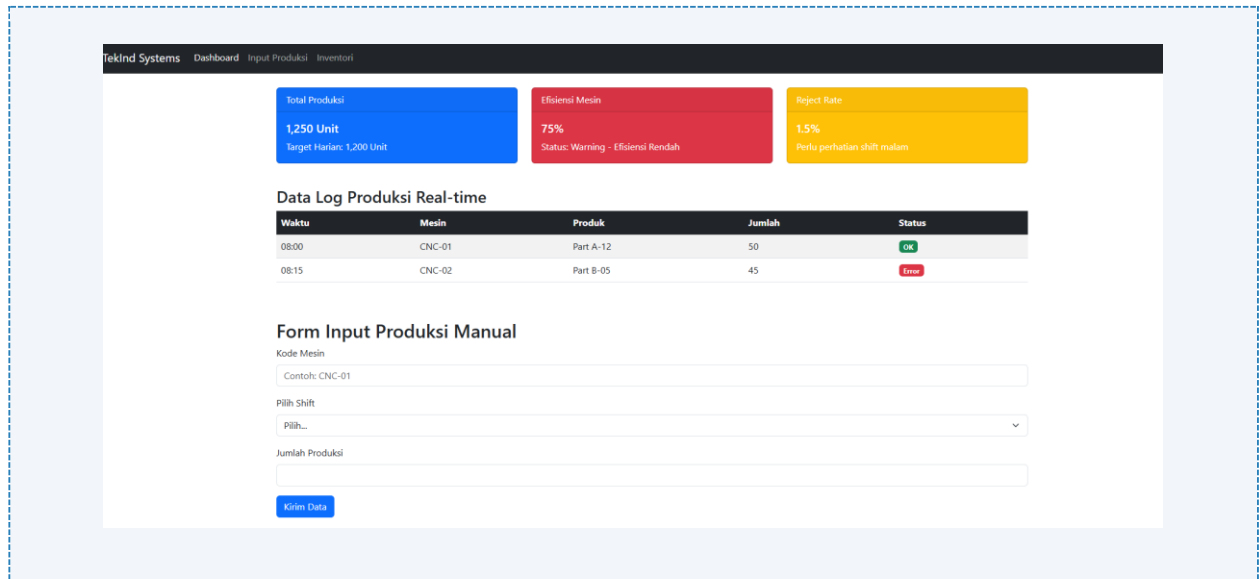
<!-- Bootstrap JS -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstra

<!-- Script Logika Perubahan Warna -->
<script>
  const nilaiElement = document.getElementById("nilaiEfisiensi")
  const cardElement = document.getElementById("cardEfisiensi")
  const statusElement = document.getElementById("statusEfisien

  let nilai = parseInt(nilaiElement.textContent.replace("%", "

  if (nilai < 80) {
    cardElement.classList.remove("bg-success");
    cardElement.classList.add("bg-danger");
    statusElement.textContent = "Status: Warning - Efisiensi R
  } else {
    statusElement.textContent = "Status: Optimal";
  }
</script>

</body>
</html>
```



Gambar F.2 – Output dari Latihan 2

F.3 Tugas Proyek Mini – Industri

Deskripsi proyek mini:

Buatlah struktur dashboard baru bernama `dashboard\_inventori.html`.

- Gunakan Bootstrap Grid.
- Bagian atas: Judul "Gudang Bahan Baku".
- Bagian kiri (4 kolom): Form Stok Masuk (Input nama barang, qty, supplier).
- Bagian kanan (8 kolom): Tabel Stok Saat Ini (Gunakan tabel Bootstrap).

Penjelasan pendekatan/solusi yang digunakan:

Pada project mini ini, saya membuat file dashboard_inventori.html dengan memanfaatkan sistem grid dari Bootstrap sebagai kerangka utama layout. Struktur halaman diawali dengan container dan row, kemudian dibagi menjadi dua bagian menggunakan col-md-4 untuk form di sisi kiri dan col-md-8 untuk tabel di sisi kanan. Bagian paling atas saya beri judul “Gudang Bahan Baku” agar halaman memiliki identitas sebagai dashboard inventori. Dengan pembagian 12 kolom ini, tampilan terlihat proporsional di desktop dan tetap menyesuaikan saat dibuka di layar yang lebih kecil.

Untuk bagian kiri, saya menggunakan komponen form bawaan Bootstrap seperti form-control dan form-label agar input nama barang, quantity, dan supplier tampil rapi dan konsisten. Sementara di bagian kanan, tabel stok dibuat menggunakan class table Bootstrap supaya data lebih mudah dibaca tanpa perlu banyak styling tambahan. Beberapa elemen dibungkus dalam card agar tampil seperti panel dashboard yang terstruktur. Pendekatan ini saya pilih karena lebih efisien, cepat dibuat, dan sudah mendukung responsivitas tanpa harus menulis CSS dari awal.

Kode Utama:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-sca
  <title>Dashboard Inventori - AMDK DAXU</title>

  <!-- Bootstrap CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/
rel="stylesheet">
</head>

<body>

  <!-- NAVBAR -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
      <a class="navbar-brand fw-bold" href="#">DAXU Water Sy

      <button class="navbar-toggler" type="button" data-bs-t
target="#navbarNav">
        <span class="navbar-toggler-icon"></span>
      </button>

      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav ms-auto">
          <li class="nav-item">
            <a class="nav-link" href="index.html">Dash
          </li>
          <li class="nav-item">
            <a class="nav-link" href="#">Input Produksi
          </li>
          <li class="nav-item">
            <a class="nav-link active" href="#">Invent
          </li>
        </ul>
      </div>
    </div>
  </nav>

  <!-- CONTAINER -->
  <div class="container mt-4">

    <!-- HEADER -->
```

```

<div class="row mb-4 text-center">
  <div class="col-12">
    <h2 class="fw-bold">Dashboard Monitoring Persediaan
    <p class="text-muted">Industri Air Minum Dalam Kemasan
    <hr>
  </div>
</div>

<!-- KPI -->
<div class="row mb-4">
  <div class="col-12 col-md-4">
    <div class="card text-white bg-primary shadow-sm">
      <div class="card-body text-center">
        <h5>Total Bahan Baku</h5>
        <h3>7 Item</h3>
      </div>
    </div>
  </div>

  <div class="col-12 col-md-4">
    <div class="card text-white bg-danger shadow-sm">
      <div class="card-body text-center">
        <h5>Bahan Baku Kritis</h5>
        <h3>2 Item</h3>
      </div>
    </div>
  </div>

  <div class="col-12 col-md-4">
    <div class="card text-white bg-success shadow-sm">
      <div class="card-body text-center">
        <h5>Produksi Hari Ini</h5>
        <h3>12.500 Botol</h3>
      </div>
    </div>
  </div>
</div>

<!-- GRID UTAMA -->
<div class="row">

  <!-- FORM (4 KOLOM) -->
  <div class="col-12 col-md-4 mb-4">
    <div class="card shadow">
      <div class="card-header bg-info text-white">
        Form Stok Masuk

```

```

        </div>
        <div class="card-body">
            <form>
                <div class="mb-3">
                    <label class="form-label">Nama Bahan
Preform 600ml">
                    <input type="text" class="form-control">
                </div>

                <div class="mb-3">
                    <label class="form-label">Jumlah (
placeholder="Masukkan jumlah">
                    <input type="number" class="form-control">
                </div>

                <div class="mb-3">
                    <label class="form-label">Supplier
supplier">
                    <input type="text" class="form-control">
                </div>

                <button type="submit" class="btn btn-primary">
                    Simpan Data
                </button>
            </form>
        </div>
    </div>
</div>

<!-- TABEL (8 KOLOM) -->
<div class="col-12 col-md-8">
    <div class="card shadow">
        <div class="card-header bg-secondary text-white">
            Data Stok Bahan Baku
        </div>
        <div class="card-body">
            <table class="table table-striped table-hover">
                <thead class="table-dark">
                    <tr>
                        <th>Kode</th>
                        <th>Nama Bahan</th>
                        <th>Qty</th>
                        <th>Supplier</th>
                        <th>Status</th>
                    </tr>
                </thead>
            </table>
        </div>
    </div>
</div>

```

```

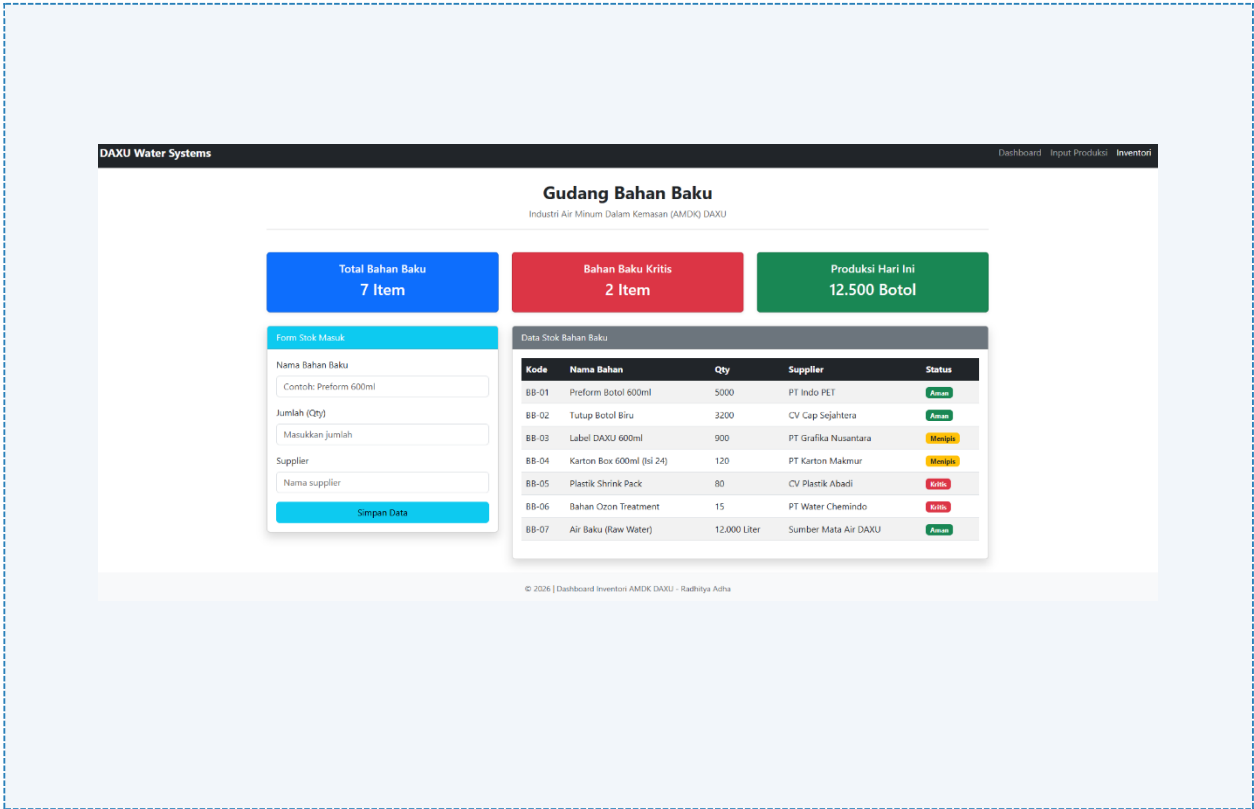
<tbody>
  <tr>
    <td>BB-01</td>
    <td>Preform Botol 600ml</td>
    <td>5000</td>
    <td>PT Indo PET</td>
    <td><span class="badge bg-success">Menipis</span></td>
  </tr>
  <tr>
    <td>BB-02</td>
    <td>Tutup Botol Biru</td>
    <td>3200</td>
    <td>CV Cap Sejahtera</td>
    <td><span class="badge bg-success">Menipis</span></td>
  </tr>
  <tr>
    <td>BB-03</td>
    <td>Label DAXU 600ml</td>
    <td>900</td>
    <td>PT Grafika Nusantara</td>
    <td><span class="badge bg-warning">Menipis</span></td>
  </tr>
  <tr>
    <td>BB-04</td>
    <td>Karton Box 600ml (Isi 24)</td>
    <td>120</td>
    <td>PT Karton Makmur</td>
    <td><span class="badge bg-warning">Menipis</span></td>
  </tr>
  <tr>
    <td>BB-05</td>
    <td>Plastik Shrink Pack</td>
    <td>80</td>
    <td>CV Plastik Abadi</td>
    <td><span class="badge bg-danger">Menipis</span></td>
  </tr>
  <tr>
    <td>BB-06</td>
    <td>Bahan Ozon Treatment</td>
    <td>15</td>
    <td>PT Water Chemindo</td>
    <td><span class="badge bg-danger">Menipis</span></td>
  </tr>
  <tr>
    <td>BB-07</td>
    <td>Karton Box 600ml (Isi 24)</td>
    <td>120</td>
    <td>PT Karton Makmur</td>
    <td><span class="badge bg-warning">Menipis</span></td>
  </tr>

```

```
        <td>BB-07</td>
        <td>Air Baku (Raw Water)</td>
        <td>12.000 Liter</td>
        <td>Sumber Mata Air DAXU</td>
        <td><span class="badge bg-success">OK</span></td>
    </tr>
</tbody>
</table>
</div>
</div>
</div>
</div>

<!-- FOOTER -->
<footer class="text-center py-3 bg-light mt-4">
    <small class="text-muted">
        © 2026 | Dashboard Inventori AMDK DAXU - Radhitya Adha
    </small>
</footer>

<!-- Bootstrap JS -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.js">
</script>
</body>
</html>
```



Gambar F.3 – Output dari Proyek Mini Industri

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum menunjukkan kesesuaian dengan teori mengenai pemisahan struktur dan desain melalui penggunaan CSS eksternal. Penerapan class selector, pengaturan tipografi, warna, serta konsep box model terbukti mampu meningkatkan keterbacaan dan keteraturan tampilan antarmuka. Penggunaan properti seperti max-width, margin: auto, serta box-sizing juga mendukung prinsip layout responsif sebagaimana dijelaskan dalam konsep UI/UX. Dengan demikian, hasil implementasi tidak hanya memperbaiki aspek visual, tetapi juga meningkatkan kenyamanan serta efisiensi interaksi pengguna terhadap sistem.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Warna atau efek hover tidak muncul	Memeriksa kembali selector dan spesifikitas CSS
Tampilan form berubah saat di-zoom	Menggunakan max-width dan box-sizing: border-box

G.3 Kaitan dengan Penerapan di Industri

Keterampilan CSS yang dipelajari dalam modul ini bisa diterapkan secara nyata dalam bidang Teknik Industri, terutama dalam merancang sistem informasi yang digunakan di area produksi. Saat ini banyak aktivitas industri yang sudah terdigitalisasi, seperti dashboard monitoring produksi, pencatatan OEE, hingga form input data kualitas. Tampilan sistem yang rapi, terstruktur, dan mudah dibaca akan sangat membantu operator dalam memahami informasi tanpa harus berpikir terlalu lama. Penyesuaian warna, ukuran teks, jarak antar elemen, serta tata letak yang proporsional dapat membuat informasi penting lebih mudah dipahami dan mengurangi potensi kesalahan saat memasukkan data.

Sebagai contoh konkret, pada sistem pelaporan produk cacat di lini produksi, operator harus mengisi jumlah cacat, jenis cacat, dan tindakan perbaikan. Jika form dirancang dengan layout yang jelas, tombol aksi yang dibedakan secara visual, serta tabel yang tertata rapi, proses input menjadi lebih cepat dan minim kesalahan. Selain itu, desain yang responsif memastikan sistem tetap nyaman digunakan baik melalui komputer di ruang kontrol maupun perangkat mobile di area produksi. Dengan demikian, penguasaan CSS tidak hanya berpengaruh pada aspek tampilan, tetapi juga mendukung efisiensi kerja dan kelancaran proses operasional di industri.

H. KESIMPULAN

1	Mahasiswa memahami bahwa CSS tidak hanya berfungsi untuk memperindah tampilan, tetapi juga berperan dalam meningkatkan kenyamanan pengguna (usability) dan efektivitas suatu sistem berbasis web.
2	Mahasiswa mampu menerapkan CSS eksternal serta memanfaatkan class selector untuk menghasilkan tampilan yang lebih terstruktur, konsisten, dan mudah dikembangkan.
3	Mahasiswa menguasai konsep Box Model sebagai dasar dalam mengatur jarak, ukuran, dan tata letak elemen agar halaman web terlihat rapi dan proporsional.
4	Praktikum ini mengarajkan pemahaman bahwa desain antarmuka yang baik merupakan bagian penting dalam perancangan sistem industri yang efisien, responsif, dan minim kesalahan operasional.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Mardiansyah, A., Kasah, B. N., Zamzami, H. R., Arabu, M. Y., Nasro, M. A., Kristanto, N., ... & Wulandari, Y. (2025). Pengenalan dasar HTML dan CSS: Langkah pertama dalam pengembangan web. Abdi Jurnal Publikasi, 3(3), 165-170.
[3]	Nopal, H. (2024). Pelatihan Pelatihan Pemrograman Web Menggunakan Html Dan Css Di Smk N 2 Slawi. Jurnal Abdimas Universitas Insan Pembangunan Indonesia, 2(2), 31-34.
[4]	Sinlae, F., Bowono, P., Christian, H. R., & Elfan, A. (2024). Pelatihan Dalam Desain Website Menggunakan HTML dan CSS. Jurnal Ilmu Multidisiplin, 3(2), 190-198.
[5]	Kurniawan, D. (2023). Belajar Pemrograman Web Dasar HTML, CSS, & Javascript Untuk Pemula. Penerbit Yayasan Prima Agus Teknik, 1-228.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
Radhitya Adha Fadhlur Rohman NIM: 23051430005	Maessa Andrea Vallenia NIM: 23051430044	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

Modul 4 JavaScript Dasar (Variabel, Tipe Data, Operator, Control Flow)

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 4 (Pertemuan 4)
Topik Utama	JavaScript Dasar (Variabel, Tipe Data, Operator, Control Flow)
Sub-Topik	<i>Logika pemrograman, variabel, tipe data, operator aritmatika/logika, dan percabangan.</i>
Durasi Praktikum	340 Menit
Tanggal Praktikum	6 Maret 2026
Nama Mahasiswa	Radhitya Adha Fadhlur Rohman
NIM	23051430005
Kelas	A2

B. TUJUAN PEMBELAJARAN

1	Memahami peran JavaScript sebagai "otak" dari halaman web.
2	Mampu mendeklarasikan variabel dan memahami aturan penamaannya.
3	Menguasai operator aritmatika untuk perhitungan sederhana.
4	Mampu menggunakan Control Flow (`if/else`, `switch`) untuk pengambilan keputusan otomatis.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	File `index.html` dari pertemuan sebelumnya.	Versi Terbaru	Hasil pekerjaan

			pertemuan sebelumnya
4	Ekstensi VS Code: LiveServer	Versi terbaru	Melihat perubahan web secara real-time.
5	Javascript	Versi terbaru	Membuat halaman web menjadi lebih interaktif dan fungsional

D. DASAR TEORI

JavaScript merupakan salah satu bahasa pemrograman yang cukup penting dalam pengembangan aplikasi web. Dalam sebuah halaman web, HTML biasanya digunakan untuk membangun struktur dasar halaman, sedangkan CSS digunakan untuk mengatur tampilan agar lebih menarik. Namun, tanpa adanya JavaScript, halaman web tersebut hanya bersifat statis dan tidak dapat merespons interaksi pengguna. Oleh karena itu, JavaScript digunakan untuk menambahkan logika pemrograman sehingga halaman web dapat menjadi lebih interaktif, misalnya dengan menampilkan pesan, memproses data yang dimasukkan pengguna, atau melakukan perhitungan secara otomatis. JavaScript dapat dianggap sebagai komponen yang membuat halaman web “hidup” dan mampu merespons berbagai tindakan dari pengguna.

Dalam JavaScript terdapat beberapa konsep dasar yang perlu dipahami, salah satunya adalah variabel. Variabel digunakan sebagai tempat untuk menyimpan data yang akan digunakan dalam program. Data tersebut bisa berupa angka, teks, maupun nilai logika. Misalnya angka untuk menghitung jumlah produksi, teks untuk menyimpan nama barang, atau nilai benar dan salah untuk menentukan suatu kondisi.

Variabel dalam JavaScript umumnya ditulis menggunakan kata kunci `let` atau `const`. Selain variabel, JavaScript juga memiliki operator yang berfungsi untuk melakukan berbagai operasi seperti penjumlahan, pengurangan, perbandingan nilai, maupun operasi logika. Operator ini sangat membantu ketika program perlu melakukan proses perhitungan atau evaluasi terhadap suatu data.

D.1 Konsep Utama

Konsep utama pada modul ini berfokus pada dasar logika pemrograman menggunakan JavaScript yang meliputi penggunaan variabel, tipe data, operator, serta percabangan untuk pengambilan keputusan. Variabel digunakan sebagai tempat untuk menyimpan data yang akan diproses oleh program, biasanya menggunakan kata kunci *let* untuk nilai yang dapat berubah dan *const* untuk nilai yang bersifat tetap. Data yang disimpan juga memiliki tipe tertentu, seperti *number* untuk angka, *string* untuk teks, dan *boolean* untuk nilai benar atau salah. Pemahaman mengenai tipe data ini penting karena akan mempengaruhi bagaimana data tersebut diproses di dalam program.

Selain itu, modul ini juga memperkenalkan penggunaan operator dan *control flow* seperti *if* dan *else* yang memungkinkan program mengambil keputusan berdasarkan kondisi tertentu. Operator digunakan untuk melakukan perhitungan maupun perbandingan nilai, sedangkan percabangan digunakan untuk menentukan tindakan apa yang harus dijalankan oleh program ketika suatu kondisi terpenuhi. Hasil dari proses tersebut biasanya ditampilkan melalui *console* pada browser menggunakan *console.log()* sehingga programmer dapat melihat output sekaligus melakukan pengecekan terhadap logika program yang dibuat.

D.2 Keterkaitan dengan Teknik Industri

Materi JavaScript pada modul ini juga cukup relevan dengan penerapan di bidang Teknik Industri, khususnya dalam pengolahan data operasional dan pembuatan sistem informasi sederhana. Dalam kegiatan industri, banyak data yang perlu dicatat dan dianalisis, seperti jumlah produksi, waktu operasi mesin, maupun pencapaian target kerja. Melalui konsep dasar pemrograman seperti variabel, operasi perhitungan, dan logika percabangan, data-data tersebut dapat diolah secara otomatis oleh sistem sehingga proses analisis menjadi lebih cepat dan risiko kesalahan akibat perhitungan manual dapat dikurangi.

Penerapan konsep ini dapat dilihat pada sistem monitoring produksi berbasis web. Data produksi yang dimasukkan oleh operator dapat langsung diproses oleh program untuk menghitung total output, mengevaluasi apakah target produksi sudah tercapai, serta menampilkan informasi tersebut secara langsung kepada pengguna. Selain itu, logika pemrograman juga dapat dimanfaatkan untuk membantu menghitung indikator performa produksi atau memberikan notifikasi ketika terjadi penyimpangan dari target yang telah ditetapkan.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Hello World & Variable di Console

```
//komentar single line
// 1. variabel & tipe data
let namaMesin = "CNC-Mazak-01";
let targetHarian = 500;
let isOperational = true;

//menampilkan ke console browser
console.log("Mesin: " + namaMesin);
console.log("Target: " + targetHarian);

// 2. operator aritmatika
let produksiPagi = 200
let produksiSiang = 150;
let totalProduksi = produksiPagi + produksiSiang;
console.log("Total saat ini: " + totalProduksi);

//hitung sisa kekurangan target
let kekurangan = targetHarian - totalProduksi;
console.log("Kekurangan target: " + kekurangan);
```

Screenshot Hasil:

Mesin: CNC-Mazak-01	script.js:8
Target: 500	script.js:9
Total saat ini: 350	script.js:15
Kekurangan target: 150	script.js:19

Gambar E.1 – Kode dari Langkah 1

E.2 Langkah 2 – Control Flow (Pengambilan Keputusan)

```
// simulasi data pembacaan sensor (jam operasional)
let jamOperasional = 1250;
let batasMaksimal = 1200;

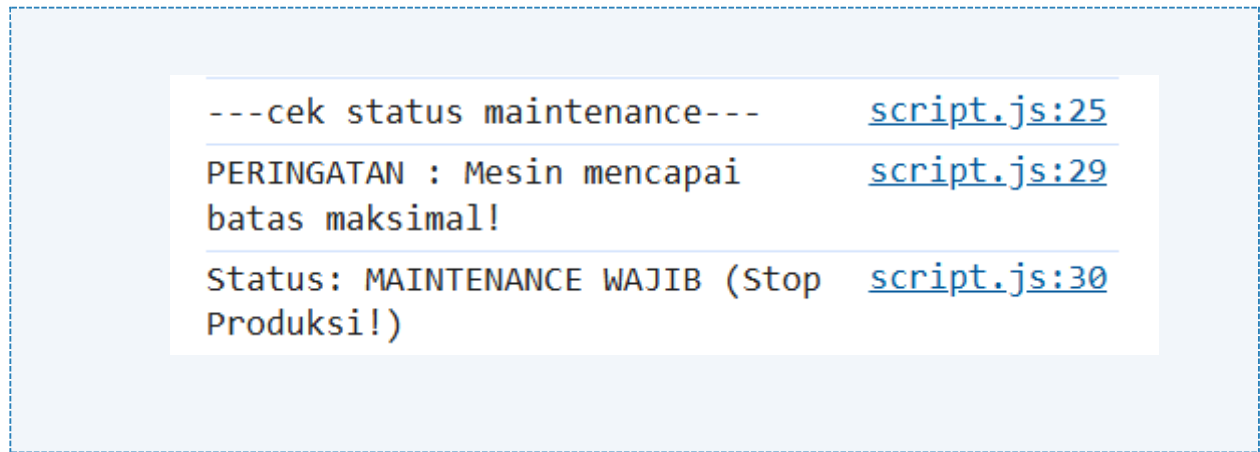
console.log("---cek status maintenance---")

// logika if/else
if (jamOperasional >= batasMaksimal) {
    console.log("PERINGATAN : Mesin mencapai batas maksimal!");
    console.log("Status: MAINTENANCE WAJIB (Stop Produksi!)");
} else if (jamOperasional > 1000) {
    console.log("Status: SIAP HATI-HATI (Segera jadwalkan maintenance)");
} else {
```



```
        console.log("Status: Berjalan Normal");  
    }  
}
```

Screenshot Hasil:



```
---cek status maintenance--- script.js:25  
PERINGATAN : Mesin mencapai script.js:29  
batas maksimal!  
Status: MAINTENANCE WAJIB (Stop script.js:30  
Produksi!)
```

Gambar E.2 – Output dari Langkah 2

E.3 Langkah 3 – Studi Kasus Industri (Perhitungan OEE Sederhana)

```
// data input  
let jamKerjaPlanned = 8;  
let jamKerjaAktual = 6.5;  
  
// perhitungan  
let availability = (jamKerjaAktual / jamKerjaPlanned)*100;  
  
//pembulatan 2 angka dibelakang koma  
availability = availability.toFixed(2);  
  
console.log("Planned time: " + jamKerjaPlanned + " jam");  
console.log("Actual Time: " + jamKerjaAktual + " jam");  
console.log("Availability: " + availability + " %");  
// logika penilaian kualitas availability  
if (availability >= 90) {  
    console.log("Kategori : WORLD CLASS")  
} else if (availability >= 80) {  
    console.log("Kategori: BAIK (Tetap monitor)");  
} else {  
    console.log("Kategori: BURUK (Perlu investigasi penyebab breakdown)");  
}
```

Screenshot Hasil:

---cek status maintenance---	script.js:25
PERINGATAN : Mesin mencapai batas maksimal!	script.js:29
Status: MAINTENANCE WAJIB (Stop Produksi!)	script.js:30
Planned time: 8 jam	script.js:47
Actual Time: 6.5 jam	script.js:48
Availability: 81.25 %	script.js:49
Kategori: BAIK (Tetap monitor)	script.js:54

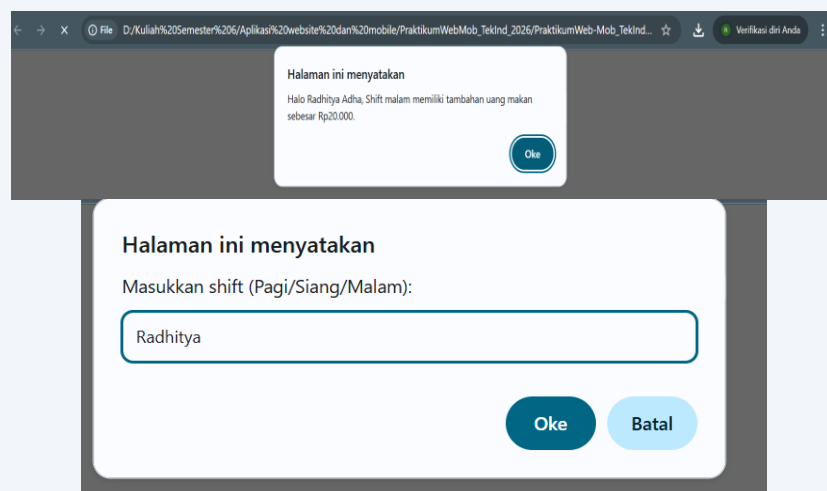
Gambar E.3 – Output dari Langkah 3

E.4 Langkah 4 – Interaksi DOM Sederhana (Alert & Prompt)

```
let namaOperator = prompt("Masukkan Nama Operator: ");
let shiftKerja = prompt("Masukkan shift (Pagi/Siang/Malam): ");

if (shiftKerja === "Malam") {
    alert("Halo " + namaOperator + ", Shift malam memiliki tambahan uang makan sebesar Rp20.000.");
} else {
    alert("Halo " + namaOperator + ", Selamat bekerja. Tetap Semangat!");
}
```

Screenshot Hasil:



Gambar E.4 – Output dari Langkah 4

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Matematika

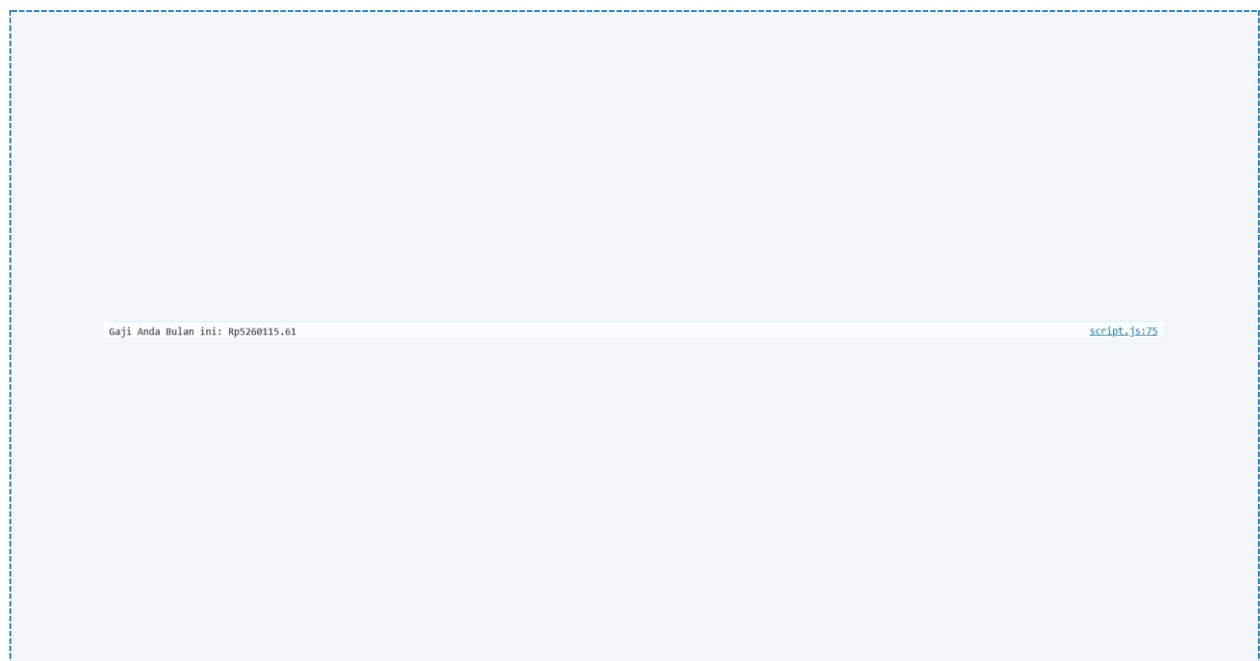
Pertanyaan / Instruksi Latihan:

Buat variabel `gajiPokok` dan `jamLembur`. Hitung `totalGaji` dengan asumsi upah lembur per jam adalah 1.5x dari gaji pokok dibagi 173 (jam kerja bulanan). Tampilkan hasilnya di console.

Jawaban / Kode Solusi:

```
let gajiPokok = 5000000
let jamLembur = 6
let totalGaji = (1.5*jamLembur*gajiPokok)/173 + gajiPokok

totalGaji = totalGaji.toFixed(2)
console.log("Gaji Anda Bulan ini: " + "Rp" + totalGaji)
```



Gambar F.1 – Output dari Latihan 1

F.2 Latihan 2 – Logika Switch

Pertanyaan / Instruksi Latihan:

Gunakan `switch` untuk mengkonversi kode shift angka (1, 2, 3) menjadi nama shift (Pagi, Siang, Malam). Jika input selain 1-3, tampilkan "Shift Tidak Valid".

Jawaban / Kode Solusi:

```
//LATIHAN 2 (LOGIKA SWITCH)
let kodeShift = 2;
switch (kodeShift) {
  case 1 :
    console.log("Shift anda adalah : Pagi")
    break;
  case 2:
    console.log("Shift anda adalah : Siang")
  case 3:
    console.log("Shift anda adalah : Malam")
  default:
    console.log("Shift tidak valid")
}
```

Shift anda adalah : Siang	script.js:84
Shift anda adalah : Malam	script.js:86
Shift tidak valid	script.js:88

Gambar F.2 – Output dari Latihan 2

F.3 Tugas Proyek Mini – Kalkulator Biaya Produksi

Deskripsi proyek mini:

- Buat program di `script.js` yang menghitung biaya produksi total per unit.
- Variabel: `biayaBahanBaku`, `biayaTenagaKerja`, `biayaOverhead`, `jumlahProduksi`.
 - Rumus: $\text{totalPerUnit} = (\text{biayaBahanBaku} + \text{biayaTenagaKerja} + \text{biayaOverhead}) / \text{jumlahProduksi}$.
 - Gunakan `if/else` untuk mengecek: jika `jumlahProduksi` < 100, tampilkan "Biaya Tinggi (Ekonomi Skala Kecil)". Jika ≥ 100 , tampilkan "Biaya Efisien".

Penjelasan pendekatan/solusi yang digunakan:

Pendekatan yang digunakan pada modul ini adalah membuat program sederhana menggunakan JavaScript untuk menghitung biaya produksi per unit. Program diawali dengan mendefinisikan beberapa variabel yang mewakili komponen biaya produksi seperti biaya bahan baku, biaya tenaga kerja, biaya overhead, serta jumlah unit yang diproduksi. Selanjutnya program melakukan perhitungan dengan menjumlahkan seluruh biaya tersebut kemudian membaginya dengan jumlah produksi untuk mendapatkan nilai biaya per unit. Setelah proses perhitungan selesai, digunakan logika percabangan *if/else* untuk mengevaluasi apakah jumlah produksi masih tergolong kecil atau sudah cukup efisien. Hasil perhitungan beserta keterangan evaluasinya kemudian ditampilkan melalui *console.log()* sehingga dapat dilihat langsung pada console browser saat program dijalankan

Kode Utama:

```
// PROYEK MINI KALKULATOR BIAYA PRODUKSI
let biayaBahanBaku = ("Rp" + 750000);
let biayaTenagaKerja = ("Rp" + 500000);
let biayaOverhead = ("Rp" + 350000);
let jumlahProduksi = (1500 + " Unit");

//menampilkan data-data
console.log("Biaya Bahan Baku : " + biayaBahanBaku)
console.log("Biaya Tenaga Kerja : " + biayaTenagaKerja)
console.log("Biaya Overhead : " + biayaOverhead)
console.log("Jumlah Produksi saat ini : " + jumlahProduksi)

//perhitungan totalperunit
let totalperunit = (biayaBahanBaku+biayaTenagaKerja+biayaOverhead)/ jumlahProduksi
totalperunit = totalperunit.toFixed(2)

// if/else pengecekan jumlah produksi
if (jumlahProduksi < 100) {
    console.log("Biaya Tinggi (Ekonomi Skala Kecil)")
} else {
    console.log("Biaya Efisien")
}
```

Biaya Bahan Baku : Rp7500000	scriptpro.js:8
Biaya Tenaga Kerja : Rp500000	scriptpro.js:9
Biaya Overhead : Rp350000	scriptpro.js:10
Jumlah Produksi saat ini : 1500 Unit	scriptpro.js:11
Biaya Efisien	scriptpro.js:21

Gambar F.3 – Output dari Projek Mini Industri

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

hasil dari praktikum ini sudah sesuai dengan teori dasar pemrograman JavaScript yang memanfaatkan variabel, operasi aritmatika, serta percabangan *if/else*. Program yang dibuat mampu menghitung biaya produksi per unit dengan menjumlahkan seluruh komponen biaya produksi kemudian membaginya dengan jumlah unit yang dihasilkan. Selain itu, penggunaan percabangan juga sudah berjalan dengan baik untuk mengevaluasi kondisi jumlah produksi dan menampilkan kategori yang sesuai berdasarkan batas yang telah ditentukan.

Meskipun demikian, secara konsep masih ada beberapa hal yang perlu diperhatikan agar lebih sesuai dengan teori pemrograman. Salah satunya adalah memastikan bahwa data yang digunakan dalam proses perhitungan memiliki tipe data numerik (*number*), karena jika data masih berbentuk teks (*string*) maka dapat mempengaruhi hasil perhitungan. Dengan penggunaan tipe data yang tepat serta penerapan logika program yang benar, maka hasil perhitungan yang dihasilkan oleh program akan lebih akurat dan sesuai dengan prinsip dasar pemrograman JavaScript.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Console pada browser tidak menampilkan hasil dan muncul error saat program dijalankan.	- Mengecek kembali penulisan kode JavaScript agar sesuai dengan sintaks yang benar. - Memastikan file JavaScript sudah terhubung dengan benar pada file HTML.

G.3 Kaitan dengan Penerapan di Industri

Konsep yang dipelajari pada praktikum ini sebenarnya cukup dekat dengan penerapan di dunia industri, terutama dalam hal pengolahan data produksi dan perhitungan biaya. Di perusahaan manufaktur, perhitungan biaya produksi per unit merupakan hal penting karena berkaitan dengan penentuan harga jual produk serta evaluasi efisiensi proses produksi. Dengan memanfaatkan logika pemrograman seperti variabel, operasi perhitungan, dan percabangan kondisi, proses pengolahan data tersebut dapat dilakukan secara lebih cepat dan sistematis melalui sistem berbasis komputer.

Sebagai contoh, dalam sebuah pabrik perusahaan perlu menghitung total biaya produksi yang berasal dari bahan baku, tenaga kerja, dan biaya operasional lainnya. Data tersebut kemudian diolah oleh sistem untuk mengetahui biaya produksi per unit dan melihat apakah proses produksi sudah berjalan efisien atau belum. Program sederhana seperti yang dibuat pada praktikum ini memang masih sangat dasar, tetapi konsepnya sudah menggambarkan bagaimana sistem informasi di industri bekerja dalam membantu proses pengolahan data dan pengambilan keputusan.

H. KESIMPULAN

1	Praktikum modul 4 membantu memahami dasar penggunaan JavaScript seperti variabel, tipe data, dan operator untuk melakukan perhitungan sederhana pada program.
2	Penggunaan <i>control flow</i> seperti <i>if/else</i> memungkinkan program mengambil keputusan otomatis berdasarkan kondisi tertentu.
3	Program yang dibuat mampu menghitung biaya produksi per unit dari beberapa komponen biaya yang dimasukkan.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Mozilla Foundation. (n.d.). <i>JavaScript guide</i> . MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide
[3]	Flanagan, D. (2020). <i>JavaScript: The Definitive Guide</i> (7th ed.). O'Reilly Media.
[4]	Kadir, A. (2018). <i>Dasar Pemrograman Web Dinamis Menggunakan JavaScript</i> . Yogyakarta: Andi Publisher.
[5]	

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
Radhitya Adha Fadhlur Rohman NIM: 23051430005	Maessa Andrea Vallenia NIM: 23051430044	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

Modul 5 Function, Array, Object di JavaScript

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 5 Function, Array, Object di JavaScript
Topik Utama	Function, Array, Object di JavaScript
Sub-Topik	Modularitas kode (Function), pengelolaan data list (Array), dan struktur data kompleks (Object).
Durasi Praktikum	340 Menit
Tanggal Praktikum	13 Maret 2026
Nama Mahasiswa	Radhitya Adha Fadhlur Rohman
NIM	23051430005
Kelas	A1

B. TUJUAN PEMBELAJARAN

1	Memahami konsep Function untuk menghindari pengulangan kode (DRY - Don't Repeat Yourself).
2	Menguasai penggunaan Array untuk menyimpan sekumpulan data (misal: daftar cacat produksi).
3	Memahami Object untuk merepresentasikan entitas nyata (misal: Spesifikasi Mesin) dalam bentuk data.
4	Mampu melakukan iterasi (perulangan) terhadap Array dan Object.

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	HTML	-	Bahasa Markup
4	Node.js & npm	-	Runtime JS (Modul tertentu)
5	Live Server	-	Local Development Server
6	Github	Web-Based	Menyimpan & Mengelola Source Code

D. DASAR TEORI

Dalam pemrograman JavaScript, penulisan kode yang terstruktur sangat penting agar program lebih mudah dipahami dan dikelola. Salah satu cara untuk membuat kode lebih terorganisir adalah dengan menggunakan *function*. Function merupakan sekumpulan perintah yang ditulis dalam satu blok kode dan dapat dipanggil kembali ketika diperlukan. Dengan adanya function, penulisan kode menjadi lebih efisien karena programmer tidak perlu menuliskan perintah yang sama secara berulang. Konsep ini juga membantu menjaga kerapian kode serta mempermudah proses perbaikan dan pengembangan program di kemudian hari.

JavaScript juga memiliki struktur data yang disebut *array*. Array digunakan untuk menyimpan beberapa data dalam satu variabel yang sama. Struktur ini sangat berguna ketika program harus mengelola banyak data sekaligus, misalnya daftar produk, data transaksi, atau catatan hasil produksi. Setiap data dalam array memiliki posisi atau indeks tertentu yang dapat digunakan untuk mengakses data tersebut. Dalam penggunaannya, array sering diproses menggunakan perulangan (*looping*) sehingga setiap elemen data dapat ditampilkan atau diolah sesuai dengan kebutuhan program.

Object digunakan untuk menyimpan data yang lebih kompleks karena terdiri dari beberapa pasangan properti dan nilai (*key* dan *value*). Dengan menggunakan object, sebuah data dapat direpresentasikan lebih jelas karena setiap informasi memiliki nama atribut masing-masing. Contohnya seperti data mesin yang memiliki informasi id, nama mesin, status, maupun spesifikasinya. Dalam pengembangan aplikasi web, object sering dikombinasikan dengan array sehingga membentuk kumpulan object dalam satu struktur data. Cara ini umum digunakan untuk menyimpan dan mengelola data yang lebih terstruktur dalam suatu aplikasi.

D.1 Konsep Utama

Konsep utama pada praktikum ini berfokus pada pemanfaatan *function*, *array*, dan *object* dalam bahasa pemrograman JavaScript untuk membantu pengelolaan kode dan data secara lebih terstruktur. Function digunakan untuk membungkus serangkaian perintah menjadi satu bagian yang dapat digunakan kembali ketika diperlukan. Dengan cara ini, program menjadi lebih ringkas dan mudah dipahami karena tidak terjadi pengulangan kode yang sama di banyak tempat. Penggunaan function juga membantu programmer dalam mengorganisasi logika program sehingga alur kerja aplikasi menjadi lebih jelas.

Selain itu, praktikum ini juga menekankan penggunaan array dan object sebagai cara untuk menyimpan serta mengelola data dalam program. Array digunakan untuk menyimpan kumpulan data yang sejenis, sedangkan object digunakan untuk merepresentasikan suatu entitas yang memiliki beberapa atribut atau informasi. Dalam praktik pengembangan aplikasi, kedua struktur data ini sering digabungkan dalam bentuk *array of objects* sehingga data dapat dikelola dengan lebih fleksibel dan mendekati struktur data yang digunakan pada sistem nyata seperti database atau JSON. Melalui konsep ini, mahasiswa dapat memahami bagaimana data di dalam aplikasi web disimpan, diproses, dan ditampilkan secara sistematis.

D.2 Keterkaitan dengan Teknik Industri

Konsep *function*, *array*, dan *object* dalam JavaScript memiliki keterkaitan dengan bidang Teknik Industri, terutama dalam pengolahan dan pengelolaan data pada sistem produksi maupun operasional. Dalam praktiknya, data seperti daftar produk cacat, informasi mesin, atau stok material sering perlu dicatat dan dianalisis secara sistematis. Penggunaan *function* dapat dianalogikan seperti prosedur kerja atau SOP yang dapat dijalankan berulang untuk melakukan perhitungan atau proses tertentu. Sementara itu, *array* dan *object* membantu merepresentasikan kumpulan data produksi maupun spesifikasi suatu mesin secara terstruktur. Dengan memahami konsep ini, mahasiswa Teknik Industri dapat melihat bagaimana teknologi pemrograman dapat dimanfaatkan untuk mendukung pengolahan data, monitoring proses, hingga pengembangan sistem informasi sederhana dalam lingkungan industri.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

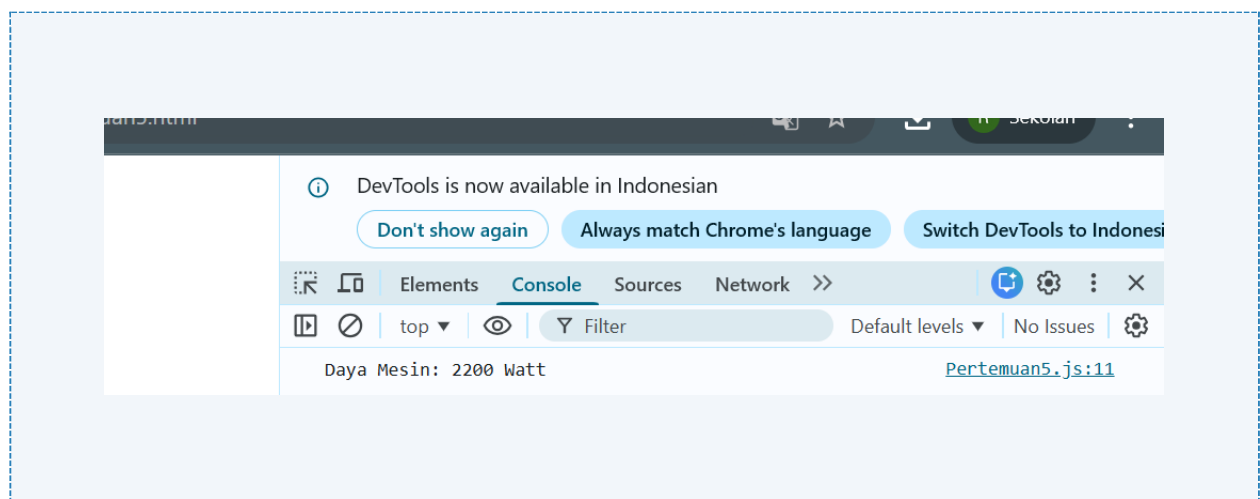
Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

E.1 Langkah 1: Membuat Function Perhitungan

Buat file baru `pertemuan5.html` dan `pertemuan5.js`. Kita akan membuat kalkulator energi listrik mesin. Di `pertemuan5.js`:

```
// 1. FUNCTION Declaration
// Fungsi untuk menghitung konsumsi daya (P = V x I)
function hitungDaya(tegangan, arus) {
  let daya = tegangan * arus;
  return daya; // Mengembalikan hasil
}
// Pemanggilan Function
let teganganMesin = 220; // Volt
let arusMesin = 10; // Ampere
let totalDaya = hitungDaya(teganganMesin, arusMesin);
console.log("Daya Mesin: " + totalDaya + " Watt");
```

Screenshot Hasil:

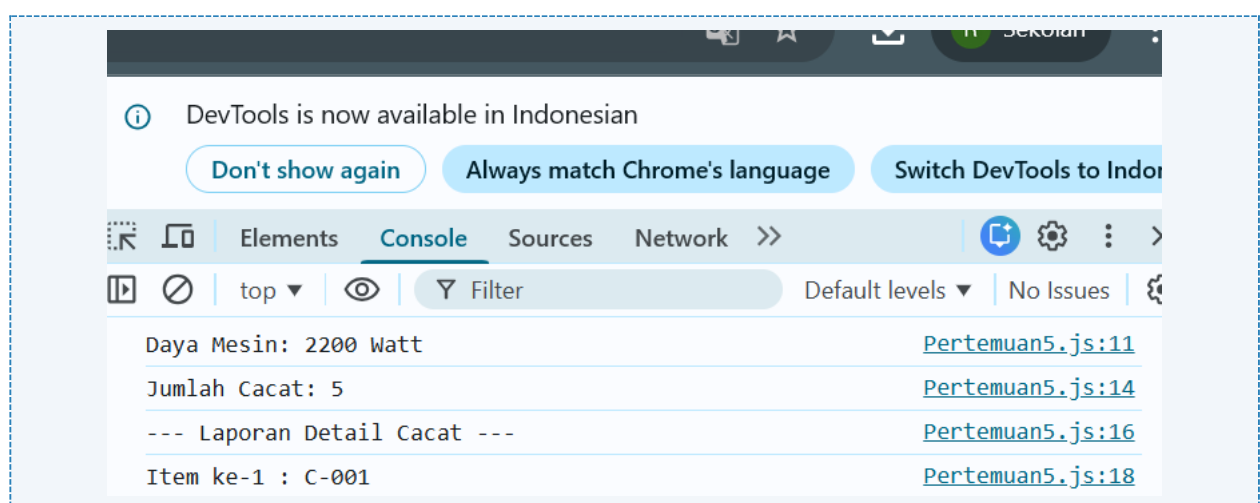


Gambar E.1 – Hasil Function Perhitungan di Console

E.2 Langkah 2 – Array dan Looping (Daftar Produk Cacat)

```
// 1. FUNCTION Declaration
// Fungsi untuk menghitung konsumsi daya (P = V x I)
function hitungDaya(tegangan, arus) {
  let daya = tegangan * arus;
  return daya; // Mengembalikan hasil
}
// Pemanggilan Function
let teganganMesin = 220; // Volt
let arusMesin = 10; // Ampere
let totalDaya = hitungDaya(teganganMesin, arusMesin);
console.log("Daya Mesin: " + totalDaya + " Watt");
// 2. ARRAY
let daftarCacat = ["C-001", "C-005", "C-012", "C-001", "C-020"]; // C-001 terjadi 2 kali
console.log("Jumlah Cacat: " + daftarCacat.length);
// Looping (Perulangan) untuk menampilkan setiap cacat
console.log("--- Laporan Detail Cacat ---");
for (let i = 0; i < daftarCacat.length; i++) {
  console.log("Item ke-" + (i + 1) + " : " + daftarCacat[i]);
}
// Menambahkan data baru ke array
daftarCacat.push("C-099");
console.log("Setelah penambahan: " + daftarCacat);
```

Screenshot Hasil:



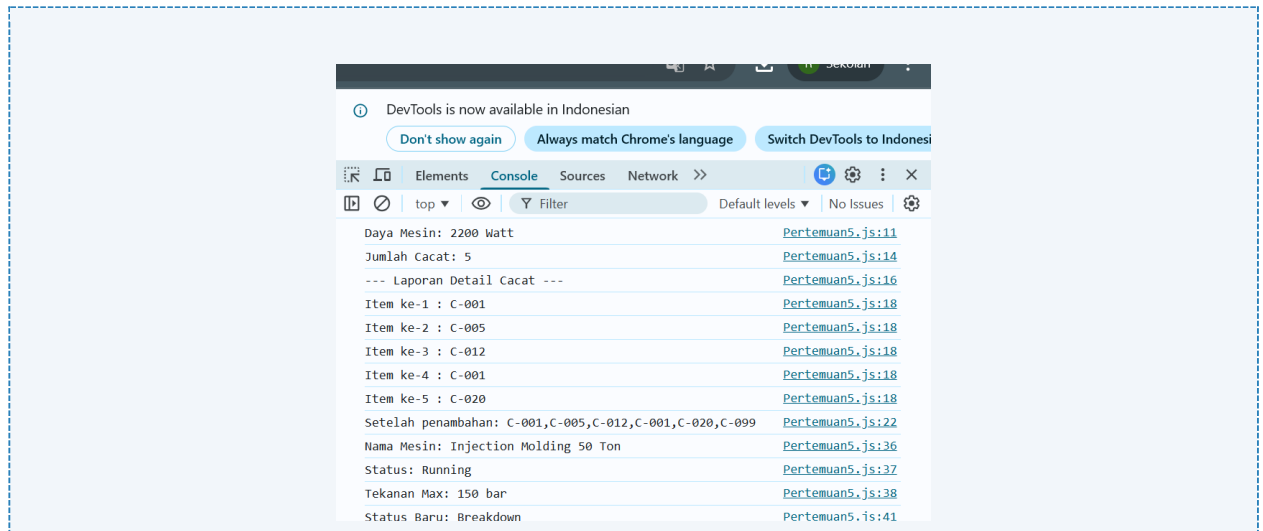
Gambar E.2 – Hasil Array dan Looping

E.3 Langkah 3 – Object (Rekaman Data Mesin)

```
// 1. FUNCTION Declaration
// Fungsi untuk menghitung konsumsi daya (P = V x I)
function hitungDaya(tegangan, arus) {
  let daya = tegangan * arus;
  return daya; // Mengembalikan hasil
}
// Pemanggilan Function
let teganganMesin = 220; // Volt
let arusMesin = 10; // Ampere
let totalDaya = hitungDaya(teganganMesin, arusMesin);
console.log("Daya Mesin: " + totalDaya + " Watt");
// 2. ARRAY
let daftarCacat = ["C-001", "C-005", "C-012", "C-001", "C-020"]; // C-001 terjadi 2 kali
console.log("Jumlah Cacat: " + daftarCacat.length);
// Looping (Perulangan) untuk menampilkan setiap cacat
console.log("--- Laporan Detail Cacat ---");
for (let i = 0; i < daftarCacat.length; i++) {
  console.log("Item ke-" + (i + 1) + " : " + daftarCacat[i]);
}
// Menambahkan data baru ke array
daftarCacat.push("C-099");
console.log("Setelah penambahan: " + daftarCacat);
// 3. OBJECT
let mesinA = {
  id: "M-01",
  nama: "Injection Molding 50 Ton",
  merk: "Toshiba",
  tahunBeli: 2018,
  status: "Running",
  spesifikasi: {
    kecepatanMax: 200, // mm/s
    tekananMax: 150 // bar
  }
};
// Mengakses data object
console.log("Nama Mesin: " + mesinA.nama);
console.log("Status: " + mesinA.status);
console.log("Tekanan Max: " + mesinA.spesifikasi.tekananMax + " bar");
// Mengubah data object (Simulasi mesin rusak)
mesinA.status = "Breakdown";
console.log("Status Baru: " + mesinA.status);
```



Screenshot Hasil:



Gambar E.3 – Tampilan Data Object Mesin dan Perubahan Status Mesin

E.4 Langkah 4: Gabungan (Array of Objects)

```
// 1. FUNCTION Declaration
// Fungsi untuk menghitung konsumsi daya (P = V x I)
function hitungDaya(tegangan, arus) {
  let daya = tegangan * arus;
  return daya; // Mengembalikan hasil
}
// Pemanggilan Function
let teganganMesin = 220; // Volt
let arusMesin = 10; // Ampere
let totalDaya = hitungDaya(teganganMesin, arusMesin);
console.log("Daya Mesin: " + totalDaya + " Watt");
// 2. ARRAY
let daftarCacat = ["C-001", "C-005", "C-012", "C-001", "C-020"]; // C-001 terjadi 2 kali
console.log("Jumlah Cacat: " + daftarCacat.length);
// Looping (Perulangan) untuk menampilkan setiap cacat
console.log("--- Laporan Detail Cacat ---");
for (let i = 0; i < daftarCacat.length; i++) {
```

```
    console.log("Item ke-" + (i + 1) + " : " + daftarCacat[i]);
  }
  // Menambahkan data baru ke array
  daftarCacat.push("C-099");
  console.log("Setelah penambahan: " + daftarCacat);
  // 3. OBJECT
  let mesinA = {
    id: "M-01",
    nama: "Injection Molding 50 Ton",
    merk: "Toshiba",
    tahunBeli: 2018,
    status: "Running",
    spesifikasi: {
      kecepatanMax: 200, // mm/s
      tekananMax: 150 // bar
    }
  };
  // Mengakses data object
  console.log("Nama Mesin: " + mesinA.nama);
  console.log("Status: " + mesinA.status);
  console.log("Tekanan Max: " + mesinA.spesifikasi.tekananMax +
    " bar");
  // Mengubah data object (Simulasi mesin rusak)
  mesinA.status = "Breakdown";
  console.log("Status Baru: " + mesinA.status);
  let gudangMaterial = [
    { kode: "MAT-01", nama: "Baja Ringan", stok: 50, satuan:
      "Batang" },
    { kode: "MAT-02", nama: "Plastik ABS", stok: 120, satuan:
      "Kg" },
    { kode: "MAT-03", nama: "Oli Mesin", stok: 10, satuan:
      "Liter" }
  ];
  console.log("--- Cek Stok Gudang ---");
  // Menggunakan forEach untuk iterasi array object
  gudangMaterial.forEach(function(item) {
    console.log(item.kode + " - " + item.nama + " : " + item.stok
      + " " + item.satuan);

    // Logika Re-order
    if (item.stok < 20) {
      console.log(" >>> PERINGATAN: Stok " + item.nama + " menipis!
        Segera PO.");
    }
  });
});
```


Screenshot Hasil:

Daya Mesin: 2200 Watt	Pertemuan5.js:11
Jumlah Cacat: 5	Pertemuan5.js:14
--- Laporan Detail Cacat ---	Pertemuan5.js:16
Item ke-1 : C-001	Pertemuan5.js:18
Item ke-2 : C-005	Pertemuan5.js:18
Item ke-3 : C-012	Pertemuan5.js:18
Item ke-4 : C-001	Pertemuan5.js:18
Item ke-5 : C-020	Pertemuan5.js:18
Setelah penambahan: C-001,C-005,C-012,C-001,C-020,C-099	Pertemuan5.js:22
Nama Mesin: Injection Molding 50 Ton	Pertemuan5.js:36
Status: Running	Pertemuan5.js:37
Tekanan Max: 150 bar	Pertemuan5.js:38
Status Baru: Breakdown	Pertemuan5.js:41
--- Cek Stok Gudang ---	Pertemuan5.js:47
MAT-01 - Baja Ringan : 50 Batang	Pertemuan5.js:50
MAT-02 - Plastik ABS : 120 Kg	Pertemuan5.js:50
MAT-03 - Oli Mesin : 10 Liter	Pertemuan5.js:50
>>> PERINGATAN: Stok Oli Mesin menipis! Segera PO.	Pertemuan5.js:54

Gambar E.4 – Hasil iterasi

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Function

Pertanyaan / Instruksi Latihan:

Buat function `hitungLingkaran(jariJari)` yang mengembalikan nilai Luas dan Keliling. Tampilkan kedua nilai tersebut.

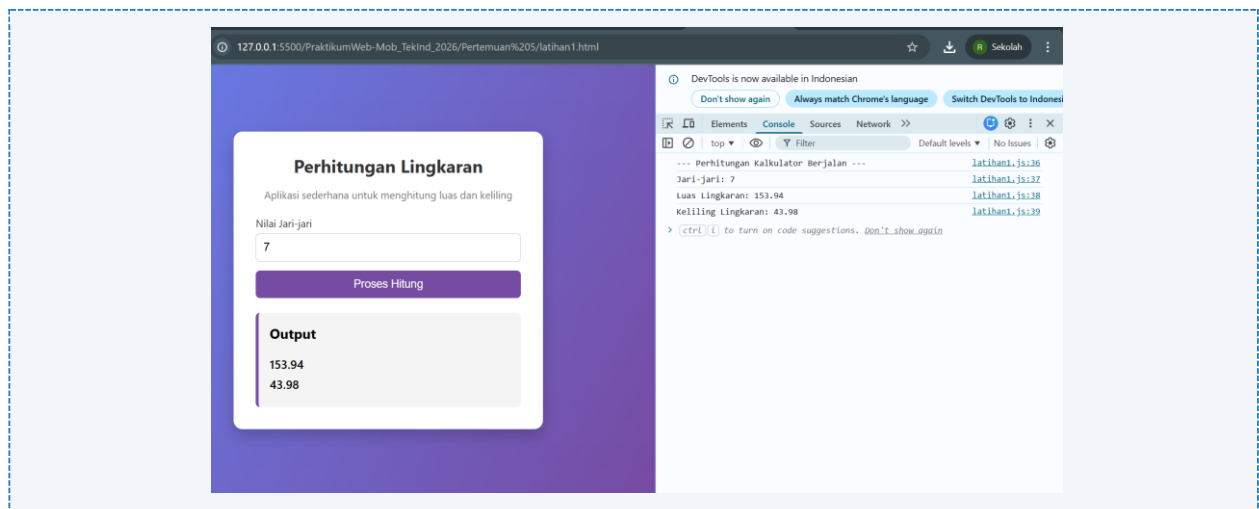
Jawaban / Kode Solusi:

```
function hitungLingkaran(jariJari) {
  const luas = Math.PI * jariJari * jariJari;
  const keliling = 2 * Math.PI * jariJari;

  return {
    luasLingkaran: luas,
    kelilingLingkaran: keliling
  };
}

// 2. FUNGSI PENGHUBUNG JS KE HTML
function hitung() {
  const inputJariJari =
document.getElementById("jariJari").value;
  // Mencegah error jika user menekan tombol tapi kotaknya
  masih kosong
```

```
if (!inputJariJari) {  
    alert("Harap masukkan panjang jari-jari terlebih dahulu!");  
    return;  
}  
  
// Mengubah teks dari input HTML menjadi tipe data Angka (Number)  
const r = Number(inputJariJari);  
  
// Jalankan fungsi hitung  
const hasil = hitungLingkaran(r);  
  
// === MENAMPILKAN HASIL KE LAYAR HTML ===  
// Mengisi ID "luas" dan "keliling" dengan hasil perhitungan  
document.getElementById("luas").innerText =  
hasil.luasLingkaran.toFixed(2);  
document.getElementById("keliling").innerText =  
hasil.kelilingLingkaran.toFixed(2);  
  
// === MENAMPILKAN HASIL KE CONSOLE (Biar dosen bisa cek) ===  
console.log("--- Perhitungan Kalkulator Berjalan ---");  
console.log("Jari-jari: " + r);  
console.log("Luas Lingkaran: " +  
hasil.luasLingkaran.toFixed(2));  
console.log("Keliling Lingkaran: " +  
hasil.kelilingLingkaran.toFixed(2));  
}
```



Gambar F.1 – Hasil tampilan latihan 1 Function

F.2 Latihan 2 – Array Manipulation

Pertanyaan / Instruksi Latihan:

Dari array `daftarCacat` di atas, carilah berapa kali cacat "C-001" muncul (Gunakan loop dan if/counter).

Jawaban / Kode Solusi:

```
// Latihan 2 - Menghitung jumlah kemunculan cacat C-001

let dataCacat = ["C-001", "C-005", "C-012", "C-001", "C-020"];

// 1. FUNGSI MENAMPILKAN ARRAY KE HTML (Otomatis jalan saat web dibuka)
function renderDataLayar() {
    let wadah = document.getElementById("dataList");
    let htmlBadges = "";

    dataCacat.forEach(function(kode) {
        let kelasCSS = (kode === "C-001") ? "badge alert" : "badge normal";
        htmlBadges += `<span class="${kelasCSS}">${kode}</span>`;
    });

    wadah.innerHTML = htmlBadges;
}

// Panggil fungsi render agar data langsung muncul
renderDataLayar();

// 2. FUNGSI ANALISIS DATA (Jalan saat tombol diklik)
function analisisData() {
    console.log("Total data cacat: " + dataCacat.length);
    console.log("=== Detail Data Cacat ===");

    dataCacat.forEach(function(kode, index) {
        console.log("Data ke-" + (index + 1) + " : " + kode);
    });

    let hitung = 0;
    // Logika pencarian C-001 menggunakan if dan counter
    for(let i = 0; i < dataCacat.length; i++){
        if(dataCacat[i] === "C-001"){
            hitung++;
        }
    }
}
```

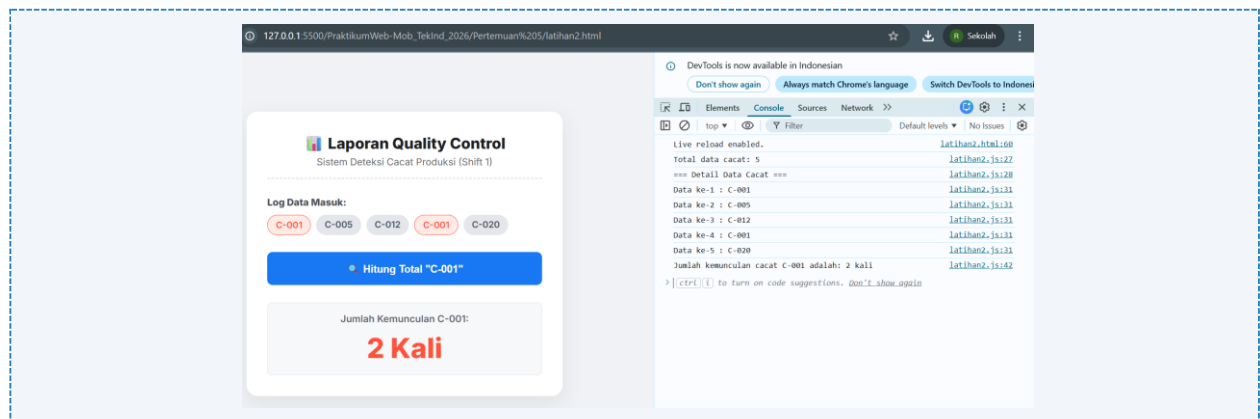
```

    }
}

    console.log("Jumlah kemunculan cacat C-001 adalah: " +
hitung + " kali");
    // =====

    document.getElementById("hasilHitung").innerText = hitung
+ " Kali";
}

```



Gambar F.2 – Hasil Tampilan Array Manipulation

F.3 Tugas Proyek Mini – (Sistem Antrian Job Shop):

Deskripsi proyek mini:

Buat program simulasi antrian 3 pekerjaan mesin.

- Buat Array of Objects bernama `antrianMesin`. Isi dengan 3 object job: `{idJob: "J01", namaProses: "Drilling", durasi: 30}`.
- Buat function `prosesAntrian(antrian)` yang menerima parameter array tersebut.
- Di dalam function, tampilkan pesan: "Memproses Job [ID] - [Nama] selama [durasi] menit" untuk setiap job dalam antrian.
- Tambahkan 1 job baru ke array dan panggil function lagi.

Penjelasan pendekatan/solusi yang digunakan:

Pendekatan yang digunakan dalam menyelesaikan proyek mini simulasi Sistem Antrian *Job Shop* ini adalah dengan memanfaatkan struktur data *array of objects* pada JavaScript. Pendekatan ini dipilih karena mampu merepresentasikan data pekerjaan secara lebih terstruktur, di mana setiap *job* dalam antrian memiliki beberapa informasi seperti ID pekerjaan, nama proses, dan durasi pengerjaan. Dengan menggunakan objek, setiap data pekerjaan dapat disimpan dalam satu kesatuan yang jelas, sedangkan array digunakan untuk menampung beberapa objek tersebut sehingga membentuk sebuah daftar antrian pekerjaan. Cara ini cukup relevan dengan konsep pengelolaan data dalam sistem produksi yang sering melibatkan beberapa pekerjaan yang harus diproses secara berurutan.

Untuk memproses data antrian tersebut digunakan sebuah function yang berfungsi membaca dan menampilkan setiap pekerjaan dalam daftar antrian. Proses pembacaan data dilakukan dengan metode iterasi seperti *forEach()* sehingga setiap objek di dalam array dapat diproses satu per satu sesuai urutan antrian. Melalui cara ini, program dapat menampilkan informasi proses pekerjaan secara berurutan, sehingga menyerupai proses antrian pada sistem produksi di industri. Pendekatan ini membantu menunjukkan bagaimana konsep dasar JavaScript seperti function, array, dan object dapat digunakan untuk membuat simulasi sederhana yang menggambarkan alur kerja dalam sistem manufaktur.

Kode Utama:

```
//
=====
// TUGAS PROYEK MINI: Sistem Antrian Job Shop
//
=====

// a. Buat Array of Objects bernama 'antrianMesin' dengan 3
object job awal
let antrianMesin = [
  { idJob: "J01", namaProses: "Drilling", durasi: 30 },
  { idJob: "J02", namaProses: "Milling", durasi: 45 },
  { idJob: "J03", namaProses: "Turning", durasi: 60 }
];

// b. Buat function 'prosesAntrian(antrian)'
function prosesAntrian(antrian) {
  let hasilRender = "";

  antrian.forEach(function(job) {
    // c. Tampilkan pesan di Console sesuai instruksi modul
    let pesan = `Memproses Job ${job.idJob} -
    ${job.namaProses} selama ${job.durasi} menit`;
  });
}
```

```
    console.log(pesan);

    // Menyimpan teks untuk ditampilkan ke layar HTML nanti
    hasilRender += pesan + "<br>";
  });

  return hasilRender; // Mengembalikan gabungan teks untuk
  HTML
}

//
=====
// EKSEKUSI OTOMATIS

console.log("=== SIMULASI ANTRIAN AWAL (3 JOB) ===");
prosesAntrian(antrianMesin); // Menjalankan fungsi dengan 3
job

// d. Tambahkan 1 job baru ke array dan panggil function lagi
console.log("\n=== SIMULASI PENAMBAHAN JOB BARU (POIN D)
===");
let jobOtomatis = { idJob: "J04", namaProses: "Grinding",
durasi: 20 };
antrianMesin.push(jobOtomatis); // Menambahkan job ke-4
prosesAntrian(antrianMesin); // Memanggil fungsi lagi

// Fungsi ini dipanggil saat tombol "Tampilkan Antrian Saat
Ini" diklik
function jalankanProses() {
  console.log("\n--- Menampilkan Antrian ke Layar HTML ---");
  let outputLayar = prosesAntrian(antrianMesin);
  document.getElementById("output").innerHTML = outputLayar;
}

// Fungsi ini dipanggil saat tombol "Tambah ke Antrian" diklik
function tambahJob() {
  let inputId = document.getElementById("idJob").value;
  let inputProses =
document.getElementById("namaProses").value;
  let inputDurasi = document.getElementById("durasi").value;

  // Cek jika ada isian yang kosong
  if (!inputId || !inputProses || !inputDurasi) {
    alert("Gagal: Harap isi ID Job, Nama Proses, dan
Durasi!");
  }
}
```

```

    return;
  }
  // Membuat object dari inputan form
  let jobDariForm = {
    idJob: inputId,
    namaProses: inputProses,
    durasi: Number(inputDurasi)
  };

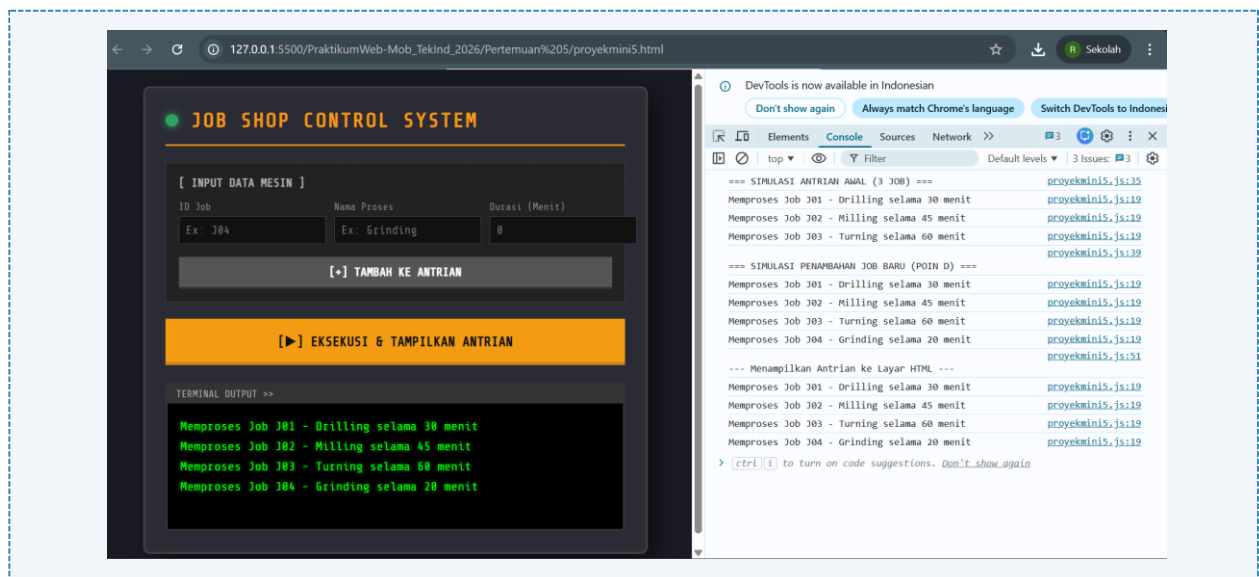
  // Memasukkan ke array
  antrianMesin.push(jobDariForm);

  // Mengosongkan form input kembali
  document.getElementById("idJob").value = "";
  document.getElementById("namaProses").value = "";
  document.getElementById("durasi").value = "";

  alert("Job " + inputId + " berhasil ditambahkan ke antrian!");

  // Langsung perbarui tampilan layar secara otomatis
  jalankanProses();
}

```



Gambar F.3 – Hasil Mini Proyek Sistem Antrian Job Shop

G. PEMBAHASAN

Analisis hasil praktikum secara kritis. Jawab pertanyaan berikut dengan lengkap dan mendalam (bukan hanya "ya/tidak").

G.1 Analisis Kesesuaian Hasil dengan Teori

Berdasarkan hasil praktikum yang telah dilakukan, output program yang dihasilkan sudah sesuai dengan konsep yang dipelajari pada modul. Pada bagian *function*, program berhasil melakukan perhitungan daya listrik dengan menggunakan nilai tegangan dan arus yang dimasukkan. Hasil yang ditampilkan pada *console* menunjukkan bahwa *function* dapat menerima parameter, memproses perhitungan, dan mengembalikan nilai menggunakan *return* sesuai dengan teori yang telah dijelaskan.

Selain itu, penggunaan *array* dan *object* pada program juga berjalan sesuai dengan konsep yang dipelajari. *Array* dapat menyimpan beberapa data dalam satu variabel dan ditampilkan menggunakan perulangan sehingga setiap data dapat terlihat secara berurutan. Sementara itu, *object* mampu menyimpan data yang lebih kompleks seperti informasi mesin dan spesifikasinya, serta memungkinkan perubahan nilai pada salah satu propertinya. Dari hasil tersebut dapat disimpulkan bahwa implementasi kode pada praktikum sudah sesuai dengan teori mengenai penggunaan *function*, *array*, dan *object* dalam JavaScript.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Kebingungan saat mengakses data pada <i>array</i> menggunakan indeks atau saat melakukan perulangan.	Melakukan percobaan langsung pada kode dengan menampilkan setiap elemen <i>array</i> menggunakan <i>looping</i> sehingga dapat memahami bagaimana data diakses dan ditampilkan.
Output proses tidak muncul di <i>Console</i> browser meskipun data sudah tampil di halaman HTML.	Menambahkan perintah <code>console.log()</code> di dalam perulangan <code>forEach()</code> agar data dapat ditampilkan sekaligus di halaman dan di <i>Console</i> .
-	-

G.3 Kaitan dengan Penerapan di Industri

Keterampilan yang dipelajari pada modul ini, seperti penggunaan *function*, *array*, dan *object*, dapat diterapkan dalam pengembangan sistem sederhana untuk membantu pengolahan data di lingkungan industri. Dalam praktik Teknik Industri, banyak aktivitas yang melibatkan pencatatan dan pengolahan data, seperti data produksi, data mesin, maupun data kualitas produk. Dengan menggunakan konsep *function*, suatu perhitungan atau proses tertentu dapat dibuat secara otomatis, misalnya untuk menghitung total produksi, tingkat cacat, atau konsumsi energi mesin dalam suatu periode.

Selain itu, *array* dan *object* dapat digunakan untuk menyimpan serta mengelola data yang berkaitan dengan proses produksi. Contohnya, sebuah sistem monitoring sederhana dapat menyimpan daftar mesin dalam bentuk *object* yang berisi informasi seperti nama mesin, status operasional, dan spesifikasinya. Data tersebut kemudian dapat disimpan dalam *array* untuk merepresentasikan beberapa mesin dalam satu lini produksi. Dengan cara ini, data dapat dipantau dan diolah secara lebih sistematis sehingga membantu pengambilan keputusan dalam pengelolaan proses produksi di industri.

H. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1	Praktikum ini membantu memahami penggunaan <i>function</i> dalam JavaScript untuk membuat proses perhitungan atau instruksi yang dapat digunakan kembali tanpa harus menuliskan kode yang sama secara berulang.
2	Mahasiswa dapat mempelajari cara menggunakan <i>array</i> untuk menyimpan dan mengelola beberapa data dalam satu variabel serta menampilkannya menggunakan perulangan (<i>looping</i>).
3	Praktikum juga memperkenalkan konsep <i>object</i> yang digunakan untuk merepresentasikan data yang lebih kompleks, seperti informasi mesin yang memiliki beberapa atribut atau properti.
4	Melalui penggabungan <i>array</i> dan <i>object</i> , mahasiswa dapat memahami bagaimana data dapat disusun dalam bentuk yang lebih terstruktur, mirip dengan penyimpanan data pada aplikasi web nyata.
5	

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Mozilla Developer Network. (2023). JavaScript Guide. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide
[3]	W3Schools. (2023). JavaScript Tutorial. https://www.w3schools.com/js/
[4]	Flanagan, D. (2020). <i>JavaScript: The Definitive Guide</i> (7th ed.). O'Reilly Media.
[5]	Kadir, A. (2018). <i>Dasar Pemrograman Web Dinamis Menggunakan JavaScript</i> . Yogyakarta: Andi Publisher.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			

TOTAL NILAI AKHIR 100%

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
Radhitya Adha Fadhlur Rohman NIM: 23051430005	Maessa Andrea Vallenia NIM: 23051430044	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

Modul 6 DOM Manipulation & Event Handling

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 6 (Pertemuan_6)
Topik Utama	DOM Manipulation & Event Handling
Sub-Topik	Seleksi DOM, Event Click, Event Input, dan Manipulasi Elemen HTML
Durasi Praktikum	340 Menit
Tanggal Praktikum	27 Maret 2026
Nama Mahasiswa	Radhitya Adha Fadhlur Rohman
NIM	23051430005
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami struktur DOM dan bagaimana JavaScript melihat halaman web.
2	Mampu memanipulasi elemen HTML (mengubah teks, warna, kelas) secara dinamis menggunakan JavaScript
3	Mampu menangani interaksi pengguna (klik tombol, ketik keyboard) menggunakan Event Listeners.
4	Menerapkan konsep ini untuk membuat simulasi Control Panel mesin industri.

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging

3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	Laptop	Ram 8 GB	Perangkat untuk membuat dan menjalankan program
5	Javascripttutorial.net	Versi terbaru	Referensi pembelajaran JavaScript dan contoh kode

D. DASAR TEORI

Document Object Model (DOM) merupakan representasi suatu halaman web dalam bentuk struktur objek yang dapat diakses dan dimanipulasi menggunakan JavaScript. Melalui DOM, setiap elemen pada halaman web, seperti teks, tombol, gambar, maupun formulir, dapat dikenali sebagai objek yang memiliki atribut dan fungsi tertentu. Dengan adanya DOM, halaman web tidak hanya menampilkan informasi secara statis, tetapi juga dapat diperbarui secara dinamis sesuai dengan kebutuhan dan interaksi pengguna.

Selain DOM, konsep penting yang digunakan dalam pengembangan web interaktif adalah *event handling*. *Event* merupakan suatu kejadian yang terjadi pada halaman web, seperti klik tombol, pergerakan mouse, atau perubahan nilai pada kolom input. Untuk merespons kejadian tersebut, JavaScript menggunakan *event listener* yang berfungsi mendeteksi *event* dan menjalankan perintah tertentu. Penerapan *event handling* memungkinkan sistem memberikan respons secara langsung terhadap tindakan pengguna tanpa perlu melakukan pemuatan ulang (*reload*) halaman.

Pada praktikum ini, konsep DOM dan *event handling* diterapkan dalam pembuatan panel kontrol mesin dan kalkulator tagihan listrik. JavaScript digunakan untuk mengakses elemen-elemen HTML, mengubah tampilan status, memvalidasi data masukan, serta menampilkan hasil perhitungan secara *real-time*. Melalui penerapan kedua konsep tersebut, halaman web menjadi lebih interaktif dan responsif sehingga dapat memberikan pengalaman penggunaan yang lebih baik dibandingkan halaman web yang bersifat statis.

D.1 Konsep Utama

Konsep utama yang dipelajari pada modul ini adalah *Document Object Model* (DOM) dan *Event Handling*. DOM merupakan struktur yang digunakan oleh browser untuk merepresentasikan elemen-elemen pada halaman web sebagai objek yang dapat diakses dan dimanipulasi menggunakan JavaScript. Dengan memanfaatkan DOM, pengembang dapat

mengubah isi teks, atribut, warna, maupun tampilan suatu elemen secara dinamis tanpa perlu memuat ulang halaman web.

Selain DOM, modul ini juga membahas *event handling*, yaitu mekanisme untuk menangani berbagai interaksi yang dilakukan oleh pengguna pada halaman web. Interaksi tersebut dapat berupa klik tombol (*click*), perubahan nilai pada kolom input (*input*), maupun pergerakan kursor (*mouseover* dan *mouseout*). Melalui *event listener*, JavaScript dapat mendeteksi setiap kejadian tersebut dan menjalankan fungsi tertentu sesuai dengan kondisi yang diinginkan.

Penerapan kedua konsep tersebut memungkinkan halaman web menjadi lebih interaktif dan responsif. Pada praktikum ini, DOM dan *event handling* digunakan untuk membuat simulasi panel kontrol mesin yang dapat mengubah status dan suhu secara otomatis, serta kalkulator tagihan listrik yang menampilkan hasil perhitungan secara *real-time*. Dengan demikian, pengguna dapat berinteraksi langsung dengan sistem dan memperoleh umpan balik secara cepat sesuai tindakan yang dilakukan.

D.2 Keterkaitan dengan Teknik Industri

Materi mengenai *Document Object Model* (DOM) dan *event handling* memiliki keterkaitan yang erat dengan bidang Teknik Industri, terutama dalam pengembangan sistem informasi, *dashboard* monitoring, dan otomatisasi proses kerja. Seorang insinyur industri tidak hanya berfokus pada perancangan sistem produksi, tetapi juga perlu memahami bagaimana data dan informasi dapat ditampilkan secara efektif kepada pengguna. Dengan memanfaatkan DOM dan JavaScript, informasi operasional dapat ditampilkan secara dinamis sehingga memudahkan proses pemantauan dan pengambilan keputusan.

Salah satu contoh penerapannya adalah pada sistem pemantauan mesin produksi di pabrik. Operator dapat melihat status mesin, suhu operasi, kecepatan produksi, atau kondisi peralatan secara *real-time* melalui *dashboard* berbasis web. Ketika terjadi kondisi tertentu, seperti suhu mesin yang melebihi batas aman, sistem dapat secara otomatis mengubah warna indikator atau menampilkan peringatan kepada operator. Konsep ini serupa dengan simulasi panel kontrol mesin yang dibuat pada praktikum, di mana tampilan berubah sesuai kondisi yang terjadi pada sistem.

Selain itu, konsep *event handling* juga banyak digunakan dalam sistem administrasi dan pengolahan data industri. Sebagai contoh, pada sistem perencanaan produksi atau manajemen persediaan, hasil perhitungan dapat diperbarui secara otomatis ketika pengguna memasukkan data baru. Penerapan seperti ini membantu meningkatkan efisiensi kerja, mengurangi kesalahan perhitungan manual, serta mempercepat proses pengambilan keputusan. Oleh karena itu, pemahaman mengenai DOM dan *event handling* menjadi keterampilan yang penting bagi mahasiswa Teknik Industri dalam menghadapi perkembangan teknologi informasi di dunia industri.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

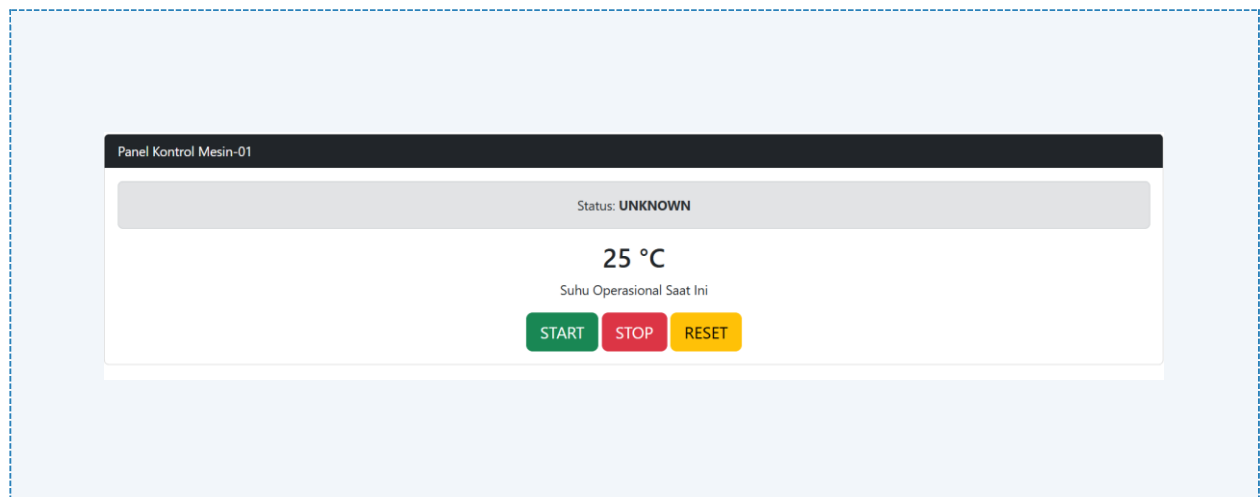
E.1 Langkah 1 – Membuat UI Panel Kontrol di HTML

Buka `pertemuan6.html`, tambahkan struktur HTML berikut di dalam container:

```
<div class="card mt-4">
  <div class="card-header bg-dark text-white">Panel Kontrol Mesin-
01</div>
  <div class="card-body text-center">
    <!-- Indikator Status -->
    <div id="statusIndicator" class="alert alert-secondary"
role="alert">
      Status: <strong>UNKNOWN</strong>
    </div>

    <h2 id="suhuMesin">25 °C</h2>
    <p>Suhu Operasional Saat Ini</p>
    <div class="d-grid gap-2 d-md-block">
      <button id="btnStart" class="btn btn-success btn-
lg">START</button>
      <button id="btnStop" class="btn btn-danger btn-
lg">STOP</button>
      <button id="btnReset" class="btn btn-warning btn-
lg">RESET</button>
    </div>
  </div>
</div>
</body>
</html>
```

Screenshot Hasil:



Gambar E.1 – Tampilan awal UI panel kontrol mesin

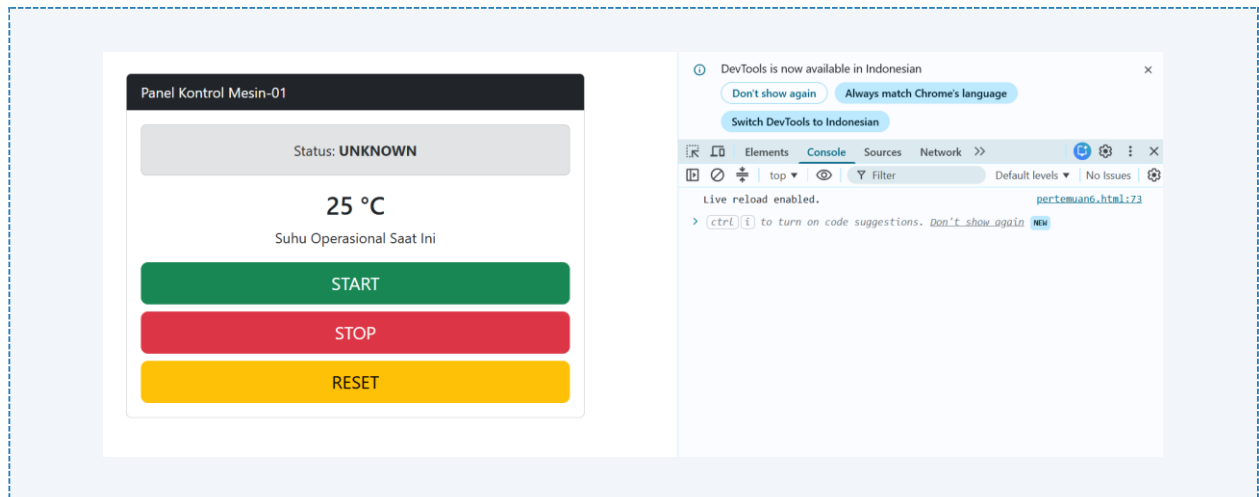
E.2 Langkah 2 – Seleksi Elemen di JavaScript

Buka `script.js` (atau buat file baru dan link-kan). Kita akan mengambil elemen yang kita buat di atas.


```
const btnStart = document.getElementById('btnStart');
const btnStop = document.getElementById('btnStop');
const btnReset = document.getElementById('btnReset');
const statusIndicator = document.getElementById('statusIndicator');
const suhuMesin = document.getElementById('suhuMesin');
const teksStatus = statusIndicator.querySelector('strong'); //mengambil
elemen <strong> didalam alert

// variable state
let suhu = 25;
let intervalId = null; //untuk menyimpan id timer
```

Screenshot Hasil:



Gambar E.2 – Hasil Langkah 2

E.3 Langkah 3 – Menambahkan Event Listener (Click)

berikan "nyawa" pada tombol

```
// event listener tombol start
btnStart.addEventListener("click", function () {
  //ubah ui status
  statusIndicator.className = "alert alert-succes"; // Ganti class
  Bootstrap (Hijau)
  teksStatus.innerText = "RUNNING";

  // Logika simulasi kenaikan suhu
  intervalId = setInterval(function () {
    suhu += 1;
    suhuMesin.innerText = suhu + " °C";

    // Peringatan jika suhu overheat
    if (suhu > 80) {
```

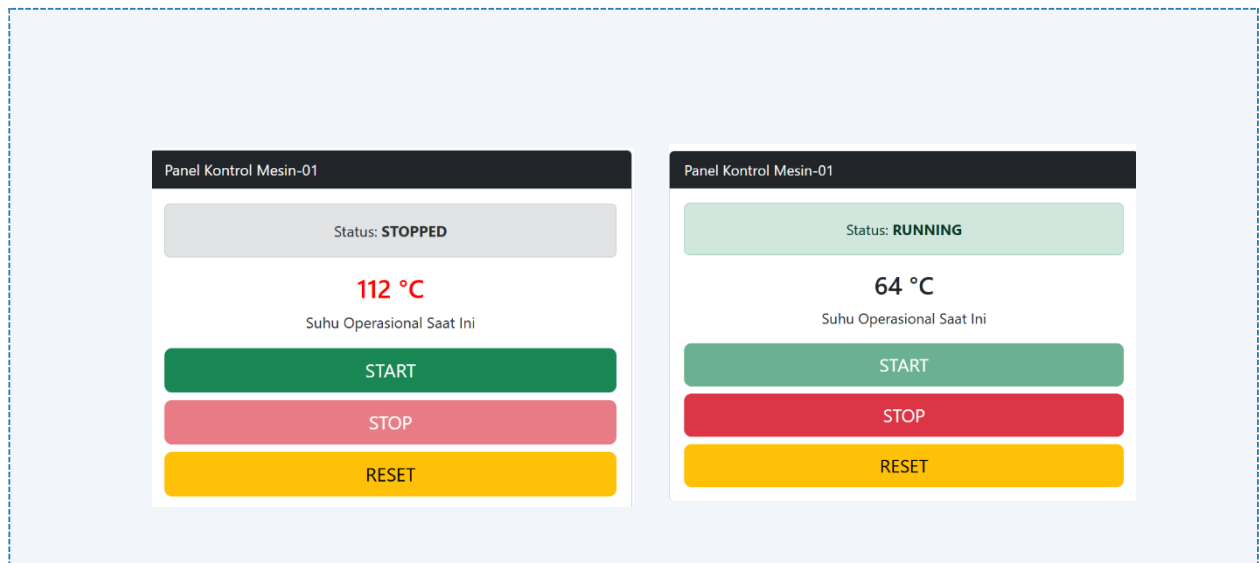
```
        statusIndicator.className = "alert alert-danger";
        teksStatus.innerText = "OVERHEAT WARNING";
        suhuMesin.style.color = "red";
    }
}, 1000); // Jalankan setiap 1 detik

// Matikan tombol Start agar tidak double click
btnStart.disabled = true;
btnStop.disabled = false;
});
// Event Listener Tombol STOP
btnStop.addEventListener("click", function () {
    clearInterval(intervalId); // Hentikan timer kenaikan suhu
    statusIndicator.className = "alert alert-secondary";
    teksStatus.innerText = "STOPPED";

    // Reset tombol
    btnStart.disabled = false;
    btnStop.disabled = true;
});
// Event Listener Tombol RESET
btnReset.addEventListener("click", function () {
    clearInterval(intervalId);
    suhu = 25;
    suhuMesin.innerText = suhu + " °C";
    suhuMesin.style.color = "black";
    statusIndicator.className = "alert alert-secondary";
    teksStatus.innerText = "UNKNOWN";

    btnStart.disabled = false;
    btnStop.disabled = true;
});
```

Screenshot Hasil:

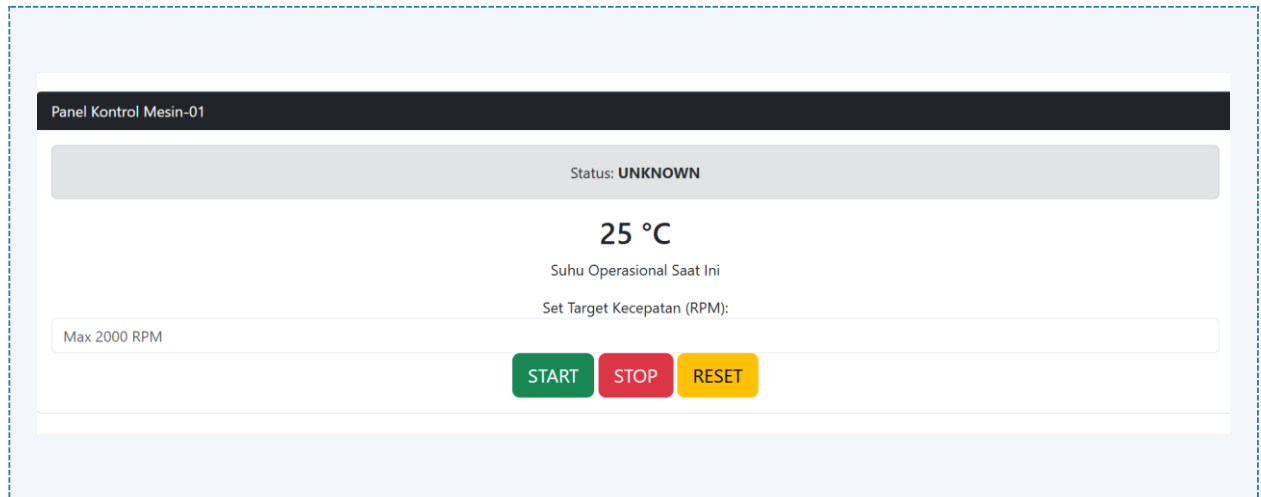


Gambar E.3 – Hasil implementasi event listener saat tombol START dan STOP ditekan

E.4 Langkah 4 – Event Input (Validasi Real-time)

```
const inputRPM = document.getElementById("inputRPM");
const pesanError = document.getElementById("pesanError");
inputRPM.addEventListener("input", function () {
  let val = parseInt(this.value);

  if (val > 2000) {
    pesanError.classList.remove("d-none"); // Tampilkan pesan
    error
    this.classList.add("is-invalid"); // Berikan border merah
    Bootstrap
  } else {
    pesanError.classList.add("d-none"); // Sembunyikan pesan
    this.classList.remove("is-invalid");
  }
});
```



Gambar E.4 - Hasil langkah 4

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – DOM Styling

Pertanyaan / Instruksi Latihan:

Buatlah tombol tambahan bernama "Maintenance Mode". Saat diklik, background dari card menjadi abu-abu (`bg-light`), dan teks status berubah menjadi "MAINTENANCE"

Jawaban / Kode Solusi:

```
// 1. Seleksi Elemen DOM
const btnStart = document.getElementById('btnStart');
const btnStop = document.getElementById('btnStop');
const btnReset = document.getElementById('btnReset');

const statusIndicator = document.getElementById('statusIndicator');
const suhuMesin = document.getElementById('suhuMesin');

// Mengambil elemen <strong> di dalam alert
const teksStatus = statusIndicator.querySelector('strong');

// Variabel State
let suhu = 25;
let intervalId = null; // Untuk menyimpan ID timer

// Event Listener Tombol START
btnStart.addEventListener('click', function () {
  // Ubah UI Status
  statusIndicator.className = 'alert alert-success';
  teksStatus.innerText = 'RUNNING';

  // Logika simulasi kenaikan suhu
```

```
intervalId = setInterval(function () {
    suhu += 1;
    suhuMesin.innerText = suhu + ' °C';

    // Peringatan jika suhu overheat
    if (suhu > 80) {
        statusIndicator.className = 'alert alert-danger';
        teksStatus.innerText = 'OVERHEAT WARNING';
        suhuMesin.style.color = 'red';
    }
}, 1000);

// Matikan tombol Start agar tidak double click
btnStart.disabled = true;
btnStop.disabled = false;
});

// Event Listener Tombol STOP
btnStop.addEventListener('click', function () {
    clearInterval(intervalId);

    statusIndicator.className = 'alert alert-secondary';
    teksStatus.innerText = 'STOPPED';

    btnStart.disabled = false;
    btnStop.disabled = true;
});

// Event Listener Tombol RESET
btnReset.addEventListener('click', function () {
    clearInterval(intervalId);

    suhu = 25;
    suhuMesin.innerText = suhu + ' °C';
    suhuMesin.style.color = 'black';

    statusIndicator.className = 'alert alert-secondary';
    teksStatus.innerText = 'UNKNOWN';

    btnStart.disabled = false;
    btnStop.disabled = true;
});

// Validasi Input RPM
const inputRPM = document.getElementById('inputRPM');
const pesanError = document.getElementById('pesanError');
```

```
inputRPM.addEventListener('input', function () {
    let val = parseInt(this.value);

    if (val > 2000) {
        pesanError.classList.remove('d-none');
        this.classList.add('is-invalid');
    } else {
        pesanError.classList.add('d-none');
        this.classList.remove('is-invalid');
    }
});

// Maintenance Mode
const btnMaintenance = document.getElementById('btnMaintenance');
const panelMesin = document.getElementById('panelMesin');

btnMaintenance.addEventListener('click', function () {
    // Hentikan simulasi jika sedang berjalan
    clearInterval(intervalId);

    // Ubah background card menjadi abu-abu
    panelMesin.classList.add('bg-light');

    // Ubah status
    statusIndicator.className = 'alert alert-warning';
    teksStatus.innerText = 'MAINTENANCE';

    // Atur tombol
    btnStart.disabled = false;
    btnStop.disabled = true;
});
```

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Panel Kontrol Mesin</title>

    <!-- Bootstrap CSS -->
    <link
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
        rel="stylesheet"
    >
</head>
```

```

<body>

    <!-- Simulasi Panel Kontrol Mesin (Meeting 6) -->
    <div id="panelMesin" class="card mt-4 mx-auto" style="max-width:
700px;">

        <div class="card-header bg-dark text-white">
            Panel Kontrol Mesin-01
        </div>

        <div class="card-body text-center">

            <!-- Indikator Status -->
            <div id="statusIndicator" class="alert alert-secondary"
role="alert">
                Status: <strong>UNKNOWN</strong>
            </div>

            <!-- Tampilan Suhu -->
            <h2 id="suhuMesin">25 °C</h2>
            <p>Suhu Operasional Saat Ini</p>

            <!-- Input RPM -->
            <div class="mt-3">
                <label class="form-label">
                    Set Target Kecepatan (RPM):
                </label>

                <input
                    type="number"
                    id="inputRPM"
                    class="form-control"
                    placeholder="Max 2000 RPM"
                >

                <small id="pesanError" class="text-danger d-none">
                    RPM melebihi batas aman!
                </small>
            </div>

            <!-- Tombol Kontrol -->
            <div class="d-grid gap-2 d-md-block mt-4">
                <button id="btnStart" class="btn btn-success btn-lg">
                    START
                </button>

                <button id="btnStop" class="btn btn-danger btn-lg" disabled>
                    STOP
                </button>

                <button id="btnReset" class="btn btn-warning btn-lg">
                    RESET
                </button>

                <button id="btnMaintenance" class="btn btn-secondary btn-
lg">
                    MAINTENANCE MODE
                </button>
            </div>
        </div>
    </div>

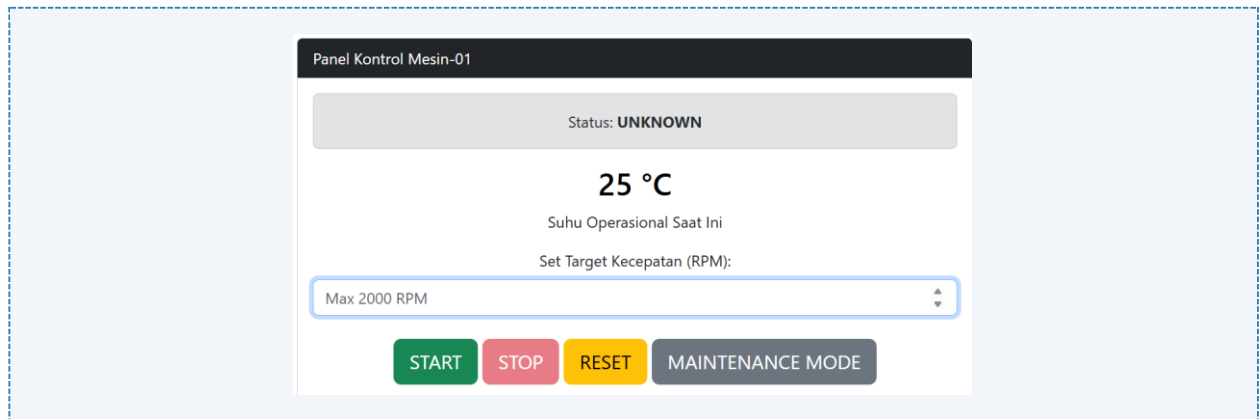
```

```
        </div>

    </div>
</div>

<!-- JavaScript -->
<script src="pertemuan6.js"></script>

</body>
</html>
```



Gambar F.1 – Hasil Latihan 1

F.2 Latihan 2 – Mouse Events

Pertanyaan / Instruksi Latihan:

[Tambahkan efek saat mouse diarahkan (`mouseover`) ke teks suhu mesin, suhu tersebut menjadi tebal dan berwarna biru. Saat mouse keluar (`mouseout`), kembali normal

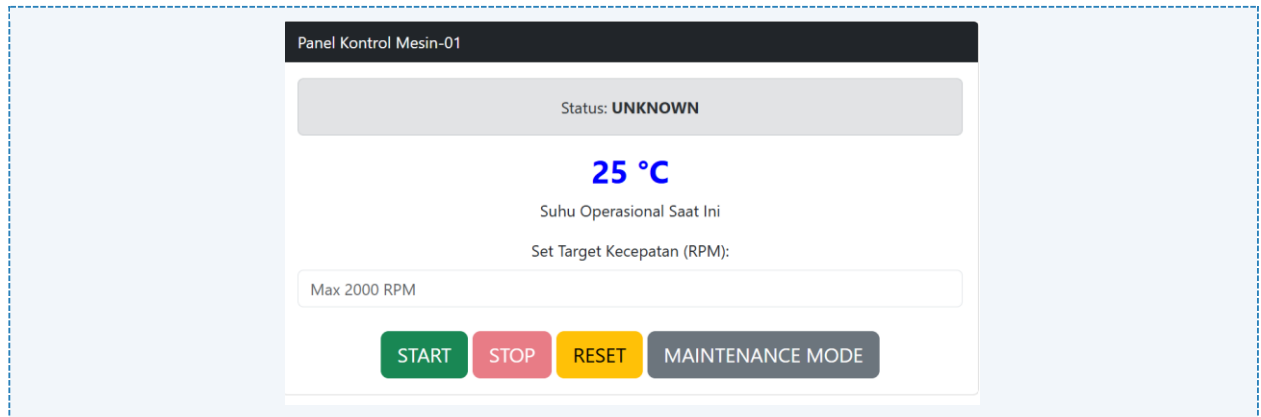
Jawaban / Kode Solusi:

Tambahkan pada file JavaScript

```
// Mouse Events pada Suhu Mesin

// Saat mouse diarahkan ke suhu
suhuMesin.addEventListener('mouseover', function () {
    suhuMesin.style.color = 'blue';
    suhuMesin.style.fontWeight = 'bold';
});

// Saat mouse keluar dari suhu
suhuMesin.addEventListener('mouseout', function () {
    suhuMesin.style.color = 'black';
    suhuMesin.style.fontWeight = 'normal';
});
```

Gambar F.2 – Hasil latihan 2

F.3 Tugas Proyek Mini – Kalkulator Tagihan Listrik

Deskripsi proyek mini:

- Buat UI dengan 2 input: "Daya (Watt)" dan "Jam Pemakaian".
- Saat user mengetik di salah satu input, secara real-time (tanpa tombol submit) tampilkan hasil "Total kWh" dan "Estimasi Biaya (Rp 1.500/kWh)" di elemen ` di bawahnya.

Penjelasan pendekatan/solusi yang digunakan:

Pada proyek mini ini, pendekatan yang digunakan adalah memanfaatkan *Document Object Model* (DOM) dan *event handling* untuk membuat kalkulator tagihan listrik yang dapat bekerja secara *real-time*. Langkah pertama yang dilakukan adalah membuat antarmuka (*user interface*) menggunakan HTML yang terdiri dari dua kolom input, yaitu daya listrik dalam satuan Watt dan jam pemakaian. Selain itu, disediakan area hasil yang berfungsi untuk menampilkan total energi listrik dan estimasi biaya penggunaan listrik.

Selanjutnya, JavaScript digunakan untuk mengambil data dari kedua kolom input melalui DOM. Ketika pengguna memasukkan atau mengubah nilai pada salah satu kolom, *event listener* dengan jenis *input* akan secara otomatis menjalankan fungsi perhitungan. Fungsi tersebut menghitung total energi listrik menggunakan rumus energi (kWh) = daya (Watt) × waktu (jam) ÷ 1000. Setelah itu, biaya listrik dihitung dengan mengalikan total energi yang diperoleh dengan tarif listrik sebesar Rp1.500 per kWh.

Hasil perhitungan kemudian ditampilkan secara langsung pada halaman web tanpa memerlukan tombol proses atau *submit*. Untuk meningkatkan kenyamanan pengguna, tampilan aplikasi dibuat menggunakan Bootstrap dan CSS sehingga lebih menarik, responsif, dan mudah digunakan pada berbagai ukuran layar. Dengan pendekatan ini, aplikasi mampu memberikan informasi penggunaan energi dan estimasi biaya listrik secara cepat, akurat, dan interaktif.

Kode Utama:

File JavaScript

```
const inputDaya = document.getElementById("inputDaya");
const inputJam = document.getElementById("inputJam");
const tampilHasil = document.getElementById("hasil");

// Fungsi perhitungan
function prosesPerhitungan() {
  // (default 0 kalau kosong)
  const nilaiDaya = Number(inputDaya.value) || 0;
  const nilaiJam = Number(inputJam.value) || 0;

  // Hitung energi (kWh)
  const totalEnergi = (nilaiDaya * nilaiJam) / 1000;

  // Hitung biaya
  const totalBiaya = totalEnergi * 1500;

  // Tampilkan ke UI
  tampilHasil.innerHTML = `
    Total Energi: ${totalEnergi.toFixed(2)} kWh <br>
    Estimasi Biaya: Rp ${totalBiaya.toLocaleString("id-ID")}
  `;
}

// Event real-time (tanpa tombol)
inputDaya.addEventListener("input", prosesPerhitungan);
inputJam.addEventListener("input", prosesPerhitungan);
```

HTML

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Kalkulator Tagihan Listrik</title>

  <!-- Bootstrap -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/boo
tstrap.min.css" rel="stylesheet">
  <link rel="stylesheet" href="proyek mini 6.css">
</head>

<body>
```

```

    <div class="container d-flex justify-content-center align-items-center min-vh-100">

        <div class="card p-4 shadow-lg">
            <h3 class="text-center mb-3">Kalkulator Tagihan Listrik</h3>

            <p class="text-center text-muted">
                Oleh: Radhitya Adha Fadhlur Rohman
                (23051430005)
            </p>

            <!-- Input Daya -->
            <div class="mb-3">
                <label>Daya (Watt)</label>
                <input type="number" id="inputDaya"
class="form-control" placeholder="Contoh: 900">
            </div>

            <!-- Input Jam -->
            <div class="mb-3">
                <label>Jam Pemakaian</label>
                <input type="number" id="inputJam" class="form-control" placeholder="Contoh: 5">
            </div>

            <!-- Hasil -->
            <h3 id="hasil" class="text-center text-primary">
                Hasil akan muncul di sini
            </h3>

        </div>

    </div>

    <script src="proyek mini 6.js"></script>
</body>
</html>

```

Tambahkan Kode CSS

```

/* ===== RESET ===== */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

/* ===== BODY ===== */

```

```
body {
  font-family: 'Segoe UI', Tahoma, sans-serif;
  background: linear-gradient(135deg, #dbeafe, #f8fafc);
  min-height: 100vh;
}

/* ===== CONTAINER CENTER ===== */
.container {
  padding: 20px;
}

/* ===== CARD ===== */
.card {
  border: none;
  border-radius: 18px;
  width: 100%;
  max-width: 420px;
  background: #ffffff;
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1);
  transition: 0.3s;
}

/* Hover card */
.card:hover {
  transform: translateY(-5px);
}

/* ===== TITLE ===== */
.card h3 {
  font-weight: 600;
  color: #1e293b;
}

/* Subtitle */
.card p {
  font-size: 13px;
}

/* ===== INPUT ===== */
input {
  border-radius: 10px !important;
  padding: 12px;
  border: 1px solid #d1d5db;
  transition: all 0.25s ease;
}

/* Fokus input */
input:focus {
  border-color: #2563eb;
  box-shadow: 0 0 8px rgba(37, 99, 235, 0.3);
  outline: none;
}

/* ===== LABEL ===== */
label {
  font-weight: 500;
  margin-bottom: 5px;
  color: #374151;
}
```

```
/* ===== HASIL ===== */
#hasil {
  margin-top: 20px;
  font-size: 18px;
  font-weight: 600;
  background: #f1f5f9;
  padding: 15px;
  border-radius: 12px;
  transition: all 0.3s ease;
}

/* Animasi muncul */
#hasil:hover {
  transform: scale(1.02);
}

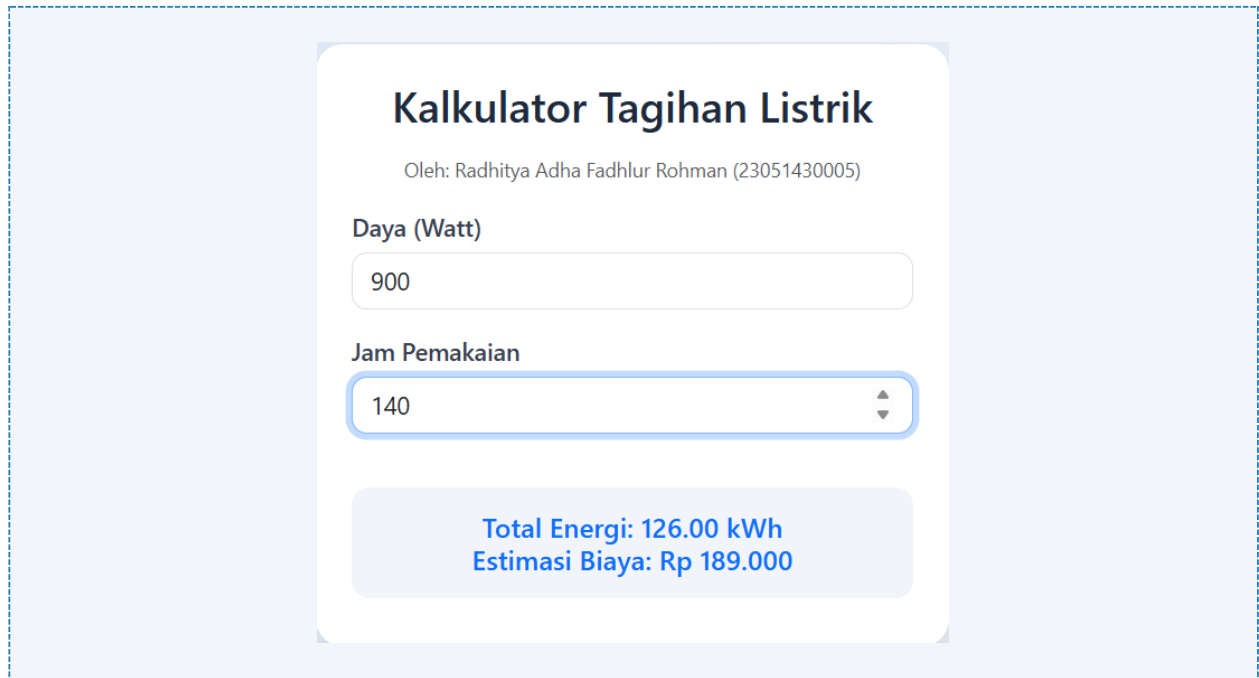
/* ===== RESPONSIVE ===== */

/* Tablet */
@media (max-width: 768px) {
  .card {
    max-width: 90%;
  }
}

/* Mobile */
@media (max-width: 576px) {
  .card {
    padding: 15px !important;
    border-radius: 14px;
  }

  #hasil {
    font-size: 16px;
    padding: 12px;
  }

  h3 {
    font-size: 18px;
  }
}
```



The image shows a web-based application titled "Kalkulator Tagihan Listrik". Below the title, it credits the developer: "Oleh: Radhitya Adha Fadhlur Rohman (23051430005)". There are two input fields: "Daya (Watt)" with a text input containing "900", and "Jam Pemakaian" with a spinner input containing "140". At the bottom, a light blue box displays the results: "Total Energi: 126.00 kWh" and "Estimasi Biaya: Rp 189.000".

Kalkulator Tagihan Listrik

Oleh: Radhitya Adha Fadhlur Rohman (23051430005)

Daya (Watt)

900

Jam Pemakaian

140

Total Energi: 126.00 kWh
Estimasi Biaya: Rp 189.000

Gambar F.3 – Hasil proyek mini kalkulator listrik

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang diperoleh telah sesuai dengan teori *Document Object Model* (DOM) dan *event handling* yang dipelajari pada modul ini. Pada program kalkulator tagihan listrik, JavaScript berhasil mengakses elemen HTML menggunakan DOM melalui fungsi `getElementById()`, kemudian memanipulasi isi elemen hasil menggunakan `innerHTML`. Selain itu, penggunaan *event listener* bertipe input memungkinkan sistem mendeteksi setiap perubahan nilai pada kolom daya dan jam pemakaian. Ketika pengguna mengetik data, program secara otomatis menjalankan fungsi perhitungan dan memperbarui hasil pada halaman web tanpa perlu menekan tombol tertentu. Hal ini menunjukkan bahwa konsep hubungan antara *event*, logika program, dan pembaruan tampilan telah diterapkan dengan baik. Kesesuaian dengan teori juga terlihat pada hasil perhitungan yang ditampilkan. Program menggunakan rumus energi listrik, yaitu daya (Watt) dikalikan waktu pemakaian (jam) kemudian dibagi 1000 untuk mendapatkan nilai dalam satuan kWh. Nilai energi tersebut selanjutnya dikalikan dengan tarif Rp1.500 per kWh untuk memperoleh estimasi biaya listrik. Sebagai contoh, ketika pengguna memasukkan daya 900 Watt dan waktu pemakaian 5 jam, program menghasilkan total energi sebesar 4,50 kWh dan estimasi biaya sebesar Rp6.750. Hasil tersebut sesuai dengan perhitungan teoritis sehingga menunjukkan bahwa logika perhitungan dan implementasi kode telah berjalan dengan benar.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Hasil perhitungan tidak muncul saat pengguna mengisi data.	Menambahkan <i>event listener</i> bertipe input pada kedua kolom masukan agar fungsi perhitungan berjalan secara otomatis.
Status dan warna indikator tidak berubah sesuai kondisi mesin.	Memeriksa kembali penggunaan DOM dan memastikan elemen yang dipanggil memiliki id yang sesuai dengan kode JavaScript.

G.3 Kaitan dengan Penerapan di Industri

Keterampilan yang dipelajari pada modul ini memiliki banyak penerapan dalam dunia Teknik Industri, terutama pada sistem pemantauan dan pengendalian proses berbasis digital. Konsep *Document Object Model* (DOM) dan *event handling* memungkinkan pengembang membuat antarmuka yang dapat menampilkan informasi secara dinamis serta merespons tindakan pengguna secara langsung. Kemampuan ini sangat penting dalam pengembangan *dashboard* operasional yang digunakan untuk memantau kinerja mesin, produksi, maupun penggunaan sumber daya di perusahaan. Salah satu contoh penerapannya adalah pada sistem pemantauan mesin produksi di pabrik. Operator dapat melihat informasi seperti suhu mesin, kecepatan produksi, atau status operasi

melalui *dashboard* berbasis web. Ketika operator menekan tombol *start*, *stop*, atau *maintenance*, sistem akan langsung memperbarui tampilan status mesin tanpa perlu memuat ulang halaman. Selain itu, jika sensor mendeteksi suhu yang melebihi batas aman, sistem dapat secara otomatis menampilkan peringatan visual berupa perubahan warna indikator atau pesan peringatan agar operator dapat segera mengambil tindakan.

Contoh lainnya adalah pada sistem manajemen energi di perusahaan manufaktur. Data konsumsi listrik dari berbagai peralatan dapat diinput dan dihitung secara otomatis untuk memperkirakan penggunaan energi dan biaya operasional. Konsep yang sama seperti pada proyek mini kalkulator tagihan listrik dapat digunakan untuk membantu manajer atau engineer dalam memantau penggunaan energi secara *real-time*, sehingga perusahaan dapat mengidentifikasi pemborosan energi dan mengambil langkah perbaikan guna meningkatkan efisiensi operasional.

H. KESIMPULAN

1	Praktikum ini berhasil meningkatkan pemahaman mengenai <i>Document Object Model</i> (DOM) sebagai media untuk mengakses, mengubah, dan memperbarui elemen HTML secara dinamis menggunakan JavaScript.
2	Konsep <i>event handling</i> berhasil diterapkan melalui penggunaan berbagai <i>event listener</i> seperti <i>click</i> , <i>input</i> , <i>mouseover</i> , dan <i>mouseout</i> untuk menciptakan interaksi antara pengguna dan aplikasi web.
3	Simulasi panel kontrol mesin berhasil dibuat dengan fitur <i>START</i> , <i>STOP</i> , <i>RESET</i> , validasi <i>RPM</i> , serta <i>Maintenance Mode</i> yang mampu mengubah status dan tampilan antarmuka sesuai kondisi yang terjadi.
4	Proyek mini kalkulator tagihan listrik berhasil menampilkan hasil perhitungan energi listrik (kWh) dan estimasi biaya secara <i>real-time</i> berdasarkan data yang dimasukkan pengguna tanpa memerlukan tombol <i>submit</i> .

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Mozilla Developer Network. (2024). <i>Document Object Model (DOM)</i> . https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
[3]	Mozilla Developer Network. (2024). <i>EventTarget: addEventListener() method</i> . https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
Radhitya Adha Fadhlur Rohman NIM: 23051430005	Maessa Andrea Vallenia NIM: 23051430044	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

Modul 7 Form Validation & Local Storage

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 7 (Pertemuan_7)
Topik Utama	Form Validation & Local Storage
Sub-Topik	Validasi form tingkat lanjut dan penyimpanan data persisten di browser (Client-side Storage).
Durasi Praktikum	340 Menit
Tanggal Praktikum	03 April 2026
Nama Mahasiswa	Radhitya Adha Fadhlur Rohman
NIM	23051430005
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Menerapkan validasi form sebelum data dikirim untuk mencegah data kotor..
2	Memahami cara kerja Web Storage API (LocalStorage).
3	Mampu menyimpan, mengambil, mengupdate, dan menghapus (CRUD sederhana) data di LocalStorage.
4	Membuat aplikasi "Buku Harian Produksi" sederhana yang datanya tidak hilang saat browser direfresh.

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor

2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	Laptop	Ram 16 GB	Perangkat untuk membuat dan menjalankan program
5			

D. DASAR TEORI

Pada modul ini dibahas mengenai konsep *Document Object Model* (DOM), yaitu suatu cara yang digunakan oleh browser untuk merepresentasikan halaman web dalam bentuk struktur yang dapat diakses dan dikelola menggunakan JavaScript. Melalui DOM, elemen-elemen HTML seperti teks, tombol, dan input diubah menjadi objek yang tersusun menyerupai struktur pohon. Dengan demikian, setiap bagian dari halaman web dapat diambil, diubah, maupun dimanipulasi sesuai kebutuhan. Kehadiran DOM membuat halaman web tidak lagi bersifat statis, melainkan dapat berubah secara dinamis berdasarkan kondisi tertentu atau perintah yang diberikan melalui JavaScript.

Selain itu, pada modul ini juga dipelajari tentang manipulasi elemen pada halaman web. Manipulasi tersebut dilakukan dengan cara mengubah isi, tampilan, maupun atribut dari suatu elemen. Contohnya seperti mengganti teks menggunakan `innerText`, mengubah tampilan melalui properti `style`, serta menambah atau menghapus kelas menggunakan `classList`. Dengan teknik ini, tampilan halaman dapat menyesuaikan dengan kondisi yang terjadi, misalnya ketika sistem mengalami perubahan status atau menampilkan pesan peringatan. Hal ini menjadikan tampilan web lebih interaktif serta mampu memberikan informasi yang lebih jelas kepada pengguna.

Konsep lain yang tidak kalah penting adalah event handling, yaitu mekanisme yang digunakan untuk merespon interaksi pengguna. Interaksi tersebut dapat berupa klik tombol, pengisian data, maupun pergerakan kursor. Dengan memanfaatkan event listener, JavaScript dapat mendeteksi setiap kejadian tersebut dan menjalankan fungsi tertentu sebagai bentuk respon. Pendekatan ini dikenal sebagai event-driven, di mana alur program dipengaruhi oleh tindakan pengguna. Dengan menggabungkan konsep DOM dan event handling, aplikasi web dapat memberikan respon secara langsung tanpa perlu melakukan reload halaman, sehingga sistem menjadi lebih responsif dan efisien.

D.1 Konsep Utama

Konsep utama pada modul ini berfokus pada bagaimana JavaScript dapat berinteraksi langsung dengan halaman web melalui Document Object Model (DOM). DOM dapat dipahami sebagai struktur dari halaman HTML yang diubah menjadi objek oleh browser, sehingga setiap elemen seperti tombol, teks, maupun input bisa diakses dan dimodifikasi. Dengan menggunakan metode seperti `getElementById()` atau `querySelector()`, kita bisa memilih elemen tertentu lalu mengubah isinya, tampilannya, atau atributnya sesuai kebutuhan.

Selain itu, modul ini juga menekankan pada konsep manipulasi DOM yang memungkinkan perubahan tampilan secara dinamis. Setelah elemen berhasil diakses, kita bisa melakukan berbagai perubahan seperti mengganti teks menggunakan `innerText`, mengatur tampilan dengan `style`, atau menambah dan menghapus kelas menggunakan `classList`. Hal ini membuat halaman web menjadi lebih fleksibel, karena tampilan dapat berubah mengikuti kondisi tertentu, misalnya perubahan status sistem atau nilai yang dihasilkan.

Konsep berikutnya adalah event handling, yaitu bagaimana sistem merespon setiap aksi yang dilakukan oleh pengguna. Event dapat berupa klik tombol, input data, maupun interaksi mouse. Dengan menggunakan event listener, JavaScript dapat mendeteksi kejadian tersebut dan menjalankan fungsi tertentu sebagai respon. Dalam praktikum ini, konsep tersebut juga dikombinasikan dengan penggunaan state sederhana, seperti variabel suhu dan status mesin, sehingga halaman web tidak hanya interaktif tetapi juga mampu merepresentasikan kondisi sistem secara real-time.

D.2 Keterkaitan dengan Teknik Industri

Materi yang dipelajari dalam modul ini memiliki hubungan yang cukup kuat dengan bidang Teknik Industri, terutama dalam pengembangan sistem monitoring dan kontrol berbasis digital. Dalam penerapannya di dunia industri, banyak proses produksi yang sudah memanfaatkan dashboard atau antarmuka berbasis web untuk memantau kondisi mesin secara real-time. Dengan adanya konsep manipulasi DOM, tampilan pada dashboard tersebut dapat berubah secara otomatis mengikuti kondisi sistem, misalnya saat terjadi perubahan status mesin atau nilai parameter tertentu.

Sebagai ilustrasi, pada sistem kontrol mesin di pabrik, operator biasanya menggunakan panel untuk mengoperasikan mesin, seperti menyalakan atau mematikan, sekaligus memantau kondisi operasional seperti suhu dan kecepatan. Hal ini sejalan dengan konsep event handling, di mana setiap interaksi pengguna, seperti menekan tombol start atau stop, akan memicu respon tertentu dari sistem. Selain itu, ketika terjadi kondisi yang tidak normal, misalnya suhu melebihi batas yang ditentukan, sistem dapat langsung memberikan peringatan melalui perubahan indikator visual seperti warna atau teks pada layar. Mekanisme ini serupa dengan simulasi panel kontrol yang dibuat dalam praktikum.

Penerapan konsep ini juga mendukung perkembangan menuju era Industri 4.0, di mana sistem cenderung lebih terintegrasi dan berbasis teknologi digital. Dengan memanfaatkan platform web, sistem monitoring dapat diakses secara lebih fleksibel dan responsif. Hal ini tidak hanya meningkatkan efisiensi kerja, tetapi juga membantu meminimalkan kesalahan manusia karena informasi disajikan secara jelas dan real-time. Oleh karena itu, pemahaman

mengenai DOM dan event handling menjadi salah satu dasar penting dalam pengembangan sistem kontrol dan monitoring di bidang Teknik Industri.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat Form Input Produksi

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Log Produksi Harian</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body class="container mt-5">
  <div class="row">

    <!-- Kolom Kiri: Form Input -->
    <div class="col-md-4">
      <div class="card">
        <div class="card-header">Input Data</div>
        <div class="card-body">
          <form id="formProduksi">
            <div class="mb-3">
              <label>Tanggal</label>
              <input type="date" id="tanggal" class="form-control" required>
            </div>
            <div class="mb-3">
              <label>Nama Operator</label>
              <input type="text" id="operator" class="form-control" placeholder="Nama Lengkap" required>
            </div>
            <div class="mb-3">
              <label>Shift</label>
              <select id="shift" class="form-select">
                <option value="Pagi">Pagi</option>
                <option value="Siang">Siang</option>
                <option value="Malam">Malam</option>
              </select>
            </div>
          </form>
        </div>
      </div>
    </div>
  </div>
</body>
```

```

        <option value="Siang">Siang</option>
        <option value="Malam">Malam</option>
    </select>
</div>

<div class="mb-3">
    <label>Jumlah Produksi (Unit)</label>
    <input type="number" id="jumlah" class="form-control" min="1" required>
</div>

<button type="submit" class="btn btn-primary">
    Simpan Log
</button>

<button type="button" id="btnHapusSemua" class="btn btn-danger w-100 mt-2">
    Hapus Semua Data
</button>

</form>
</div>
</div>
</div>

<!-- Kolom Kanan: Tabel Data -->
<div class="col-md-8">
    <h3>Riwayat Produksi</h3>
    <table class="table table-bordered table-striped">
        <thead class="table-dark">
            <tr>
                <th>Tanggal</th>
                <th>Operator</th>
                <th>Shift</th>
                <th>Jumlah</th>
                <th>Aksi</th>
            </tr>
        </thead>
        <tbody id="tabelBody">
            <!-- Data akan masuk di sini lewat JS -->
        </tbody>
    </table>
</div>
</div>

```

```
<script src="script_log.js"></script>
</body>
</html>
```

Screenshot Hasil:

The screenshot displays a web application interface for production data management. On the left, there is a form titled 'Input Data' with the following fields: 'Tanggal' (Date) with a calendar icon, 'Nama Operator' (Operator Name) with a text input, 'Shift' (Shift) with a dropdown menu showing 'Pagi', and 'Jumlah Produksi (Unit)' (Production Quantity in Units) with a text input. Below these fields are two buttons: 'Simpan Log' (Save Log) in blue and 'Hapus Semua Data' (Delete All Data) in red. On the right, there is a section titled 'Riwayat Produksi' (Production History) which contains a table with the following columns: 'Tanggal', 'Operator', 'Shift', 'Jumlah', and 'Aksi'.

Gambar E.1 – Form input produksi

E.2 Langkah 2 – Logika JavaScript (Simpan & Baca)

Buka `script.js` (atau buat file baru dan link-kan). Kita akan mengambil elemen yang kita buat di atas.

```
// 1. Seleksi Elemen
const formProduksi = document.getElementById('formProduksi');
const tabelBody = document.getElementById('tabelBody');
const btnHapusSemua = document.getElementById('btnHapusSemua');

// Kunci untuk LocalStorage
const STORAGE_KEY = 'DATA_PRODUKSI_INDUSTRI';
```



```
// Fungsi Load Data saat halaman dibuka
document.addEventListener('DOMContentLoaded', function () {
    loadDataFromStorage();
});

// 2. Event Listener: Submit Form
formProduksi.addEventListener('submit', function (event) {
    event.preventDefault(); // Mencegah refresh halaman

    // Ambil Value dari Form
    const tanggal = document.getElementById('tanggal').value;
    const operator = document.getElementById('operator').value;
    const shift = document.getElementById('shift').value;
    const jumlah = document.getElementById('jumlah').value;

    // Validasi Sederhana (JavaScript)
    if (jumlah <= 0) {
        alert("Jumlah produksi harus lebih dari 0!");
        return;
    }

    // Buat Object Data
    const dataBaru = {
        id: Date.now(), // ID unik berdasarkan waktu
        tanggal: tanggal,
        operator: operator,
        shift: shift,
        jumlah: parseInt(jumlah)
    };

    // Simpan ke LocalStorage
    saveData(dataBaru);

    // Reset Form
    formProduksi.reset();

    // Refresh Tampilan Tabel
    loadDataFromStorage();
});

// 3. Fungsi Simpan ke LocalStorage
function saveData(data) {
    // Ambil data lama (jika ada), jika tidak ada array kosong
    let dataLama = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
```

```
// Tambah data baru ke array
dataLama.push(data);

// Simpan kembali ke LocalStorage (Convert ke JSON String)
localStorage.setItem(STORAGE_KEY, JSON.stringify(dataLama));
}

// 4. Fungsi Baca & Render Tabel
function loadDataFromStorage() {
  // Ambil data
  let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

  // Kosongkan tabel dulu
  tabelBody.innerHTML = '';

  // Loop data dan buat baris tabel
  data.forEach(function (item) {
    const row = document.createElement('tr');

    row.innerHTML = `
      <td>${item.tanggal}</td>
      <td>${item.operator}</td>
      <td>${item.shift}</td>
      <td>${item.jumlah}</td>
      <td>
        <button class="btn btn-sm btn-danger"
onclick="hapusData(${item.id})">
          Hapus
        </button>
      </td>
    `;

    tabelBody.appendChild(row);
  });
}

// 5. Fungsi Hapus Data Spesifik
// Kita pasang di window agar bisa dipanggil dari inline HTML onclick
window.hapusData = function (id) {
  if (confirm('Yakin ingin menghapus log ini?')) {
    let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

    // Filter data: hapus item yang id-nya cocok
    let dataBaru = data.filter(item => item.id !== id);

    // Simpan ulang
```

```

        localStorage.setItem(STORAGE_KEY, JSON.stringify(dataBaru));

        // Refresh tampilan
        loadDataFromStorage();
    }
};

// 6. Event Hapus Semua
btnHapusSemua.addEventListener('click', function () {
    if (confirm('PERINGATAN: Semua data akan dihapus permanen!')) {
        localStorage.removeItem(STORAGE_KEY);
        loadDataFromStorage();
    }
});

```

Screenshot Hasil:

The screenshot displays a web application interface. On the left, there is a form titled 'Input Data' with the following fields: 'Tanggal' (Date) with a calendar icon, 'Nama Operator' (Operator Name) with a text input, 'Shift' (Shift) with a dropdown menu, and 'Jumlah Produksi (Unit)' (Production Quantity) with a text input. Below the form are two buttons: 'Simpan Log' (Save Log) in blue and 'Hapus Semua Data' (Delete All Data) in red. On the right, there is a table titled 'Riwayat Produksi' (Production History) with the following data:

Tanggal	Operator	Shift	Jumlah	Aksi
2026-03-29	Joko	Pagi	2000	<button>Hapus</button>
2026-03-29	Supri	Pagi	2500	<button>Hapus</button>
2026-03-29	Budi	Pagi	4500	<button>Hapus</button>
2026-03-29	Basuki	Pagi	3200	<button>Hapus</button>

Gambar E.2 – Logika JavaScript (simpan dan baca)

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Filter Data

Pertanyaan / Instruksi Latihan:

Tambahkan fitur filter atau pencarian. Buat input "Cari Nama Operator". Saat user mengetik, tabel hanya menampilkan data operator yang namanya cocok (Gunakan `filter` array).

Jawaban / Kode Solusi:

HTML

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Log Produksi Harian</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body class="container mt-5">
  <div class="row">

    <!-- Kolom Kiri: Form Input -->
    <div class="col-md-4">
      <div class="card">
        <div class="card-header">Input Data</div>
        <div class="card-body">
          <form id="formProduksi">

            <div class="mb-3">
              <label>Tanggal</label>
              <input type="date" id="tanggal" class="form-
control" required>
            </div>

            <div class="mb-3">
              <label>Nama Operator</label>
              <input type="text" id="operator" class="form-
control" placeholder="Nama Lengkap" required>
            </div>

            <div class="mb-3">
              <label>Shift</label>
              <select id="shift" class="form-select" required>
                <option value="Pagi">Pagi</option>
                <option value="Siang">Siang</option>
                <option value="Malam">Malam</option>
              </select>
            </div>

            <div class="mb-3">
              <label>Jumlah Produksi (Unit)</label>
              <input type="number" id="jumlah" class="form-
control" min="1" required>
```

```

        </div>

        <button type="submit" class="btn btn-primary w-100">
            Simpan Log
        </button>

        <button type="button" id="btnHapusSemua" class="btn
btn-danger w-100 mt-2">
            Hapus Semua Data
        </button>

    </form>
</div>
</div>
</div>

<!-- Kolom Kanan: Tabel + Filter -->
<div class="col-md-8">
    <h3>Riwayat Produksi</h3>
    <div class="mb-3">
        <label>Cari Nama Operator</label>
        <input type="text" id="inputCari" class="form-control"
placeholder="Ketik nama operator...">
    </div>
    <table class="table table-bordered table-striped">
        <thead class="table-dark">
            <tr>
                <th>Tanggal</th>
                <th>Operator</th>
                <th>Shift</th>
                <th>Jumlah</th>
                <th>Aksi</th>
            </tr>
        </thead>
        <tbody id="tabelBody">
            <!-- Data akan masuk di sini lewat JS -->
        </tbody>
    </table>
</div>

</div>

<script src="latihan 1.js"></script>
</body>
</html>

```

JavaScript

```
// 1. Seleksi Elemen
const formProduksi = document.getElementById('formProduksi');
const tabelBody = document.getElementById('tabelBody');
const btnHapusSemua = document.getElementById('btnHapusSemua');
const inputCari = document.getElementById('inputCari');

// Kunci untuk LocalStorage
const STORAGE_KEY = 'DATA_PRODUKSI_INDUSTRI';

// Fungsi Load Data saat halaman dibuka
document.addEventListener('DOMContentLoaded', function () {
    loadDataFromStorage();
});

// 2. Event Listener: Submit Form
formProduksi.addEventListener('submit', function (event) {
    event.preventDefault(); // Mencegah refresh halaman

    // Ambil Value dari Form
    const tanggal = document.getElementById('tanggal').value;
    const operator = document.getElementById('operator').value;
    const shift = document.getElementById('shift').value;
    const jumlah = document.getElementById('jumlah').value;

    // Validasi Sederhana (JavaScript)
    if (jumlah <= 0) {
        alert("Jumlah produksi harus lebih dari 0!");
        return;
    }

    // Buat Object Data
    const dataBaru = {
        id: Date.now(), // ID unik berdasarkan waktu
        tanggal: tanggal,
        operator: operator,
        shift: shift,
        jumlah: parseInt(jumlah)
    };

    // Simpan ke LocalStorage
    saveData(dataBaru);

    // Reset Form
    formProduksi.reset();
});
```

```
// Refresh Tampilan Tabel
loadDataFromStorage(inputCari.value);
});

// 3. Event Listener: Filter/Pencarian
inputCari.addEventListener('input', function () {
    loadDataFromStorage(this.value);
});

// 4. Fungsi Simpan ke LocalStorage
function saveData(data) {
    let dataLama = JSON.parse(localStorage.getItem(STORAGE_KEY))
    || [];
    dataLama.push(data);
    localStorage.setItem(STORAGE_KEY, JSON.stringify(dataLama));
}

// 5. Fungsi Baca, Filter & Render Tabel
function loadDataFromStorage(keyword = '') {
    let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) ||
    [];

    // Filter berdasarkan nama operator
    let dataFilter = data.filter(function (item) {
        return
        item.operator.toLowerCase().includes(keyword.toLowerCase());
    });

    tabelBody.innerHTML = '';

    dataFilter.forEach(function (item) {
        const row = document.createElement('tr');

        row.innerHTML = `
            <td>${item.tanggal}</td>
            <td>${item.operator}</td>
            <td>${item.shift}</td>
            <td>${item.jumlah}</td>
            <td>
                <button class="btn btn-sm btn-danger"
onclick="hapusData(${item.id})">
                    Hapus
                </button>
            </td>
        `;
    });
}
```

```
        tabelBody.appendChild(row);
    });
}

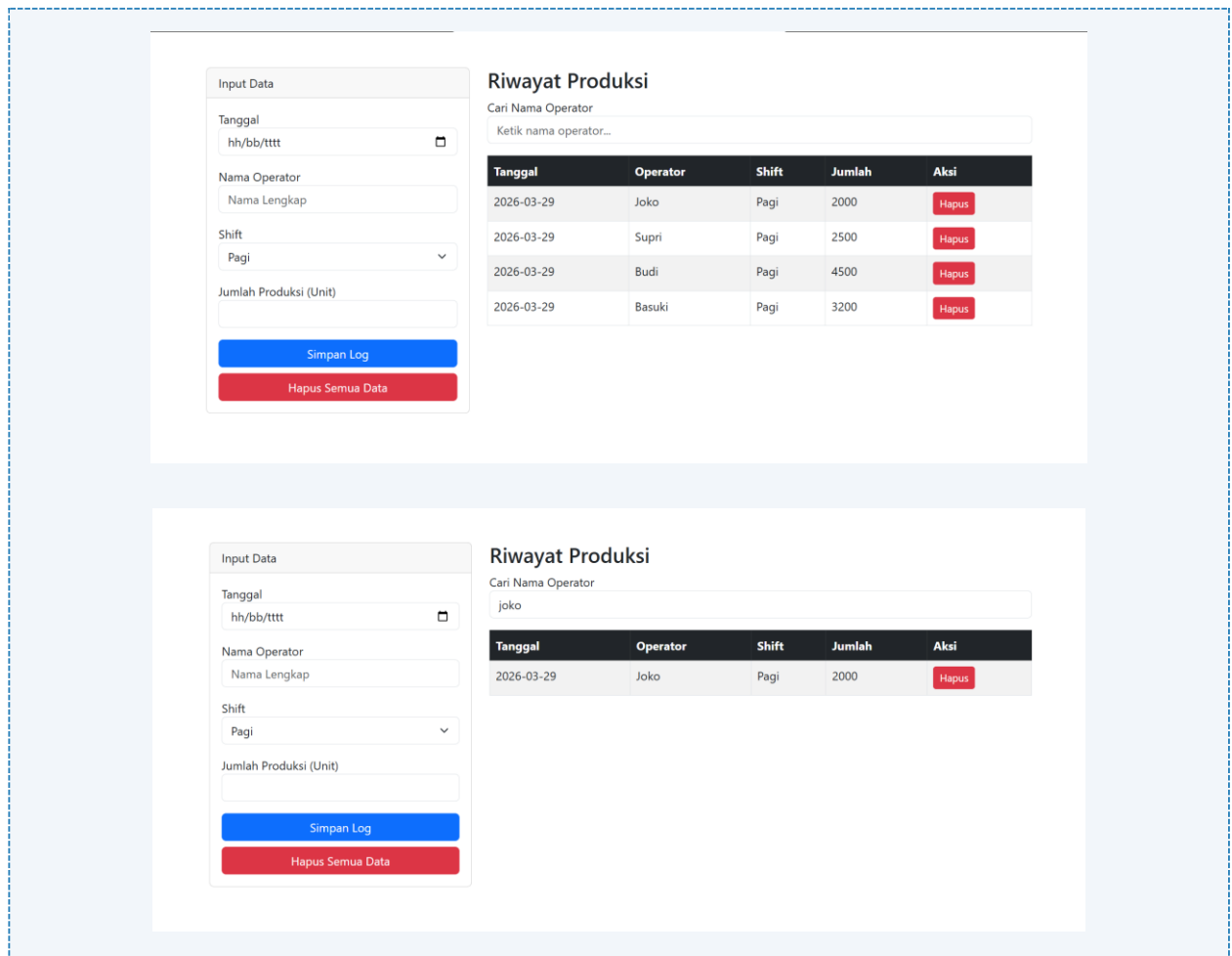
// 6. Fungsi Hapus Data Spesifik
window.hapusData = function (id) {
    if (confirm('Yakin ingin menghapus log ini?')) {
        let data = JSON.parse(localStorage.getItem(STORAGE_KEY))
        || [];

        let dataBaru = data.filter(item => item.id !== id);

        localStorage.setItem(STORAGE_KEY,
        JSON.stringify(dataBaru));

        loadDataFromStorage(inputCari.value);
    }
};

// 7. Event Hapus Semua
btnHapusSemua.addEventListener('click', function () {
    if (confirm('PERINGATAN: Semua data akan dihapus permanen!'))
    {
        localStorage.removeItem(STORAGE_KEY);
        loadDataFromStorage();
    }
});
```

Gambar F.1 – Filter Data

F.2 Latihan 2 – Sortir Data

Pertanyaan / Instruksi Latihan:

Tambahkan tombol "Sortir berdasarkan Jumlah (Terbesar)" yang mengurutkan tabel dari jumlah produksi terbanyak ke terkecil.

Jawaban / Kode Solusi:

HTML

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Latihan 2 – Log Produksi</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
```

```
</head>

<body class="container mt-5">
  <div class="row">

    <!-- Kolom Kiri: Form Input -->
    <div class="col-md-4">
      <div class="card">
        <div class="card-header">Input Data</div>
        <div class="card-body">
          <form id="formProduksi">

            <div class="mb-3">
              <label>Tanggal</label>
              <input type="date" id="tanggal" class="form-
control" required>
            </div>

            <div class="mb-3">
              <label>Nama Operator</label>
              <input type="text" id="operator" class="form-
control" placeholder="Nama Lengkap" required>
            </div>

            <div class="mb-3">
              <label>Shift</label>
              <select id="shift" class="form-select" required>
                <option value="Pagi">Pagi</option>
                <option value="Siang">Siang</option>
                <option value="Malam">Malam</option>
              </select>
            </div>

            <div class="mb-3">
              <label>Jumlah Produksi (Unit)</label>
              <input type="number" id="jumlah" class="form-
control" min="1" required>
            </div>

            <button type="submit" class="btn btn-primary w-100">
              Simpan Log
            </button>

            <button type="button" id="btnHapusSemua" class="btn
btn-danger w-100 mt-2">
              Hapus Semua Data
```

```

        </button>

    </form>
</div>
</div>
</div>

<!-- Kolom Kanan: Filter + Sort + Tabel -->
<div class="col-md-8">
    <h3>Riwayat Produksi</h3>

    <!-- Filter -->
    <div class="mb-3">
        <label>Cari Nama Operator</label>
        <input type="text" id="inputCari" class="form-control"
placeholder="Ketik nama operator...">
    </div>

    <!-- Tombol Sort -->
    <button id="btnSort" class="btn btn-success mb-3">
        Sortir berdasarkan Jumlah (Terbesar)
    </button>

    <table class="table table-bordered table-striped">
        <thead class="table-dark">
            <tr>
                <th>Tanggal</th>
                <th>Operator</th>
                <th>Shift</th>
                <th>Jumlah</th>
                <th>Aksi</th>
            </tr>
        </thead>
        <tbody id="tabelBody">
            <!-- Data akan masuk lewat JS -->
        </tbody>
    </table>
</div>

</div>

<script src="latihan 2.js"></script>
</body>
</html>

```

JavaScript

```
<!DOCTYPE html>
// 1. Seleksi Elemen
const formProduksi = document.getElementById('formProduksi');
const tabelBody = document.getElementById('tabelBody');
const btnHapusSemua = document.getElementById('btnHapusSemua');
const inputCari = document.getElementById('inputCari');
const btnSort = document.getElementById('btnSort');

// Kunci LocalStorage
const STORAGE_KEY = 'DATA_PRODUKSI_INDUSTRI';

// Status sorting
let isSorted = false;

// Load data saat halaman dibuka
document.addEventListener('DOMContentLoaded', function () {
    loadDataFromStorage();
});

// Event submit form
formProduksi.addEventListener('submit', function (event) {
    event.preventDefault();

    const tanggal = document.getElementById('tanggal').value;
    const operator = document.getElementById('operator').value;
    const shift = document.getElementById('shift').value;
    const jumlah = document.getElementById('jumlah').value;

    if (jumlah <= 0) {
        alert("Jumlah produksi harus lebih dari 0!");
        return;
    }

    const dataBaru = {
        id: Date.now(),
        tanggal: tanggal,
        operator: operator,
        shift: shift,
        jumlah: parseInt(jumlah)
    };
});
```

```
        saveData(dataBaru);
        formProduksi.reset();

        loadDataFromStorage(inputCari.value);
    });

    // Event filter
    inputCari.addEventListener('input', function () {
        loadDataFromStorage(this.value);
    });

    // Event sorting
    btnSort.addEventListener('click', function () {
        isSorted = true;
        loadDataFromStorage(inputCari.value);
    });

    // Simpan data
    function saveData(data) {
        let dataLama = JSON.parse(localStorage.getItem(STORAGE_KEY))
        || [];
        dataLama.push(data);
        localStorage.setItem(STORAGE_KEY, JSON.stringify(dataLama));
    }

    // Load + Filter + Sort + Render
    function loadDataFromStorage(keyword = '') {
        let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) ||
        [];

        // Filter
        let dataFilter = data.filter(function (item) {
            return
            item.operator.toLowerCase().includes(keyword.toLowerCase());
        });

        // Sorting
        if (isSorted) {
            dataFilter.sort((a, b) => b.jumlah - a.jumlah);
        }

        tabelBody.innerHTML = '';

        dataFilter.forEach(function (item) {
            const row = document.createElement('tr');
```

```
        row.innerHTML = `
            <td>${item.tanggal}</td>
            <td>${item.operator}</td>
            <td>${item.shift}</td>
            <td>${item.jumlah}</td>
            <td>
                <button class="btn btn-sm btn-danger"
onclick="hapusData(${item.id})">
                    Hapus
                </button>
            </td>
        `;

        tabelBody.appendChild(row);
    });
}

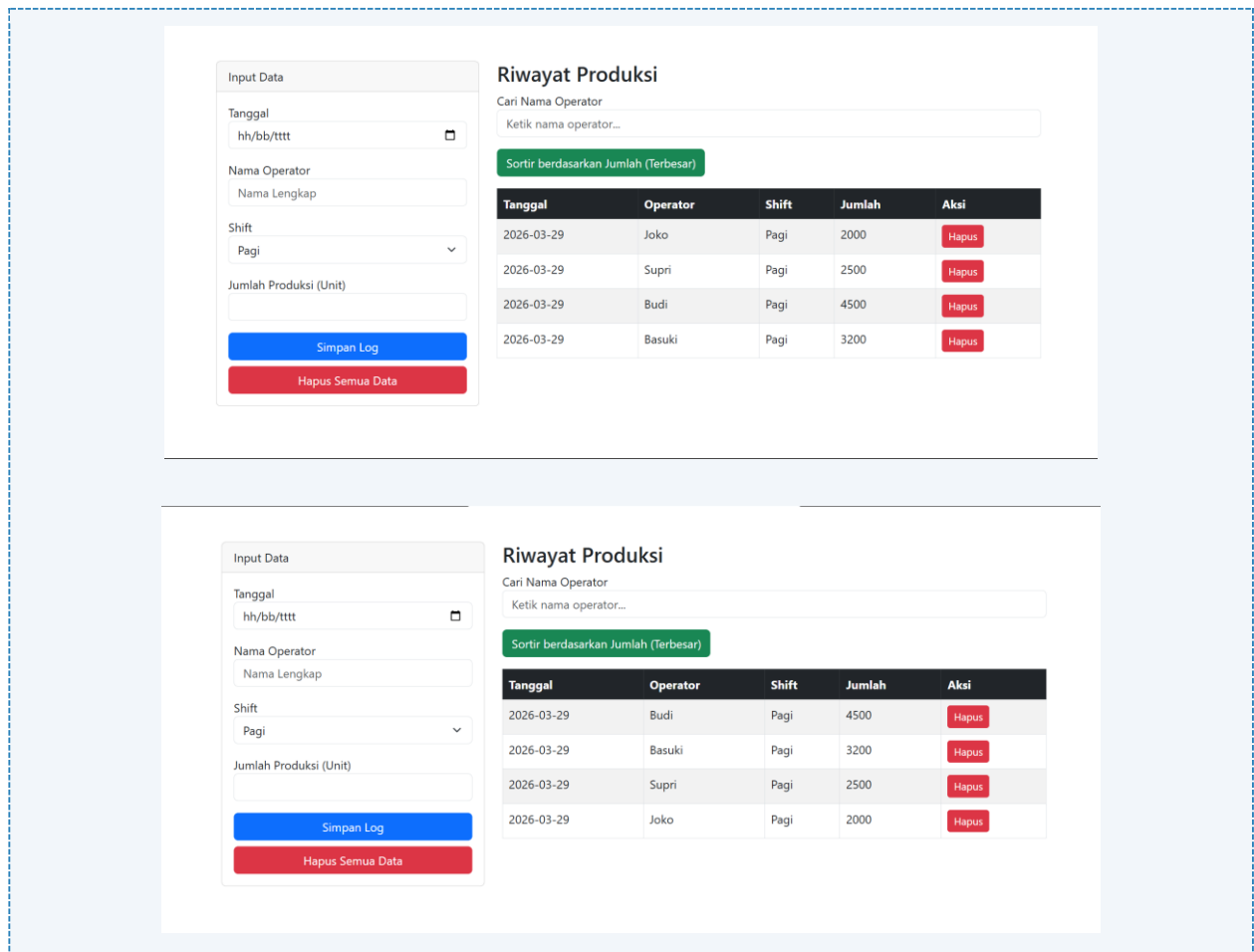
// Hapus data per item
window.hapusData = function (id) {
    if (confirm('Yakin ingin menghapus data ini?')) {
        let data = JSON.parse(localStorage.getItem(STORAGE_KEY))
|| [];

        let dataBaru = data.filter(item => item.id !== id);

        localStorage.setItem(STORAGE_KEY,
JSON.stringify(dataBaru));

        loadDataFromStorage(inputCari.value);
    }
};

// Hapus semua data
btnHapusSemua.addEventListener('click', function () {
    if (confirm('PERINGATAN: Semua data akan dihapus!')) {
        localStorage.removeItem(STORAGE_KEY);
        loadDataFromStorage();
    }
});
```



Gambar F.2 – Sortir Data

F.3 Tugas Proyek Mini – Aplikasi Checklist 5S

Deskripsi proyek mini:

- Buat aplikasi checklist sederhana audit 5S (Seiri, Seiton, Seiso, Seiketsu, Shitsuke).
- Form berisi 5 checkbox (boolean) dan input nama auditor.
 - Saat disimpan, hitung skor (jumlah checklist dicentang / 5 * 100).
 - Simpan riwayat audit ke LocalStorage dan tampilkan di tabel (Tanggal, Auditor, Skor).

Penjelasan pendekatan/solusi yang digunakan:

Pada proyek mini ini, pendekatan yang saya gunakan adalah dengan menyediakan dua input utama, yaitu daya listrik (Watt) dan lama pemakaian (jam), kemudian memanfaatkan JavaScript untuk melakukan perhitungan secara otomatis. Setiap perubahan pada input langsung diproses menggunakan event input tanpa memerlukan tombol submit, sehingga hasil dapat diperoleh secara instan. Secara logika, perhitungan dilakukan dengan mengalikan daya dan waktu pemakaian, kemudian dibagi 1000 untuk memperoleh nilai kWh, yang selanjutnya dikalikan dengan tarif listrik sebesar Rp1.500 guna mendapatkan estimasi biaya. Hasil tersebut ditampilkan secara real-time pada elemen HTML melalui manipulasi DOM,

sehingga pengguna dapat melihat perubahan nilai dengan cepat dan membuat aplikasi terasa lebih interaktif serta mudah digunakan.

Kode Utama:

HTML

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Checklist Audit 5S</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body class="container mt-5">

  <h2 class="mb-4">Checklist Audit 5S</h2>

  <div class="row">

    <!-- Form Input -->
    <div class="col-md-4">
      <div class="card">
        <div class="card-header">Form Audit</div>
        <div class="card-body">

          <form id="formAudit">

            <div class="mb-3">
              <label>Nama Auditor</label>
              <input type="text" id="auditor" class="form-
control" required>
            </div>

            <label class="fw-bold">Checklist 5S</label>

            <div class="form-check">
```



```

        <input class="form-check-input" type="checkbox"
id="seiri">
        <label class="form-check-label">Seiri
(Ringkas)</label>
    </div>

    <div class="form-check">
        <input class="form-check-input" type="checkbox"
id="seiton">
        <label class="form-check-label">Seiton
(Rapi)</label>
    </div>

    <div class="form-check">
        <input class="form-check-input" type="checkbox"
id="seiso">
        <label class="form-check-label">Seiso
(Resik)</label>
    </div>

    <div class="form-check">
        <input class="form-check-input" type="checkbox"
id="seiketsu">
        <label class="form-check-label">Seiketsu
(Rawat)</label>
    </div>

    <div class="form-check mb-3">
        <input class="form-check-input" type="checkbox"
id="shitsuke">
        <label class="form-check-label">Shitsuke
(Rajin)</label>
    </div>

    <button type="submit" class="btn btn-primary w-100">
        Simpan Audit
    </button>

    <button type="button" id="btnHapusSemua" class="btn
btn-danger w-100 mt-2">
        Hapus Semua Data
    </button>

</form>
</div>
</div>
```

```
</div>

<!-- Tabel -->
<div class="col-md-8">
  <h4>Riwayat Audit 5S</h4>

  <table class="table table-bordered table-striped">
    <thead class="table-dark">
      <tr>
        <th>Tanggal</th>
        <th>Auditor</th>
        <th>Skor (%)</th>
      </tr>
    </thead>
    <tbody id="tabelBody">
    </tbody>
  </table>
</div>

</div>

<script src="checklist5s.js"></script>
</body>
</html>
```

JavaScript

```
// Seleksi Elemen
const formAudit = document.getElementById('formAudit');
const tabelBody = document.getElementById('tabelBody');
const btnHapusSemua = document.getElementById('btnHapusSemua');

const STORAGE_KEY = 'DATA_AUDIT_5S';

// Load data saat halaman dibuka
document.addEventListener('DOMContentLoaded', function () {
  loadData();
});

// Event submit
formAudit.addEventListener('submit', function (e) {
  e.preventDefault();

  const auditor = document.getElementById('auditor').value;
```

```
const checklist = [
  document.getElementById('seiri').checked,
  document.getElementById('seiton').checked,
  document.getElementById('seiso').checked,
  document.getElementById('seiketsu').checked,
  document.getElementById('shitsuke').checked
];

// Hitung jumlah checklist yang dicentang
const jumlahCeklis = checklist.filter(item => item).length;

// Hitung skor (%)
const skor = (jumlahCeklis / 5) * 100;

const dataBaru = {
  tanggal: new Date().toLocaleDateString(),
  auditor: auditor,
  skor: skor
};

saveData(dataBaru);

formAudit.reset();

loadData();
});

// Simpan ke LocalStorage
function saveData(data) {
  let dataLama = JSON.parse(localStorage.getItem(STORAGE_KEY))
  || [];
  dataLama.push(data);
  localStorage.setItem(STORAGE_KEY, JSON.stringify(dataLama));
}

// Load & tampilkan data
function loadData() {
  let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) ||
  [];

  tabelBody.innerHTML = '';

  data.forEach(function (item) {
    const row = document.createElement('tr');

    row.innerHTML = `
      <td>${item.tanggal}</td>
    `
  })
}
```

```

        <td>${item.auditor}</td>
        <td>${item.skor}%</td>
    `;

    tabelBody.appendChild(row);
});
}

// Hapus semua data
btnHapusSemua.addEventListener('click', function () {
    if (confirm('Semua data audit akan dihapus!')) {
        localStorage.removeItem(STORAGE_KEY);
        loadData();
    }
});
});

```

Checklist Audit 5S

Form Audit

Nama Auditor

Checklist 5S
☒ Seiri (Ringkas)
☒ Seiton (Rapi)
☒ Seiso (Resik)
☐ Seiketsu (Rawat)
☒ Shitsuke (Rajin)

Simpan Audit

Hapus Semua Data

Riwayat Audit 5S

Tanggal	Auditor	Skor (%)
3/4/2026	dhitya	100%
3/4/2026	farrel	40%

Checklist Audit 5S

Form Audit

Nama Auditor

Checklist 5S
☐ Seiri (Ringkas)
☐ Seiton (Rapi)
☐ Seiso (Resik)
☐ Seiketsu (Rawat)
☐ Shitsuke (Rajin)

Simpan Audit

Hapus Semua Data

Riwayat Audit 5S

Tanggal	Auditor	Skor (%)
3/4/2026	dhitya	100%
3/4/2026	farrel	40%
4/4/2026	Bama	80%

Gambar F.3 – Hasil penerapan audit

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Berdasarkan praktikum yang telah dilakukan, hasil yang diperoleh sudah sesuai dengan konsep DOM manipulation dan event handling yang dipelajari pada modul ini. Hal ini terlihat dari perubahan tampilan yang dapat terjadi secara langsung tanpa perlu reload halaman, seperti perubahan status mesin, warna indikator, dan nilai suhu. Penggunaan fungsi seperti `getElementById()` dan `querySelector()` juga sudah tepat untuk mengakses elemen HTML, sehingga manipulasi seperti mengubah teks (`innerText`) dan style dapat berjalan dengan baik. Selain itu, penerapan event listener menggunakan `addEventListener()` juga sudah sesuai dengan teori, di mana setiap aksi pengguna seperti klik tombol start, stop, dan reset dapat memicu respon sistem secara langsung. Penggunaan `setInterval()` pada simulasi kenaikan suhu juga berhasil menunjukkan konsep proses yang berjalan secara real-time, dan dapat dihentikan dengan `clearInterval()`. Pada bagian input RPM dan proyek mini, penggunaan event input memungkinkan sistem merespon secara otomatis saat pengguna mengetik, tanpa perlu tombol tambahan. Secara keseluruhan, hasil output yang dihasilkan sudah mencerminkan alur event-driven programming, yaitu dari aksi pengguna, kemudian diproses oleh JavaScript, lalu menghasilkan perubahan pada tampilan. Hal ini menunjukkan bahwa konsep yang dipelajari tidak hanya dipahami secara teori, tetapi juga berhasil diterapkan dalam bentuk aplikasi sederhana berbasis web.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Status mesin tidak berubah saat tombol START/STOP diklik	Memastikan <code>addEventListener()</code> sudah terpasang dengan benar dan ID elemen sesuai dengan HTML
Tampilan (warna/status) tidak berubah	Mengecek manipulasi DOM seperti <code>textContent</code> dan <code>style</code> sudah mengarah ke elemen yang benar

G.3 Kaitan dengan Penerapan di Industri

Menurut saya, materi di dalam modul ini cukup sesuai dengan penerapannya di dunia kerja, terutama dalam sistem monitoring dan kontrol yang berbasis digital. Misalnya seperti layar pemantau mesin di pabrik yang menampilkan kondisi kerja secara langsung, termasuk tanda warna dan data seperti suhu atau kondisi mesin. Dengan konsep DOM, tampilan bisa diperbarui sendiri secara otomatis, sedangkan pengelolaan event digunakan untuk merespons tindakan pengguna seperti menekan tombol start atau stop. Selain itu, konsep ini juga bisa digunakan dalam menghitung penggunaan energi atau biaya operasional secara langsung, sehingga membantu perusahaan mengelola penggunaan sumber daya dengan lebih efisien dan mengurangi risiko kesalahan.

H. KESIMPULAN

1	Pada praktikum ini, saya memahami bagaimana DOM digunakan untuk mengakses dan mengubah elemen pada halaman web secara langsung.
2	Saya dapat menerapkan event handling seperti klik dan input sehingga halaman web bisa merespon aksi pengguna secara real-time tanpa perlu reload.
3	Simulasi panel kontrol mesin berhasil dibuat dengan beberapa kondisi seperti running, stop, dan overheat yang ditampilkan secara dinamis.
4	Saya juga mampu membuat validasi input dan perhitungan sederhana yang berjalan otomatis sesuai input pengguna.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Mozilla Developer Network. (2023). JavaScript Guide. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide
[3]	W3Schools. (2023). JavaScript Tutorial. https://www.w3schools.com/js/
[4]	Duckett, J. (2014). <i>JavaScript and JQuery: Interactive front-end web development</i> . John Wiley & Sons.
[5]	Kadir, A. (2018). <i>Dasar Pemrograman Web Dinamis Menggunakan JavaScript</i> . Yogyakarta: Andi Publisher.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
Radhitya Adha Fadhlur Rohman NIM: 23051430005	Maessa Andrea Vallenia NIM: 23051430044	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

Modul 8 Asynchronous JavaScript (Fetch API, JSON, REST API Sederhana)

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 8 (Pertemuan 8)
Topik Utama	Asynchronous JavaScript (Fetch API, JSON, REST API Sederhana)
Sub-Topik	<i>Konsep Asynchronous, Fetch API, Promise, Async/Await, dan integrasi REST API eksternal.</i>
Durasi Praktikum	340 Menit
Tanggal Praktikum	17 April 2026
Nama Mahasiswa	Radhitya Adha Fadhlur Rohman
NIM	23051430005
Kelas	A1

B. TUJUAN PEMBELAJARAN

1	Memahami perbedaan kode sinkronus dan asinkronus.
2	Mampu menggunakan `fetch()` untuk mengambil data dari server eksternal (API).
3	Memahami format data JSON (JavaScript Object Notation).
4	Menerapkan penanganan error saat request API gagal.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	File `index.html`	Versi Terbaru	Hasil pekerjaan pertemuan sebelumnya

4	Ekstensi VS Code: LiveServer	Versi terbaru	Melihat perubahan web secara real-time.
5	Javascript	Versi terbaru	Membuat halaman web menjadi lebih interaktif dan fungsional

D. DASAR TEORI

Perkembangan teknologi web sekarang ini menuntut aplikasi yang tidak hanya bersifat statis, tetapi juga mampu berinteraksi secara dinamis dengan berbagai sumber data. Salah satu konsep penting adalah *asynchronous JavaScript*, yaitu mekanisme yang memungkinkan program tetap berjalan tanpa harus menunggu proses lain selesai terlebih dahulu. Dengan cara ini, aplikasi web tetap bisa merespons pengguna dengan cepat meskipun sedang melakukan proses pengambilan data dari server, sehingga pengalaman pengguna terasa lebih nyaman.

Dalam penerapannya, komunikasi antara aplikasi web dan server umumnya menggunakan konsep *REST API (Representational State Transfer Application Programming Interface)*. *REST API* memungkinkan pertukaran data antar sistem melalui protokol *HTTP* dengan metode seperti *GET*, *POST*, *PUT*, dan *DELETE*. Data yang digunakan biasanya dalam format *JSON (JavaScript Object Notation)* karena lebih ringan, mudah dibaca, dan praktis untuk diolah menggunakan JavaScript. Untuk mengambil data dari API, biasanya digunakan *Fetch API*, yaitu fitur modern JavaScript yang memudahkan pengembang dalam melakukan *request* ke server secara *asynchronous* dengan penulisan kode yang lebih sederhana dan fleksibel.

Selain itu, dalam mengelola proses *asynchronous* terdapat dua pendekatan utama, yaitu *Promise* dan *async/await*. *Promise* digunakan untuk menangani hasil dari proses *asynchronous* melalui metode seperti *.then()*, *.catch()*, dan *.finally()*. Sementara itu, *async/await* merupakan bentuk yang lebih sederhana dari *Promise* karena membuat kode terlihat seperti berjalan secara berurutan, sehingga lebih mudah dipahami. Penggunaan kedua pendekatan ini penting agar alur program tetap rapi dan terstruktur, serta mampu menangani error dengan baik jika terjadi masalah saat proses komunikasi dengan server.

D.1 Konsep Utama

Konsep utama dalam modul ini adalah pemanfaatan *asynchronous JavaScript* untuk mengintegrasikan aplikasi web dengan sumber data eksternal melalui API. Dengan menggunakan *Fetch API*, aplikasi dapat mengambil data dari server tanpa harus memuat ulang halaman. Data yang diterima dalam format JSON kemudian diproses dan ditampilkan ke dalam tampilan web menggunakan manipulasi DOM.

Selain itu, modul ini juga menekankan pentingnya pengelolaan alur data menggunakan *Promise* dan *async/await*. Kedua pendekatan ini memungkinkan untuk mengatur proses pengambilan data secara lebih terstruktur, termasuk dalam hal penanganan error. Dengan demikian, aplikasi tidak hanya mampu menampilkan data, tetapi juga tetap stabil ketika terjadi gangguan seperti koneksi internet terputus atau server tidak merespon.

D.2 Keterkaitan dengan Teknik Industri

Konsep *asynchronous JavaScript* dan integrasi API sangat relevan dalam bidang Teknik Industri, terutama dalam pengembangan sistem informasi yang mendukung proses operasional. Dalam dunia industri, data biasanya tidak tersimpan di satu tempat saja, melainkan tersebar di berbagai sistem seperti database pusat, sensor *IoT*, atau sistem produksi. Dengan adanya *REST API*, data dari berbagai sumber tersebut dapat diakses dan ditampilkan secara *real-time* pada sebuah *dashboard* tanpa harus mengganggu kinerja sistem, sehingga pengguna tetap dapat berinteraksi dengan aplikasi secara lancar.

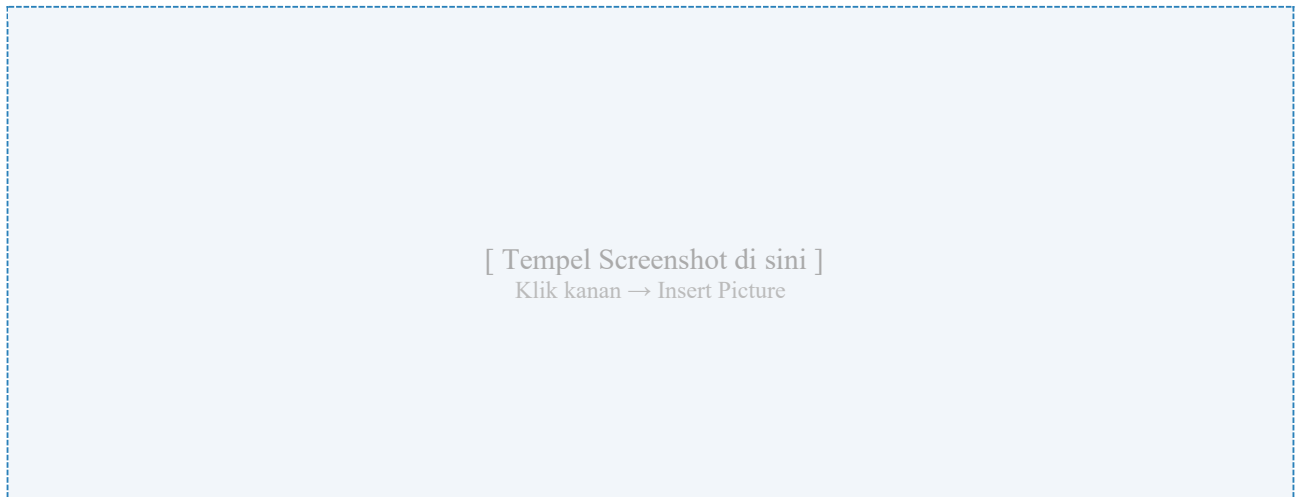
Sebagai contoh, pada sistem monitoring produksi, data seperti jumlah *output*, *downtime* mesin, atau laporan insiden dapat diambil langsung dari server dan ditampilkan kepada operator di lapangan. Dengan pendekatan *asynchronous*, proses pengambilan data tersebut tidak membuat sistem menjadi lambat. Hal ini membantu proses pengambilan keputusan menjadi lebih cepat dan akurat, yang merupakan salah satu tujuan utama dalam Teknik Industri, yaitu meningkatkan efisiensi dan efektivitas sistem kerja.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Mengetahui JSONPlaceholder

```
https://jsonplaceholder.typicode.com/users
```

Screenshot Hasil:



Gambar E.1 – Kode dari Langkah 1

E.2 Langkah 2 – Membuat HTML untuk Menampilkan Data

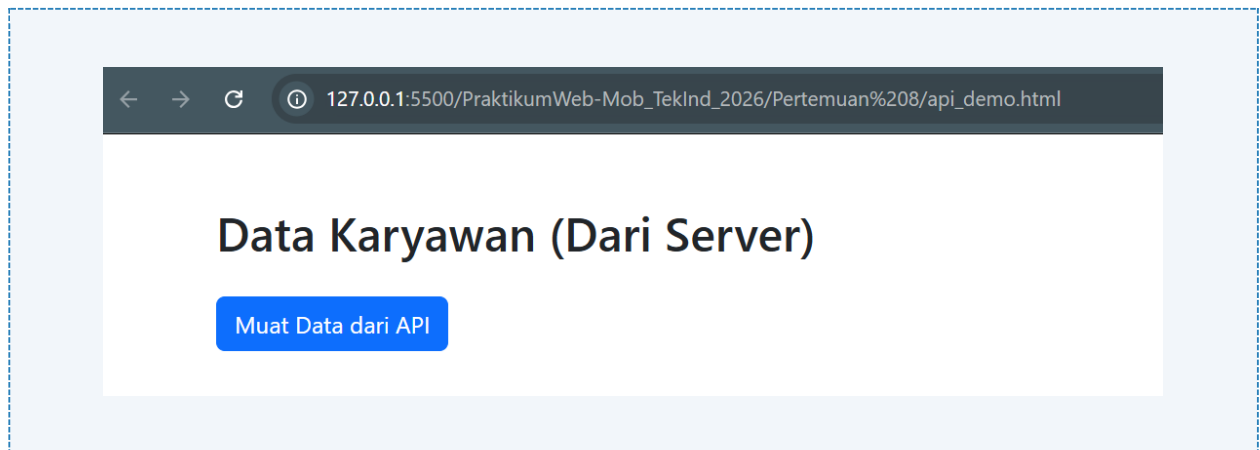
```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Integrasi API Data Karyawan</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="container mt-5">
  <h2 class="mb-4">Data Karyawan (Dari Server)</h2>

  <div class="mb-3">
    <button id="btnLoad" class="btn btn-primary">Muat Data
dari API</button>
    <div id="loading" class="spinner-border text-primary
d-none" role="status"></div>
  </div>

  <div class="row" id="containerKaryawan">
    </div>

  <script src="api_script.js"></script>
</body>
</html>
```

Screenshot Hasil:



Gambar E.2 – Output dari Langkah 2

E.3 Langkah 3 – Fetch Data dengan .then() (Promise)

```
const btnLoad = document.getElementById('btnLoad');
const container =
document.getElementById('containerKaryawan');
const loading = document.getElementById('loading');

// Endpoint API (Simulasi Database)
const API_URL = 'https://jsonplaceholder.typicode.com/users';

btnLoad.addEventListener('click', function () {
  // Tampilkan loading
  loading.classList.remove('d-none');
  container.innerHTML = ''; // Bersihkan konten lama

  // Fetch Data
  fetch(API_URL)
    .then(function (response) {
      // Cek jika response sukses (kode 200-299)
      if (!response.ok) {
        throw new Error('Gagal mengambil data');
      }

      // Parsing data JSON
      return response.json();
    })
    .then(function (dataKaryawan) {
      // Data berhasil didapat
      console.log(dataKaryawan);
      renderData(dataKaryawan);
    })
    .catch(function (error) {
      // Jika ada error
    })
  });
```

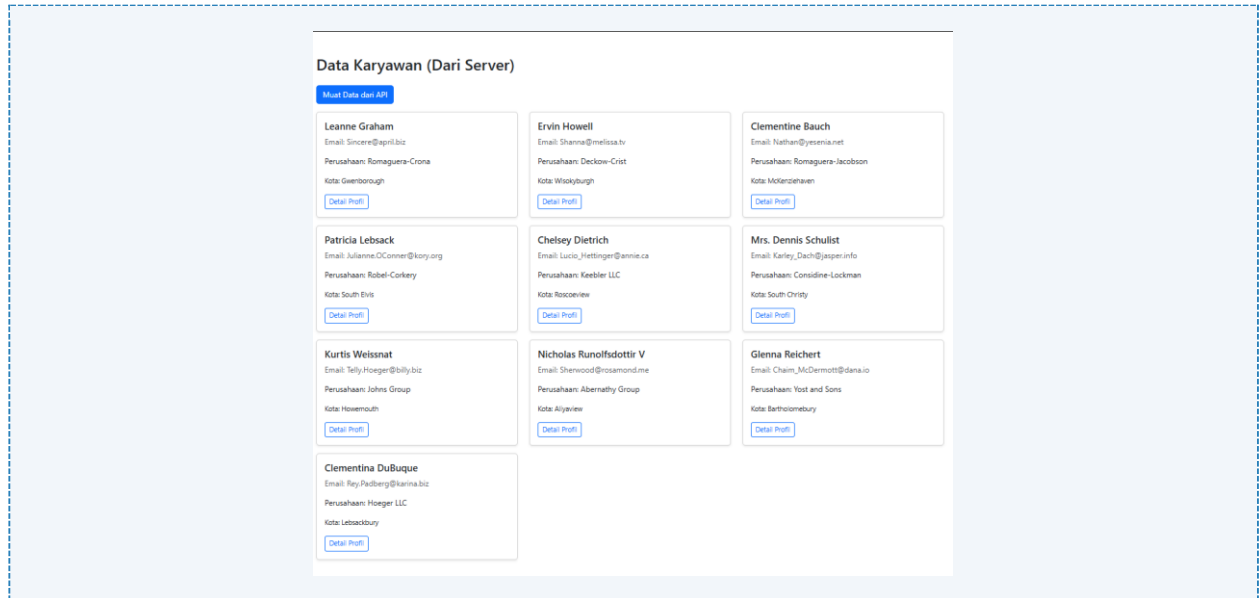
```
        container.innerHTML = `
            <div class="alert alert-danger">
                Error: ${error.message}
            </div>
        `;
    })
    .finally(function () {
        // Sembunyikan loading
        loading.classList.add('d-none');
    });
});

// Fungsi untuk render semua data
function renderData(data) {
    data.forEach(function (karyawan) {
        const col = document.createElement('div');
        col.className = 'col-md-4 mb-3';

        col.innerHTML = `
            <div class="card h-100 shadow-sm">
                <div class="card-body">
                    <h5 class="card-
title">${karyawan.name}</h5>
                    <p class="card-text text-muted">
                        Email: ${karyawan.email}
                    </p>
                    <p class="card-text">
                        Perusahaan: ${karyawan.company.name}
                    </p>
                    <p class="card-text">
                        <small>Kota:
${karyawan.address.city}</small>
                    </p>
                    <a href="" class="btn btn-sm btn-outline-
primary">
                        Detail Profil
                    </a>
                </div>
            </div>
        `;

        container.appendChild(col);
    });
}
```

Screenshot Hasil:



Gambar E.3 – Output dari Langkah 3

E.4 Langkah 4 – Fetch dengan Async/Await (Modern Style)

```
// Fungsi Async untuk mencari data spesifik
async function cariKaryawan(id) {
  try {
    console.log(`Mencari data ID: ${id}...`);

    const response = await fetch(`${API_URL}/${id}`);

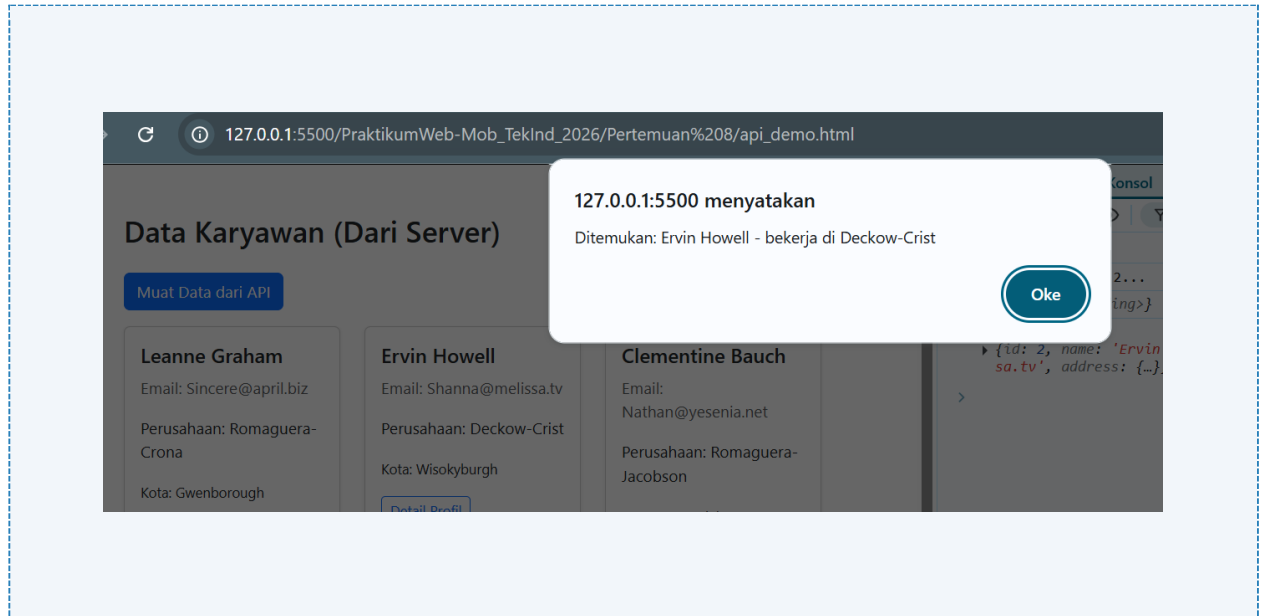
    if (!response.ok) {
      throw new Error('Data tidak ditemukan');
    }

    const data = await response.json();
    console.log("Ditemukan:", data);

    alert(`Ditemukan: ${data.name} - bekerja di ${data.company.name}`);
  } catch (error) {
    console.error(error);
    alert(error.message);
  }
}

// Contoh pemanggilan:
// cariKaryawan(2);
```

Screenshot Hasil:



Gambar E.4 – Output dari Langkah 4

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Posting Data

Pertanyaan / Instruksi Latihan:

Buat tombol "Tambah Karyawan Baru". Gunakan method `fetch` dengan opsi `{ method: 'POST', body: ... }` untuk mengirim data dummy ke API. Cek response di console.

Jawaban / Kode Solusi:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Integrasi API Data Karyawan</title>
  <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.0/dist/css/bo
otstrap.min.css" rel="stylesheet">
  <link rel="stylesheet" href="stylelatihan.css">
</head>

<body class="container mt-5">

  <h2 class="mb-4">Data Karyawan (Dari Server)</h2>
```



```

<div class="mb-3">
  <button id="btnLoad" class="btn btn-primary">Muat Data
dari API</button>
  <button class="btn btn-success" data-bs-toggle="modal"
data-bs-target="#modalTambah">
    Tambah Karyawan Baru
  </button>

  <div id="loading" class="spinner-border text-primary
d-none" role="status"></div>
</div>

<div class="row" id="containerKaryawan"></div>

<div class="modal fade" id="modalTambah" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content">

      <div class="modal-header">
        <h5 class="modal-title">Tambah Karyawan</h5>
        <button type="button" class="btn-close"
data-bs-dismiss="modal"></button>
      </div>

      <div class="modal-body">
        <input type="text" id="inputNama"
class="form-control mb-2" placeholder="Nama">
        <input type="email" id="inputEmail"
class="form-control mb-2" placeholder="Email">
        <input type="text" id="inputPerusahaan"
class="form-control" placeholder="Perusahaan">
        <input type="text" id="inputKota"
class="form-control" placeholder="Kota">
      </div>

      <div class="modal-footer">
        <button class="btn btn-secondary" data-bs-
dismiss="modal">Batal</button>
        <button id="btnSubmitTambah" class="btn
btn-primary">Simpan</button>
      </div>

    </div>
  </div>
</div>

```

```
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/boot
strap.bundle.min.js"></script>

<script src="latihan_api_1.js"></script>
</body>
</html>
```

Kode Java Script

```
document.addEventListener("DOMContentLoaded", function () {

    console.log("JS TERBACA");

    const btnLoad = document.getElementById('btnLoad');
    const btnSubmitTambah =
document.getElementById('btnSubmitTambah');
    const container = document.getElementById('containerKaryawan');
    const loading = document.getElementById('loading');

    const API_URL = 'https://jsonplaceholder.typicode.com/users';

    btnLoad.addEventListener('click', function () {

        console.log("TOMBOL LOAD DIKLIK");

        loading.classList.remove('d-none');
        container.innerHTML = '';

        fetch(API_URL)
            .then(response => {
                if (!response.ok) {
                    throw new Error('Gagal mengambil data');
                }
                return response.json();
            })
            .then(data => {
                console.log("DATA GET:", data);
                renderData(data);
            })
            .catch(error => {
                console.error("ERROR GET:", error);
                container.innerHTML = `<div class="alert alert-
danger">${error.message}</div>`;
            })
            .finally(() => {
```

```
        loading.classList.add('d-none');
    });
});
btnSubmitTambah.addEventListener('click', async function () {

    console.log("TOMBOL SIMPAN DIKLIK");

    const nama = document.getElementById('inputNama').value;
    const email = document.getElementById('inputEmail').value;
    const perusahaan =
document.getElementById('inputPerusahaan').value;
    const kota = document.getElementById('inputKota').value;

    if (!nama || !email) {
        alert('Nama dan Email wajib diisi!');
        return;
    }

    try {
        const response = await fetch(API_URL, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify({
                name: nama,
                email: email,
                company: {
                    name: perusahaan || 'Tidak diisi'
                },
                address: {
                    city: kota || 'Tidak diisi'
                }
            })
        });

        if (!response.ok) {
            throw new Error('Gagal POST data');
        }

        const result = await response.json();

        console.log("=== HASIL POST ===");
        console.log("Data berhasil ditambahkan:", result);
        console.table(result);

        renderData([result]);
    }
});
```

```
        alert(`Berhasil! Data "${result.name}" dikirim`);

        // Tutup modal
        const modal =
bootstrap.Modal.getInstance(document.getElementById('modalTambah'))
        if (modal) modal.hide();

        // Reset input
        document.getElementById('inputNama').value = '';
        document.getElementById('inputEmail').value = '';
        document.getElementById('inputPerusahaan').value = '';
        document.getElementById('inputKota').value = '';

    } catch (error) {
        console.error("ERROR POST:", error);
        alert(error.message);
    }
});

function renderData(data) {

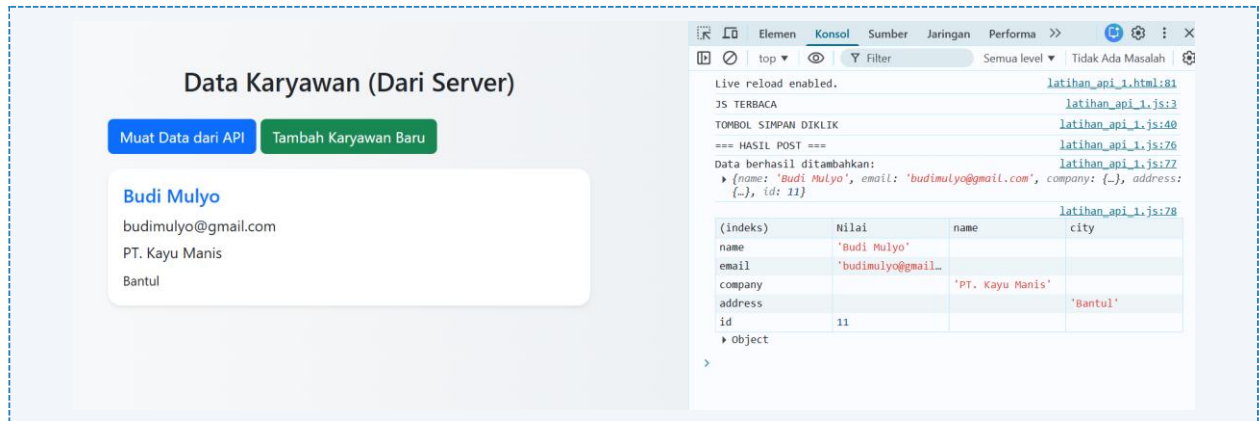
    data.forEach(karyawan => {

        const col = document.createElement('div');
        col.className = 'col-md-4 mb-3';

        col.innerHTML = `
            <div class="card h-100 shadow-sm">
                <div class="card-body">
                    <h5>${karyawan.name || '-'}</h5>
                    <p>${karyawan.email || '-'}</p>
                    <p>${karyawan.company?.name || '-'}</p>
                    <small>${karyawan.address?.city || '-'}
'</small>
                </div>
            </div>
        `;

        container.appendChild(col);
    });
}

});
```



Gambar F.1 – Output dari Latihan 1

F.2 Latihan 2 – Filter Array

Pertanyaan / Instruksi Latihan:

Sebelum data dirender ke layar dengan `forEach`, lakukan filter terlebih dahulu. Tampilkan HANYA karyawan yang tinggal di kota yang mengandung huruf "s" (misal: "Southside", "London"). Petunjuk: Gunakan `data.filter(...)` sebelum `data.forEach(...)`.

Jawaban / Kode Solusi:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Data Karyawan (Dari Server)</title>
  <meta name="viewport" content="width=device-width,
initial-scale=1">

  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bo
otstrap.min.css" rel="stylesheet">
  <link rel="stylesheet" href="stylelatihan.css">
</head>

<body>

<div class="container mt-5">

  <h2 class="mb-4">Data Karyawan (Dari Server)</h2>
  <div class="d-flex flex-wrap gap-2 align-items-center mb-
4">
    <button id="btnLoad" class="btn btn-primary">Muat Data
dari API</button>
```

```

        <button id="btnFilter" class="btn btn-warning">Filter
Kota "s"</button>
        <button class="btn btn-success ms-auto" data-bs-
toggle="modal" data-bs-target="#modalTambah">
            Tambah Karyawan Baru
        </button>
        <div id="loading" class="spinner-border text-primary
d-none"></div>
    </div>
    <div class="row" id="containerKaryawan"></div>
</div>
<!-- MODAL -->
<div class="modal fade" id="modalTambah">
    <div class="modal-dialog modal-dialog-centered">
        <div class="modal-content">

            <div class="modal-header">
                <h5 class="modal-title">Tambah Karyawan</h5>
                <button class="btn-close" data-bs-
dismiss="modal"></button>
            </div>

            <div class="modal-body">
                <input type="text" id="inputNama" class="form-
control mb-2" placeholder="Nama">
                <input type="email" id="inputEmail"
class="form-control mb-2" placeholder="Email">
                <input type="text" id="inputPerusahaan"
class="form-control mb-2" placeholder="Perusahaan">
                <input type="text" id="inputKota" class="form-
control" placeholder="Kota">
            </div>

            <div class="modal-footer">
                <button class="btn btn-secondary" data-bs-
dismiss="modal">Batal</button>
                <button id="btnSubmitTambah" class="btn btn-
primary">Simpan</button>
            </div>

        </div>
    </div>
</div>

```

```
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/boot
strap.bundle.min.js"></script>
<script src="latihan_api_2.js"></script>

</body>
</html>
```

Kode Java Script

```
document.addEventListener("DOMContentLoaded", function () {

    console.log("JS TERBACA");

    const btnLoad = document.getElementById('btnLoad');
    const btnFilter = document.getElementById('btnFilter');
    const btnSubmitTambah =
document.getElementById('btnSubmitTambah');
    const container = document.getElementById('containerKaryawan');
    const loading = document.getElementById('loading');

    const API_URL = 'https://jsonplaceholder.typicode.com/users';

    let dataLocal = [];
    btnLoad.addEventListener('click', function () {

        console.log("TOMBOL LOAD DIKLIK");

        loading.classList.remove('d-none');
        container.innerHTML = '';

        fetch(API_URL)
            .then(response => {
                if (!response.ok) {
                    throw new Error('Gagal mengambil data');
                }
                return response.json();
            })
            .then(data => {

                console.log("DATA GET:", data);
                console.table(data);

                dataLocal = data;

                renderData(dataLocal); // tampil semua data
```

```
    })
    .catch(error => {
        console.error("ERROR GET:", error);
        container.innerHTML = `

Radhitya Adha Fadhlur Rohman – 23051430005 | Modul Praktikum



Halaman 164 dari 349


```



```
    } catch (error) {
        console.error("ERROR FILTER:", error);
    } finally {
        loading.classList.add('d-none');
    }
});

// =====
// POST DATA
// =====
btnSubmitTambah.addEventListener('click', async function () {

    console.log("TOMBOL SIMPAN DIKLIK");

    const nama = document.getElementById('inputNama').value;
    const email = document.getElementById('inputEmail').value;
    const perusahaan =
document.getElementById('inputPerusahaan').value;
    const kota = document.getElementById('inputKota').value;

    if (!nama || !email) {
        alert('Nama dan Email wajib diisi!');
        return;
    }

    try {
        const response = await fetch(API_URL, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify({
                name: nama,
                email: email,
                company: {
                    name: perusahaan || 'Tidak diisi'
                },
                address: {
                    city: kota || 'Tidak diisi'
                }
            })
        });

        if (!response.ok) {
            throw new Error('Gagal POST data');
        }
    }
});
```

```
const result = await response.json();

console.log("=== HASIL POST ===");
console.log("Data berhasil ditambahkan:", result);
console.table(result); //

// Tambah ke data lokal
dataLocal.push(result);

renderData(dataLocal);

alert(`Berhasil! Data "${result.name}" dikirim`);

// Tutup modal
const modal =
bootstrap.Modal.getInstance(document.getElementById('modalTambah'))
if (modal) modal.hide();

// Reset input
document.getElementById('inputNama').value = '';
document.getElementById('inputEmail').value = '';
document.getElementById('inputPerusahaan').value = '';
document.getElementById('inputKota').value = '';

} catch (error) {
  console.error("ERROR POST:", error);
  alert(error.message);
}
});
function renderData(data) {

  data.forEach(karyawan => {

    const col = document.createElement('div');
    col.className = 'col-md-4 mb-3';

    col.innerHTML = `
      <div class="card h-100 shadow-sm">
        <div class="card-body">
          <h5>${karyawan.name || '-'}</h5>
          <p>${karyawan.email || '-'}</p>
          <p>${karyawan.company?.name || '-'}</p>
          <small>${karyawan.address?.city || '-'}
    `;

    col.innerHTML += `
      </div>
    `;

    `</small>
      </div>
    `;

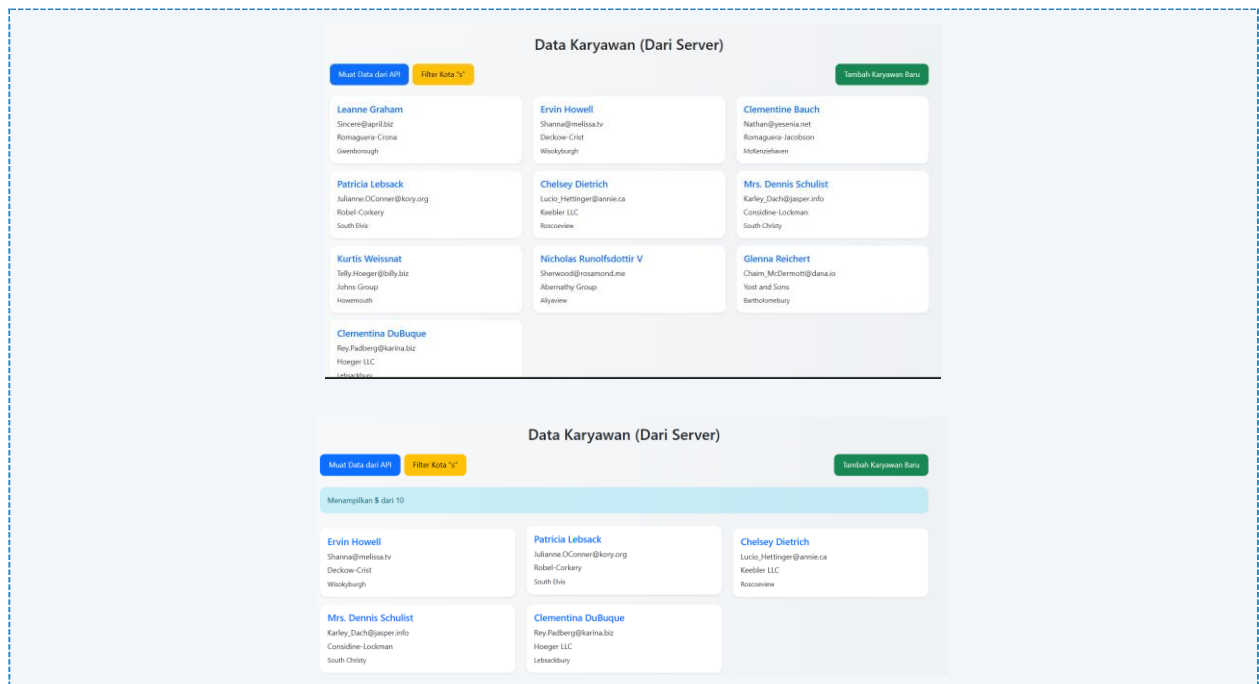
    `;
  });
}
```

```

        container.appendChild(col);
    });
}

});

```



Gambar F.2 – Output dari Latihan 2

F.3 Tugas Proyek Mini –

Deskripsi proyek mini:

- Tampilkan daftar 10 laporan insiden terbaru (Title = Judul Insiden, Body = Deskripsi).
- Di setiap card, tambahkan tombol "Tindak Lanjut". Saat diklik, `alert()` menampilkan "Tiket ID sedang diproses oleh Tim Maintenance".

Penjelasan pendekatan/solusi yang digunakan:

Pendekatan yang digunakan dalam proyek mini ini adalah memanfaatkan konsep *asynchronous JavaScript* dengan metode *async/await* untuk mengambil data dari API jsonplaceholder. Data yang diperoleh dianggap sebagai laporan insiden, kemudian diproses dengan mengambil 10 data terakhir menggunakan *method slice()* dan ditampilkan secara terbalik (*reverse*) agar data terbaru muncul di bagian atas. Hal ini bertujuan agar informasi yang ditampilkan lebih relevan seperti pada sistem monitoring di dunia industri. Selanjutnya, data ditampilkan ke halaman web menggunakan manipulasi DOM dalam bentuk *card* yang berisi judul dan deskripsi insiden. Setiap *card* dilengkapi tombol “Tindak Lanjut” yang ketika diklik akan menampilkan notifikasi dan mengubah status menjadi sedang diproses. Selain itu, ditambahkan juga pengelolaan *state* seperti *loading* dan *error* untuk meningkatkan

interaksi pengguna, sehingga sistem tidak hanya menampilkan data tetapi juga memberikan pengalaman yang lebih realistis dan terstruktur.

Kode Utama:

```
=== KODE DI HTML ===
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Sistem Andon & Insiden Produksi</title>

  <!-- Font -->
  <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;
500;600;700;800&display=swap" rel="stylesheet">

  <!-- Bootstrap -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bo
otstrap.min.css" rel="stylesheet">

  <!-- Icon -->
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap-
icons@1.11.1/font/bootstrap-icons.css">

  <!-- CSS -->
  <link rel="stylesheet" href="style_insiden.css">
</head>

<body>
```

```

<!-- NAVBAR -->
<nav class="navbar ind-navbar sticky-top">
  <div class="container-fluid px-4 text-white">
    <span class="navbar-brand fw-bold text-white fs-5">
      <i class="bi bi-gear-wide-connected text-warning
me-2"></i>
      SISTEM MONITORING PRODUKSI
    </span>

    <div class="d-flex align-items-center">
      <span class="badge bg-dark border border-secondary
me-3 px-3 py-2">
        Shift: <span class="text-warning">2 (14:00 -
22:00)</span>
      </span>

      <span class="badge ind-status-online px-3 py-2">
        <i class="bi bi-broadcast"></i> Sensor: Aktif
      </span>
    </div>
  </div>
</nav>

<div class="container-fluid px-5 my-4">

  <!-- HEADER -->
  <div class="d-flex justify-content-between align-items-end
mb-4 border-bottom pb-3">
    <div>
      <h2 class="fw-bold m-0 header-title">Daftar
Insiden Lini Produksi</h2>
      <p class="text-muted small m-0 mt-1">
        Menampilkan 10 data terbaru dari sistem pusat.
      </p>
    </div>

    <button class="btn btn-outline-secondary btn-sm fw-
bold shadow-sm" onclick="muatDataLaporan()">
      <i class="bi bi-arrow-repeat"></i> Sinkronisasi
    </button>
  </div>

  <!-- KPI -->
  <div id="statsBar" class="row g-3 mb-4 d-none">
    <div class="col-md-4">
      <div class="kpi-card border-left-blue">

```

```

        <div>
            <h6 class="kpi-title">Total Insiden</h6>
            <h3 class="kpi-value"
id="statTotal">0</h3>
        </div>
        <i class="bi bi-clipboard-data kpi-icon text-
primary"></i>
    </div>
</div>

<div class="col-md-4">
    <div class="kpi-card border-left-red">
        <div>
            <h6 class="kpi-title">Belum Ditangani</h6>
            <h3 class="kpi-value text-danger"
id="statPending">0</h3>
        </div>
        <i class="bi bi-exclamation-octagon kpi-icon
text-danger"></i>
    </div>
</div>

<div class="col-md-4">
    <div class="kpi-card border-left-green">
        <div>
            <h6 class="kpi-title">Sedang Diproses</h6>
            <h3 class="kpi-value text-success"
id="statProses">0</h3>
        </div>
        <i class="bi bi-tools kpi-icon text-
success"></i>
    </div>
</div>
</div>

<!-- LOADING -->
<div id="loadingState" class="d-none d-flex justify-
content-center align-items-center py-5">
    <div class="spinner-border text-primary me-3"></div>
    <span class="fw-bold text-secondary">Memuat
data...</span>
</div>

<!-- ERROR -->
<div id="errorState" class="alert alert-danger d-none fw-
bold">

```

```

        <i class="bi bi-cone-striped me-2"></i>
        <span id="errorMsg"></span>
    </div>

    <!-- EMPTY -->
    <div id="emptyState" class="alert alert-success d-none fw-
bold">
        <i class="bi bi-check-circle-fill me-2"></i>
        Tidak ada insiden.
    </div>

    <!-- DATA -->
    <div class="row g-3 pb-5" id="containerLaporan"></div>

</div>

<script src="insiden.js"></script>

</body>
</html>

```

Kode JS

```

const container    =
document.getElementById('containerLaporan');
const loadingState = document.getElementById('loadingState');
const errorState   = document.getElementById('errorState');
const errorMsg     = document.getElementById('errorMsg');
const emptyState   = document.getElementById('emptyState');
const statsBar     = document.getElementById('statsBar');
const statTotal    = document.getElementById('statTotal');
const statPending  = document.getElementById('statPending');
const statProses   = document.getElementById('statProses');

const API_URL = `https://jsonplaceholder.typicode.com/posts` ;
const JUMLAH_LAPORAN = 10;
const ticketsProses = new Set();

function tampilkanState(state) {
    // reset semua state
    loadingState.classList.add('d-none');
    loadingState.classList.remove('d-flex');
    errorState.classList.add('d-none');
    emptyState.classList.add('d-none');

    if (state === 'loading') {

```

```
        loadingState.classList.remove('d-none');
        loadingState.classList.add('d-flex');
    }
    else if (state === 'error') {
        errorState.classList.remove('d-none');
    }
    else if (state === 'empty') {
        emptyState.classList.remove('d-none');
    }
    // state 'data' = semua disembunyikan (default)
}

/* =====
   FETCH DATA
   ===== */
async function muatDataLaporan() {
    container.innerHTML = '';
    statsBar.classList.add('d-none');
    ticketsProses.clear();

    tampilkanState('loading');

    try {
        const response = await fetch(API_URL);

        if (!response.ok) {
            throw new Error(`Gagal mengambil data (HTTP
${response.status})`);
        }

        const data = await response.json();

        // Ambil 10 data terbaru
        const laporan = data.slice(-JUMLAH_LAPORAN).reverse();

        if (laporan.length === 0) {
            tampilkanState('empty');
            return;
        }

        renderData(laporan);
        tampilkanStatistik(laporan.length);

        statsBar.classList.remove('d-none');

        tampilkanState('data');
```



```

    } catch (error) {
        errorMsg.textContent = error.message;
        tampilkanState('error');
    }
}

/* =====
   RENDER DATA
===== */
function renderData(data) {
    container.innerHTML = '';

    data.forEach((laporan) => {
        const judul = sanitasi(laporan.title);
        const deskripsi = sanitasi(laporan.body);
        const formatId = `WO-${String(laporan.id).padStart(3,
'0')}}`;

        const col = document.createElement('div');
        col.className = 'col-md-6 col-lg-4 col-xl-3';

        col.innerHTML = `
            <div class="incident-card" id="card-${laporan.id}">

                <div class="card-header-custom">
                    <span class="wo-id">
                        <i class="bi bi-upc-scan me-1"></i>
${formatId}
                    </span>
                    <div id="badgeContainer-${laporan.id}">
                        <span class="badge-ind badge-ind-
pending">
                            <i class="bi bi-exclamation-
triangle"></i> GANGGUAN
                        </span>
                    </div>
                </div>

                <div class="card-body-custom">
                    <h5 class="card-title-custom">${judul}</h5>
                    <p class="card-desc-
custom">${deskripsi}</p>
                </div>

                <div class="card-footer-custom">
                    <button

```

```

                                class="btn btn-tindak btn-tindak-
pending"
                                id="btn-`${laporan.id}`"
                                onclick="tindakLanjut(`${laporan.id}`)">
                                <i class="bi bi-wrench"></i> Tindak
Lanjut
                                </button>
                                </div>

                                </div>
                                `;

                                container.appendChild(col);
                                });
                                }

function tindakLanjut(id) {
    alert("Tiket ID sedang diproses oleh Tim Maintenance");

    if (!tiketsProses.has(id)) {
        ticketsProses.add(id);

        const card = document.getElementById(`card-${id}`);
        if (card) card.classList.add('card-diproses');

        const badge = document.getElementById(`badgeContainer-
${id}`);
        if (badge) {
            badge.innerHTML = `
                <span class="badge-ind badge-ind-proses">
                    <i class="bi bi-check2-all"></i> SEDANG
DIPROSES
                </span>
            `;
        }

        const btn = document.getElementById(`btn-${id}`);
        if (btn) {
            btn.classList.remove('btn-tindak-pending');
            btn.classList.add('btn-tindak-done');
            btn.innerHTML = `<i class="bi bi-lock-fill"></i>
Sedang Ditangani`;
        }

        perbaruiStatistik();
    }
}

```

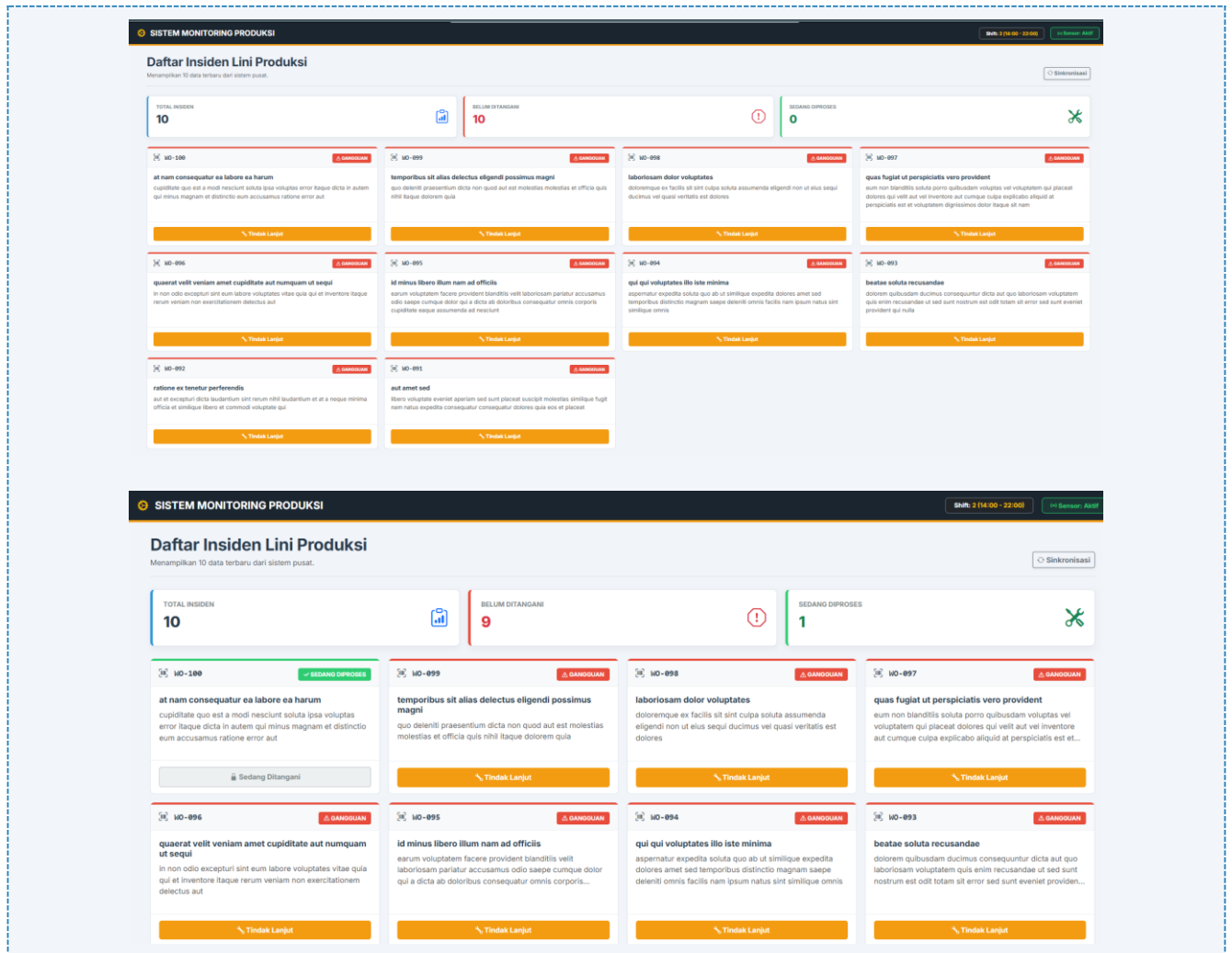
```
function tampilkanStatistik(total) {
    statTotal.textContent = total;
    statPending.textContent = total;
    statProses.textContent = 0;
}

function perbaruiStatistik() {
    const proses = ticketsProses.size;
    const total = parseInt(statTotal.textContent);

    statProses.textContent = proses;
    statPending.textContent = total - proses;
}

function sanitasi(str) {
    const div = document.createElement('div');
    div.appendChild(document.createTextNode(str));
    return div.innerHTML;
}

document.addEventListener("DOMContentLoaded", muatDataLaporan);
```



Gambar F.3 – Output dari Proyek Mini Industri

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang diperoleh sudah sesuai dengan konsep *asynchronous JavaScript* yang dipelajari. Hal ini terlihat dari proses pengambilan data menggunakan *Fetch API* yang tidak menghambat jalannya aplikasi. Saat tombol ditekan, sistem menampilkan *loading* terlebih dahulu, kemudian data dari API berhasil ditampilkan dalam bentuk *card* tanpa membuat halaman menjadi *freeze*. Selain itu, penggunaan *Promise* dan *async/await* juga berjalan sesuai teori, terutama dengan adanya pengecekan *response* serta penanganan *error* menggunakan *catch* atau *try-catch*.

Pada bagian proyek mini, data dari API berhasil diproses dan ditampilkan secara dinamis setelah melalui tahap pengolahan seperti *slice()* dan *reverse()*. Fitur interaksi seperti tombol “Tindak Lanjut” juga berjalan dengan baik dan memberikan respon berupa perubahan status serta notifikasi. Secara keseluruhan, hasil yang diperoleh tidak hanya sesuai dengan teori, tetapi juga menunjukkan bahwa konsep yang dipelajari sudah dapat diterapkan dalam bentuk sistem sederhana yang interaktif dan mendekati kondisi nyata.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Data tidak langsung muncul saat halaman dibuka karena proses <i>fetch</i> membutuhkan waktu	Menambahkan indikator <i>loading</i> agar pengguna mengetahui bahwa sistem sedang mengambil data dari server

G.3 Kaitan dengan Penerapan di Industri

Keterampilan yang dipelajari pada modul ini, seperti penggunaan *asynchronous JavaScript* dan integrasi API, sangat relevan dalam pengembangan sistem informasi di dunia industri. Teknik ini dapat digunakan untuk membangun dashboard monitoring yang menampilkan data secara *real-time* tanpa mengganggu kinerja sistem. Misalnya, pada sistem produksi, data seperti jumlah output, kondisi mesin, atau laporan gangguan dapat diambil dari server pusat dan langsung ditampilkan ke operator melalui dashboard berbasis web.

Sebagai contoh konkret, pada sistem *andon* di pabrik, ketika terjadi gangguan mesin, data insiden akan langsung dikirim ke server dan ditampilkan pada dashboard monitoring. Operator atau tim maintenance dapat melihat informasi tersebut dan melakukan tindak lanjut secara cepat. Konsep yang dipraktikkan dalam modul ini, seperti pengambilan data dari API, pengolahan data, serta interaksi melalui tombol “Tindak Lanjut”, mencerminkan alur kerja sistem tersebut. Dengan demikian, kemampuan ini sangat penting bagi mahasiswa Teknik Industri dalam mendukung pengembangan sistem yang efisien, responsif, dan terintegrasi.

H. KESIMPULAN

1	Praktikum ini menunjukkan bahwa penggunaan <i>asynchronous JavaScript</i> memungkinkan aplikasi web tetap responsif meskipun melakukan pengambilan data dari server eksternal.
2	Implementasi <i>Fetch API</i> berhasil digunakan untuk mengambil data dari API dan mengolahnya dalam format <i>JSON</i> sebelum ditampilkan ke halaman web.
3	Penggunaan <i>Promise</i> dan <i>async/await</i> membantu dalam mengatur alur program serta mempermudah penanganan error saat terjadi kegagalan dalam proses <i>request</i> .
4	Data yang diperoleh dari API dapat diproses lebih lanjut menggunakan <i>method</i> seperti <code>slice()</code> dan <code>reverse()</code> sehingga dapat disesuaikan dengan kebutuhan tampilan, seperti menampilkan data terbaru.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Doctoral dissertation, University of California, Irvine).
[3]	Laudon, K. C., & Laudon, J. P. (2020). Management information systems: Managing the digital firm (16th ed.). Pearson.
[4]	Mozilla Developer Network. (2023). Using the Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
[5]	Sommerville, I. (2016). Software engineering (10th ed.). Pearson.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
Radhitya Adha Fadhlur Rohman NIM: 23051430005	Maessa Andrea Vallenia NIM: 23051430044	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

Modul 9 Pengenalan React.js (Component, JSX, Props)

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 9 (Pertemuan 9)
Topik Utama	Pengenalan React.js (Component, JSX, Props)
Sub-Topik	<i>Setup Lingkungan React, Konsep Component, JSX Syntax, dan Props (Data Passing).</i>
Durasi Praktikum	340 Menit
Tanggal Praktikum	24 April 2026
Nama Mahasiswa	Radhitya Adha Fadhlur Rohman
NIM	23051430005
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami konsep Library vs Framework dan mengapa React dipilih untuk aplikasi skala besar.
2	Mampu menyiapkan environment pengembangan React menggunakan Vite.
3	Memahami sintaks JSX (JavaScript XML).
4	Mampu membuat Functional Component yang menerima dan menampilkan data melalui Props.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	File `index.html`	Versi Terbaru	Hasil pekerjaan pertemuan sebelumnya

4	Ekstensi VS Code: LiveServer	Versi terbaru	Melihat perubahan web secara real-time.
5	Javascript	Versi terbaru	Membuat halaman web menjadi lebih interaktif dan fungsional
6	Node.js	Versi Terbaru	<i>runtime environment</i> JavaScript

D. DASAR TEORI

React merupakan *library* JavaScript yang digunakan untuk membangun tampilan antarmuka (*user interface*) pada aplikasi web. React banyak digunakan karena mampu menangani aplikasi yang kompleks dengan cara yang lebih terstruktur dibandingkan *vanilla JavaScript*. Konsep utama yang digunakan adalah membagi tampilan menjadi bagian-bagian kecil sehingga lebih mudah dikembangkan dan dikelola.

Dalam React, terdapat konsep *component* yaitu bagian kecil dari tampilan yang dapat digunakan kembali (*reusable*). Komponen biasanya ditulis dalam bentuk fungsi yang mengembalikan tampilan menggunakan JSX (*JavaScript XML*). JSX memungkinkan penulisan kode yang menyerupai HTML di dalam JavaScript sehingga lebih mudah dibaca dan dipahami, terutama dalam menampilkan data secara dinamis.

Selain itu, React menggunakan *props* sebagai cara untuk mengirim data dari komponen induk ke komponen anak. Props ini berfungsi seperti parameter pada fungsi, sehingga satu komponen bisa digunakan berulang kali dengan data yang berbeda. Penggunaan teknik seperti *destructuring* dan pemberian nilai default pada props dapat membantu membuat kode menjadi lebih ringkas, rapi, dan mengurangi potensi error.

D.1 Konsep Utama

Konsep utama pada praktikum ini adalah penggunaan React untuk membangun tampilan web berbasis *component*. Dengan React, tampilan tidak dibuat secara utuh dalam satu bagian, tetapi dipecah menjadi komponen-komponen kecil yang lebih terstruktur dan bisa digunakan kembali. Hal ini membuat pengembangan aplikasi menjadi lebih rapi dan efisien. Selain itu, digunakan JSX (*JavaScript XML*) yang memungkinkan penulisan tampilan seperti HTML di dalam JavaScript, sehingga lebih mudah dipahami dan mendukung penampilan data secara dinamis.

Selain komponen dan JSX, konsep penting lainnya adalah *props*, yaitu cara mengirim data dari komponen induk ke komponen anak. Props memungkinkan satu komponen digunakan berulang kali dengan data yang berbeda. Dalam praktikum ini juga diterapkan *destructuring props* untuk menyederhanakan penulisan kode serta *default props* untuk memberikan nilai

awal jika data tidak dikirim. Dengan konsep-konsep ini, aplikasi menjadi lebih fleksibel, rapi, dan meminimalisir kesalahan.

D.2 Keterkaitan dengan Teknik Industri

Materi pada praktikum ini sangat relevan dengan bidang Teknik Industri, terutama dalam pengembangan sistem informasi untuk mendukung proses operasional. Dalam dunia industri, sering dibutuhkan dashboard atau sistem monitoring untuk memantau kondisi produksi secara real-time, seperti status mesin, jumlah output, atau kondisi proses. Dengan menggunakan React dan konsep komponen, tampilan data tersebut dapat dibuat lebih terstruktur, dinamis, dan mudah diperbarui sesuai kebutuhan.

Sebagai contoh, pada lini produksi di sebuah pabrik, setiap mesin memiliki status seperti *running*, *maintenance*, atau *stop*, serta data jumlah produksi. Informasi ini dapat ditampilkan dalam bentuk kartu (card) seperti pada praktikum, di mana setiap komponen merepresentasikan satu mesin. Dengan memanfaatkan props, data dari sistem dapat dikirim ke setiap komponen sehingga dashboard dapat menampilkan kondisi terbaru secara akurat. Pendekatan ini membantu pengambilan keputusan menjadi lebih cepat dan tepat, misalnya saat terjadi penurunan produksi atau kerusakan mesin.

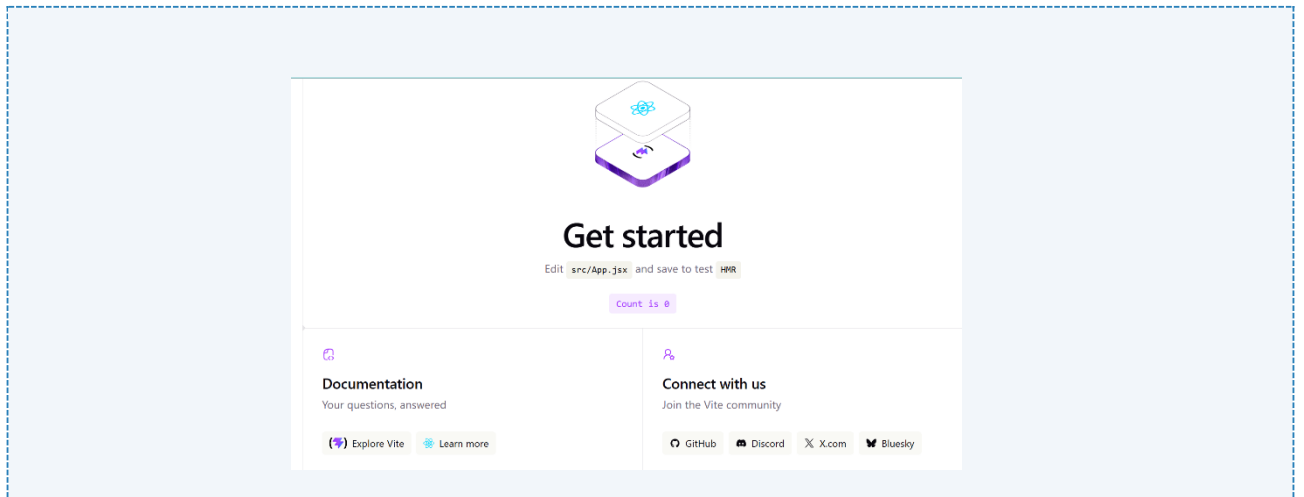
E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Setup Project dengan Vite

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
VITE v8.0.10  ready in 3673 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Screenshot Hasil:

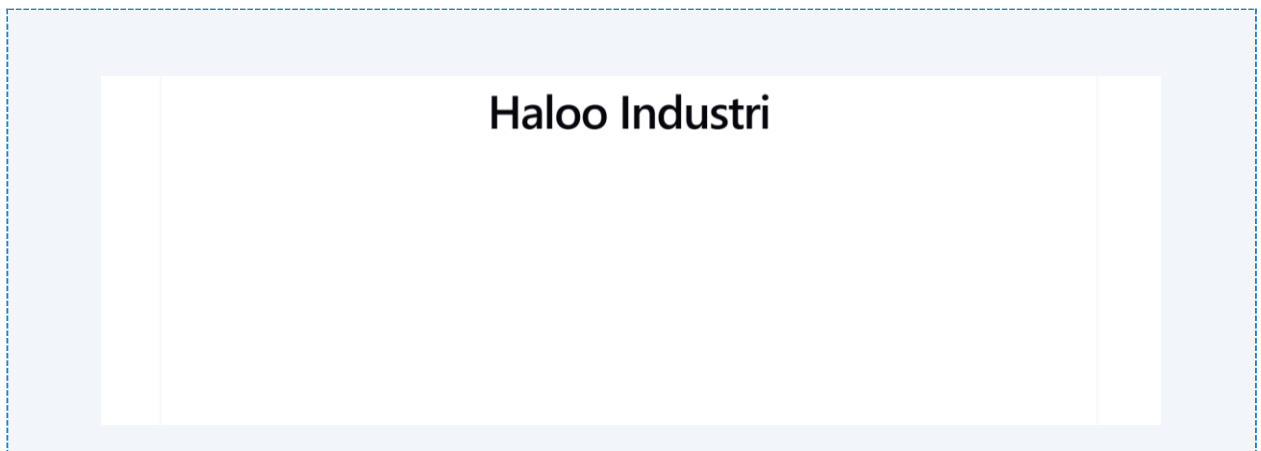


Gambar E.1 – Kode dari Langkah 1

E.2 Langkah 2 – Membersihkan Project

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */  
<h1>Halo Industri</h1>
```

Screenshot Hasil:



Gambar E.2 – Output dari Langkah 2

E.3 Langkah 3 – Membuat Komponen Pertama: KartuMesin

```
// 1. Import React  
import React from 'react';  
  
// 2. Membuat Komponen Fungsi yang menerima 'props'  
function KartuMesin(props) {  
  
  // Menerima data dari props  
  const namaMesin = props.nama;
```

```
const status = props.status;
const produksi = props.produksi;

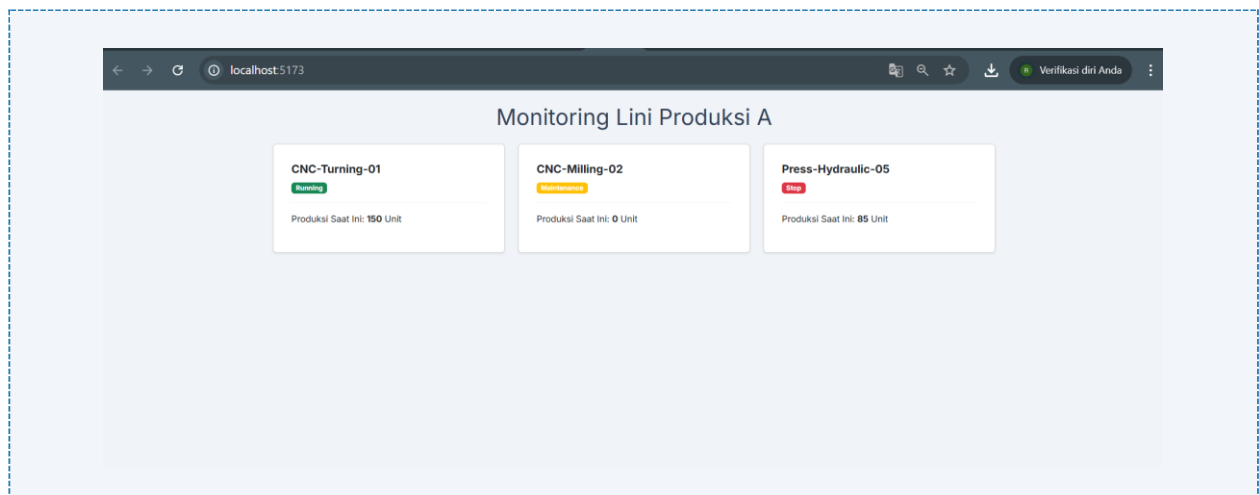
// Logika penentuan warna badge berdasarkan status
let badgeColor = 'bg-secondary';

if (status === 'Running') badgeColor = 'bg-success';
if (status === 'Stop') badgeColor = 'bg-danger';
if (status === 'Maintenance') badgeColor = 'bg-warning';

return (
  // Menggunakan class -> className di JSX
  <div className="card shadow-sm p-3 mb-3">
    <div className="card-body">
      <h5 className="card-title">{namaMesin}</h5>
      <span className={`badge ${badgeColor}`}>{status}</span>
      <hr />
      <p>
        Produksi Saat Ini: <strong>{produksi}</strong> Unit
      </p>
    </div>
  </div>
);
}

// 3. Export agar bisa dipakai di file lain
export default KartuMesin;
```

Screenshot Hasil:



Gambar E.3 – Output dari Langkah 3

E.4 Langkah 4 – Menggunakan Komponen di App.jsx

```
/* ===== KODE / OUTPUT LANGKAH 4 ===== */
import React from 'react';
// Pastikan file KartuMesin.js ada di folder ./Komponen/
import KartuMesin from './Komponen/KartuMesin';

function App() {
  return (
    <div style={{ padding: '20px', fontFamily: 'Arial, sans-serif' }}>
      <h1>Monitoring Lini Produksi A</h1>

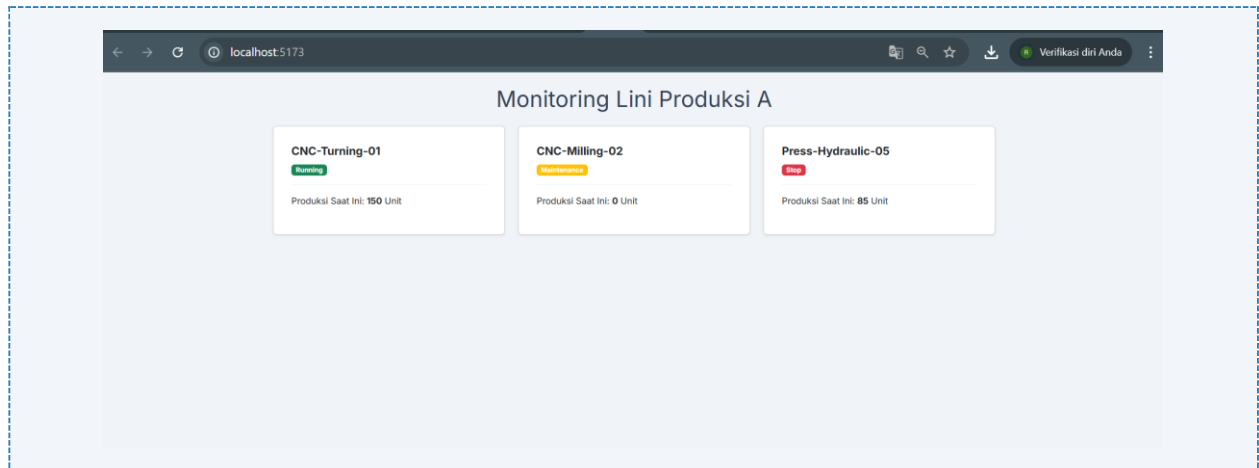
      <div style={{ display: 'flex', gap: '20px', marginTop: '20px' }}>
        /* Kolom 1: Menggunakan komponen dengan data berbeda */
        <div className="column">
          <KartuMesin
            nama="Mesin CNC 01"
            status="Berjalan"
            suhu={45}
          />
        </div>

        /* Kolom 2 */
        <div className="column">
          <KartuMesin
            nama="Mesin Bubut 02"
            status="Berhenti"
            suhu={25}
          />
        </div>

        /* Kolom 3 */
        <div className="column">
          <KartuMesin
            nama="Mesin Milling 03"
            status="Peringatan"
            suhu={85}
          />
        </div>
      </div>
    </div>
  );
}
```

```
export default App;
```

Screenshot Hasil:



Gambar E.4 – Output dari Langkah 4

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Destructuring Props

Pertanyaan / Instruksi Latihan:

Ubah cara deklarasi props di `KartuMesin.jsx` menggunakan destructuring: `function KartuMesin({ nama, status, produksi }) { ... }`. Sesuaikan kode di dalamnya.

Jawaban / Kode Solusi:

Membuat file didalam folder KOMPonen dengan nama KartuMesinLat1.jsx

```
// 1. Import React
import React from 'react';

// 2. Komponen dengan destructuring props
function KartuMesinLat1({ nama, status, produksi }) {

  // Logika penentuan warna badge berdasarkan status
  let badgeColor = 'bg-secondary';

  if (status === 'Running') badgeColor = 'bg-success';
  if (status === 'Stop') badgeColor = 'bg-danger';
```

```

    if (status === 'Maintenance') badgeColor = 'bg-warning';

    return (
      <div className="card shadow-sm p-3 mb-3">
        <div className="card-body">
          /* langsung pakai nama */
          <h5 className="card-title">{nama}</h5>
          <span className={`badge
${badgeColor}`}>{status}</span>
          <hr />
          <p>
            Produksi Saat Ini: <strong>{produksi}</strong> Unit
          </p>
        </div>
      </div>
    );
  }

// 3. Export
export default KartuMesinLat1;

```

Kemudian Dalam APP.jsx Tambahkan untuk KartuMesinLat1

```

import React, { useState } from 'react';
import './App.css';

// Import komponen
import KartuMesin from './Komponen/KartuMesin';
import KartuMesinLat1 from './Komponen/KartuMesinLat1';

function App() {
  const [halaman, setHalaman] = useState('Praktikum');

  return (
    <div className="container mt-5 mb-5">
      <h2 className="text-center mb-2">Monitoring Lini
Produksi</h2>
      <p className="text-center text-muted mb-5">
        Radhitya Adha Fadhlur Rohman (23051430005)
      </p>

      /* ===== NAVIGASI ===== */
      <div className="d-flex justify-content-center gap-3 mb-5
flex-wrap">

```

```

<button
  className={`btn tombol-navigasi ${
    halaman === 'Praktikum' ? 'btn-primary' : 'btn-
outline-primary'
  }}
  onClick={() => setHalaman('Praktikum')}
>
  Versi Praktikum
</button>

<button
  className={`btn tombol-navigasi ${
    halaman === 'latihan1' ? 'btn-info text-white' :
'btn-outline-info'
  }}
  onClick={() => setHalaman('latihan1')}
>
  Latihan 1
</button>

<button
  className={`btn tombol-navigasi ${
    halaman === 'latihan2' ? 'btn-success' : 'btn-
outline-success'
  }}
  onClick={() => setHalaman('latihan2')}
>
  /* ===== 1. PRAKTIKUM ===== */
  {halaman === 'Praktikum' && (
    <div>
      <h4 className="text-center mb-4">1. Versi Praktikum
(Props)</h4>
      <div className="row g-4">
        <div className="col-md-4">
          <KartuMesin nama="CNC-Turning-01"
status="Running" produksi={150} />
        </div>
        <div className="col-md-4">
          <KartuMesin nama="CNC-Milling-02"
status="Maintenance" produksi={0} />
        </div>
        <div className="col-md-4">
          <KartuMesin nama="Press-Hydraulic-05"
status="Stop" produksi={85} />
        </div>
      </div>
    </div>
  )}

```



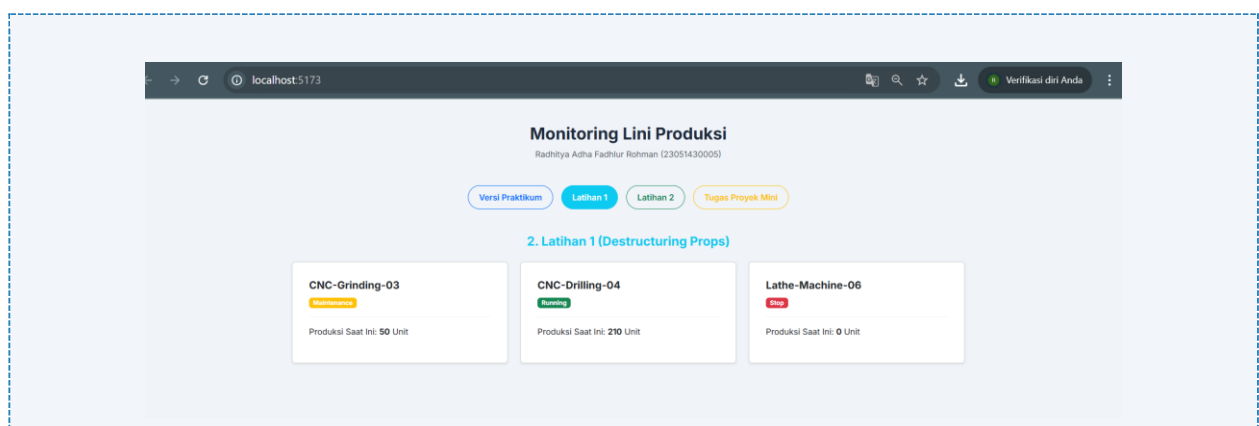
```

    ) }

    { /* ===== 2. LATIHAN 1 ===== */ }
    { halaman === 'latihan1' && (
      <div>
        <h4 className="text-center mb-4 text-info">
          2. Latihan 1 (Destructuring Props)
        </h4>
        <div className="row g-4">
          <div className="col-md-4">
            <KartuMesinLat1 nama="CNC-Grinding-03"
status="Maintenance" produksi={50} />
          </div>
          <div className="col-md-4">
            <KartuMesinLat1 nama="CNC-Drilling-04"
status="Running" produksi={210} />
          </div>
          <div className="col-md-4">
            <KartuMesinLat1 nama="Lathe-Machine-06"
status="Stop" produksi={0} />
          </div>
        </div>
      </div>
    ) }
  );
}

export default App;

```



Gambar F.1 – Output dari Latihan 1

F.2 Latihan 2 – Default Props

Pertanyaan / Instruksi Latihan:

Berikan nilai default pada prop `produksi` menjadi 0, jika user lupa mengirim prop tersebut.

Jawaban / Kode Solusi:

Membuat file didalam folder KOmponen dengan nama KartuMesinLat2.jsx

```
// 1. Import React
import React from 'react';

// 2. Komponen dengan destructuring + default props
function KartuMesinLat2({ nama, status, produksi = 0 }) {

  // Logika penentuan warna badge berdasarkan status
  let badgeColor = 'bg-secondary';

  if (status === 'Running') badgeColor = 'bg-success';
  if (status === 'Stop') badgeColor = 'bg-danger';
  if (status === 'Maintenance') badgeColor = 'bg-warning';

  return (
    <div className="card shadow-sm p-3 mb-3">
      <div className="card-body">
        <h5 className="card-title">{nama}</h5>
        <span className={`badge
${badgeColor}`}>{status}</span>
        <hr />
        <p>
          Produksi Saat Ini: <strong>{produksi}</strong> Unit
        </p>
      </div>
    </div>
  );
}

// 3. Export
export default KartuMesinLat2;
```

```
import React, { useState } from 'react';
import './App.css';

// Import komponen
import KartuMesin from './Komponen/KartuMesin';
import KartuMesinLat1 from './Komponen/KartuMesinLat1';
```

```
import KartuMesinLat2 from './Komponen/KartuMesinLat2';

function App() {
  const [halaman, setHalaman] = useState('Praktikum');

  return (
    <div className="container mt-5 mb-5">
      <h2 className="text-center mb-2">Monitoring Lini Produksi</h2>
      <p className="text-center text-muted mb-5">
        Radhitya Adha Fadhlur Rohman (23051430005)
      </p>

      { /* ===== NAVIGASI ===== */ }
      <div className="d-flex justify-content-center gap-3 mb-5 flex-wrap">
        <button
          className={`btn tombol-navigasi ${
            halaman === 'Praktikum' ? 'btn-primary' : 'btn-outline-primary'
          }`}
          onClick={() => setHalaman('Praktikum')}
        >
          Versi Praktikum
        </button>

        <button
          className={`btn tombol-navigasi ${
            halaman === 'latihan1' ? 'btn-info text-white' : 'btn-outline-info'
          }`}
          onClick={() => setHalaman('latihan1')}
        >
          Latihan 1
        </button>

        <button
          className={`btn tombol-navigasi ${
            halaman === 'latihan2' ? 'btn-success' : 'btn-outline-success'
          }`}
          onClick={() => setHalaman('latihan2')}
        >
          Latihan 2
        </button>

        <button
          className={`btn tombol-navigasi ${
            halaman === 'tugas' ? 'btn-warning text-dark' : 'btn-outline-
warning'
          }`}
          onClick={() => setHalaman('tugas')}
        >
```

```

    >
    { /* ===== 1. PRAKTIKUM ===== */ }
    { halaman === 'Praktikum' && (
      <div>
        <h4 className="text-center mb-4">1. Versi Praktikum (Props)</h4>
        <div className="row g-4">
          <div className="col-md-4">
            <KartuMesin nama="CNC-Turning-01" status="Running" produksi={150}
          />
          </div>
          <div className="col-md-4">
            <KartuMesin nama="CNC-Milling-02" status="Maintenance"
produksi={0} />
          </div>
          <div className="col-md-4">
            <KartuMesin nama="Press-Hydraulic-05" status="Stop" produksi={85}
          />
          </div>
        </div>
      </div>
    )}

    { /* ===== 2. LATIHAN 1 ===== */ }
    { halaman === 'latihan1' && (
      <div>
        <h4 className="text-center mb-4 text-info">
          2. Latihan 1 (Destructuring Props)
        </h4>
        <div className="row g-4">
          <div className="col-md-4">
            <KartuMesinLat1 nama="CNC-Grinding-03" status="Maintenance"
produksi={50} />
          </div>
          <div className="col-md-4">
            <KartuMesinLat1 nama="CNC-Drilling-04" status="Running"
produksi={210} />
          </div>
          <div className="col-md-4">
            <KartuMesinLat1 nama="Lathe-Machine-06" status="Stop"
produksi={0} />
          </div>
        </div>
      </div>
    )}

    { /* ===== 3. LATIHAN 2 ===== */ }
    { halaman === 'latihan2' && (

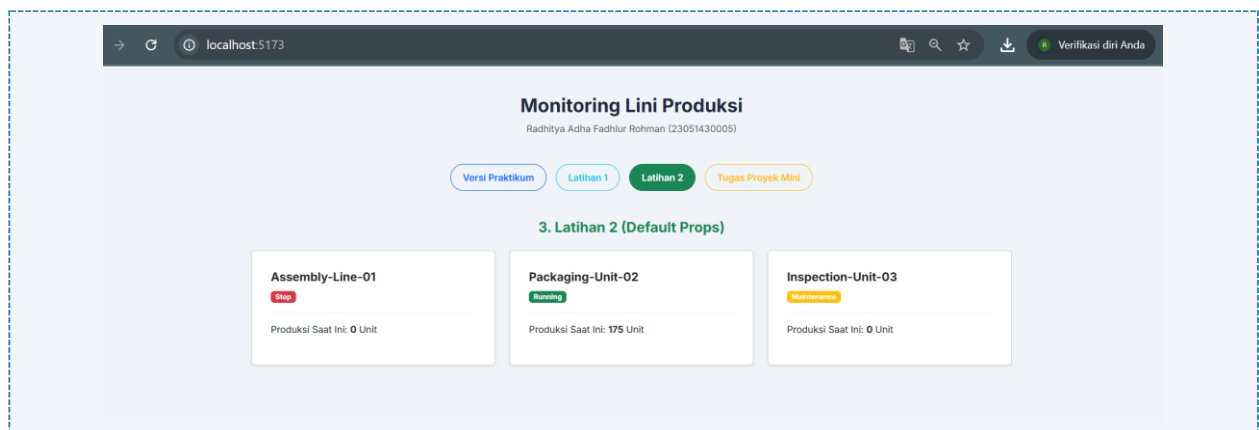
```

```

    <div>
      <h4 className="text-center mb-4 text-success">
        3. Latihan 2 (Default Props)
      </h4>
      <div className="row g-4">
        <div className="col-md-4">
          <KartuMesinLat2 nama="Assembly-Line-01" status="Stop" />
        </div>
        <div className="col-md-4">
          <KartuMesinLat2 nama="Packaging-Unit-02" status="Running"
produksi={175} />
        </div>
        <div className="col-md-4">
          <KartuMesinLat2 nama="Inspection-Unit-03" status="Maintenance" />
        </div>
      </div>
    </div>
  )}
);
}

export default App;

```



Gambar F.2 – Output dari Latihan 2

F.3 Tugas Proyek Mini – Kartu Karyawan

Deskripsi proyek mini:

- Buat komponen baru bernama `KartuKaryawan.jsx`. Komponen ini menerima props: `nama`, `jabatan`, `bagian`.
- Tampilkan komponen ini 3 kali di `App.jsx` dengan data karyawan berbeda (Manager, Operator, QC).

Penjelasan pendekatan/solusi yang digunakan:

Pendekatan yang digunakan pada proyek mini ini adalah dengan menerapkan konsep *component-based* pada React untuk menampilkan data karyawan dalam bentuk kartu (*card*). Dibuat komponen khusus yaitu *KartuKaryawan* yang berfungsi menampilkan informasi seperti nama, jabatan, dan bagian kerja. Komponen ini dirancang agar bersifat *reusable*, sehingga dapat digunakan berulang kali hanya dengan mengganti data yang dikirim melalui *props* dari file utama (*App.jsx*).

Pada bagian *App.jsx*, data karyawan ditampilkan dengan cara memanggil komponen *KartuKaryawan* sebanyak tiga kali dengan isi data yang berbeda (Manager, Operator, dan QC). Selain itu, digunakan *state* (*useState*) untuk mengatur navigasi sederhana sehingga pengguna dapat berpindah antara halaman praktikum, latihan, dan tugas proyek mini. Dengan pendekatan ini, tampilan menjadi lebih terstruktur, mudah dikembangkan, dan sesuai dengan konsep aplikasi modern yang dinamis serta modular.

Kode Utama:

```
// 1. Import React
import React from 'react';

// 2. Komponen dengan destructuring props
function KartuKaryawan({ nama, jabatan, bagian }) {
  // Logika penentuan warna badge berdasarkan jabatan
  let badgeColor = 'bg-primary';
  if (jabatan === 'Manager') badgeColor = 'bg-danger';
  if (jabatan === 'QC') badgeColor = 'bg-success';
  if (jabatan === 'Operator') badgeColor = 'bg-info text-
dark';

  return (
    <div className="card shadow-sm p-3 mb-3 kartu-kustom">
      <div className="card-body">

        {/* Nama */}
        <h5 className="card-title fw-bold mb-3">{nama}</h5>

        {/* Jabatan (label + badge, tidak double) */}
        <p className="mb-2 text-muted">
          Jabatan: { ' ' }
          <span className={`badge ${badgeColor}`}>
            {jabatan}
          </span>
        </p>

        {/* Bagian */}
        <p className="mb-0 text-muted">
```

```

        Bagian: <strong className="text-
dark">{bagian}</strong>
      </p>
    </div>
  </div>
);
}

// 3. Export
export default KartuKaryawan;

```

```

Membuat dan melengkapi app.jsx
import React, { useState } from 'react';
import './App.css';

// Import komponen
import KartuMesin from './Komponen/KartuMesin';
import KartuMesinLat1 from './Komponen/KartuMesinLat1';
import KartuMesinLat2 from './Komponen/KartuMesinLat2';
import KartuKaryawan from './Komponen/KartuKaryawan';

function App() {
  const [halaman, setHalaman] = useState('Praktikum');

  return (
    <div className="container mt-5 mb-5">
      <h2 className="text-center mb-2">Monitoring Lini
Produksi</h2>
      <p className="text-center text-muted mb-5">
        Radhitya Adha Fadhlur Rohman (23051430005)
      </p>

      {/* ===== NAVIGASI ===== */}
      <div className="d-flex justify-content-center gap-3 mb-5
flex-wrap">
        <button
          className={`btn tombol-navigasi ${
            halaman === 'Praktikum' ? 'btn-primary' : 'btn-
outline-primary'
          }`}
          onClick={() => setHalaman('Praktikum')}
        >
          Versi Praktikum

```

```

</button>

<button
  className={`btn tombol-navigasi ${
    halaman === 'latihan1' ? 'btn-info text-white' :
    'btn-outline-info'
 }`}
  onClick={() => setHalaman('latihan1')}
>
  Latihan 1
</button>

<button
  className={`btn tombol-navigasi ${
    halaman === 'latihan2' ? 'btn-success' : 'btn-
outline-success'
 }`}
  onClick={() => setHalaman('latihan2')}
>
  Latihan 2
</button>

<button
  className={`btn tombol-navigasi ${
    halaman === 'tugas' ? 'btn-warning text-dark' :
    'btn-outline-warning'
 }`}
  onClick={() => setHalaman('tugas')}
>
  Tugas Proyek Mini
</button>
</div>

{/* ===== 1. PRAKTIKUM ===== */}
{halaman === 'Praktikum' && (
  <div>
    <h4 className="text-center mb-4">1. Versi Praktikum
(Props)</h4>
    <div className="row g-4">
      <div className="col-md-4">
        <KartuMesin nama="CNC-Turning-01"
status="Running" produksi={150} />
      </div>
      <div className="col-md-4">
        <KartuMesin nama="CNC-Milling-02"
status="Maintenance" produksi={0} />
      </div>
    </div>
  </div>

```



```

        <div className="col-md-4">
            <KartuMesin nama="Press-Hydraulic-05"
status="Stop" produksi={85} />
        </div>
    </div>
</div>
)}

{/* ===== 2. LATIHAN 1 ===== */}
{halaman === 'latihan1' && (
    <div>
        <h4 className="text-center mb-4 text-info">
            2. Latihan 1 (Destructuring Props)
        </h4>
        <div className="row g-4">
            <div className="col-md-4">
                <KartuMesinLat1 nama="CNC-Grinding-03"
status="Maintenance" produksi={50} />
            </div>
            <div className="col-md-4">
                <KartuMesinLat1 nama="CNC-Drilling-04"
status="Running" produksi={210} />
            </div>
            <div className="col-md-4">
                <KartuMesinLat1 nama="Lathe-Machine-06"
status="Stop" produksi={0} />
            </div>
        </div>
    </div>
)}

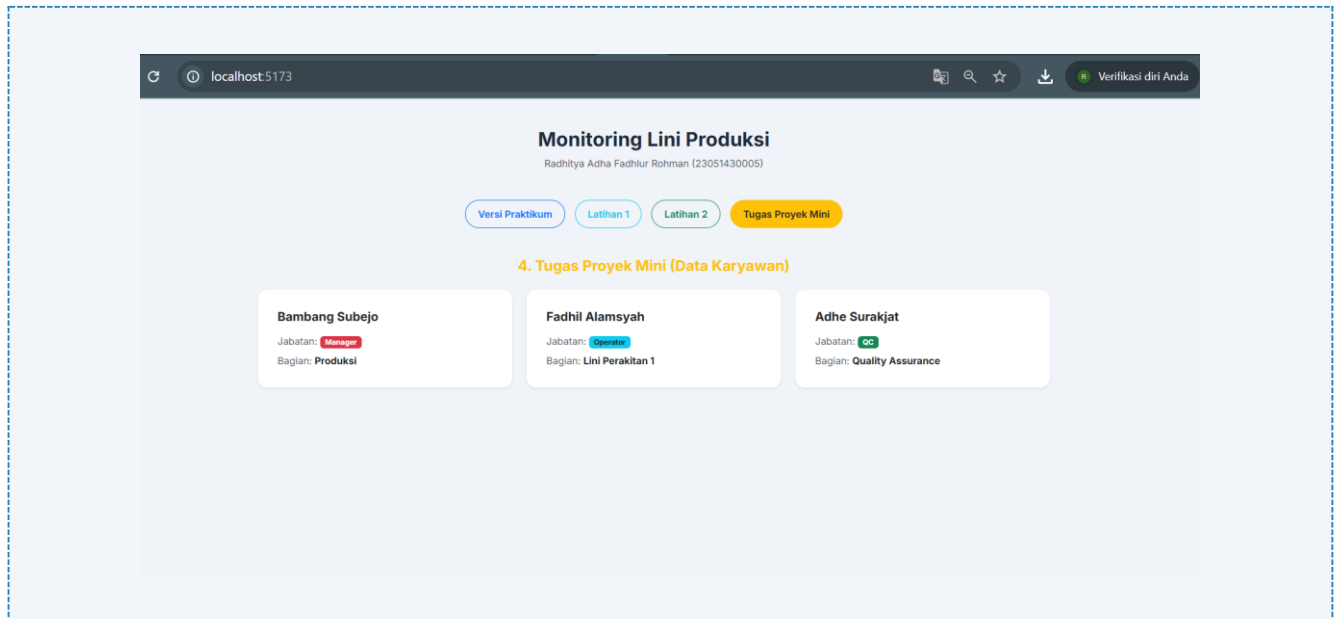
{/* ===== 3. LATIHAN 2 ===== */}
{halaman === 'latihan2' && (
    <div>
        <h4 className="text-center mb-4 text-success">
            3. Latihan 2 (Default Props)
        </h4>
        <div className="row g-4">
            <div className="col-md-4">
                <KartuMesinLat2 nama="Assembly-Line-01"
status="Stop" />
            </div>
            <div className="col-md-4">
                <KartuMesinLat2 nama="Packaging-Unit-02"
status="Running" produksi={175} />
            </div>
            <div className="col-md-4">

```

```
        <KartuMesinLat2 nama="Inspection-Unit-03"
status="Maintenance" />
    </div>
</div>
</div>
)}

{/* ===== 4. TUGAS ===== */}
{halaman === 'tugas' && (
    <div>
        <h4 className="text-center mb-4 text-warning">
            4. Tugas Proyek Mini (Data Karyawan)
        </h4>
        <div className="row g-4">
            <div className="col-md-4">
                <KartuKaryawan nama="Bambang Subejo"
jabatan="Manager" bagian="Produksi" />
            </div>
            <div className="col-md-4">
                <KartuKaryawan nama="Fadhil Alamsyah"
jabatan="Operator" bagian="Lini Perakitan 1" />
            </div>
            <div className="col-md-4">
                <KartuKaryawan nama="Adhe Surakjat" jabatan="QC"
bagian="Quality Assurance" />
            </div>
        </div>
    </div>
)}
</div>
);
}

export default App;
```



Gambar F.3 – Output dari Proyek Mini Industri

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang diperoleh pada pertemuan ini pada dasarnya sudah sesuai dengan konsep yang dipelajari, khususnya terkait penggunaan *component*, *JSX*, dan *props* dalam React. Hal ini dapat dilihat dari pembuatan komponen seperti *KartuMesin* dan *KartuKaryawan* yang dipisahkan dari file utama dan kemudian digunakan kembali di dalam *App.jsx*. Setiap komponen mampu menampilkan data yang berbeda melalui *props*, sehingga sesuai dengan teori bahwa komponen bersifat *reusable* dan dapat menerima data layaknya parameter pada fungsi.

Penggunaan *JSX* juga sudah diterapkan dengan baik, di mana struktur tampilan ditulis menyerupai HTML namun tetap berada dalam JavaScript. Data seperti nama mesin, status, dan jumlah produksi ditampilkan secara dinamis menggunakan tanda {}, yang menunjukkan integrasi antara logika dan tampilan. Penerapan kondisi untuk menentukan warna *badge* berdasarkan status mesin juga menunjukkan bahwa logika JavaScript dapat langsung digunakan di dalam *JSX*, sesuai dengan konsep yang dijelaskan dalam teori.

Pada bagian latihan, penerapan *destructuring props* pada *KartuMesinLat1* dan *default props* pada *KartuMesinLat2* juga sudah sesuai dengan konsep yang dipelajari. *Destructuring* membantu membuat penulisan kode menjadi lebih ringkas dan mudah dibaca, sedangkan *default props* memastikan komponen tetap berjalan meskipun ada data yang tidak dikirim, seperti pada kasus nilai produksi yang otomatis menjadi 0. Secara keseluruhan, hasil praktikum tidak hanya sesuai dengan teori, tetapi juga menunjukkan pemahaman yang lebih dalam melalui pengembangan seperti penggunaan *state* dan *conditional rendering* pada *App.jsx*.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Tampilan awal terlihat kurang rapi karena semua komponen tampil memanjang dan tidak terstruktur.	Menggunakan <i>layout</i> dari Bootstrap seperti <i>container</i> , <i>row</i> , dan <i>col-md-4</i> agar tampilan terbagi menjadi beberapa kolom sehingga lebih rapi dan menyerupai dashboard.

G.3 Kaitan dengan Penerapan di Industri

Keterampilan yang dipelajari pada praktikum ini, khususnya penggunaan React dengan konsep *component*, *props*, dan *state*, sangat relevan dalam dunia Teknik Industri, terutama dalam pengembangan sistem informasi dan *dashboard* monitoring. Dalam industri, sering dibutuhkan sistem yang mampu menampilkan data produksi secara real-time, seperti status mesin, jumlah output, hingga kondisi operasional. Dengan pendekatan berbasis *component*, tampilan tersebut dapat dibuat lebih terstruktur dan mudah dikembangkan sesuai kebutuhan.

Sebagai contoh, pada sebuah pabrik manufaktur, setiap mesin di lini produksi dapat dimonitor melalui *dashboard* digital. Setiap mesin direpresentasikan sebagai satu komponen (seperti KartuMesin) yang menampilkan informasi status (*running*, *maintenance*, atau *stop*) serta jumlah produksi. Data ini bisa diambil dari sistem sensor atau database, lalu ditampilkan secara dinamis menggunakan *props*. Dengan sistem seperti ini, operator atau manajer produksi dapat dengan cepat mengetahui kondisi lapangan dan mengambil keputusan, misalnya saat terjadi penurunan produksi atau kerusakan mesin.

Selain itu, konsep yang sama juga dapat diterapkan pada manajemen sumber daya manusia, seperti pada proyek mini KartuKaryawan. Data karyawan dapat ditampilkan dalam bentuk kartu untuk memudahkan pemantauan jabatan dan penempatan kerja. Dengan demikian, keterampilan yang dipelajari tidak hanya terbatas pada pembuatan tampilan, tetapi juga mendukung pengambilan keputusan berbasis data dalam lingkungan industri yang nyata.

H. KESIMPULAN

1	Praktikum ini berhasil mengimplementasikan konsep dasar React seperti <i>component</i> , <i>JSX</i> , dan <i>props</i> dalam pembuatan tampilan monitoring data mesin secara dinamis.
2	Penggunaan <i>component</i> terbukti membuat kode lebih terstruktur dan dapat digunakan kembali (<i>reusable</i>), sehingga memudahkan dalam pengembangan aplikasi dengan data yang berbeda.
3	Penerapan <i>destructuring props</i> dan <i>default props</i> membantu menyederhanakan penulisan kode serta meningkatkan keandalan aplikasi ketika terdapat data yang tidak dikirim.
4	Penggunaan <i>state</i> dan <i>conditional rendering</i> pada <i>App.jsx</i> memungkinkan pembuatan navigasi sederhana, sehingga aplikasi menjadi lebih interaktif dan mendekati konsep aplikasi modern.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Meta Platforms, Inc. (2024). <i>React documentation</i> . React.dev. https://react.dev
[3]	W3Schools. (2024). <i>React props</i> . https://www.w3schools.com/react/react_props.asp
[4]	React Documentation (2024).

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
Radhitya Adha Fadhlur Rohman NIM: 23051430005	Maessa Andrea Vallenia NIM: 23051430044	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

Modul 10 State, Hooks (useState, useEffect), dan Conditional Rendering

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 10
Topik Utama	State, Hooks (useState, useEffect), dan Conditional Rendering
Sub-Topik	<i>State Management, Lifecycle Hooks, dan Rendering Bersyarat.</i>
Durasi Praktikum	340 Menit
Tanggal Praktikum	25 April 2026
Nama Mahasiswa	Radhitya Adha Fadhlur Rohman
NIM	23051430005
Kelas	A2

B. TUJUAN PEMBELAJARAN

1	Memahami apa itu State dan perbedaannya dengan Props.
2	Menguasai penggunaan `useState` untuk menyimpan data yang berubah-ubah.
3	Memahami `useEffect` untuk menangani side effects (seperti timer atau API call).
4	Mampu melakukan Conditional Rendering (menyembunyikan/menampilkan elemen berdasarkan kondisi).

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	File `index.html`	Versi Terbaru	Hasil pekerjaan pertemuan sebelumnya
4	Ekstensi VS Code: LiveServer	Versi terbaru	Melihat perubahan web secara real-time.

5	Javascript	Versi terbaru	Membuat halaman web menjadi lebih interaktif dan fungsional
6	Node.js	Versi Terbaru	<i>runtime environment</i> JavaScript

D. DASAR TEORI

React.js merupakan salah satu library JavaScript yang banyak digunakan untuk membangun tampilan website yang interaktif dan dinamis. React menggunakan konsep component-based, yaitu tampilan aplikasi dibagi menjadi beberapa komponen kecil yang dapat digunakan kembali sehingga proses pengembangan menjadi lebih rapi dan mudah dikelola. Selain itu, React juga mampu memperbarui tampilan secara otomatis ketika data berubah tanpa perlu me-refresh halaman, sehingga aplikasi terasa lebih cepat dan responsif bagi pengguna.

Pada praktikum ini dipelajari konsep state dan hooks pada React, khususnya useState dan useEffect. State digunakan untuk menyimpan data yang dapat berubah selama aplikasi berjalan, misalnya jumlah produksi, status mesin, atau data input pengguna. Ketika state berubah, React akan melakukan render ulang secara otomatis sehingga tampilan langsung menyesuaikan data terbaru. Sementara itu, useEffect digunakan untuk menjalankan proses tertentu secara otomatis, seperti membuat jam digital yang terus berjalan atau mengubah title browser sesuai input pengguna.

Selain itu, praktikum ini juga membahas conditional rendering, yaitu teknik menampilkan tampilan berdasarkan kondisi tertentu. Konsep ini membuat aplikasi menjadi lebih interaktif karena sistem dapat memberikan respon berbeda sesuai kondisi data yang ada. Contohnya pada simulasi produksi, sistem dapat menampilkan pesan target tercapai, status darurat, maupun perubahan warna pada hasil perhitungan OEE. Dengan praktikum ini, mahasiswa dapat memahami bagaimana React digunakan untuk membuat dashboard monitoring dan aplikasi web modern yang lebih dinamis serta mudah digunakan.

D.1 Konsep Utama

Konsep utama pada praktikum ini adalah penggunaan state dan hooks pada React.js untuk membuat aplikasi web yang lebih interaktif dan dinamis. State digunakan untuk menyimpan data yang dapat berubah selama aplikasi berjalan, seperti jumlah produksi, status mesin, maupun data input pengguna. Ketika data berubah, React akan otomatis memperbarui tampilan tanpa perlu refresh halaman. Selain itu, digunakan juga hooks seperti useState untuk mengelola data dan useEffect untuk menjalankan proses otomatis, misalnya membuat jam digital atau mengubah title browser sesuai input pengguna.

Praktikum ini juga mengenalkan conditional rendering, yaitu teknik menampilkan tampilan berdasarkan kondisi tertentu. Dengan konsep ini, aplikasi dapat memberikan respon visual yang berbeda sesuai keadaan sistem, seperti menampilkan pesan target tercapai, status

emergency, atau perubahan warna pada hasil OEE. Melalui materi ini, mahasiswa dapat memahami dasar pembuatan aplikasi React yang responsif dan real-time seperti dashboard monitoring pada sistem industri modern.

D.2 Keterkaitan dengan Teknik Industri

Materi pada praktikum ini memiliki keterkaitan yang cukup erat dengan dunia Teknik Industri, khususnya dalam bidang monitoring produksi, sistem informasi industri, dan dashboard manufaktur. Konsep state dan real-time update pada React dapat digunakan untuk menampilkan data produksi yang selalu berubah secara langsung, seperti jumlah output produksi, status mesin, hingga performa operator. Dengan tampilan yang interaktif dan otomatis diperbarui, proses monitoring menjadi lebih cepat dan memudahkan pengambilan keputusan di lingkungan industri.

Contoh penerapannya dapat dilihat pada dashboard monitoring pabrik yang menampilkan kondisi mesin secara real-time, misalnya status running, maintenance, atau stop. Selain itu, proyek mini kalkulator OEE juga sangat relevan karena OEE merupakan salah satu indikator penting dalam evaluasi efektivitas mesin produksi di industri manufaktur. Sistem seperti ini membantu perusahaan dalam memantau efisiensi produksi, mendeteksi masalah lebih cepat, dan meningkatkan produktivitas kerja secara keseluruhan.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat Komponen Counter Produksi

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
import React, { useState } from 'react';

function CounterProduksi() {
  const [jumlah, setJumlah] = useState(0);
  const [target, setTarget] = useState(100);

  const tambahProduksi = () => {
    setJumlah(jumlah + 1);
  };

  const reset = () => {
    setJumlah(0);
  };

  return (
    <div className="card shadow-sm p-3 mb-3">
      <div className="card-body">
```

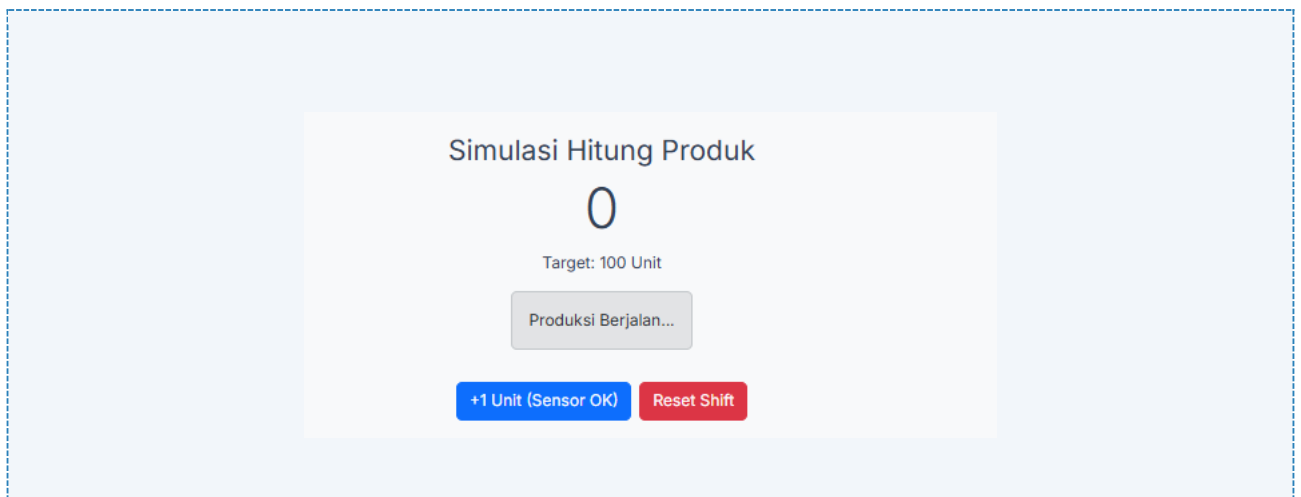
```
<h3 className="card-title">Simulasi Hitung Produk</h3>
<h1 className="display-2 fw-bold">{jumlah}</h1>
<p className="text-muted">Target: {target} Unit</p>

    {jumlah >= target ? (
      <div className="alert alert-success">Target
Tercapai!</div>
    ) : (
      <div className="alert alert-info">Produksi
Berjalan...</div>
    )}

    <button className="btn btn-primary me-2"
onClick={tambahProduksi}>
      +1 Unit (Sensor OK)
    </button>
    <button className="btn btn-outline-danger"
onClick={reset}>
      Reset Shift
    </button>
  </div>
</div>
);
}

export default CounterProduksi;
```

Screenshot Hasil:



Gambar E.1 – Kode dari Langkah 1

E.2 Langkah 2 – Menggunakan useEffect untuk Real-time Clock

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */
import React, { useState, useEffect } from 'react';

function JamDigital() {
  // State untuk menyimpan waktu saat ini
  const [waktu, setWaktu] = useState(new Date());

  // useEffect untuk menjalankan timer otomatis
  useEffect(() => {
    // Membuat interval agar waktu update setiap 1 detik (1000 ms)
    const timerID = setInterval(() => {
      setWaktu(new Date());
    }, 1000);

    // Fungsi pembersihan (Cleanup) agar memory tidak bocor
    return () => {
      clearInterval(timerID);
    };
  }, []); // Array kosong [] artinya efek ini hanya jalan sekali saat komponen muncul

  return (
    <div className="card shadow-sm p-3 bg-light">
      <div className="card-body text-center">
        <h5 className="text-muted mb-2">Waktu Sistem
        Produksi</h5>
        <h1 className="display-4 fw-bold text-primary">
          {waktu.toLocaleTimeString()}
        </h1>
        <p className="text-secondary mb-0">
          {waktu.toLocaleDateString('id-ID', {
            weekday: 'long',
            year: 'numeric',
            month: 'long',
            day: 'numeric'
          })}
        </p>
      </div>
    </div>
  );
}

// Pastikan baris export ini ada agar tidak error di App.jsx
export default JamDigital;
```

Screenshot Hasil:



Waktu Sistem Server: 15.44.00

Gambar E.2 – Output dari Langkah 2

E.3 Langkah 3 – Contoh Form Input State

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
import React, { useState } from 'react';

function KartuMesin(props) {

  const namaMesin = props.nama;
  const status = props.status;
  const produksi = props.produksi;

  // State lokal
  const [statusLokal, setStatusLokal] = useState(status);

  // Badge berdasarkan status lokal
  let badgeColor = 'bg-secondary';

  if (statusLokal === 'Running') badgeColor = 'bg-success';
  if (statusLokal === 'Stop') badgeColor = 'bg-danger';
  if (statusLokal === 'Maintenance') badgeColor = 'bg-
warning';

  return (
    <div className="card shadow-sm p-3 mb-3">
      <div className="card-body">

        <h5 className="card-title">{namaMesin}</h5>

        <span className={`badge ${badgeColor}`}>
          {statusLokal}
        </span>

        <hr />

        <p>
```

```

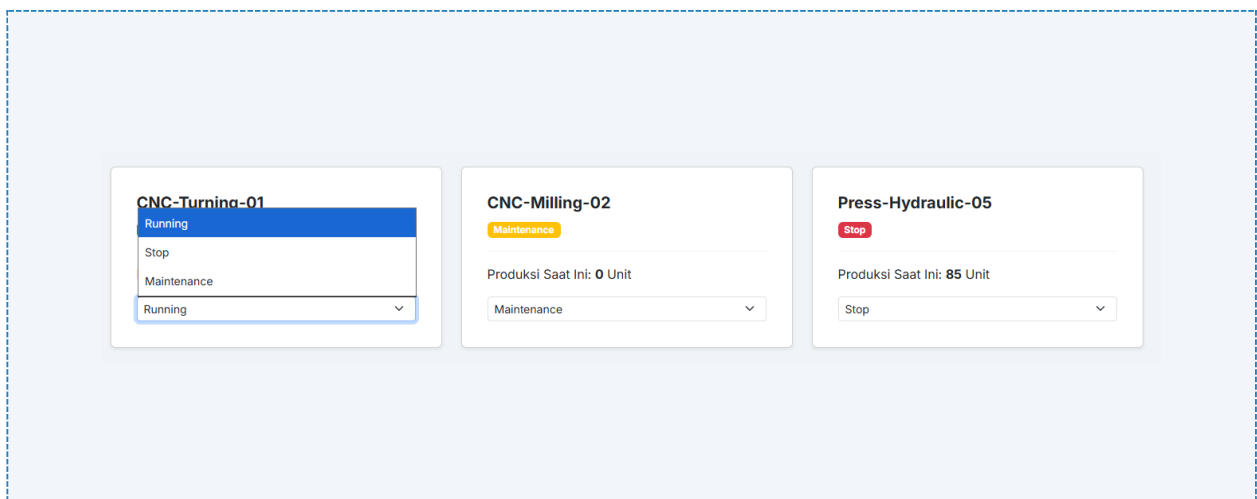
        Produksi Saat Ini: <strong>{produksi}</strong> Unit
      </p>

      {/* Select status */}
      <div className="mt-3">
        <select
          className="form-select form-select-sm"
          value={statusLokal}
          onChange={(e) => setStatusLokal(e.target.value)}
        >
          <option value="Running">Running</option>
          <option value="Stop">Stop</option>
          <option value="Maintenance">Maintenance</option>
        </select>
      </div>
    </div>
  </div>
);
}

export default KartuMesin;

```

Screenshot Hasil:



Gambar E.3 – Output dari Langkah 3

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Dependency Array useEffect

Pertanyaan / Instruksi Latihan:

Modifikasi `JamDigital`. Tambahkan input text untuk memasukkan nama kota. Gunakan `useEffect` untuk mengubah `document.title` menjadi "Jam [Nama Kota]". Gunakan state kota sebagai dependency array `[kota]`.

Jawaban / Kode Solusi:

```
import React, { useState, useEffect } from "react";

function JamDigital() {

  // State waktu
  const [waktu, setWaktu] = useState(new Date());

  // State kota
  const [kota, setKota] = useState("Yogyakarta");

  // useEffect untuk jam digital
  useEffect(() => {

    // Membuat interval timer
    const timerID = setInterval(() => {
      setWaktu(new Date());
    }, 1000);

    // Cleanup function
    return () => {
      clearInterval(timerID);
    };

  }, []);

  // useEffect untuk mengubah title browser
  useEffect(() => {
    document.title = `Jam ${kota}`;
  }, [kota]);

  return (
    <div className="alert alert-info text-center p-4">

      <h3 className="mb-3">
        Jam Digital Dunia Industri
      </h3>

      { /* Input Nama Kota */ }
      <div className="mb-4">
        <input
          type="text"
          className="form-control"
        >
      </div>
    </div>
  );
}
```

```
        placeholder="Masukkan Nama Kota"
        value={kota}
        onChange={ (e) => setKota(e.target.value) }
      />
    </div>

    { /* Tampilan Kota */ }
    <h5 className="mb-3">
      Kota: <strong>{kota}</strong>
    </h5>

    { /* Tampilan Jam */ }
    <h2>
      {waktu.toLocaleTimeString()}
    </h2>

  </div>
);
}

export default JamDigital;
```



Gambar F.1 – Output dari Latihan 1

F.2 Latihan 2 – Conditional Rendering

Pertanyaan / Instruksi Latihan:

Pada `CounterProduksi`, buat tombol "Emergency Stop". Saat diklik, ubah state status menjadi "EMERGENCY", tombol "+1" menjadi disabled (`disabled`), dan tampilkan pesan merah.

Jawaban / Kode Solusi:

```
import React, { useState } from "react";

function CounterProduksi() {

  // State jumlah produksi
  const [jumlah, setJumlah] = useState(0);

  // State target produksi
  const [target, setTarget] = useState(100);

  // State status sistem
  const [status, setStatus] = useState("NORMAL");

  // Fungsi menambah produksi
  const tambahProduksi = () => {
    setJumlah(jumlah + 1);
  };

  // Fungsi reset
  const reset = () => {
    setJumlah(0);
    setStatus("NORMAL");
  };

  // Fungsi Emergency Stop
  const emergencyStop = () => {
    setStatus("EMERGENCY");
  };

  return (
    <div className="text-center p-4 border rounded bg-light">

      <h3>Simulasi Hitung Produk</h3>

      { /* Jumlah produksi */ }
      <h1 className="display-4">
        {jumlah}
      </h1>

      { /* Target */ }
      <p>
        Target: {target} Unit
      </p>

      { /* Status Sistem */ }
      <h5 className="mb-3">
        Status Sistem :
        { " " }
      </h5>
    </div>
  );
}
```



```

    <span
      className={
        status === "EMERGENCY"
          ? "text-danger"
          : "text-success"
      }
    >
      {status}
    </span>
  </h5>

  {/* Conditional Rendering */}
  {status === "EMERGENCY" ? (

    <div className="alert alert-danger d-inline-block">
      SISTEM DARURAT AKTIF!
    </div>

  ) : jumlah >= target ? (

    <div className="alert alert-success d-inline-block">
      Target Tercapai!
    </div>

  ) : (

    <div className="alert alert-secondary d-inline-block">
      Produksi Berjalan...
    </div>

  )}

  {/* Tombol */}
  <div className="mt-4">

    {/* Tombol tambah produksi */}
    <button
      className="btn btn-primary me-2"
      onClick={tambahProduksi}
      disabled={status === "EMERGENCY"}
    >
      +1 Unit (Sensor OK)
    </button>

    {/* Tombol reset */}
    <button
      className="btn btn-danger me-2"
      onClick={reset}

```

```
>
  Reset Shift
</button>

{/* Tombol emergency */}
<button
  className="btn btn-dark"
  onClick={emergencyStop}
>
  Emergency Stop
</button>

</div>

</div>
);
}

export default CounterProduksi;
```



Gambar F.2 – Output dari Latihan 2

F.3 Tugas Proyek Mini – Kalkulator OEE Sederhana

Deskripsi proyek mini:

- Buat komponen form input: `Plan Time`, `Run Time`, `Total Parts`, `Good Parts`.
- Gunakan State untuk menyimpan nilai-nilai ini.
- Hitung OEE secara otomatis (Real-time) setiap kali input berubah.
- Rumus OEE = (Availability x Performance x Quality).
- Tampilkan hasil OEE dalam persen. Jika OEE < 50%, warnanya merah. Jika > 85%, warnanya hijau.

Penjelasan pendekatan/solusi yang digunakan:

Solusi yang digunakan pada Proyek Mini Kalkulator OEE adalah pendekatan real-time reactive interface menggunakan React.js. Sistem dirancang agar setiap data yang dimasukkan pengguna pada form input seperti Plan Time, Run Time, Total Parts, dan Good Parts langsung disimpan ke dalam state menggunakan useState. Ketika nilai input berubah, React secara otomatis melakukan render ulang sehingga hasil perhitungan OEE dapat diperbarui secara langsung tanpa perlu menekan tombol tambahan.

Variabel kalkulasi OEE akan menghitung ulang berdasarkan rumus standar ($\text{Availability} \times \text{Performance} \times \text{Quality}$) dan langsung memperbarui tampilan persentase serta warna indikator (merah/kuning/hijau) tanpa perlu menekan tombol “Hitung”. Selain itu, sistem juga menerapkan conditional rendering untuk menampilkan indikator visual sesuai hasil perhitungan. Jika nilai OEE rendah maka sistem menampilkan warna merah sebagai tanda performa produksi kurang baik, sedangkan nilai tinggi akan ditampilkan dengan warna hijau sebagai indikator performa mesin yang optimal. Pendekatan ini membuat dashboard menjadi lebih interaktif, responsif, dan mudah dipahami pengguna.

Kode Utama:

```
import React, { useState } from "react";

function KalkulatorOEE() {

  // ===== STATE =====

  // Waktu produksi yang direncanakan
  const [planTime, setPlanTime] = useState(0);

  // Waktu mesin berjalan
  const [runTime, setRunTime] = useState(0);

  // Total seluruh produk
  const [totalParts, setTotalParts] = useState(0);

  // Total produk baik
  const [goodParts, setGoodParts] = useState(0);

  // ===== PERHITUNGAN =====

  // Availability
  const availability =
    planTime > 0
      ? runTime / planTime
      : 0;

  // Performance
  const performance =
    planTime > 0
```

```
    ? totalParts / planTime
    : 0;

// Quality
const quality =
  totalParts > 0
    ? goodParts / totalParts
    : 0;

// OEE
const oee =
  availability *
  performance *
  quality *
  100;

// ===== WARNA OEE =====

let warnaOEE = "text-warning";

if (oee < 50) {
  warnaOEE = "text-danger";
}

if (oee > 85) {
  warnaOEE = "text-success";
}

return (
  <div className="container mt-4">

    <div className="card shadow p-4">

      {/* ===== JUDUL ===== */}
      <h2 className="text-center mb-4">
        Kalkulator OEE Sederhana
      </h2>

      {/* ===== INPUT ===== */}
      <div className="row g-4">

        {/* Plan Time */}
        <div className="col-md-6">
          <label className="form-label">
            Plan Time
          </label>

          <input
```

```
        type="number"
        className="form-control"
        placeholder="Masukkan Plan Time"
        value={planTime}
        onChange={ (e) =>
            setPlanTime (Number(e.target.value))
        }
    />
</div>

{/* Run Time */}
<div className="col-md-6">
    <label className="form-label">
        Run Time
    </label>

    <input
        type="number"
        className="form-control"
        placeholder="Masukkan Run Time"
        value={runTime}
        onChange={ (e) =>
            setRunTime (Number(e.target.value))
        }
    />
</div>

{/* Total Parts */}
<div className="col-md-6">
    <label className="form-label">
        Total Parts
    </label>

    <input
        type="number"
        className="form-control"
        placeholder="Masukkan Total Parts"
        value={totalParts}
        onChange={ (e) =>
            setTotalParts (Number(e.target.value))
        }
    />
</div>

{/* Good Parts */}
<div className="col-md-6">
    <label className="form-label">
        Good Parts
```

```

        </label>

        <input
            type="number"
            className="form-control"
            placeholder="Masukkan Good Parts"
            value={goodParts}
            onChange={ (e) =>
                setGoodParts (Number (e.target.value))
            }
        />
    </div>

</div>

{/* ===== HASIL ===== */}
<div className="mt-5">

    <h4 className="mb-3">
        Hasil Perhitungan
    </h4>

    {/* Availability */}
    <p>
        <strong>Availability :</strong>{" "}
        {(availability * 100).toFixed(2)}%
    </p>

    {/* Performance */}
    <p>
        <strong>Performance :</strong>{" "}
        {(performance * 100).toFixed(2)}%
    </p>

    {/* Quality */}
    <p>
        <strong>Quality :</strong>{" "}
        {(quality * 100).toFixed(2)}%
    </p>

    <hr />

    {/* OEE */}
    <h3>
        Nilai OEE :
    </h3>

    <h1 className={warnaOEE}>

```

```

        {oee.toFixed(2)}%
      </h1>

      {/* Conditional Rendering */}
      {oee < 50 ? (

        <div className="alert alert-danger mt-3">
          OEE Rendah! Perlu Evaluasi Mesin Produksi.
        </div>

      ) : oee > 85 ? (

        <div className="alert alert-success mt-3">
          OEE Sangat Baik! Produksi Optimal.
        </div>

      ) : (

        <div className="alert alert-warning mt-3">
          OEE Cukup Baik, Namun Masih Bisa Ditingkatkan.
        </div>

      )}

    </div>

  </div>

</div>
);
}

export default KalkulatorOEE;

```

Kode final App.jsx

```

import React, { useState } from 'react';
import './App.css';

// ===== IMPORT KOMPONEN PERTEMUAN 9 =====
import KartuMesin from './Komponen/KartuMesin';
import KartuMesinLat1 from './Komponen/KartuMesinLat1';
import KartuMesinLat2 from './Komponen/KartuMesinLat2';
import KartuKaryawan from './Komponen/KartuKaryawan';

// ===== IMPORT KOMPONEN PERTEMUAN 10 =====
import CounterProduksi from './Komponen/CounterProduksi';
import JamDigital from './Komponen/JamDigital';
import KalkulatorOEE from './Komponen/KalkulatorOEE';

```

```

function App() {

  // ===== STATE NAVIGASI =====
  const [halaman, setHalaman] = useState('Praktikum');

  return (
    <div className="container mt-5 mb-5">

      /* =====
*/}
      /* ===== HEADER =====
*/}
      /* =====
*/}

      <h2 className="text-center mb-2">
        Monitoring Lini Produksi
      </h2>

      <p className="text-center text-muted mb-5">
        Radhitya Adha Fadhlur Rohman (23051430005)
      </p>

      /* =====
*/}
      /* ===== NAVIGASI =====
*/}
      /* =====
*/}

      <div className="d-flex justify-content-center gap-3 mb-5
flex-wrap">

        /* Praktikum */}
        <button
          className={`btn tombol-navigasi ${
            halaman === 'Praktikum'
              ? 'btn-primary'
              : 'btn-outline-primary'
          }`}
          onClick={() => setHalaman('Praktikum')}
        >
          Versi Praktikum
        </button>

        /* Latihan 1 */}
        <button
          className={`btn tombol-navigasi ${

```



```

        halaman === 'latihan1'
        ? 'btn-info text-white'
        : 'btn-outline-info'
    `}`
    onClick={() => setHalaman('latihan1')}
>
    Latihan 1
</button>

{/* Latihan 2 */}
<button
    className={`btn tombol-navigasi ${
        halaman === 'latihan2'
        ? 'btn-success'
        : 'btn-outline-success'
    }`}
    onClick={() => setHalaman('latihan2')}
>
    Latihan 2
</button>

{/* Tugas Mini */}
<button
    className={`btn tombol-navigasi ${
        halaman === 'tugas'
        ? 'btn-warning text-dark'
        : 'btn-outline-warning'
    }`}
    onClick={() => setHalaman('tugas')}
>
    Tugas Proyek Mini
</button>

{/* Pertemuan 10 */}
<button
    className={`btn tombol-navigasi ${
        halaman === 'state'
        ? 'btn-danger'
        : 'btn-outline-danger'
    }`}
    onClick={() => setHalaman('state')}
>
    Pertemuan 10
</button>

</div>

{/* =====
*/}

```

```

    { /* ===== 1. PRAKTIKUM =====
*/ }

{ /* =====
*/ }

{ halaman === 'Praktikum' && (
    <div>

        <h4 className="text-center mb-4">
            1. Versi Praktikum (Props)
        </h4>

        <div className="row g-4">

            <div className="col-md-4">
                <KartuMesin
                    nama="CNC-Turning-01"
                    status="Running"
                    produksi={150}
                />
            </div>

            <div className="col-md-4">
                <KartuMesin
                    nama="CNC-Milling-02"
                    status="Maintenance"
                    produksi={0}
                />
            </div>

            <div className="col-md-4">
                <KartuMesin
                    nama="Press-Hydraulic-05"
                    status="Stop"
                    produksi={85}
                />
            </div>

        </div>

    </div>
) }

{ /* =====
*/ }

{ /* ===== 2. LATIHAN 1 =====
*/ }

{ /* =====
*/ }

```

```

{halaman === 'latihan1' && (
  <div>

    <h4 className="text-center mb-4 text-info">
      2. Latihan 1 (Destructuring Props)
    </h4>

    <div className="row g-4">

      <div className="col-md-4">
        <KartuMesinLat1
          nama="CNC-Grinding-03"
          status="Maintenance"
          produksi={50}
        />
      </div>

      <div className="col-md-4">
        <KartuMesinLat1
          nama="CNC-Drilling-04"
          status="Running"
          produksi={210}
        />
      </div>

      <div className="col-md-4">
        <KartuMesinLat1
          nama="Lathe-Machine-06"
          status="Stop"
          produksi={0}
        />
      </div>

    </div>

  </div>
)}

{ /* =====
*/}

{ /* ===== 3. LATIHAN 2 =====
*/}

{ /* =====
*/}

{halaman === 'latihan2' && (
  <div>

```

```

<h4 className="text-center mb-4 text-success">
  3. Latihan 2 (Default Props)
</h4>

<div className="row g-4">

  <div className="col-md-4">
    <KartuMesinLat2
      nama="Assembly-Line-01"
      status="Stop"
    />
  </div>

  <div className="col-md-4">
    <KartuMesinLat2
      nama="Packaging-Unit-02"
      status="Running"
      produksi={175}
    />
  </div>

  <div className="col-md-4">
    <KartuMesinLat2
      nama="Inspection-Unit-03"
      status="Maintenance"
    />
  </div>

</div>

</div>
)}

{ /* =====
*/ }

{ /* ===== 4. TUGAS MINI =====
*/ }

{ /* =====
*/ }

{halaman === 'tugas' && (
  <div>

    <h4 className="text-center mb-4 text-warning">
      4. Tugas Proyek Mini (Data Karyawan)
    </h4>

    <div className="row g-4">

```

```

        <div className="col-md-4">
            <KartuKaryawan
                nama="Bambang Subejo"
                jabatan="Manager"
                bagian="Produksi"
            />
        </div>

        <div className="col-md-4">
            <KartuKaryawan
                nama="Fadhil Alamsyah"
                jabatan="Operator"
                bagian="Lini Perakitan 1"
            />
        </div>

        <div className="col-md-4">
            <KartuKaryawan
                nama="Adhe Surakjat"
                jabatan="QC"
                bagian="Quality Assurance"
            />
        </div>

    </div>

</div>
)}

{/* =====
*/}

{/* ===== 5. PERTEMUAN 10 =====
*/}

{/* =====
*/}

{halaman === 'state' && (
    <div>

        <h4 className="text-center mb-4 text-danger">
            5. Pertemuan 10 (State, Hooks, Conditional Rendering)
        </h4>

        {/* ===== JAM DIGITAL ===== */}
        <div className="mb-5">
            <JamDigital />
        </div>
    </div>

```

```

        { /* ===== COUNTER PRODUKSI =====
*/}

        <div className="mb-5">
            <CounterProduksi />
        </div>

        { /* ===== KARTU MESIN DINAMIS
===== */}
        <div className="mb-5">

            <h4 className="text-center mb-4">
                Monitoring Status Mesin
            </h4>

            <div className="row g-4">

                <div className="col-md-4">
                    <KartuMesin
                        nama="CNC-Turning-01"
                        status="Running"
                        produksi={150}
                    />
                </div>

                <div className="col-md-4">
                    <KartuMesin
                        nama="CNC-Milling-02"
                        status="Maintenance"
                        produksi={0}
                    />
                </div>

                <div className="col-md-4">
                    <KartuMesin
                        nama="Press-Hydraulic-05"
                        status="Stop"
                        produksi={85}
                    />
                </div>

            </div>

        </div>

        { /* ===== KALKULATOR OEE =====
*/}

        <div className="mt-5">

            <h4 className="text-center mb-4 text-primary">

```

```

        Proyek Mini - Kalkulator OEE
    </h4>

    <KalkulatorOEE />

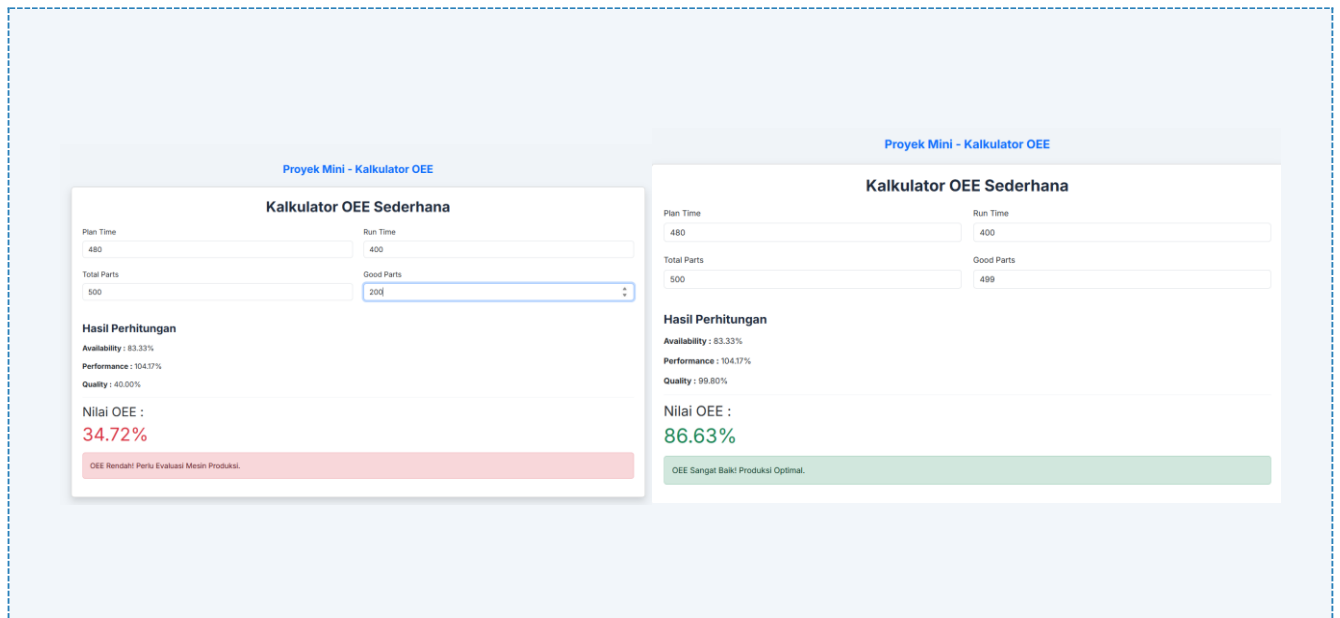
  </div>

</div>
)}

</div>
);
}

export default App;

```



Gambar F.3 – Output dari Proyek Mini Industri

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang diperoleh sudah sesuai dengan teori dan konsep React.js yang dipelajari pada modul ini, khususnya mengenai penggunaan state, hooks, dan conditional rendering. Hal tersebut terlihat pada implementasi useState yang mampu menyimpan serta memperbarui data secara real-time ketika pengguna melakukan input atau menekan tombol tertentu. Contohnya pada komponen CounterProduksi, nilai produksi langsung bertambah setiap kali tombol “+1 Unit” ditekan tanpa perlu melakukan refresh halaman. Selain itu, ketika tombol “Emergency Stop” ditekan, status sistem berubah menjadi “EMERGENCY”, tombol produksi otomatis disabled, dan muncul alert merah sesuai kondisi yang telah dibuat pada conditional rendering.

Hal ini menunjukkan bahwa konsep perubahan state dan render ulang otomatis pada React berjalan dengan baik sesuai teori.

Pada komponen JamDigital, penggunaan `useEffect` juga berjalan sesuai konsep yang dipelajari. Waktu sistem dapat diperbarui setiap detik secara otomatis menggunakan interval timer, kemudian title browser berubah mengikuti nama kota yang diinput pengguna melalui dependency array [kota]. Ini membuktikan bahwa `useEffect` berhasil digunakan untuk menjalankan side effect setelah proses render komponen berlangsung. Selain itu, pada proyek mini Kalkulator OEE, hasil perhitungan dapat berubah secara langsung ketika nilai input diubah. Variabel kalkulasi OEE menghitung ulang berdasarkan rumus standar ($\text{Availability} \times \text{Performance} \times \text{Quality}$) dan langsung memperbarui persentase beserta warna indikator tanpa tombol “Hitung”. Warna merah muncul ketika nilai OEE rendah dan warna hijau muncul ketika nilai tinggi, sehingga implementasi conditional rendering dan real-time rendering telah sesuai dengan teori yang dipelajari pada modul praktikum.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Nilai input terbaca sebagai string sehingga perhitungan OEE tidak akurat	Menggunakan <code>Number(e.target.value)</code> agar input terbaca sebagai angka
Tombol produksi masih bisa ditekan saat status emergency aktif	Menambahkan properti <code>disabled={status === "EMERGENCY"}</code> pada tombol produksi

G.3 Kaitan dengan Penerapan di Industri

Keterampilan yang dipelajari pada modul ini dapat diterapkan dalam pembuatan dashboard monitoring produksi dan sistem informasi industri berbasis web. Konsep state dan real-time update sangat berguna untuk menampilkan data produksi yang terus berubah, seperti jumlah output, status mesin, maupun performa operator secara langsung. Dengan sistem seperti ini, pengawas produksi dapat memantau kondisi lini produksi secara lebih cepat dan efisien tanpa harus melakukan pengecekan manual.

Contoh penerapan nyata dapat ditemukan pada dashboard monitoring pabrik manufaktur. Status mesin dapat berubah secara real-time menjadi running, maintenance, atau stop sesuai kondisi aktual di lapangan. Selain itu, perhitungan OEE seperti pada proyek mini juga banyak digunakan di industri untuk mengukur efektivitas mesin produksi. Jika nilai OEE rendah, sistem dapat memberikan indikator peringatan sehingga perusahaan dapat segera melakukan evaluasi terhadap downtime, kualitas produk, atau performa produksi agar produktivitas tetap optimal.

H. KESIMPULAN

1	Praktikum ini berhasil menerapkan konsep useState untuk menyimpan dan mengelola data dinamis seperti jumlah produksi, status sistem, dan data input pengguna secara real-time.
2	Conditional rendering berhasil digunakan untuk menampilkan tampilan berbeda berdasarkan kondisi tertentu, seperti status emergency dan indikator warna hasil OEE.
3	Proyek mini Kalkulator OEE dapat menghitung nilai OEE secara otomatis berdasarkan input pengguna dan menampilkan hasil persentase beserta indikator performa mesin secara interaktif

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	McConnell, S. (2004). <i>Code Complete: A Practical Handbook of Software Construction</i> . Microsoft Press.
[3]	Meta Platforms, Inc. (2024). <i>React Documentation: Thinking in React</i> . Diakses dari https://react.dev

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
Radhitya Adha Fadhlur Rohman NIM: 23051430005	Maessa Andrea Vallenia NIM: 23051430044	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

Modul 11 React Routing & Multi-Page App + Integrasi API

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 11 (Pertemuan 11)
Topik Utama	React Routing & Multi-Page App + Integrasi API
Sub-Topik	Single Page Application (SPA) Navigation, React Router DOM, dan Fetch API dalam React.
Durasi Praktikum	340 Menit
Tanggal Praktikum	1 Mei 2026
Nama Mahasiswa	Radhitya Adha Fadhlur Rohman
NIM	23051430005
Kelas	A2

B. TUJUAN PEMBELAJARAN

1	Memahami konsep SPA (Single Page Application).
2	Mampu mengatur navigasi antar halaman tanpa reload browser menggunakan `react router-dom`.
3	Menerapkan `fetch` API di dalam `useEffect` untuk mengambil data eksternal.
4	Membuat struktur aplikasi multi-halaman (Dashboard, Inventori, Settings).

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	File `index.html`	Versi Terbaru	Hasil pekerjaan pertemuan sebelumnya
4	Ekstensi VS Code: LiveServer	Versi terbaru	Melihat perubahan web secara real-time.

5	Javascript	Versi terbaru	Membuat halaman web menjadi lebih interaktif dan fungsional
6	Node.js	Versi Terbaru	<i>runtime environment</i> JavaScript

D. DASAR TEORI

React Router DOM merupakan library pada React yang digunakan untuk membuat navigasi antar halaman pada aplikasi berbasis Single Page Application (SPA). Dengan React Router, perpindahan halaman dapat dilakukan tanpa melakukan reload browser sehingga aplikasi menjadi lebih cepat dan responsif. Konsep utama yang digunakan pada modul ini meliputi Routes, Route, dan Link untuk menentukan halaman mana yang akan ditampilkan berdasarkan URL yang diakses pengguna.

Selain navigasi dasar, modul ini juga membahas dynamic route yang digunakan untuk menampilkan halaman detail berdasarkan parameter tertentu pada URL. Dynamic route biasanya ditulis menggunakan format seperti `/inventori/:id`, di mana nilai id dapat berubah sesuai data yang dipilih. React menyediakan hook `useParams()` untuk mengambil parameter tersebut sehingga aplikasi dapat menampilkan data yang berbeda pada setiap halaman detail.

Modul ini juga mempelajari penggunaan state loading dan fetch API untuk mengambil data dari server secara asynchronous. Loading state digunakan untuk memberikan informasi kepada pengguna bahwa data sedang diproses atau dimuat. Dengan adanya loading state, tampilan aplikasi menjadi lebih interaktif dan meningkatkan pengalaman pengguna karena sistem memberikan feedback saat proses pengambilan data berlangsung.

D.1 Konsep Utama

Konsep utama pada modul ini adalah penggunaan React Router DOM untuk membuat navigasi halaman pada aplikasi React. Dengan React Router, perpindahan halaman seperti dashboard, inventori, dan laporan kualitas dapat dilakukan tanpa reload browser sehingga aplikasi terasa lebih cepat dan nyaman digunakan.

Modul ini juga mempelajari dynamic route, yaitu route yang dapat menampilkan halaman detail berdasarkan ID tertentu pada URL. Selain itu, digunakan juga `useState` dan `useEffect` untuk mengambil data dari API serta menampilkan loading saat data masih diproses agar pengguna mengetahui bahwa sistem sedang memuat data.

D.2 Keterkaitan dengan Teknik Industri

Materi pada modul ini relevan dengan Teknik Industri karena sistem navigasi dan pengolahan data pada aplikasi web banyak digunakan dalam proses monitoring industri dan manajemen produksi. Contohnya pada sistem inventori bahan baku, monitoring status mesin, hingga laporan kualitas produksi yang memerlukan tampilan data secara cepat dan terstruktur.

Dalam dunia industri, aplikasi berbasis web seperti ini dapat digunakan untuk membantu operator maupun manajer memantau kondisi produksi secara realtime. Misalnya, halaman

inventori digunakan untuk mengecek ketersediaan bahan baku, sedangkan halaman laporan kualitas digunakan untuk melihat jumlah produk cacat agar proses evaluasi dan pengambilan keputusan dapat dilakukan lebih cepat dan efisien.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

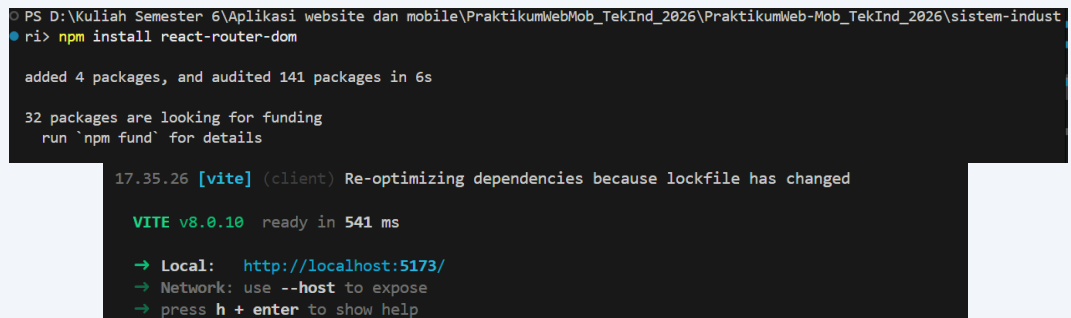
E.1 Langkah 1 – Setup Router

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'

import './index.css'
import App from './App2.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
)
```

Screenshot Hasil:



```
PS D:\Kuliah Semester 6\Aplikasi website dan mobile\PraktikumWebMob_TekInd_2026\PraktikumWeb-Mob_TekInd_2026\sistem-indust
ri> npm install react-router-dom

added 4 packages, and audited 141 packages in 6s

32 packages are looking for funding
  run `npm fund` for details

17.35.26 [vite] (client) Re-optimizing dependencies because lockfile has changed

VITE v8.0.10 ready in 541 ms

  → Local:   http://localhost:5173/
  → Network: use --host to expose
  → press h + enter to show help
```

Gambar E.1 – Kode dari Langkah 1

E.2 Langkah 2 – Membuat Halaman (Pages)

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */
=== Dashboard.jsx ===

import React from 'react'
import { Link } from 'react-router-dom'

function Dashboard() {
```

```
return (  
  <div className="p-5">  
    <h1>Dashboard Utama Pabrik</h1>  
  
    <p>  
      Selamat datang di sistem monitoring terpadu.  
    </p>  
  
    <Link  
      to="/inventori"  
      className="btn btn-primary"  
    >  
      Ke Halaman Inventori  
    </Link>  
  
  </div>  
  
)  
}  
  
export default Dashboard  
  
=== Inventori.jsx ===  
import React, { useState, useEffect } from 'react'  
import { Link } from 'react-router-dom'  
  
function Inventori() {  
  
  const [products, setProducts] = useState([])  
  
  // Fetch data saat halaman dibuka  
  useEffect(() => {  
  
    fetch('https://jsonplaceholder.typicode.com/posts')  
  
    .then(res => res.json())  
  
    .then(data => {  
  
      // Ambil 5 data pertama  
      setProducts(data.slice(0, 5))  
  
    })  
  
    .catch(err => console.log(err))  
  
  }, [])  
  
  return (  
  
    <div className="container mt-4">  
  
      <h1>Data Inventori Bahan Baku</h1>  
  
      <Link
```

```

        to="/"
        className="btn btn-secondary mb-3"
      >
        Kembali ke Dashboard
      </Link>

      <table className="table table-striped">

        <thead>
          <tr>
            <th>ID Item</th>
            <th>Nama Bahan</th>
            <th>Status Supplier</th>
          </tr>
        </thead>

        <tbody>

          {products.map((item) => (

            <tr key={item.id}>

              <td>{item.id}</td>

              <td>{item.title}</td>

              <td>
                <span className="badge bg-success">
                  Available
                </span>
              </td>

            </tr>

          ))}

        </tbody>

      </table>

    </div>

  )
}

export default Inventori

=== NotFound.jsx ===

import React from 'react'
import { Link } from 'react-router-dom'

function NotFound() {

  return (

    <div className="text-center mt-5">

```

```

    <h1 className="display-1">
      404
    </h1>

    <p>
      Halaman tidak ditemukan dalam sistem pabrik.
    </p>

    <Link
      to="/"
      className="btn btn-dark"
    >
      Kembali ke Home
    </Link>

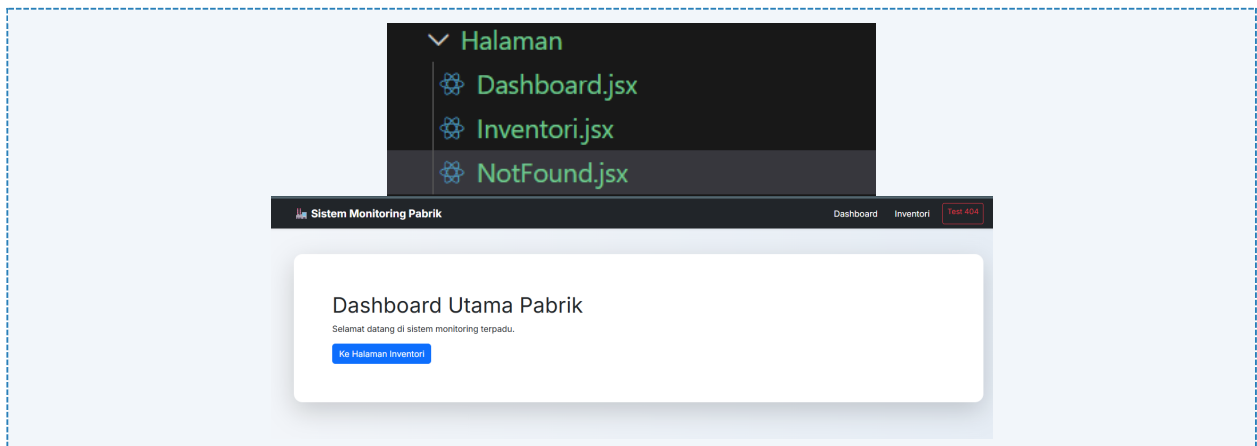
  </div>

)
}

export default NotFound

```

Screenshot Hasil:



Gambar E.2 – Output dari Langkah 2

E.3 Langkah 3 – Mengatur Route di App.jsx

```

/* ===== KODE / OUTPUT LANGKAH 3 ===== */
import React from 'react'

import {
  Routes,
  Route,
  Link
} from 'react-router-dom'

import Dashboard from './Halaman/Dashboard'
import Inventori from './Halaman/Inventori'
import NotFound from './Halaman/NotFound'

```



```
// ===== NAVBAR =====
function Navbar() {

    return (

        <nav className="navbar navbar-expand-lg navbar-dark bg-dark shadow-sm">

            <div className="container">

                {/* Logo / Judul */}
                <Link
                    className="navbar-brand fw-bold"
                    to="/"
                >
                     Sistem Monitoring Pabrik
                </Link>

                {/* Menu */}
                <div className="navbar-nav ms-auto d-flex flex-row gap-3">

                    <Link
                        className="nav-link text-white"
                        to="/"
                    >
                        Dashboard
                    </Link>

                    <Link
                        className="nav-link text-white"
                        to="/inventori"
                    >
                        Inventori
                    </Link>

                    {/* Tombol Test 404 */}
                    <Link
                        className="btn btn-outline-danger btn-sm"
                        to="/halaman-error"
                    >
                        Test 404
                    </Link>

                </div>

            </div>

        </nav>

    )
}

// ===== APP =====
function App() {

    return (
```

```
<div
  style={{
    minHeight: '100vh',
    background:
      'linear-gradient(to right, #f5f7fa, #e4ecf5)'
  }}
>

{/* Navbar */}
<Navbar />

{/* Container Isi */}
<div className="container py-5">

  {/* Card Wrapper */}
  <div
    className="card border-0 shadow-lg rounded-4 p-4"
  >

    <Routes>

      {/* Dashboard */}
      <Route
        path="/"
        element={<Dashboard />}
      />

      {/* Inventori */}
      <Route
        path="/inventori"
        element={<Inventori />}
      />

      {/* 404 */}
      <Route
        path="*"
        element={<NotFound />}
      />

    </Routes>

  </div>

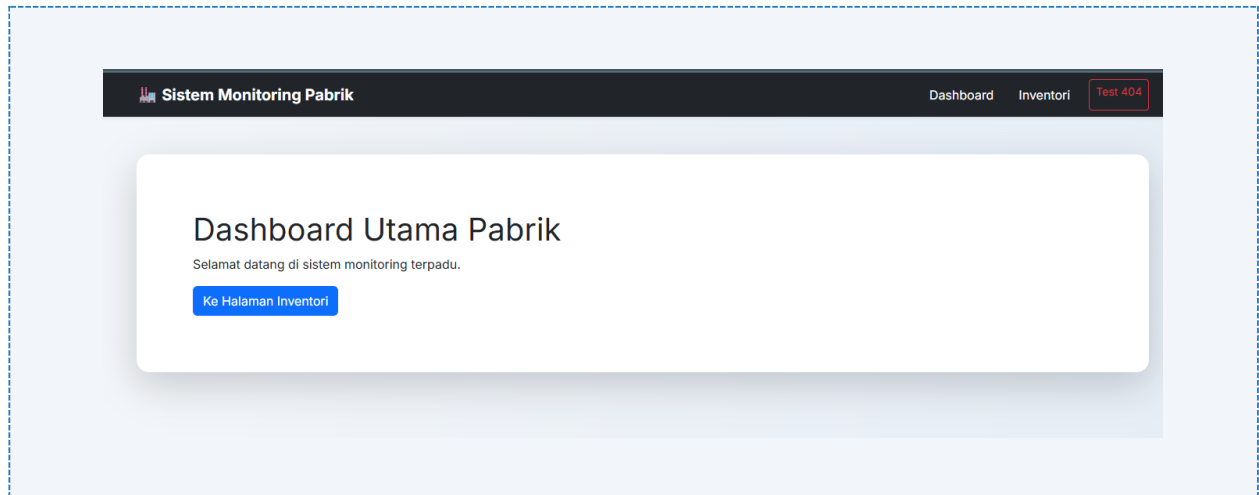
</div>

</div>

)
}

export default App
```

Screenshot Hasil:



Gambar E.3 – Output dari Langkah 3

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Dynamic Route

Pertanyaan / Instruksi Latihan:

Tambahkan Route untuk detail item. Misal path `/inventori/:id`. Di halaman Inventori, ubah "Nama Bahan" menjadi Link ke halaman detail.

Jawaban / Kode Solusi:

membuat file baru DetailInventori

```
// LATIHAN 1
import React from 'react'

import {
  useParams,
  Link
} from 'react-router-dom'

function DetailInventori() {

  // Mengambil parameter id dari URL
  const { id } = useParams()

  return (

    <div className="container text-center">

      <h1 className="mb-4">
        Detail Inventori
      </h1>

      <div className="card shadow p-4 border-0 rounded-4">
```

```
        <h3>
          ID Item: {id}
        </h3>

        <p className="text-muted">
          Data detail bahan baku dengan ID {id}
        </p>

        <div className="mt-3">

          <span className="badge bg-success fs-6">
            Status Supplier Available
          </span>

        </div>

        <Link
          to="/inventori"
          className="btn btn-primary mt-4"
        >
          Kembali ke Inventori
        </Link>

      </div>

    </div>

  )
}

export default DetailInventori
```

Edit Inventori.jsx

```
import React, { useState, useEffect } from 'react'

import { Link } from 'react-router-dom'

function Inventori() {

  // ===== STATE DATA =====
  const [products, setProducts] = useState([])

  // ===== FETCH API =====
  useEffect(() => {

    fetch('https://jsonplaceholder.typicode.com/posts')

    .then(res => res.json())

    .then(data => {

      // Ambil 5 data pertama
      setProducts(data.slice(0, 5))

    })

  })

}
```

```

        .catch(err => console.log(err))

    }, [])

    return (

        <div className="container">

            { /* ===== HEADER ===== */ }
            <div className="mb-4">

                <h1 className="fw-bold">
                    Data Inventori Bahan Baku
                </h1>

                <p className="text-muted">
                    Monitoring data inventori supplier pabrik
                </p>

            </div>

            { /* ===== BUTTON ===== */ }
            <Link
                to="/"
                className="btn btn-secondary mb-4"
            >
                ← Kembali ke Dashboard
            </Link>

            { /* ===== TABLE ===== */ }
            <div className="card shadow border-0 rounded-4">

                <div className="card-body">

                    <table className="table table-hover align-middle">

                        <thead className="table-dark">

                            <tr>
                                <th>ID Item</th>
                                <th>Nama Bahan</th>
                                <th>Status Supplier</th>
                            </tr>

                        </thead>

                        <tbody>

                            {products.map((item) => (

                                <tr key={item.id}>

                                    { /* ID */ }
                                    <td>
                                        <strong>
                                            #{item.id}
                                        </strong>
                                    </td>
                                </tr>
                            ))}

                        </tbody>

                    </table>

                </div>

            </div>

        </div>
    )
}

```

```

                { /* Nama Bahan + Dynamic Route */}
                <td>

                    <Link
                        to={`/inventori/${item.id}`}
                        className="text-decoration-none fw-semibold text-
primary"
                    >
                        {item.title}
                    </Link>

                </td>

                { /* Status */}
                <td>

                    <span className="badge bg-success">
                        Available
                    </span>

                </td>

            </tr>

        }}}

    </tbody>

</table>

</div>

</div>

</div>

)
}

export default Inventori

```

Tambahkan pada file App2.jsx

```

Tambahkan Import
import DetailInventori from './Halaman/DetailInventori'
Tambahkan Route Baru
<Routes>

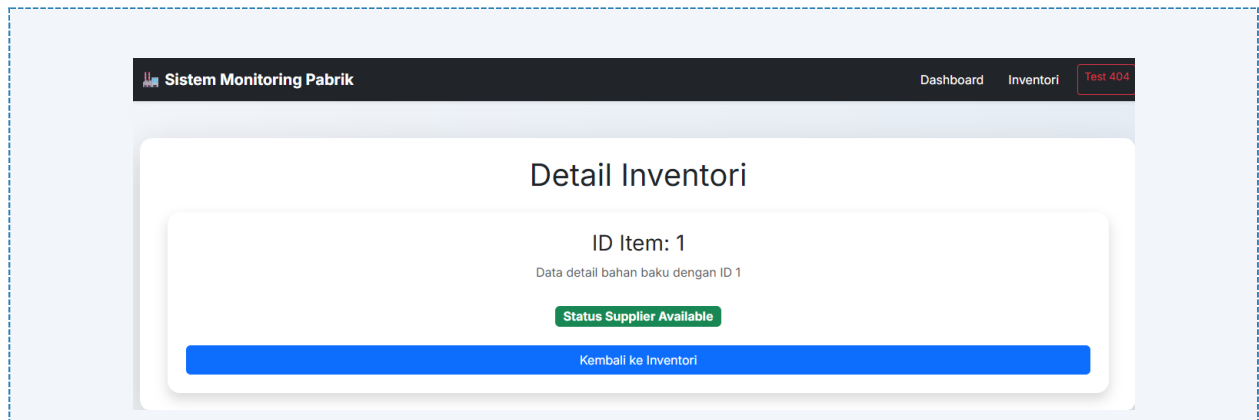
    <Route
        path="/"
        element={<Dashboard />}
    />

    <Route
        path="/inventori"
        element={<Inventori />}
    />

    <Route
        path="/inventori/:id"

```

```
    element={<DetailInventori />}  
  />  
  
  <Route  
    path="*"  
    element={<NotFound />}  
  />  
  
</Routes>
```



Gambar F.1 – Output dari Latihan 1

F.2 Latihan 2 – Loading State

Pertanyaan / Instruksi Latihan:

Di halaman `Inventori.jsx`, tambahkan state `loading` (boolean). Set `loading = true` sebelum fetch, dan `false` setelah data masuk. Tampilkan teks "Memuat data..." jika loading true.

Jawaban / Kode Solusi:

Pada File Inventory.jsx kode final sebagai berikut:

```
import React, { useState, useEffect } from 'react'  
  
import { Link } from 'react-router-dom'  
  
function Inventori() {  
  // ===== STATE DATA =====  
  const [products, setProducts] = useState([])  
  
  // ===== STATE LOADING =====  
  const [loading, setLoading] = useState(true)  
  
  // ===== FETCH API =====  
  useEffect(() => {  
    // Loading dimulai  
    setLoading(true)  
  
    fetch('https://jsonplaceholder.typicode.com/posts')
```

```
.then(res => res.json())

.then(data => {

    // Simulasi loading 2 detik
    setTimeout(() => {

        // Ambil 5 data pertama
        setProducts(data.slice(0, 5))

        // Loading selesai
        setLoading(false)

    }, 2000)

})

.catch(err => {

    console.log(err)

    // Jika error loading tetap dihentikan
    setLoading(false)

})

}, [])

// ===== TAMPILAN LOADING =====
if (loading) {

    return (

        <div className="container text-center mt-5">

            <div
                className="card shadow border-0 rounded-4 p-5"
            >

                <h3 className="mb-3">
                    ⌚ Memuat Data Inventori...
                </h3>

                <p className="text-muted">
                    Sistem sedang mengambil data supplier pabrik
                </p>

                { /* Spinner Bootstrap */ }
                <div className="mt-3">

                    <div
                        className="spinner-border text-primary"
                        role="status"
                    >

                        <span className="visually-hidden">
                            Loading...
                        </span>

                    </div>

                </div>

            </div>

        )

    )

}
```



```

        </div>

    </div>

</div>

</div>

)

}

return (

<div className="container">

    { /* ===== HEADER ===== */ }
    <div className="mb-4">

        <h1 className="fw-bold">
            Data Inventori Bahan Baku
        </h1>

        <p className="text-muted">
            Monitoring data inventori supplier pabrik
        </p>

    </div>

    { /* ===== BUTTON ===== */ }
    <Link
        to="/"
        className="btn btn-secondary mb-4"
    >
        ← Kembali ke Dashboard
    </Link>

    { /* ===== TABLE ===== */ }
    <div className="card shadow border-0 rounded-4">

        <div className="card-body">

            <table className="table table-hover align-middle">

                <thead className="table-dark">

                    <tr>
                        <th>ID Item</th>
                        <th>Nama Bahan</th>
                        <th>Status Supplier</th>
                    </tr>

                </thead>

                <tbody>

                    {products.map((item) => (

```

```

        <tr key={item.id}>

            { /* ID */ }
            <td>
                <strong>
                    #{item.id}
                </strong>
            </td>

            { /* Nama Bahan + Dynamic Route */ }
            <td>

                <Link
                    to={` /inventori/${item.id}`}
                    className="text-decoration-none fw-semibold text-
primary"
                >
                    {item.title}
                </Link>

            </td>

            { /* Status */ }
            <td>

                <span className="badge bg-success">
                    Available
                </span>

            </td>

        </tr>

    </tbody>

</table>

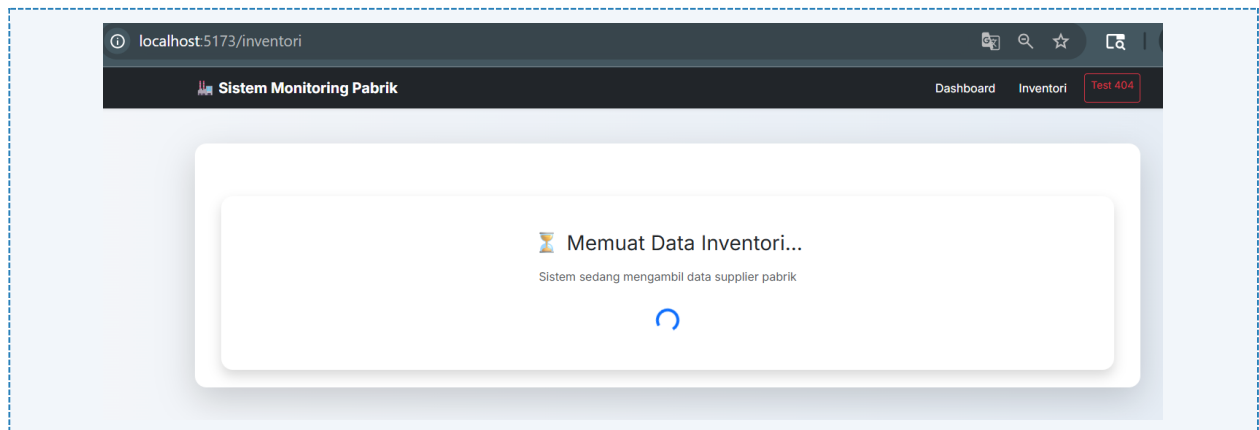
</div>

</div>

</div>

)
}

export default Inventori
```



Gambar F.2 – Output dari Latihan 2

F.3 Tugas Proyek Mini – Navigasi Modular

Deskripsi proyek mini:

- Buat halaman baru bernama `LaporanKualitas.jsx`. Halaman ini menampilkan data "Cacat" (mock data array manual).
- Tambahkan link di Navbar untuk menuju halaman ini.
- Pastikan transisi antar halaman berjalan mulus tanpa reload browser.

Penjelasan pendekatan/solusi yang digunakan:

Pendekatan yang digunakan pada proyek mini ini adalah membuat aplikasi berbasis React dengan sistem navigasi menggunakan React Router DOM. Setiap fitur dipisahkan menjadi beberapa halaman seperti dashboard, inventori, detail inventori, dan laporan kualitas agar struktur aplikasi lebih rapi dan mudah dikembangkan.

Data inventori diambil menggunakan fetch API, sedangkan data laporan kualitas dibuat menggunakan mock data array manual. Selain itu, digunakan dynamic route untuk menampilkan detail item berdasarkan ID serta loading state agar pengguna mengetahui saat data sedang dimuat. Tampilan aplikasi juga dibuat responsif menggunakan Bootstrap sehingga nyaman digunakan pada desktop maupun mobile.

Kode Utama:

Buat file baru LaporanKualitas.jsx

```
import React from 'react'

import { Link } from 'react-router-dom'

function LaporanKualitas() {

  // ===== MOCK DATA =====
  const dataCacat = [

    {
      id: 1,
      namaProduk: 'Gear Box Type A',
      jumlahCacat: 5,
      status: 'Perlu Inspeksi'
    },

    {
      id: 2,
      namaProduk: 'Bearing Unit B',
      jumlahCacat: 2,
      status: 'Aman'
    },

    {
      id: 3,
      namaProduk: 'Poros Baja C',
      jumlahCacat: 7,
      status: 'Critical'
    },

  ],
```

```

    {
      id: 4,
      namaProduk: 'Hydraulic Pump D',
      jumlahCacat: 1,
      status: 'Aman'
    }
  ]

  return (

    <div className="container">

      { /* ===== HEADER ===== */ }
      <div className="mb-4">

        <h1 className="fw-bold">
          Laporan Kualitas Produksi
        </h1>

        <p className="text-muted">
          Monitoring data cacat produksi harian
        </p>

      </div>

      { /* ===== BUTTON ===== */ }
      <Link
        to="/"
        className="btn btn-secondary mb-4"
      >
        ← Kembali ke Dashboard
      </Link>

      { /* ===== TABLE ===== */ }
      <div className="card shadow border-0 rounded-4">

        <div className="card-body">

          <table className="table table-hover align-middle">

            <thead className="table-dark">

              <tr>

                <th>ID</th>
                <th>Nama Produk</th>
                <th>Jumlah Cacat</th>
                <th>Status</th>

              </tr>

            </thead>

            <tbody>

              {dataCacat.map((item) => (

```

```

        <tr key={item.id}>
          <td>
            <strong>
              #{item.id}
            </strong>
          </td>
          <td>
            {item.namaProduk}
          </td>
          <td>
            {item.jumlahCacat} Unit
          </td>
          <td>
            <span
              className={
                item.status === 'Critical'
                  ? 'badge bg-danger'
                  : item.status === 'Perlu Inspeksi'
                  ? 'badge bg-warning text-dark'
                  : 'badge bg-success'
              }
            >
              {item.status}
            </span>
          </td>
        </tr>
      </tbody>
    </table>
  </div>
</div>
</div>
)
}

export default LaporanKualitas

```

Lengkapi File App2.jsx dengan kode berikut

```
import React from 'react'
```

```
import {
  Routes,
```

```
Route,
Link
} from 'react-router-dom'

import Dashboard from './Halaman/Dashboard'
import Inventori from './Halaman/Inventori'
import DetailInventori from './Halaman/DetailInventori'
import LaporanKualitas from './Halaman/LaporanKualitas'
import NotFound from './Halaman/NotFound'

// ===== NAVBAR =====
function Navbar() {

  return (

    <nav className="navbar navbar-expand-lg navbar-dark bg-dark shadow-sm">

      <div className="container">

        {/* Logo */}
        <Link
          className="navbar-brand fw-bold"
          to="/"
        >
           Sistem Monitoring
        </Link>

        {/* Tombol Hamburger Mobile */}
        <button
          className="navbar-toggler"
          type="button"
          data-bs-toggle="collapse"
          data-bs-target="#navbarMenu"
        >
          <span className="navbar-toggler-icon"></span>
        </button>

        {/* Menu Navbar */}
        <div
          className="collapse navbar-collapse"
          id="navbarMenu"
        >

          <div className="navbar-nav ms-auto">

            <Link
              className="nav-link"
              to="/"
            >
              Dashboard
            </Link>

            <Link
              className="nav-link"
              to="/inventori"
            >
              Inventori
            </Link>


```

```

        <Link
          className="nav-link"
          to="/laporan-kualitas"
        >
          Laporan Kualitas
        </Link>

        <Link
          className="nav-link text-danger"
          to="/halaman-error"
        >
          Test 404
        </Link>

      </div>

    </div>

  </div>

</nav>

)
}

// ===== APP =====
function App() {

  return (

    <div
      style={{
        minHeight: '100vh',
        background:
          'linear-gradient(to right, #f5f7fa, #e4ecf5)'
      }}
    >

      {/* Navbar */}
      <Navbar />

      {/* Isi Halaman */}
      <div className="container py-4 py-md-5">

        <div
          className="
            card
            border-0
            shadow-lg
            rounded-4
            p-3
            p-md-5
          "
        >

          <Routes>

            {/* Dashboard */}

```



```
<Route
  path="/"
  element={<Dashboard />}
/>

{/* Inventori */}
<Route
  path="/inventori"
  element={<Inventori />}
/>

{/* Detail Inventori */}
<Route
  path="/inventori/:id"
  element={<DetailInventori />}
/>

{/* Laporan Kualitas */}
<Route
  path="/laporan-kualitas"
  element={<LaporanKualitas />}
/>

{/* 404 */}
<Route
  path="*"
  element={<NotFound />}
/>

</Routes>

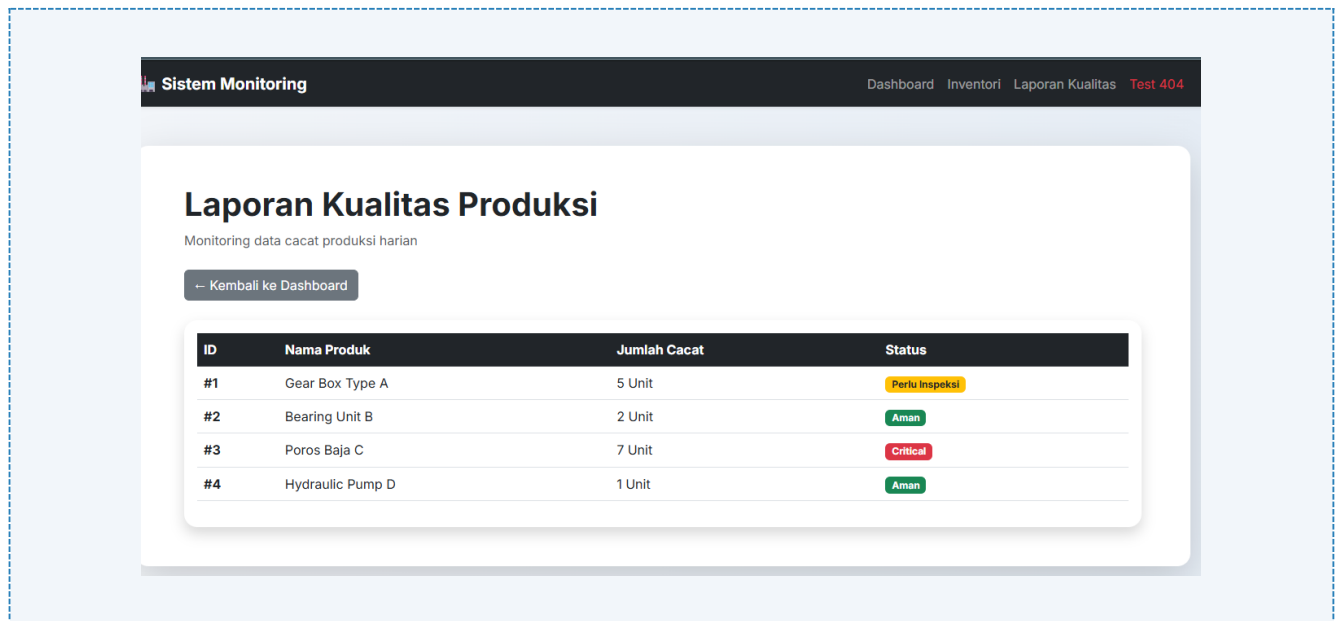
</div>

</div>

</div>

)
}

export default App
```



Gambar F.3 – Output dari Projek Mini Industri

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang diperoleh sudah sesuai dengan teori yang dipelajari pada modul *React Router DOM* dan *state management* di React. Pada praktikum ini, navigasi antar halaman berhasil dilakukan menggunakan *Routes*, *Route*, dan *Link* sehingga perpindahan halaman dapat berjalan tanpa *reload browser* sesuai konsep *Single Page Application (SPA)*. Hal ini terlihat ketika pengguna berpindah dari halaman *dashboard* ke *inventori* maupun laporan kualitas dengan transisi yang cepat dan lancar.

Selain itu, penerapan *dynamic route* juga berhasil berjalan sesuai teori. Pada halaman inventori, setiap item dapat dibuka ke halaman detail menggunakan parameter URL seperti */inventori/:id*. Parameter tersebut kemudian diproses menggunakan *useParams()* sehingga halaman dapat menampilkan detail berdasarkan ID yang dipilih. Hal ini menunjukkan bahwa konsep *dynamic routing* pada React telah diterapkan dengan benar.

Penggunaan *useState* dan *useEffect* untuk *fetch API* juga berjalan sesuai teori. Data inventori berhasil diambil dari API dan ditampilkan ke dalam tabel secara dinamis menggunakan *map()*. *Loading state* yang ditambahkan juga berhasil memberikan informasi kepada pengguna saat data sedang dimuat sehingga aplikasi menjadi lebih interaktif dan *user friendly*.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
<i>Loading state</i> terlalu cepat sehingga tidak terlihat saat data dimuat	Menambahkan <i>setTimeout()</i> agar proses loading terlihat beberapa detik sebelum data tampil.

G.3 Kaitan dengan Penerapan di Industri

Keterampilan yang dipelajari pada modul ini dapat diterapkan dalam berbagai sistem informasi di dunia Teknik Industri, terutama pada proses monitoring produksi dan manajemen data. Penggunaan *React Router DOM* memungkinkan perusahaan membuat aplikasi dengan banyak halaman seperti dashboard produksi, inventori bahan baku, laporan kualitas, dan data mesin tanpa perpindahan halaman yang lambat.

Contoh penerapannya adalah pada sistem monitoring pabrik, di mana operator dapat berpindah dari halaman inventori ke laporan kualitas atau detail mesin secara cepat tanpa *reload browser*. Selain itu, penggunaan *loading state* dan *fetch API* juga relevan untuk menampilkan data produksi secara *realtime* dari server sehingga proses pengawasan dan pengambilan keputusan dapat dilakukan lebih efisien..

H. KESIMPULAN

1	Praktikum ini berhasil menerapkan <i>React Router DOM</i> untuk membuat navigasi antar halaman pada aplikasi React tanpa melakukan <i>reload browser</i> sesuai konsep <i>Single Page Application (SPA)</i> .
2	Penerapan <i>dynamic route</i> , <i>fetch API</i> , dan <i>loading state</i> berhasil digunakan untuk menampilkan data inventori dan halaman detail secara dinamis serta lebih interaktif.
3	Aplikasi yang dibuat berhasil memiliki tampilan yang lebih rapi, responsif, dan modular sehingga sesuai digunakan sebagai simulasi sistem monitoring pada dunia industri.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Banks, A., & Porcello, E. (2017). <i>Learning React: Functional Web Development with React and Redux</i> . O'Reilly Media.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
Radhitya Adha Fadhlur Rohman NIM: 23051430005	Maessa Andrea Vallenia NIM: 23051430044	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

Modul 12 Proyek Web: Dashboard Monitoring Produksi (React + Chart.js)

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 12 (Pertemuan 12)
Topik Utama	Proyek Web: Dashboard Monitoring Produksi (React + Chart.js)
Sub-Topik	<i>Integrasi Library Visualisasi Data (Chart.js) dan Finalisasi Layout Dashboard.</i>
Durasi Praktikum	340 Menit
Tanggal Praktikum	8 Mei 2026
Nama Mahasiswa	Radhitya Adha Fadhlur Rohman
NIM	23051430005
Kelas	A2

B. TUJUAN PEMBELAJARAN

1	Mampu mengintegrasikan library pihak ketiga (`react-chartjs-2`) ke dalam React.
2	Mampu mengubah data mentah menjadi visualisasi grafik yang informatif.
3	Menerapkan `fetch` API di dalam `useEffect` untuk mengambil data eksternal.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	File `index.html`	Versi Terbaru	Hasil pekerjaan pertemuan sebelumnya
4	Ekstensi VS Code: LiveServer	Versi terbaru	Melihat perubahan web secara real-time.
5	Javascript	Versi terbaru	Membuat halaman web menjadi lebih interaktif dan fungsional

6	Node.js	Versi Terbaru	<i>runtime environment</i> JavaScript
---	---------	---------------	---------------------------------------

D. DASAR TEORI

Dashboard monitoring produksi merupakan tampilan sistem yang digunakan untuk memantau kondisi proses produksi secara lebih mudah dan cepat. Dalam dunia industri, data produksi biasanya terus bertambah setiap jam sehingga jika hanya ditampilkan dalam bentuk angka atau tabel akan sulit dibaca dan dianalisis. Oleh karena itu, dashboard digunakan untuk mengubah data tersebut menjadi informasi visual seperti grafik, kartu indikator, dan status mesin. Dengan tampilan visual, pengguna dapat mengetahui kondisi produksi, pencapaian target, serta performa mesin secara real-time tanpa harus membaca data satu per satu. Sistem dashboard seperti ini sangat penting pada era industri 4.0 karena membantu perusahaan mengambil keputusan dengan lebih cepat dan tepat.

Pada praktikum ini, dashboard dibuat menggunakan React.js yang merupakan library JavaScript untuk membangun antarmuka website secara interaktif. React bekerja dengan konsep komponen, yaitu setiap bagian tampilan dibuat terpisah agar lebih mudah dikelola dan digunakan kembali. Contohnya seperti komponen grafik produksi, kartu status mesin, dan sidebar menu. Dengan sistem ini, kode program menjadi lebih rapi dan proses pengembangan aplikasi menjadi lebih efisien. React juga mendukung penggunaan state dan props untuk mengatur serta mengirim data antar komponen sehingga tampilan dashboard dapat berubah secara dinamis sesuai data produksi yang ditampilkan.

Untuk visualisasi data, digunakan library Chart.js yang diintegrasikan melalui react-chartjs-2. Library ini memungkinkan data produksi ditampilkan dalam bentuk grafik batang, grafik garis, maupun grafik donat. Grafik batang digunakan untuk melihat jumlah produksi setiap jam, sedangkan grafik garis dapat digunakan untuk membandingkan hasil produksi dengan target yang telah ditentukan. Selain itu, grafik donat membantu menampilkan proporsi cacat produksi agar lebih mudah dianalisis. Praktikum ini juga menggunakan Bootstrap untuk membantu membuat layout dashboard menjadi lebih rapi dan responsif. Dengan Bootstrap, tampilan dashboard dapat menyesuaikan ukuran layar sehingga tetap nyaman digunakan baik pada komputer maupun perangkat mobile.

D.1 Konsep Utama

Konsep utama pada praktikum ini adalah pembuatan dashboard monitoring produksi berbasis web menggunakan React.js dan Chart.js. Dashboard digunakan untuk menampilkan data produksi dalam bentuk visual seperti grafik dan kartu informasi sehingga kondisi produksi dapat dipahami dengan lebih cepat dan mudah. Melalui dashboard, pengguna dapat melihat jumlah produksi per jam, target produksi, efisiensi kerja, serta status mesin secara real-time. Visualisasi data menjadi bagian penting karena membantu pengguna mengetahui tren produksi, membandingkan hasil dengan target, dan mendeteksi masalah produksi dengan lebih efektif dibandingkan hanya menggunakan tabel angka biasa.

Praktikum ini juga menerapkan konsep component pada React.js, yaitu membagi tampilan website menjadi beberapa bagian kecil seperti sidebar, grafik produksi, dan kartu status mesin agar kode lebih rapi dan mudah dikembangkan. Selain itu, digunakan library Chart.js

untuk mengubah data array menjadi grafik interaktif seperti bar chart dan doughnut chart. Layout dashboard dibuat menggunakan Bootstrap agar tampilan lebih modern dan responsif, sehingga aplikasi tetap nyaman digunakan baik pada komputer maupun smartphone.

D.2 Keterkaitan dengan Teknik Industri

Materi pada modul ini sangat berkaitan dengan dunia Teknik Industri, terutama dalam bidang sistem produksi, pengawasan proses, dan pengolahan data operasional. Dalam industri manufaktur, seorang engineer tidak hanya bertugas memastikan proses produksi berjalan lancar, tetapi juga harus mampu memantau performa mesin, jumlah output produksi, tingkat efisiensi, hingga jumlah produk cacat. Dashboard monitoring produksi membantu semua informasi tersebut ditampilkan secara visual dan real-time sehingga proses pengambilan keputusan dapat dilakukan lebih cepat dan tepat. Dengan adanya dashboard, perusahaan dapat mengetahui apakah target produksi tercapai atau terdapat kendala pada lini produksi.

Contoh penerapannya dapat dilihat pada pabrik manufaktur yang memiliki banyak mesin produksi. Melalui dashboard, operator dan supervisor dapat memantau status mesin seperti running, stop, atau maintenance secara langsung. Selain itu, data produksi per jam dapat divisualisasikan dalam bentuk grafik sehingga penurunan produksi atau peningkatan defect dapat segera diketahui. Sistem seperti ini sangat membantu dalam meningkatkan efisiensi kerja, mengurangi downtime mesin, serta mendukung penerapan konsep industri 4.0 yang mengutamakan penggunaan teknologi digital dalam proses industri.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Persiapan Data Grafik

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
const data = {
  labels: [
    '08:00',
    '09:00',
    '10:00',
    '11:00',
    '12:00',
    '13:00',
  ],
  datasets: [
    {
      label: 'Produksi',
      data: [
        120,
        150,
        180,
        170,
      ]
    }
  ]
}
```



```
    200,  
    210,  
  ],
```

Screenshot Hasil:

Gambar E.1 – Kode dari Langkah 1

E.2 Langkah 2 – Membuat Komponen Grafik

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */  
import React from 'react';  
  
import {  
  Chart as ChartJS,  
  CategoryScale,  
  LinearScale,  
  BarElement,  
  LineElement,  
  PointElement,  
  Title,  
  Tooltip,  
  Legend,  
} from 'chart.js';  
  
import { Chart } from 'react-chartjs-2';  
  
// ===== REGISTER =====  
ChartJS.register(  
  CategoryScale,  
  LinearScale,  
  BarElement,  
  LineElement,  
  PointElement,  
  Title,  
  Tooltip,  
  Legend  
);  
  
function GrafikProduksi() {  
  
  const data = {  
  
    labels: [  
      '08:00',  
      '09:00',  
      '10:00',  
      '11:00',  
      '12:00',  
      '13:00',  
    ],  
  
    datasets: [  
  
      {
```

```
        type: 'bar',

        label: 'Jumlah Produksi',

        data: [
            120,
            150,
            180,
            170,
            200,
            210,
        ],

        backgroundColor:
            'rgba(54, 162, 235, 0.5)',

        borderColor:
            'rgba(54, 162, 235, 1)',

        borderWidth: 1,
    },

    {
        type: 'line',

        label: 'Target',

        data: [
            150,
            150,
            150,
            150,
            150,
            150,
        ],

        borderColor:
            'rgb(255, 99, 132)',

        backgroundColor:
            'rgb(255, 99, 132)',

        borderWidth: 2,

        tension: 0.3,
    },
],
};

const options = {

    responsive: true,

    plugins: {

        legend: {
            position: 'top',
```

```

    },

    title: {
      display: true,
      text: 'Grafik Produksi Harian',
    },

  },

  scales: {

    y: {
      beginAtZero: true,
    },

  },

};

return (

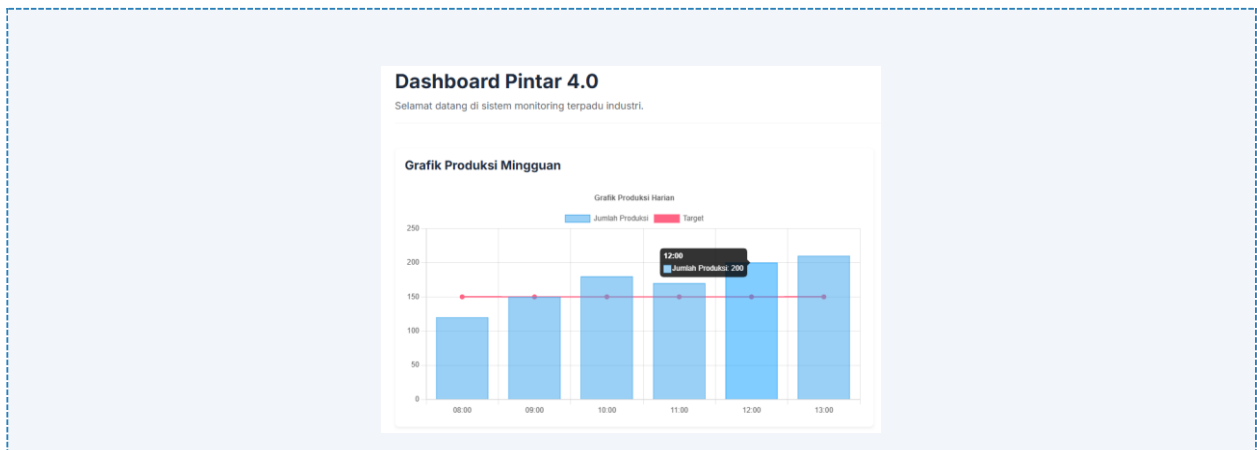
  <Chart
    type="bar"
    data={data}
    options={options}
  />

);
}

export default GrafikProduksi;

```

Screenshot Hasil:



Gambar E.2 – Output dari Langkah 2

E.3 Langkah 3 – Menggabungkan ke Dashboard Utama

```

/* ===== KODE / OUTPUT LANGKAH 3 ===== */
import React from 'react';
import { Link } from 'react-router-dom';

```

```

import GrafikProduksi from '../Komponen/GrafikProduksi';
import KartuMesin from '../Komponen/KartuMesin';

function Dashboard() {
  return (
    <div className="container-fluid py-3 py-md-4 px-3 px-md-4">

      { /* ===== */ }
      { /* ===== HEADER ===== */ }
      { /* ===== */ }

      <div className="row align-items-center mb-4">

        <div className="col-12 col-lg-8">

          <h2 className="fw-bold text-center text-lg-start mb-2">
            Dashboard Pintar Industri 4.0
          </h2>

          <p className="text-muted text-center text-lg-start mb-0">
            Sistem monitoring produksi, inventori,
            dan efisiensi mesin secara realtime.
          </p>

        </div>

        <div className="col-12 col-lg-4 text-center text-lg-end mt-3 mt-
lg-0">

          <span className="badge bg-success px-3 py-2 fs-6">
            • Sistem Online
          </span>

        </div>

      </div>

      { /* ===== */ }
      { /* ===== GRAFIK ===== */ }
      { /* ===== */ }

      <div className="row g-4 mb-4">

        { /* ===== GRAFIK ===== */ }
        <div className="col-12 col-xl-8">

          <div className="card border-0 shadow-sm rounded-4 h-100">

            <div className="card-body p-3 p-md-4">

              <div
                className="
                  d-flex
                  justify-content-between
                  align-items-center
                  flex-wrap
                  mb-4

```

```

"
>

<div>

  <h5 className="fw-bold mb-1">
    Grafik Produksi Harian
  </h5>

  <small className="text-muted">
    Monitoring output produksi lini utama
  </small>

</div>

<span
  className="
    badge
    bg-primary-subtle
    text-primary
    mt-2
    mt-md-0
  "
>
  Realtime
</span>

</div>

{/* Grafik */}
<div style={{ height: '350px' }}>
  <GrafikProduksi />
</div>

</div>

</div>

</div>

{/* ===== KPI ===== */}
<div className="col-12 col-xl-4">

  <div className="row g-4">

    {/* OUTPUT */}
    <div className="col-12">

      <div
        className="
          card
          border-0
          shadow-sm
          rounded-4
          bg-primary
          text-white
        "
      >

```

```
<div className="card-body p-4">

  <div className="d-flex justify-content-between align-
items-center">

    <div>

      <h6 className="mb-2">
        Total Output Hari Ini
      </h6>

      <h2 className="fw-bold mb-1">
        1,030
      </h2>

      <small>
        Unit Produksi
      </small>

    </div>

    <div style={{ fontSize: '40px' }}>
      📦
    </div>

  </div>

</div>

</div>

</div>

{ /* EFFICIENCY */ }
<div className="col-12">

  <div
    className="
      card
      border-0
      shadow-sm
      rounded-4
      bg-success
      text-white
    "
  >

    <div className="card-body p-4">

      <div className="d-flex justify-content-between align-
items-center">

        <div>

          <h6 className="mb-2">
            Efficiency Rate
```

```
        </h6>

        <h2 className="fw-bold mb-1">
          92.4%
        </h2>

        <small>
          +1.2% dari kemarin
        </small>

      </div>

      <div style={{ fontSize: '40px' }}>
        ⚙️
      </div>

    </div>

  </div>

</div>

{ /* MESIN AKTIF */ }
<div className="col-12">

  <div
    className="
      card
      border-0
      shadow-sm
      rounded-4
      bg-dark
      text-white
    "
  >

    <div className="card-body p-4">

      <div className="d-flex justify-content-between align-
items-center">

        <div>

          <h6 className="mb-2">
            Mesin Aktif
          </h6>

          <h2 className="fw-bold mb-1">
            12
          </h2>

          <small>
            Dari 14 mesin produksi
          </small>
```

```

        </div>

        <div style={{ fontSize: '40px' }}>
            🏭
        </div>

    </div>

</div>

</div>

</div>

</div>

</div>

</div>

</div>

/* ===== */
/* ===== STATUS MESIN ===== */
/* ===== */

<div className="row mb-5">

    <div
        className="
            col-12
            d-flex
            justify-content-between
            align-items-center
            flex-wrap
            mb-4
        "
    >

        <div>

            <h4 className="fw-bold mb-1">
                Status Mesin Aktif
            </h4>

            <small className="text-muted">
                Monitoring realtime kondisi mesin produksi
            </small>

        </div>

        <span
            className="
                badge
                bg-success-subtle
                text-success
                px-3
                py-2
                mt-2

```



```

        mt-md-0
        "
    >
        4 Mesin Dipantau
    </span>

</div>

{ /* CARD MESIN */}
<div className="col-12 col-sm-6 col-xl-3 mb-4">
    <KartuMesin
        nama="CNC-01"
        status="Running"
        produksi={320}
    />
</div>

<div className="col-12 col-sm-6 col-xl-3 mb-4">
    <KartuMesin
        nama="CNC-02"
        status="Running"
        produksi={310}
    />
</div>

<div className="col-12 col-sm-6 col-xl-3 mb-4">
    <KartuMesin
        nama="Press-01"
        status="Stop"
        produksi={150}
    />
</div>

<div className="col-12 col-sm-6 col-xl-3 mb-4">
    <KartuMesin
        nama="Weld-04"
        status="Maintenance"
        produksi={0}
    />
</div>

</div>

{ /* ===== */}
{ /* ===== NAVIGASI ===== */}
{ /* ===== */}

<div className="row">

    <div
        className="
            col-12
            d-flex
            flex-wrap
            gap-3
            justify-content-center
            justify-content-lg-start
        "

```

```
>

<Link
  to="/inventori"
  className="
    btn
    btn-primary
    px-4
    py-2
    rounded-3
    shadow-sm
  "
>
  📦 Halaman Inventori
</Link>

<Link
  to="/praktikum"
  className="
    btn
    btn-outline-dark
    px-4
    py-2
    rounded-3
  "
>
  📁 Halaman Praktikum
</Link>

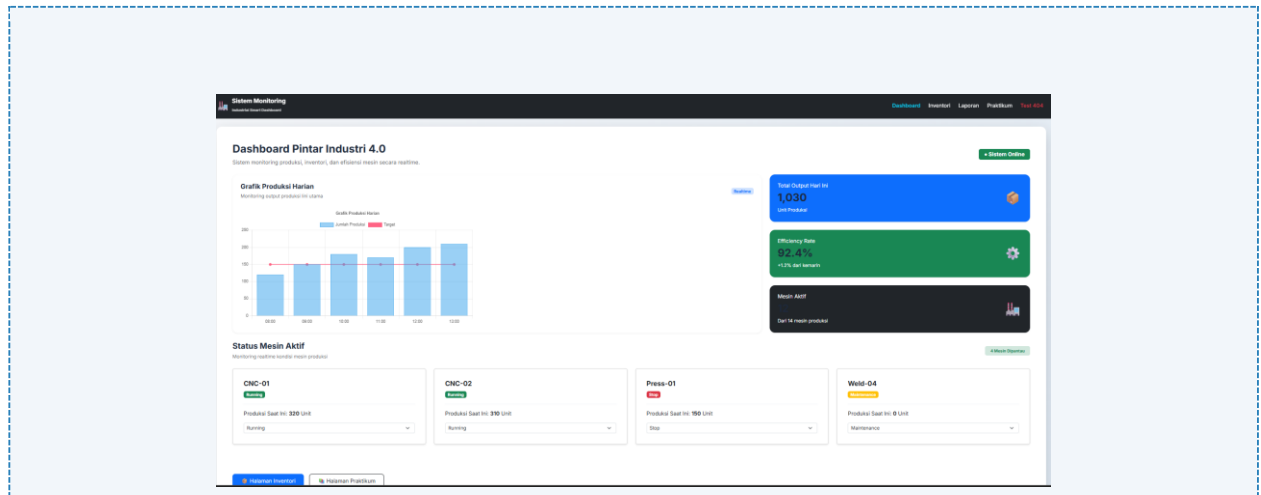
</div>

</div>

</div>
);
}

export default Dashboard;
```

Screenshot Hasil:



Gambar E.3 – Output dari Langkah 3

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Integrasi Data Realistik

Pertanyaan / Instruksi Latihan:

Ubah data statis di `GrafikProduksi` menjadi data yang diambil dari `fetch` API (gunakan mock data dari internet atau buat generator acak).

Jawaban / Kode Solusi:

```
import React, {
  useEffect,
  useState,
} from 'react';

import {
  Chart as ChartJS,
  CategoryScale,
  LinearScale,
  BarElement,
  PointElement,
  LineElement,
  Title,
  Tooltip,
  Legend,
} from 'chart.js';

import { Chart } from 'react-chartjs-2';

// ===== REGISTER CHART =====

ChartJS.register(
  CategoryScale,
```

```
LinearScale,
BarElement,
PointElement,
LineElement,
Title,
Tooltip,
Legend
);

function GrafikProduksi() {

  // =====
  // ===== STATE =====
  // =====

  const [dataProduksi, setDataProduksi] = useState([]);
  const [loading, setLoading] = useState(true);

  // =====
  // ===== FETCH DATA =====
  // =====

  useEffect(() => {

    const ambilDataProduksi = async () => {

      try {

        // ===== FETCH MOCK API =====

        const response = await fetch(
          'https://jsonplaceholder.typicode.com/posts'
        );

        const hasil = await response.json();

        // ===== GENERATE DATA =====

        const dataRandom = hasil
          .slice(0, 6)
          .map(() => {
            return Math.floor(
              Math.random() * 120 + 100
            );
          });

        // ===== SIMPAN STATE =====

        setDataProduksi(dataRandom);

      } catch (error) {

        console.log(
          'Gagal mengambil data:',
          error
        );

      } finally {
```

```
        setLoading(false);

    }

};

ambilDataProduksi();

}, []);

// =====
// ===== DATA GRAFIK =====
// =====

const data = {
  labels: [
    '08:00',
    '09:00',
    '10:00',
    '11:00',
    '12:00',
    '13:00',
  ],
  datasets: [

    // ===== BAR PRODUKSI =====

    {
      type: 'bar',
      label: 'Produksi Aktual',

      data: dataProduksi,

      backgroundColor:
        'rgba(54, 162, 235, 0.5)',

      borderColor:
        'rgba(54, 162, 235, 1)',

      borderWidth: 1,
      borderRadius: 10,
    },

    // ===== TARGET =====

    {
      type: 'line',
      label: 'Target Produksi',

      data: [
        150,
        150,
        150,
        150,
        150,
        150,
      ]
    }
  ]
};
```

```

    ],

    borderColor:
      'rgb(255, 99, 132)',

    borderWidth: 3,
    tension: 0.3,
  },

],
};

// =====
// ===== OPTIONS =====
// =====

const options = {
  responsive: true,
  maintainAspectRatio: false,

  plugins: {

    legend: {
      position: 'top',
    },

    title: {
      display: true,
      text: 'Grafik Produksi Harian - Lini 1',
    },

  },

  scales: {
    y: {
      beginAtZero: true,
    },
  },
};

// =====
// ===== LOADING =====
// =====

if (loading) {

  return (
    <div
      className="
        d-flex
        justify-content-center
        align-items-center
      "
      style={{ height: '350px' }}
    >

    <div
      className="spinner-border text-primary"

```

```

        role="status"
      ></div>

    </div>
  );
}

// =====
// ===== RETURN =====
// =====

return (
  <div style={{ height: '350px' }}>

    <Chart
      type="bar"
      data={data}
      options={options}
    />

  </div>
);
}

export default GrafikProduksi;

```



Gambar F.1 – Output dari Latihan 1

F.2 Latihan 2 – Tambahkan Grafik Donat

Pertanyaan / Instruksi Latihan:

"Proporsi Cacat" (Misal: Scratch 50%, Dent 30%, Lainnya 20%) menggunakan komponen `Doughnut` dari chart.js.

Jawaban / Kode Solusi:

Membuat File GrafikCacat.jsx

```
import React from 'react';

import {
  Chart as ChartJS,
  ArcElement,
  Tooltip,
  Legend,
} from 'chart.js';

import { Doughnut } from 'react-chartjs-2';

// =====
// ===== REGISTER =====
// =====

ChartJS.register(
  ArcElement,
  Tooltip,
  Legend
);

function GrafikCacat() {

  // =====
  // ===== DATA =====
  // =====

  const data = {
    labels: [
      'Scratch',
      'Dent',
      'Lainnya',
    ],

    datasets: [
      {
        label: 'Proporsi Cacat',

        data: [
          50,
          30,
          20,
        ],

        backgroundColor: [
          '#ff6384',
          '#36a2eb',
          '#ffce56',
        ],

        borderColor: [
          'ffffff',
          'ffffff',
          'ffffff',
        ],

        borderWidth: 3,
        hoverOffset: 10,
      }
    ]
  };
}
```



```
    },
  ],
};

// =====
// ===== OPTIONS =====
// =====

const options = {
  responsive: true,
  maintainAspectRatio: false,

  plugins: {

    legend: {
      position: 'bottom',

      labels: {
        padding: 20,

        font: {
          size: 14,
        },
      },
    },

    title: {
      display: true,
      text: 'Proporsi Cacat Produksi',

      font: {
        size: 18,
      },
    },

  },
};

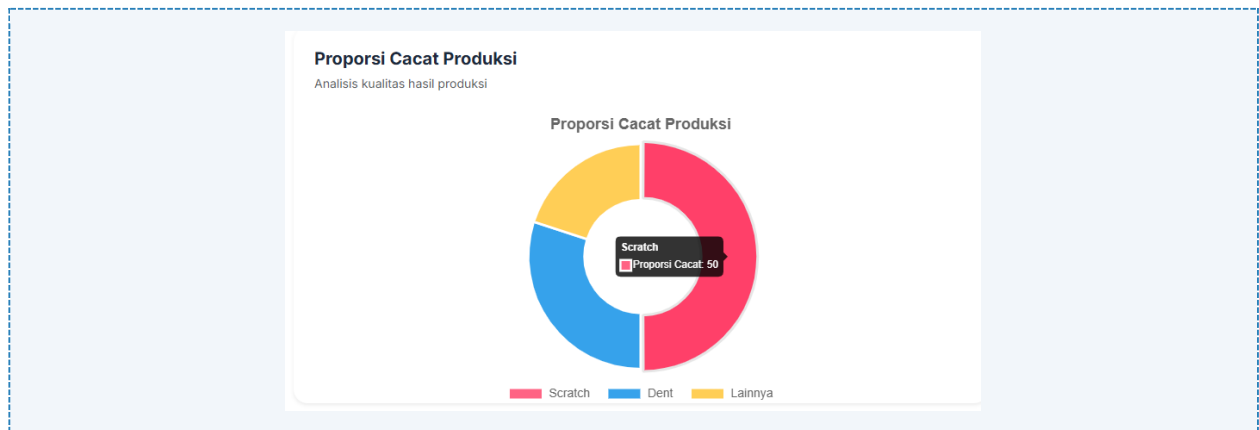
// =====
// ===== RETURN =====
// =====

return (
  <div style={{ height: '350px' }}>

    <Doughnut
      data={data}
      options={options}
    />

  </div>
);
}

export default GrafikCacat;
```



Gambar F.2 – Output dari Latihan 2

F.3 Tugas Akhir Web

Deskripsi proyek mini:

Susun ulang halaman `Dashboard` agar memiliki Sidebar (kiri) dan Main Content (kanan).

- Sidebar: Logo, Menu Dashboard, Menu Inventori, Menu Laporan.
- Main Content: Header (User Profile), Grafik, dan Tabel Data.
- Pastikan desain responsif (Sidebar bisa collapse di HP).

Penjelasan pendekatan/solusi yang digunakan:

Pada tugas akhir web ini, saya membuat tampilan dashboard industri yang dibagi menjadi dua bagian utama yaitu sidebar di sebelah kiri dan main content di sebelah kanan. Sidebar digunakan untuk navigasi menu seperti Dashboard, Inventori, dan Laporan, sedangkan bagian main content berisi header user profile, grafik produksi, grafik proporsi cacat, status mesin, dan tabel data produksi. Dalam pengerjaannya saya menggunakan React JS, React Router DOM, Bootstrap, serta Chart.js agar tampilan lebih modern dan interaktif.

Agar dashboard terasa lebih realistis, data pada grafik produksi dibuat dinamis menggunakan simulasi fetch data dan generator angka acak. Selain itu, tampilan juga dibuat responsif supaya tetap rapi saat dibuka di laptop maupun HP. Sidebar dapat di-collapse pada tampilan mobile menggunakan state useState, sehingga navigasi tetap nyaman digunakan pada layar kecil. Dengan pendekatan ini, dashboard menjadi lebih menarik, mudah digunakan, dan menyerupai sistem monitoring industri secara nyata.

Kode Utama:

```
import React, { useState } from 'react';
import { Link } from 'react-router-dom';

import GrafikProduksi from '../Komponen/GrafikProduksi';
import GrafikCacat from '../Komponen/GrafikCacat';
import KartuMesin from '../Komponen/KartuMesin';

function Dashboard() {

  // =====
  // ===== SIDEBAR STATE =====
  // =====

  const [sidebarOpen, setSidebarOpen] = useState(false);

  return (

    <div
      className="container-fluid p-0"
      style={{
        backgroundColor: '#f4f7fb',
        minHeight: '100vh',
      }}
    >
```

```

<div className="row g-0">

  { /* ===== */ }
  { /* ===== SIDEBAR ===== */ }
  { /* ===== */ }

  <div
    className={ `
      col-lg-2
      sidebar-custom
      bg-dark
      text-white
      ${sidebarOpen ? 'd-block position-fixed' : 'd-none'}
      d-lg-block
    ` }
    style={{
      minHeight: '100vh',
      width: '260px',
      zIndex: 1050,
      top: 0,
      left: 0,
    }}
  >

    { /* ===== LOGO ===== */ }

    <div className="p-4 border-bottom border-secondary">

      <h3 className="fw-bold text-center mb-1">
         Smart Factory
      </h3>

      <p className="text-center text-secondary small mb-0">
        Monitoring System
      </p>

    </div>

    { /* ===== MENU ===== */ }

    <div className="p-3">

      <div className="d-grid gap-3">

        <Link
          to="/"
          className="btn btn-light text-start fw-semibold py-3 rounded-3"
          onClick={() => setSidebarOpen(false)}
        >
           Dashboard
        </Link>

        <Link
          to="/inventori"
          className="btn btn-outline-light text-start py-3 rounded-3"
          onClick={() => setSidebarOpen(false)}
        >

```

```

         Inventori
      </Link>

      <Link
        to="/laporan-kualitas"
        className="btn btn-outline-light text-start py-3 rounded-3"
        onClick={ () => setSidebarOpen(false) }
      >
         Laporan
      </Link>

      <Link
        to="/praktikum"
        className="btn btn-outline-light text-start py-3 rounded-3"
        onClick={ () => setSidebarOpen(false) }
      >
         Praktikum
      </Link>
    </div>

  </div>

</div>

{ /* ===== */ }
{ /* ===== OVERLAY MOBILE ===== */ }
{ /* ===== */ }

{
  sidebarOpen && (
    <div
      className="
        d-lg-none
        position-fixed
        top-0
        start-0
        w-100
        h-100
        bg-dark
      "
      style={{
        opacity: 0.5,
        zIndex: 1040,
      }}
      onClick={ () => setSidebarOpen(false) }
    >
    )
  )
}

{ /* ===== */ }
{ /* ===== MAIN CONTENT ===== */ }
{ /* ===== */ }

<div className="col">

  <div

```

```

        className="p-3 p-md-4"
        style={{
            marginLeft:
                window.innerWidth >= 992
                ? '260px'
                : '0',
            transition: '0.3s',
        }}
    >

    { /* ===== */ }
    { /* ===== HEADER ===== */ }
    { /* ===== */ }

    <div className="card border-0 shadow-sm rounded-4 mb-4">

        <div className="card-body p-3 p-md-4">

            <div
                className="
                    d-flex
                    justify-content-between
                    align-items-center
                    flex-wrap
                    gap-3
                "
            >

                { /* ===== LEFT ===== */ }

                <div
                    className="
                        d-flex
                        align-items-center
                        gap-3
                    "
                >

                    { /* TOGGLE BUTTON */ }

                    <button
                        className="btn btn-dark d-lg-none rounded-3"
                        onClick={() =>
                            setSidebarOpen(!sidebarOpen)
                        }
                    >
                        ≡
                    </button>

                    <div>

                        <h3 className="fw-bold mb-1">
                            Dashboard Industri 4.0
                        </h3>

                        <small className="text-muted">
                            Smart Manufacturing Monitoring System

```

```

        </small>

    </div>

</div>

{ /* ===== RIGHT ===== */}

<div
  className="
    d-flex
    align-items-center
    gap-3
  "
>

  <div className="text-end">

    <h6 className="fw-bold mb-0">
      Radhitya Adha
    </h6>

    <small className="text-muted">
      Administrator
    </small>

  </div>

</div>

</div>

</div>

</div>

{ /* ===== */}
{ /* ===== KPI ===== */}
{ /* ===== */}

<div className="row g-4 mb-4">

  { /* ===== CARD 1 ===== */}

  <div className="col-12 col-md-6 col-xl-4">

    <div className="card border-0 shadow-sm rounded-4 bg-primary
text-white h-100">

      <div className="card-body p-4">

```

```

        <div className="d-flex justify-content-between align-items-
center">

        <div>

            <h6>Total Output</h6>

            <h2 className="fw-bold">
                1,030
            </h2>

            <small>Unit Hari Ini</small>

        </div>

        <div style={{ fontSize: '45px' }}>
            📦
        </div>

    </div>

</div>

</div>

</div>

{ /* ===== CARD 2 ===== */ }

<div className="col-12 col-md-6 col-xl-4">

    <div className="card border-0 shadow-sm rounded-4 bg-success
text-white h-100">

        <div className="card-body p-4">

            <div className="d-flex justify-content-between align-items-
center">

                <div>

                    <h6>Efficiency</h6>

                    <h2 className="fw-bold">
                        92.4%
                    </h2>

                    <small>Realtime Monitoring</small>

                </div>

                <div style={{ fontSize: '45px' }}>
                    ⚙️
                </div>

            </div>

        </div>

    </div>


```



```

        </div>

    </div>

</div>

{ /* ===== CARD 3 ===== */ }

<div className="col-12 col-md-6 col-xl-4">

    <div className="card border-0 shadow-sm rounded-4 bg-dark text-
white h-100">

        <div className="card-body p-4">

            <div className="d-flex justify-content-between align-items-
center">

                <div>

                    <h6>Mesin Aktif</h6>

                    <h2 className="fw-bold">
                        12
                    </h2>

                    <small>Dari 14 Mesin</small>

                </div>

                <div style={{ fontSize: '45px' }}>
                    🏭
                </div>

            </div>

        </div>

    </div>

</div>

</div>

{ /* ===== GRAFIK ===== */ }
{ /* ===== GRAFIK ===== */ }
{ /* ===== GRAFIK ===== */ }

<div className="row g-4 mb-4">

    { /* ===== GRAFIK PRODUKSI ===== */ }

    <div className="col-12 col-xl-8">

        <div className="card border-0 shadow-sm rounded-4 h-100">

```

```

        <div className="card-body p-3 p-md-4">

            <h5 className="fw-bold mb-3">
                Grafik Produksi Harian
            </h5>

            <div style={{ height: '350px' }}>
                <GrafikProduksi />
            </div>

        </div>

    </div>

</div>

{/* ===== GRAFIK CACAT ===== */}

<div className="col-12 col-xl-4">

    <div className="card border-0 shadow-sm rounded-4 h-100">

        <div className="card-body p-3 p-md-4">

            <h5 className="fw-bold mb-3">
                Proporsi Cacat
            </h5>

            <div style={{ height: '320px' }}>
                <GrafikCacat />
            </div>

        </div>

    </div>

</div>

</div>

{/* ===== STATUS MESIN ===== */}
{/* ===== STATUS MESIN ===== */}
{/* ===== STATUS MESIN ===== */}

<div className="mb-4">

    <h4 className="fw-bold mb-3">
        Status Mesin
    </h4>

    <div className="row g-4">

        <div className="col-12 col-sm-6 col-xl-3">

            <KartuMesin
                nama="CNC-01"
                status="Running"
                produksi={320}
            />
        </div>
    </div>
</div>

```

```

        />

</div>

<div className="col-12 col-sm-6 col-xl-3">

    <KartuMesin
        nama="CNC-02"
        status="Running"
        produksi={310}
    />

</div>

<div className="col-12 col-sm-6 col-xl-3">

    <KartuMesin
        nama="Press-01"
        status="Stop"
        produksi={150}
    />

</div>

<div className="col-12 col-sm-6 col-xl-3">

    <KartuMesin
        nama="Weld-04"
        status="Maintenance"
        produksi={0}
    />

</div>

</div>

</div>

{ /* ===== */ }
{ /* ===== TABEL ===== */ }
{ /* ===== */ }

<div className="card border-0 shadow-sm rounded-4 mb-4">

    <div className="card-body p-3 p-md-4">

        <div
            className="
                d-flex
                justify-content-between
                align-items-center
                flex-wrap
                gap-3
                mb-4
            "
        >

            <div>

```

```
<h4 className="fw-bold mb-1">
  Data Produksi
</h4>

<small className="text-muted">
  Aktivitas produksi terbaru
</small>

</div>

<button className="btn btn-primary rounded-3">
  Export Data
</button>

</div>

<div className="table-responsive">

  <table className="table table-hover align-middle">

    <thead className="table-dark">

      <tr>

        <th>Mesin</th>
        <th>Status</th>
        <th>Output</th>
        <th>Operator</th>
        <th>Shift</th>

      </tr>

    </thead>

    <tbody>

      <tr>

        <td>CNC-01</td>

        <td>
          <span className="badge bg-success">
            Running
          </span>
        </td>

        <td>320 Unit</td>
        <td>Bambang</td>
        <td>Pagi</td>

      </tr>

      <tr>

        <td>Press-01</td>

        <td>
```

```
        <span className="badge bg-danger">
          Stop
        </span>
      </td>

      <td>150 Unit</td>
      <td>Fadhil</td>
      <td>Siang</td>

    </tr>

    <tr>

      <td>Weld-04</td>

      <td>
        <span className="badge bg-warning text-dark">
          Maintenance
        </span>
      </td>

      <td>0 Unit</td>
      <td>Rizky</td>
      <td>Malam</td>

    </tr>

  </tbody>

</table>

</div>

</div>

</div>

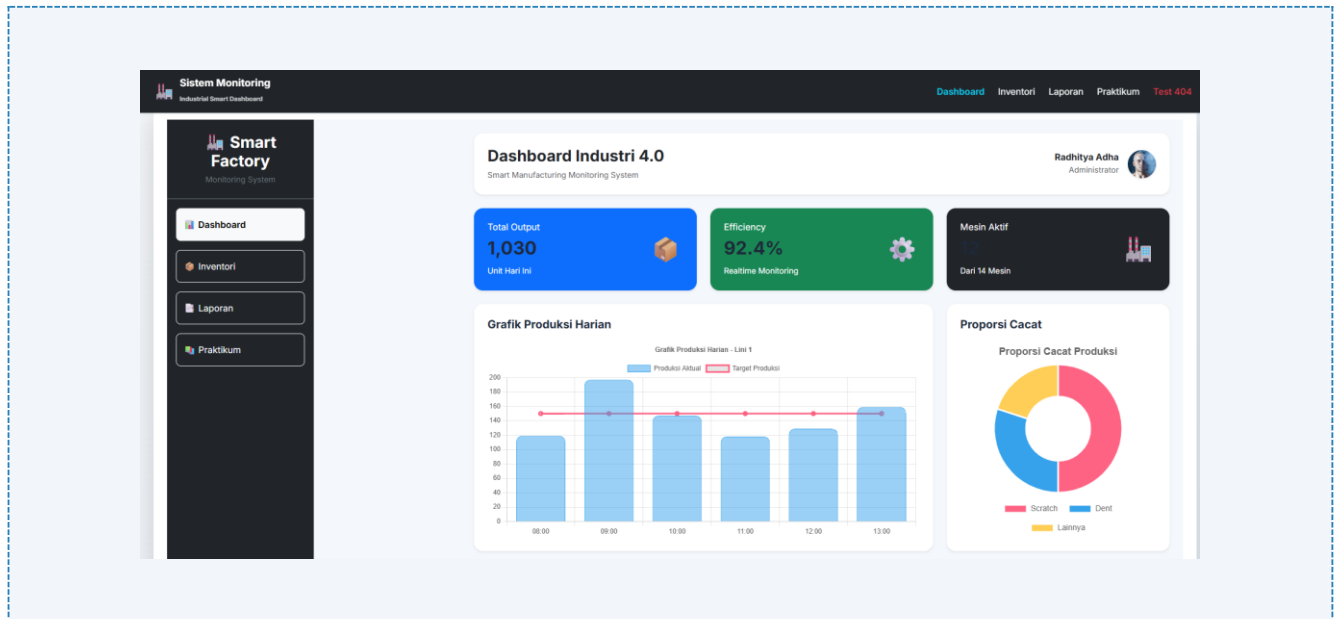
</div>

</div>

</div>

  );
}

export default Dashboard;
```



Gambar F.3 – Output dari Tugas Akhir Web

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang diperoleh sudah sesuai dengan teori dan konsep yang dipelajari pada materi React JS dan pengembangan web responsif. Pada implementasinya, saya berhasil menerapkan konsep component reusable seperti KartuMesin, GrafikProduksi, dan GrafikCacat sehingga kode menjadi lebih terstruktur dan mudah digunakan kembali. Selain itu, penggunaan useState pada sidebar juga sesuai dengan teori state management di React, karena sidebar dapat membuka dan menutup secara dinamis khususnya pada tampilan mobile.

Hasil tampilan dashboard juga sesuai dengan konsep responsive web design yang dipelajari menggunakan Bootstrap Grid System seperti col-md, col-lg, dan col-xl. Saat diuji pada ukuran layar berbeda, layout tetap menyesuaikan dengan baik tanpa merusak tampilan utama. Penggunaan Chart.js untuk grafik produksi dan grafik donat juga berhasil menampilkan data secara visual dan interaktif, sehingga dashboard terlihat lebih realistis seperti sistem monitoring industri nyata.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Tampilan dashboard berantakan dan kurang responsif saat dibuka di HP karena layout masih menggunakan ukuran statis	Menggunakan Bootstrap Grid System seperti col-md, col-lg, dan col-xl, serta menambahkan sidebar collapse dengan useState agar tampilan lebih rapi dan responsif di berbagai ukuran layar.

G.3 Kaitan dengan Penerapan di Industri

Keterampilan yang dipelajari pada modul ini sangat berkaitan dengan penerapan di dunia Teknik Industri, khususnya dalam bidang monitoring produksi dan digitalisasi sistem industri. Dashboard web seperti yang dibuat pada praktikum dapat digunakan untuk memantau kondisi mesin, jumlah output produksi, efisiensi kerja, hingga data kualitas produk secara realtime. Dengan adanya tampilan data yang visual dan interaktif, pengambilan keputusan di area produksi menjadi lebih cepat dan akurat.

Contoh penerapannya adalah pada pabrik manufaktur yang memiliki banyak mesin produksi. Supervisor dapat menggunakan dashboard untuk melihat mesin mana yang sedang running, maintenance, atau stop tanpa harus mengecek langsung ke lapangan. Selain itu, grafik produksi dan proporsi cacat juga dapat membantu bagian quality control dalam menganalisis performa produksi harian sehingga perusahaan dapat segera melakukan perbaikan apabila ditemukan penurunan output atau peningkatan cacat produk..

H. KESIMPULAN

1	Praktikum ini berhasil menerapkan React JS untuk membangun dashboard industri yang terdiri dari sidebar, header, grafik, kartu monitoring mesin, dan tabel data produksi secara terstruktur menggunakan konsep component reusable.
2	Penggunaan Bootstrap Grid System dan state useState berhasil membuat tampilan dashboard menjadi responsif, sehingga layout tetap rapi dan sidebar dapat collapse dengan baik pada tampilan HP maupun desktop.
3	Integrasi Chart.js pada grafik produksi dan grafik donat berhasil menampilkan data produksi serta proporsi cacat secara visual dan interaktif, sehingga dashboard terlihat lebih realistis seperti sistem monitoring industri di dunia nyata.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Chart.js. (2025). <i>Chart.js documentation</i> . Chart.js Official Website. https://www.chartjs.org/docs/latest/
[3]	Bootstrap. (2025). <i>Bootstrap documentation</i> . Bootstrap Official Website. https://getbootstrap.com/docs/5.3/getting-started/introduction/

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
Radhitya Adha Fadhlur Rohman NIM: 23051430005	Maessa Andrea Vallenia NIM: 23051430044	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

Modul 14 Pengenalan React Native & Setup Expo

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 14 (Pertemuan 14)
Topik Utama	Pengenalan React Native & Setup Expo
Sub-Topik	<i>Konsep Cross-Platform, Setup Lingkungan Expo, Komponen Dasar (View, Text, Image), dan Styling di Mobile.</i>
Durasi Praktikum	340 Menit
Tanggal Praktikum	15 Mei 2026
Nama Mahasiswa	Radhitya Adha Fadhlur Rohman
NIM	23051430005
Kelas	A2

B. TUJUAN PEMBELAJARAN

1	Memahami perbedaan antara Web Development (React.js) dan Mobile Development (React Native).
2	Mampu menyiapkan lingkungan kerja React Native menggunakan Expo.
3	Menguasai komponen dasar pengganti HTML: `View` (div), `Text` (p/h1), `ScrollView`.
4	Mampu menata tampilan mobile menggunakan `StyleSheet`.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	File `index.html`	Versi Terbaru	Hasil pekerjaan pertemuan sebelumnya
4	Ekstensi VS Code: LiveServer	Versi terbaru	Melihat perubahan web secara real-time.

5	Javascript	Versi terbaru	Membuat halaman web menjadi lebih interaktif dan fungsional
6	Node.js	Versi Terbaru	<i>runtime environment</i> JavaScript
7	Expo Go	Versi Terbaru	lingkungan pengembangan instan (<i>sandbox</i>) untuk menjalankan, menguji, dan meninjau kode React Native dari laptop ke perangkat HP secara <i>real-time</i>

D. DASAR TEORI

React Native merupakan framework pengembangan aplikasi mobile yang dikembangkan oleh Meta (Facebook) untuk membangun aplikasi Android maupun iOS menggunakan bahasa pemrograman JavaScript dan library React. React Native memungkinkan pengembang membuat antarmuka aplikasi menggunakan komponen bawaan seperti View, Text, Image, ScrollView, dan TouchableOpacity tanpa harus menulis kode native Android atau iOS secara terpisah. Dengan pendekatan *component-based*, proses pengembangan aplikasi menjadi lebih efisien, mudah dipelajari, serta mendukung konsep *code reusability* sehingga komponen yang sudah dibuat dapat digunakan kembali pada bagian aplikasi lain.

Dalam pengembangan tampilan aplikasi mobile, React Native menggunakan sistem tata letak Flexbox untuk mengatur posisi dan ukuran komponen pada layar. Flexbox mempermudah pengembang dalam menyusun elemen secara vertikal maupun horizontal menggunakan properti seperti `flexDirection`, `justifyContent`, dan `alignItems`. Pada praktikum ini digunakan `flexDirection: 'row'` untuk menyusun foto mesin di sisi kiri dan informasi mesin di sisi kanan sehingga tampilan aplikasi menjadi lebih terstruktur dan responsif. Selain itu, komponen ScrollView digunakan agar pengguna dapat menggulir halaman ketika jumlah informasi yang ditampilkan melebihi ukuran layar perangkat.

React Native juga menyediakan fitur interaksi pengguna melalui komponen seperti TouchableOpacity dan Alert. TouchableOpacity memungkinkan suatu elemen menerima sentuhan atau klik dari pengguna sehingga aplikasi menjadi lebih interaktif. Sedangkan Alert digunakan untuk menampilkan pesan atau informasi tertentu ketika pengguna melakukan suatu aksi. Selain itu, komponen Image digunakan untuk menampilkan gambar baik dari penyimpanan lokal maupun sumber daring (*online resource*). Pada modul ini konsep-konsep tersebut diterapkan untuk membangun aplikasi monitoring gudang dan profil mesin produksi yang menampilkan informasi mesin, status operasional, serta visualisasi gambar sehingga aplikasi menjadi lebih informatif dan mudah digunakan.

D.1 Konsep Utama

Konsep utama pada modul ini adalah pembuatan antarmuka (*user interface*) aplikasi mobile menggunakan React Native dengan memanfaatkan berbagai komponen seperti View, Text, Image, ScrollView, dan TouchableOpacity. Selain itu, modul juga mengenalkan penggunaan Flexbox untuk mengatur tata letak komponen agar tampilan aplikasi lebih rapi dan responsif. Pada praktikum ini, konsep tersebut diterapkan untuk membuat aplikasi monitoring gudang dan profil mesin sehingga aplikasi tidak hanya menampilkan informasi, tetapi juga memiliki interaksi dengan pengguna melalui tombol dan notifikasi (*alert*).

D.2 Keterkaitan dengan Teknik Industri

Materi pada modul ini relevan dengan Teknik Industri karena membantu dalam pengembangan sistem informasi yang dapat digunakan untuk mendukung proses operasional di perusahaan. Aplikasi mobile yang dibuat dapat dimanfaatkan untuk memantau kondisi gudang, status mesin produksi, maupun data operasional secara *real-time* sehingga mempermudah proses pengambilan keputusan.

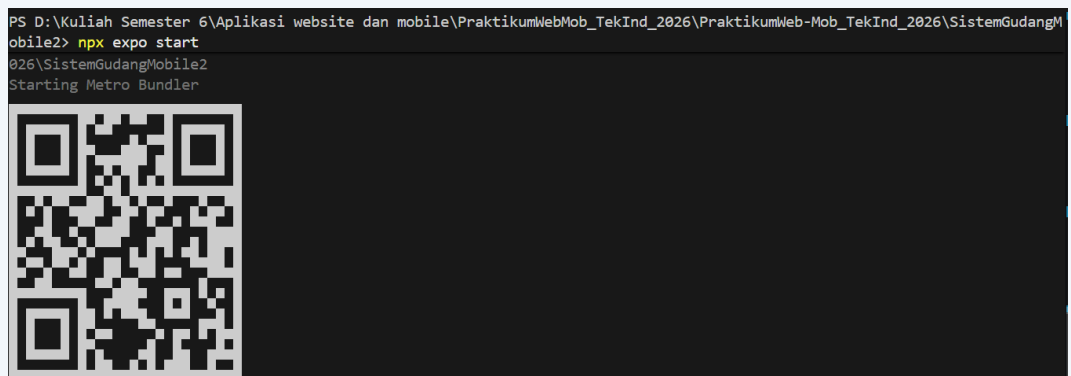
Contoh penerapannya dalam dunia Teknik Industri adalah pembuatan aplikasi monitoring gudang untuk melihat kapasitas penyimpanan material atau aplikasi profil mesin untuk memantau kondisi mesin produksi. Dengan adanya sistem seperti ini, perusahaan dapat meningkatkan efisiensi kerja, mempercepat akses informasi, serta membantu pengendalian proses produksi dan manajemen operasional.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat Project Expo

```
=== TERMINAL ===  
Npx create-expo-app SistemGudangMobile  
Cd SistemGudangMobile  
Npx expo start
```

Screenshot Hasil:



Gambar E.1 – Kode dari Langkah 1

E.2 Langkah 2 – Memodifikasi App.js (UI Dashboard Sederhana)

```
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, SafeAreaView, Platform } from 'react-native';

export default function App() {
  return (
    <SafeAreaView style={styles.container}>
      <StatusBar style="auto" />

      {/* Bagian Header */}
      <View style={styles.header}>
        <Text style={styles.headerTitle}>PT. Manufaktur Maju</Text>
        <Text style={styles.headerSubtitle}>Aplikasi Monitoring
Gudang</Text>
      </View>

      {/* Bagian Konten Utama */}
      <View style={styles.content}>
        <Text style={styles.welcomeText}>Selamat Datang, Operator!</Text>

        {/* Card Gudang A */}
        <View style={styles.card}>
          <Text style={styles.cardTitle}>Status Gudang A</Text>
          <Text style={styles.cardValue}>Kapasitas: 85%</Text>
          <Text style={styles.cardStatus}>TERSEDIA</Text>
        </View>

        {/* Card Gudang B (Dengan Warning) */}
        <View style={[styles.card, styles.cardWarning]}>
          <Text style={styles.cardTitle}>Status Gudang B</Text>
          <Text style={styles.cardValue}>Kapasitas: 95%</Text>
          <Text style={[styles.cardStatus,
styles.textWarning]}>PENUH</Text>
        </View>
      </View>
    </SafeAreaView>
  );
}

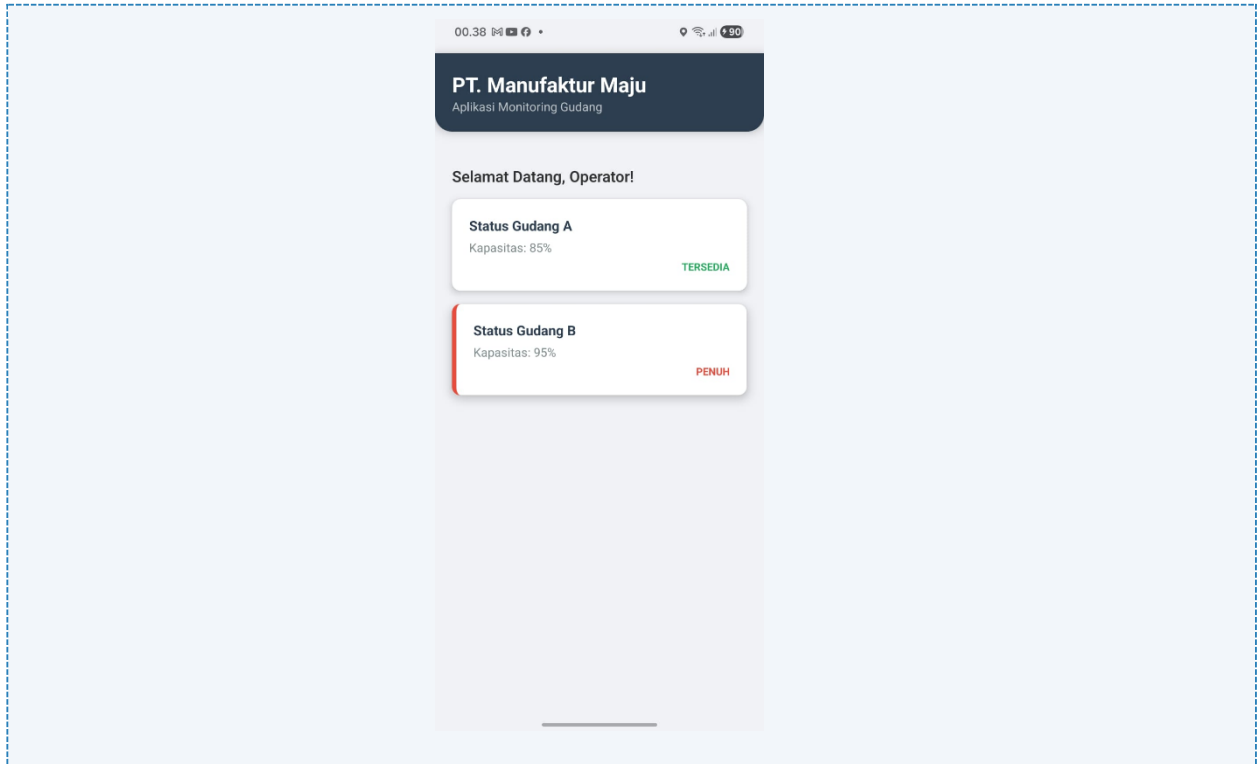
// Styling menggunakan StyleSheet
const styles = StyleSheet.create({
  container: {
    flex: 1, // Memenuhi seluruh layar
    backgroundColor: '#f0f2f5',
    paddingTop: Platform.OS === 'android' ? 40 : 0, // Mengatasi status
bar Android agar tidak menumpuk
  },
  header: {
    backgroundColor: '#2c3e50',
    padding: 20,
    borderBottomLeftRadius: 20,
    borderBottomRightRadius: 20,
    marginBottom: 20,
    elevation: 5, // Shadow untuk Android
    shadowColor: '#000', // Shadow untuk iOS
    shadowOffset: { width: 0, height: 2 },
  },
});
```

```
        shadowOpacity: 0.2,
        shadowRadius: 3,
    },
    headerTitle: {
        color: 'white',
        fontSize: 24,
        fontWeight: 'bold',
    },
    headerSubtitle: {
        color: '#bdc3c7',
        fontSize: 14,
    },
    content: {
        padding: 20,
    },
    welcomeText: {
        fontSize: 18,
        marginBottom: 15,
        color: '#333',
        fontWeight: '600',
    },
    card: {
        backgroundColor: 'white',
        padding: 20,
        borderRadius: 10,
        marginBottom: 15,
        // Shadow untuk iOS & Android
        shadowColor: "#000",
        shadowOffset: { width: 0, height: 2 },
        shadowOpacity: 0.1,
        shadowRadius: 3.84,
        elevation: 5,
    },
    cardWarning: {
        borderLeftWidth: 5,
        borderLeftColor: '#e74c3c', // Garis merah di sebelah kiri untuk
peringatan
    },
    cardTitle: {
        fontSize: 16,
        fontWeight: 'bold',
        marginBottom: 5,
        color: '#2c3e50',
    },
    cardValue: {
        fontSize: 14,
        color: '#7f8c8d',
    },
    cardStatus: {
        fontSize: 12,
        fontWeight: 'bold',
        color: '#27ae60', // Warna hijau untuk status tersedia
        marginTop: 5,
        textAlign: 'right',
    },
    textWarning: {
        color: '#e74c3c', // Warna merah untuk status penuh
    },

```

```
});
```

Screenshot Hasil:



Gambar E.2 – Output dari Langkah 2

E.3 Langkah 3 – Menambahkan Interaksi (Button & Alert)

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */

=== mengubah import paling atas ===
import {
  StyleSheet,
  Text,
  View,
  SafeAreaView,
  Platform,
  TouchableOpacity,
  Alert,
} from 'react-native';

=== mengubah salah satu kartu Gudang A menjadi bisa di klik ===

// Di dalam return(...), ganti View Card Gudang A dengan:

{/* ===== Card Gudang A ===== */}
<TouchableOpacity
  style={styles.card}
```

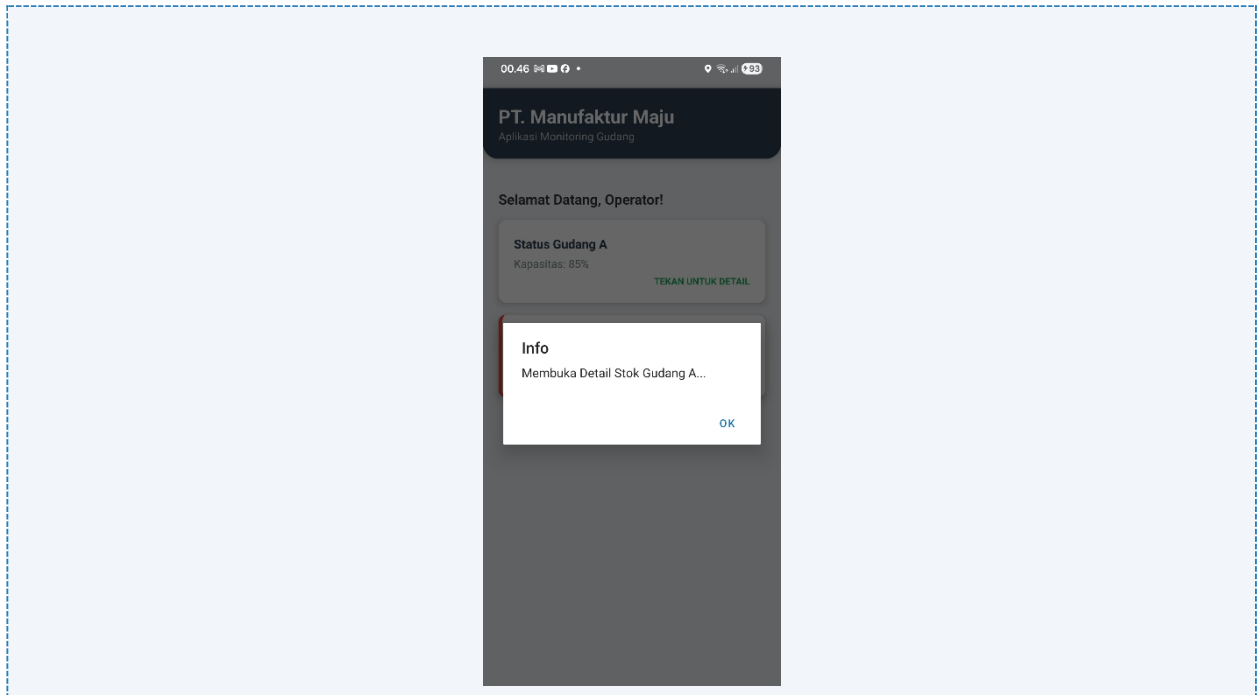
```
onPress={ () =>
  Alert.alert(
    "Info",
    "Membuka Detail Stok Gudang A..."
  )
}
>
<Text style={styles.cardTitle}>
  Status Gudang A
</Text>

<Text style={styles.cardValue}>
  Kapasitas: 85%
</Text>

<Text style={styles.cardStatus}>
  TEKAN UNTUK DETAIL
</Text>

</TouchableOpacity>
```

Screenshot Hasil:



Gambar E.3 – Output dari Langkah 3

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Gambar / Ikon

Pertanyaan / Instruksi Latihan:

Cari gambar logo perusahaan (atau gunakan placeholder), taruh di folder project, lalu tampilkan menggunakan `

Jawaban / Kode Solusi:

```
import React from "react";
import {
  StyleSheet,
  Text,
  View,
  ScrollView,
  StatusBar,
  Alert,
  TouchableOpacity,
  Image,
} from "react-native";

export default function App() {

  // Detail Gudang A
  const detailGudangA = () => {
    Alert.alert(
      "Detail Gudang A",
      "Kondisi: Aman\nKapasitas saat ini: 85%\nSisa slot penyimpanan masih tersedia untuk beban logistik baru.",
      [
        {
          text: "Tutup",
          style: "cancel",
        },
      ],
    );
  };

  // Detail Gudang B
  const detailGudangB = () => {
    Alert.alert(
      "Peringatan Gudang B",
      "Kondisi: KRITIS!\nKapasitas saat ini: 95%\nSegera lakukan pengosongan atau pemindahan material.",
      [
        {
          text: "Mengerti",
          style: "destructive",
        },
      ],
    );
  };

  return (
    <ScrollView style={styles.container}>
      <StatusBar
        barStyle="light-content"
        backgroundColor="#2c3e50"
      />
    </ScrollView>
  );
}
```



```
    { /* Header */ }
    <View style={styles.header}>

        <Image
            source={require("../pngwing.com (2).png")}
            style={styles.companyLogo}
        />

        <View>

            <Text style={styles.title}>
                PT. Manufaktur Maju
            </Text>

            <Text style={styles.subtitle}>
                Aplikasi Monitoring Gudang
            </Text>

        </View>

    </View>

    { /* Welcome */ }
    <Text style={styles.welcomeText}>
        Selamat Datang, Operator!
    </Text>

    { /* Gudang A */ }
    <TouchableOpacity
        style={styles.card}
        onPress={detailGudangA}
        activeOpacity={0.8}
    >

        <View style={styles.cardContent}>

            <View style={styles.textContainer}>

                <Text style={styles.gudangTitle}>
                    Status Gudang A
                </Text>

                <Text style={styles.gudangKapasitas}>
                    Kapasitas: 85%
                </Text>

            </View>

            <Text
                style={[
                    styles.statusBadge,
                    styles.statusAman,
                ]}
            >
                TERSEDIA
            </Text>

        </View>

    </View>
```

```
        </TouchableOpacity>

        { /* Gudang B */ }
        <TouchableOpacity
          style={ [
            styles.card,
            styles.cardAlert,
          ] }
          onPress={ detailGudangB }
          activeOpacity={ 0.8 }
        >

          <View style={ styles.cardContent }>

            <View style={ styles.textContainer }>

              <Text style={ styles.gudangTitle }>
                Status Gudang B
              </Text>

              <Text style={ styles.gudangKapasitas }>
                Kapasitas: 95%
              </Text>

            </View>

            <Text
              style={ [
                styles.statusBadge,
                styles.statusBahaya,
              ] }
            >
              PENUH
            </Text>

          </View>

        </TouchableOpacity>

      </ScrollView>
    );
  }

  const styles = StyleSheet.create({

    container: {
      flex: 1,
      backgroundColor: "#f5f6fa",
      paddingTop: 50,
      paddingHorizontal: 20,
    },

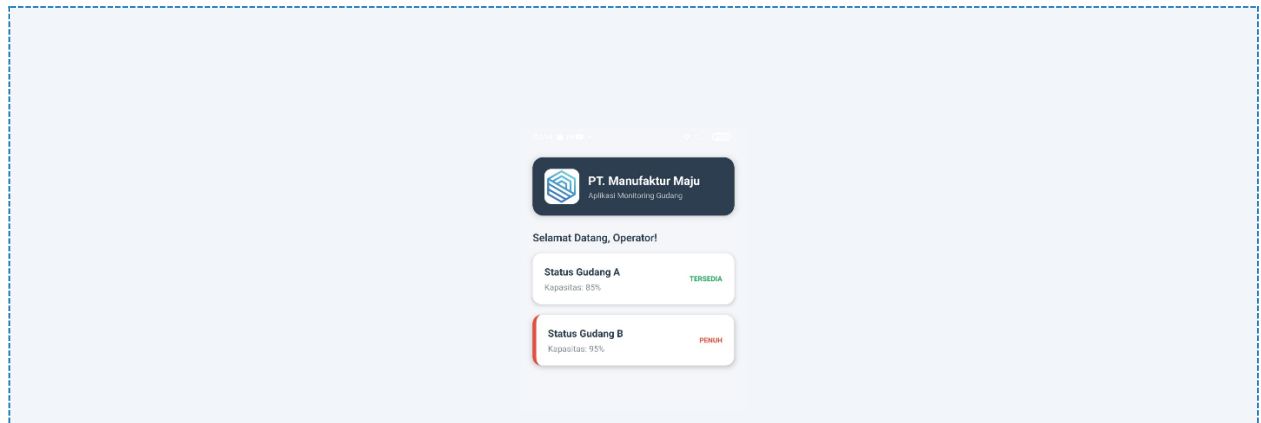
    header: {
      backgroundColor: "#2c3e50",

      padding: 20,
```

```
borderRadius: 18,  
  
marginBottom: 25,  
  
flexDirection: "row",  
  
alignItems: "center",  
  
elevation: 6,  
},  
  
companyLogo: {  
  width: 60,  
  height: 60,  
  
  marginRight: 15,  
  
  borderRadius: 12,  
  
  resizeMode: "contain",  
  
  backgroundColor: "white",  
},  
  
title: {  
  fontSize: 20,  
  
  fontWeight: "bold",  
  
  color: "white",  
},  
  
subtitle: {  
  fontSize: 13,  
  
  color: "#bdc3c7",  
  
  marginTop: 3,  
},  
  
welcomeText: {  
  fontSize: 18,  
  
  fontWeight: "600",  
  
  color: "#2c3e50",  
  
  marginBottom: 15,  
},  
  
card: {  
  backgroundColor: "white",  
  
  padding: 20,  
  
  borderRadius: 15,  
  
  marginBottom: 18,
```

```
elevation: 4,  
  
shadowColor: "#000",  
  
shadowOffset: {  
  width: 0,  
  height: 2,  
},  
  
shadowOpacity: 0.1,  
  
shadowRadius: 4,  
},  
  
cardAlert: {  
  borderLeftWidth: 6,  
  
  borderLeftColor: "#e74c3c",  
},  
  
cardContent: {  
  flexDirection: "row",  
  
  justifyContent: "space-between",  
  
  alignItems: "center",  
},  
  
textContainer: {  
  flex: 1,  
},  
  
gudangTitle: {  
  fontSize: 17,  
  
  fontWeight: "bold",  
  
  color: "#2c3e50",  
},  
  
gudangKapasitas: {  
  fontSize: 14,  
  
  marginTop: 5,  
  
  color: "#7f8c8d",  
},  
  
statusBadge: {  
  fontSize: 12,  
  
  fontWeight: "bold",  
},  
  
statusAman: {  
  color: "#27ae60",  
},
```

```
statusBahaya: {  
  color: "#e74c3c",  
},  
});
```



Gambar F.1 – Output dari Latihan 1

F.2 Latihan 2 – ScrollView

Pertanyaan / Instruksi Latihan:

Jika konten melebihi tinggi layar, gunakan `` untuk membungkus konten agar bisa digulir ke bawah.

Jawaban / Kode Solusi:

```
import React from "react";  
import {  
  StyleSheet,  
  Text,  
  View,  
  ScrollView,  
  StatusBar,  
  Alert,  
  TouchableOpacity,  
  Image,  
} from "react-native";  
  
export default function App() {  
  
  // Detail Gudang A  
  const detailGudangA = () => {  
    Alert.alert(  
      "Detail Gudang A",  
      "Kondisi: Aman\nKapasitas saat ini: 85%\nSisa slot penyimpanan masih  
tersedia untuk beban logistik baru.",  
      [  
        {  

```

```
        text: "Tutup",
        style: "cancel",
      },
    ]
  );
};

// Detail Gudang B
const detailGudangB = () => {
  Alert.alert(
    "Peringatan Gudang B",
    "Kondisi: KRITIS!\nKapasitas saat ini: 95%\nSegera lakukan pengosongan atau pemindahan material.",
    [
      {
        text: "Mengerti",
        style: "destructive",
      },
    ],
  );
};

return (
  <ScrollView style={styles.container}>
    <StatusBar
      barStyle="light-content"
      backgroundColor="#2c3e50"
    />

    {/* Header */}
    <View style={styles.header}>

      <Image
        source={require("../pngwing.com (2).png")}
        style={styles.companyLogo}
      />

      <View>

        <Text style={styles.title}>
          PT. Manufaktur Maju
        </Text>

        <Text style={styles.subtitle}>
          Aplikasi Monitoring Gudang
        </Text>

      </View>

    </View>

    {/* Welcome */}
    <Text style={styles.welcomeText}>
      Selamat Datang, Operator!
    </Text>

    {/* Gudang A */}
    <TouchableOpacity
```

```
        style={styles.card}
        onPress={detailGudangA}
        activeOpacity={0.8}
      >

      <View style={styles.cardContent}>

        <View style={styles.textContainer}>

          <Text style={styles.gudangTitle}>
            Status Gudang A
          </Text>

          <Text style={styles.gudangKapasitas}>
            Kapasitas: 85%
          </Text>

        </View>

        <Text
          style={[
            styles.statusBadge,
            styles.statusAman,
          ]}
        >
          TERSEDIA
        </Text>

      </View>

    </TouchableOpacity>

    { /* Gudang B */ }
    <TouchableOpacity
      style={[
        styles.card,
        styles.cardAlert,
      ]}
      onPress={detailGudangB}
      activeOpacity={0.8}
    >

      <View style={styles.cardContent}>

        <View style={styles.textContainer}>

          <Text style={styles.gudangTitle}>
            Status Gudang B
          </Text>

          <Text style={styles.gudangKapasitas}>
            Kapasitas: 95%
          </Text>

        </View>

        <Text
          style={ [
```

```
        styles.statusBadge,  
        styles.statusBahaya,  
      ]}  
    >  
      PENUH  
    </Text>  
  
  </View>  
  
  </TouchableOpacity>  
  
  </ScrollView>  
);  
}  
  
const styles = StyleSheet.create({  
  
  container: {  
    flex: 1,  
    backgroundColor: "#f5f6fa",  
    paddingTop: 50,  
    paddingHorizontal: 20,  
  },  
  
  header: {  
    backgroundColor: "#2c3e50",  
  
    padding: 20,  
  
    borderRadius: 18,  
  
    marginBottom: 25,  
  
    flexDirection: "row",  
  
    alignItems: "center",  
  
    elevation: 6,  
  },  
  
  companyLogo: {  
    width: 60,  
    height: 60,  
  
    marginRight: 15,  
  
    borderRadius: 12,  
  
    resizeMode: "contain",  
  
    backgroundColor: "white",  
  },  
  
  title: {  
    fontSize: 20,  
  
    fontWeight: "bold",
```



```
        color: "white",
      },

      subtitle: {
        fontSize: 13,

        color: "#bdc3c7",

        marginTop: 3,
      },

      welcomeText: {
        fontSize: 18,

        fontWeight: "600",

        color: "#2c3e50",

        marginBottom: 15,
      },

      card: {
        backgroundColor: "white",

        padding: 20,

        borderRadius: 15,

        marginBottom: 18,

        elevation: 4,

        shadowColor: "#000",

        shadowOffset: {
          width: 0,
          height: 2,
        },

        shadowOpacity: 0.1,

        shadowRadius: 4,
      },

      cardAlert: {
        borderLeftWidth: 6,

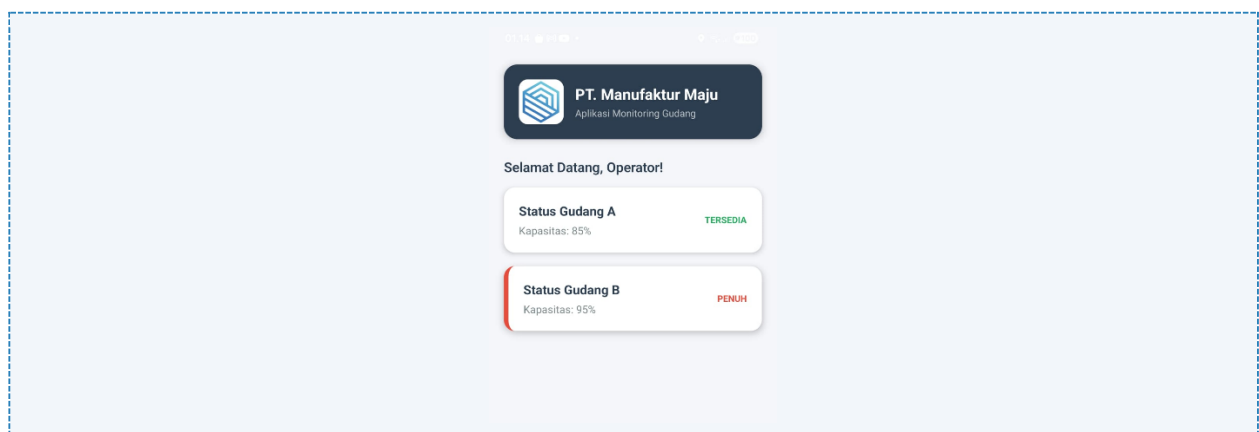
        borderLeftColor: "#e74c3c",
      },

      cardContent: {
        flexDirection: "row",

        justifyContent: "space-between",

        alignItems: "center",
      },
    },
```

```
textContainer: {  
  flex: 1,  
},  
  
gudangTitle: {  
  fontSize: 17,  
  
  fontWeight: "bold",  
  
  color: "#2c3e50",  
},  
  
gudangKapasitas: {  
  fontSize: 14,  
  
  marginTop: 5,  
  
  color: "#7f8c8d",  
},  
  
statusBadge: {  
  fontSize: 12,  
  
  fontWeight: "bold",  
},  
  
statusAman: {  
  color: "#27ae60",  
},  
  
statusBahaya: {  
  color: "#e74c3c",  
},  
  
});
```



Gambar F.2 – Output dari Latihan 2

F.3 Tugas Akhir Web

Deskripsi proyek mini:

- a. Buat halaman profil mesin yang menampilkan: Foto Mesin (gunakan placeholder), Nama Mesin, Tahun Pembuatan, dan Status.
- b. Gunakan Layout Flexbox (`flexDirection: 'row'`) untuk menyusun Foto di kiri dan Teks di kanan.

Penjelasan pendekatan/solusi yang digunakan:

Pendekatan yang digunakan pada proyek mini ini adalah membuat aplikasi mobile sederhana menggunakan React Native dengan memanfaatkan komponen bawaan seperti View, Text, Image, ScrollView, dan TouchableOpacity. Informasi mesin ditampilkan dalam bentuk kartu (*card layout*) agar tampilan lebih rapi dan mudah dibaca pengguna.

Selain itu, digunakan konsep Flexbox khususnya `flexDirection: 'row'` untuk mengatur posisi gambar mesin di sebelah kiri dan informasi mesin di sebelah kanan sesuai kebutuhan proyek. Komponen ScrollView digunakan agar halaman tetap dapat diakses dengan nyaman ketika jumlah data bertambah, sedangkan Alert dimanfaatkan untuk memberikan interaksi saat pengguna menekan bagian tertentu pada aplikasi.

Melalui pendekatan tersebut, aplikasi yang dibuat mampu menampilkan informasi mesin produksi dan monitoring gudang secara lebih terstruktur, interaktif, serta mudah digunakan pada perangkat mobile.

Kode Utama:

```
import React from "react";
import {
  StyleSheet,
  Text,
  View,
  ScrollView,
  StatusBar,
  Alert,
  TouchableOpacity,
  Image,
} from "react-native";

export default function App() {

  const mesinData = [
    {
      nama: "Mesin Filling AMDK",
      tahun: "2022",
      status: "AKTIF",
      warna: "#27ae60",
      gambar:
        "https://cdn-icons-png.flaticon.com/512/4341/4341134.png",
    },
    {
      nama: "Mesin Conveyor",
      tahun: "2020",
      status: "OPERASI",
      warna: "#3498db",
      gambar:
        "https://cdn-icons-png.flaticon.com/512/1995/1995470.png",
    },
  ],
```

```
{
  nama: "Mesin Labeling",
  tahun: "2021",
  status: "SIAP PRODUKSI",
  warna: "#8e44ad",
  gambar:
    "https://cdn-icons-png.flaticon.com/512/3082/3082037.png",
},

{
  nama: "Mesin Packaging",
  tahun: "2023",
  status: "AKTIF",
  warna: "#27ae60",
  gambar:
    "https://cdn-icons-png.flaticon.com/512/869/869636.png",
},

{
  nama: "Kompresor Udara",
  tahun: "2019",
  status: "MAINTENANCE",
  warna: "#f39c12",
  gambar:
    "https://cdn-icons-png.flaticon.com/512/2942/2942813.png",
},
];

const detailGudangA = () => {
  Alert.alert(
    "Gudang A",
    "Kapasitas 85%\nKondisi aman dan siap menerima material baru."
  );
};

const detailGudangB = () => {
  Alert.alert(
    "Gudang B",
    "Kapasitas 95%\nSegera lakukan pengurangan material."
  );
};

return (
  <ScrollView
    style={styles.container}
    showsVerticalScrollIndicator={false}
  >

    <StatusBar
      backgroundColor="#2c3e50"
      barStyle="light-content"
    />

    { /* HEADER */ }

    <View style={styles.header}>
```

```
<Image
  source={require("../pngwing.com (2).png")}
  style={styles.logo}
/>

<View>

  <Text style={styles.title}>
    PT. Manufaktur Maju
  </Text>

  <Text style={styles.subtitle}>
    Monitoring Gudang & Mesin
  </Text>

</View>

</View>

<Text style={styles.welcome}>
  Selamat Datang, Operator
</Text>

{/* GUDANG */}

<TouchableOpacity
  style={styles.card}
  onPress={detailGudangA}
>

  <View style={styles.row}>

    <View>

      <Text style={styles.cardTitle}>
        Gudang A
      </Text>

      <Text style={styles.cardInfo}>
        Kapasitas 85%
      </Text>

    </View>

    <View
      style={[
        styles.badge,
        { backgroundColor:"#27ae60" }
      ]}
    >

      <Text style={styles.badgeText}>
        TERSEDIA
      </Text>

    </View>

  </View>

</View>
```

```
</TouchableOpacity>

<TouchableOpacity
  style={[
    styles.card,
    styles.warningCard,
  ]}
  onPress={detailGudangB}
>

  <View style={styles.row}>

    <View>

      <Text style={styles.cardTitle}>
        Gudang B
      </Text>

      <Text style={styles.cardInfo}>
        Kapasitas 95%
      </Text>

    </View>

    <View
      style={[
        styles.badge,
        { backgroundColor:"#e74c3c" }
      ]}
    >

      <Text style={styles.badgeText}>
        PENUH
      </Text>

    </View>

  </View>

</TouchableOpacity>

{/* MESIN */}

<Text style={styles.section}>
  Profil Mesin Produksi
</Text>

{mesinData.map((mesin, index)=>(

  <View
    key={index}
    style={styles.machineCard}
  >

    <Image
      source={{
        uri:mesin.gambar
```

```
        }}
        style={styles.machineImage}
      />

      <View style={styles.machineInfo}>

        <Text style={styles.machineName}>
          {mesin.nama}
        </Text>

        <Text style={styles.machineText}>
          Tahun : {mesin.tahun}
        </Text>

        <View
          style={[
            styles.statusBox,
            {
              backgroundColor:
                mesin.warna
            }
          ]}
        >

          <Text style={styles.statusText}>
            {mesin.status}
          </Text>

        </View>

      </View>

    </View>

  )})

</ScrollView>
);
}

const styles=StyleSheet.create({

  container:{
    flex:1,
    backgroundColor:"#eef2f7",
    paddingTop:45,
    paddingHorizontal:18,
  },

  header:{
    backgroundColor:"#2c3e50",
    padding:18,
    borderRadius:18,
    flexDirection:"row",
    alignItems:"center",
    marginBottom:25,
    elevation:6,
  },
```

```
logo:{
width:60,
height:60,
marginRight:15,
resizeMode:"contain",
},

title:{
fontSize:20,
fontWeight:"700",
color:"white",
},

subtitle:{
color:"#dfe6e9",
marginTop:3,
},

welcome:{
fontSize:18,
fontWeight:"600",
marginBottom:18,
color:"#2c3e50",
},

card:{
backgroundColor:"white",
padding:18,
borderRadius:16,
marginBottom:15,
elevation:3,
},

warningCard:{
borderLeftWidth:5,
borderLeftColor:"#e74c3c",
},

row:{
flexDirection:"row",
justifyContent:"space-between",
alignItems:"center",
},

cardTitle:{
fontWeight:"700",
fontSize:17,
color:"#2c3e50",
},

cardInfo:{
marginTop:4,
color:"#7f8c8d",
},

badge:{
paddingHorizontal:12,
```



```
paddingVertical:7,
borderRadius:20,
},

badgeText:{
color:"white",
fontWeight:"700",
fontSize:11,
},

section:{
fontSize:20,
fontWeight:"700",
marginVertical:18,
color:"#2c3e50",
},

machineCard:{
backgroundColor:"white",
padding:16,
borderRadius:16,
flexDirection:"row",
alignItems:"center",
marginBottom:14,
elevation:3,
},

machineImage:{
width:75,
height:75,
marginRight:15,
},

machineInfo:{
flex:1,
},

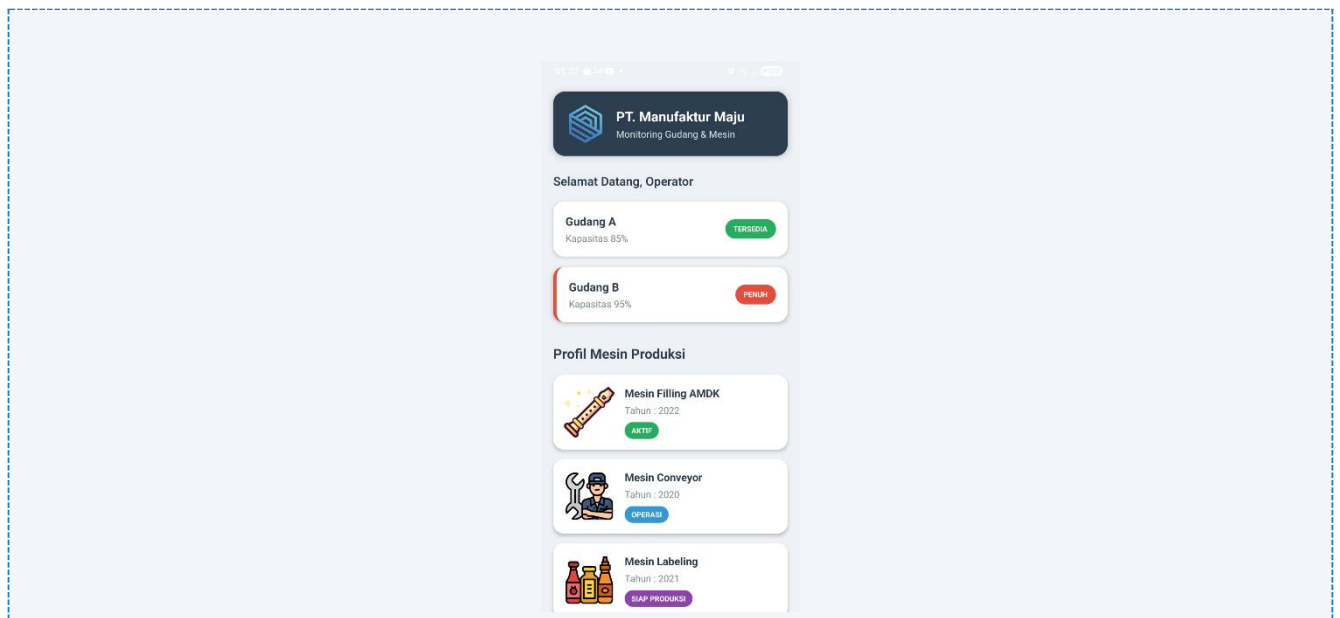
machineName:{
fontWeight:"700",
fontSize:16,
color:"#2c3e50",
},

machineText:{
marginTop:5,
color:"#7f8c8d",
},

statusBox:{
paddingVertical:5,
paddingHorizontal:10,
borderRadius:20,
marginTop:8,
alignSelf:"flex-start",
},

statusText:{
color:"white",
```

```
fontWeight:"700",  
fontSize:11,  
},  
  
});
```



Gambar F.3 – Output dari Tugas Akhir Web

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang diperoleh telah sesuai dengan teori dan konsep yang dipelajari pada modul. Hal ini terlihat dari penggunaan komponen React Native seperti View, Text, Image, ScrollView, dan TouchableOpacity yang berhasil diterapkan untuk membangun tampilan aplikasi monitoring gudang dan profil mesin. Komponen Image digunakan untuk menampilkan logo perusahaan dan gambar mesin, sehingga sesuai dengan teori mengenai penggunaan media visual pada aplikasi mobile.

Selain itu, konsep Flexbox juga berhasil diterapkan melalui penggunaan flexDirection: 'row' pada bagian profil mesin. Hasilnya, gambar mesin dapat ditampilkan di sisi kiri sedangkan informasi seperti nama mesin, tahun pembuatan, dan status berada di sisi kanan sesuai dengan tujuan praktikum. Tampilan yang dihasilkan menjadi lebih terstruktur dan mudah dipahami pengguna.

Penggunaan ScrollView juga menunjukkan kesesuaian dengan teori karena aplikasi tetap dapat menampilkan seluruh informasi ketika jumlah data mesin bertambah tanpa membuat tampilan terpotong. Komponen TouchableOpacity dan Alert juga berhasil memberikan interaksi pada kartu gudang sehingga aplikasi tidak hanya menampilkan informasi secara statis tetapi juga dapat merespons tindakan pengguna. Berdasarkan hasil tersebut, implementasi praktikum telah berjalan sesuai konsep React Native yang dipelajari pada modul baik dari sisi tampilan, tata letak, maupun interaktivitas aplikasi.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Saat awal membuat proyek Expo, file App.js tidak muncul karena menggunakan template proyek yang berbeda dari modul praktikum.	Saat awal membuat proyek Expo, file App.js tidak muncul karena menggunakan template proyek yang berbeda dari modul praktikum.

G.3 Kaitan dengan Penerapan di Industri

Keterampilan yang dipelajari pada modul ini dapat diterapkan dalam dunia Teknik Industri terutama pada pengembangan sistem monitoring dan pengelolaan informasi operasional perusahaan. Kemampuan membuat aplikasi mobile sederhana memungkinkan perusahaan memiliki sistem yang dapat digunakan untuk memantau kondisi gudang, mesin produksi, maupun proses operasional secara lebih cepat dan terintegrasi.

Contoh penerapan nyata adalah pada perusahaan manufaktur yang memiliki banyak mesin produksi. Perusahaan dapat menggunakan aplikasi mobile untuk menampilkan informasi nama mesin, status operasional, tahun penggunaan, hingga kondisi mesin secara *real-time*. Operator atau supervisor produksi dapat memantau kondisi mesin langsung melalui perangkat mobile tanpa harus melakukan pengecekan manual ke area produksi.

Selain itu, pada bidang manajemen gudang, aplikasi mobile dapat digunakan untuk memonitor kapasitas penyimpanan material, status stok, atau kondisi gudang sehingga membantu proses pengambilan keputusan yang lebih cepat. Penerapan teknologi seperti ini dapat meningkatkan efisiensi operasional, mengurangi keterlambatan informasi, serta mendukung proses digitalisasi industri yang saat ini banyak diterapkan pada perusahaan manufaktur modern.

H. KESIMPULAN

1	Pada praktikum ini berhasil dibuat aplikasi mobile sederhana menggunakan React Native dengan memanfaatkan komponen seperti View, Text, Image, ScrollView, dan TouchableOpacity untuk membangun tampilan aplikasi yang interaktif.
2	Konsep Flexbox khususnya flexDirection: 'row' berhasil diterapkan untuk mengatur tata letak foto mesin dan informasi mesin sehingga tampilan aplikasi menjadi lebih rapi, responsif, dan mudah dipahami pengguna.
3	Fitur interaksi seperti Alert serta penggunaan ScrollView berhasil diterapkan sehingga aplikasi tidak hanya menampilkan informasi secara statis, tetapi juga mampu memberikan pengalaman penggunaan yang lebih baik pada perangkat mobile

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Eisenman, B. (2020). <i>Learning React Native: Building Native Mobile Apps with JavaScript</i> . 2nd ed. Sebastopol: O'Reilly Media.
[3]	Meta Platforms, Inc. (n.d.). <i>React Native documentation</i> . React Native. https://reactnative.dev/docs/getting-started

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
Radhitya Adha Fadhlur Rohman NIM: 23051430005	Maessa Andrea Vallenia NIM: 23051430044	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

Modul 15 Component, Navigation, dan Styling di React Native + Proyek Mobile

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 15 (Pertemuan 15)
Topik Utama	Component, Navigation, dan Styling di React Native + Proyek Mobile
Sub-Topik	<i>React Navigation (Stack Navigator), Passing Data Antar Layar, dan Proyek Inventori Sederhana.</i>
Durasi Praktikum	340 Menit
Tanggal Praktikum	29 Mei 2026
Nama Mahasiswa	Radhitya Adha Fadhlur Rohman
NIM	23051430005
Kelas	A2

B. TUJUAN PEMBELAJARAN

1	Memahami konsep navigasi layar di aplikasi mobile (Stack).
2	Mampu menginstall dan mengonfigurasi `react-navigation`.
3	Membuat aplikasi multi-halaman: Beranda (List) → Detail Item.
4	Meneruskan data (params) dari satu layar ke layar lain.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Project Expo dari Pertemuan 14	-	Hasil pekerjaan pertemuan sebelumnya
4	Library Navigasi	Versi terbaru	Mengelola perpindahan, transisi,

			dan pertukaran data antar halaman
5	Javascript	Versi terbaru	Membuat halaman web menjadi lebih interaktif dan fungsional
6	Node.js	Versi Terbaru	<i>runtime environment</i> JavaScript
7	Expo Go	Versi Terbaru	lingkungan pengembangan instan (<i>sandbox</i>) untuk menjalankan, menguji, dan meninjau kode React Native dari laptop ke perangkat HP secara <i>real-time</i>

D. DASAR TEORI

Pada pengembangan aplikasi mobile menggunakan React Native, komponen merupakan bagian utama yang digunakan untuk membangun tampilan dan fungsi aplikasi. Setiap halaman atau screen dapat dibuat sebagai komponen yang memiliki tugas tertentu, misalnya menampilkan daftar data, menampilkan detail informasi, atau menerima input dari pengguna. Dengan pendekatan ini, kode program menjadi lebih terstruktur dan mudah untuk dikembangkan.

Salah satu konsep penting dalam aplikasi mobile adalah navigasi. Berbeda dengan aplikasi web yang berpindah halaman menggunakan URL, aplikasi mobile menggunakan sistem stack navigation. Setiap layar yang dibuka akan ditambahkan ke dalam tumpukan layar (stack), sehingga pengguna dapat berpindah dari halaman utama ke halaman detail dan kembali lagi dengan mudah. Pada React Native, proses navigasi dapat dilakukan menggunakan library React Navigation yang menyediakan berbagai jenis navigasi sesuai kebutuhan aplikasi.

Selain navigasi, pertukaran data antar layar juga menjadi bagian penting dalam pengembangan aplikasi mobile. Data dapat dikirim dari satu screen ke screen lain melalui parameter (params) saat proses navigasi berlangsung. Pada praktikum ini, konsep tersebut digunakan untuk menampilkan detail data yang dipilih serta memperbarui status hasil inspeksi. Dengan memanfaatkan state management sederhana menggunakan hook seperti `useState` dan `useEffect`, perubahan data dapat langsung ditampilkan pada antarmuka aplikasi secara dinamis tanpa perlu memuat ulang halaman.

D.1 Konsep Utama

Konsep utama pada modul ini adalah penerapan navigasi antar layar (screen navigation) dan pertukaran data antar halaman dalam aplikasi mobile menggunakan React Native. Navigasi memungkinkan pengguna berpindah dari satu halaman ke halaman lainnya, misalnya dari halaman daftar data menuju halaman detail informasi. Untuk mendukung proses tersebut digunakan React Navigation yang mengatur alur perpindahan layar agar aplikasi dapat berjalan secara terstruktur dan mudah digunakan.

Selain navigasi, modul ini juga membahas cara mengirim dan menerima data antar screen menggunakan parameter (params). Data yang dipilih pada halaman utama dapat diteruskan ke halaman detail untuk ditampilkan atau diolah lebih lanjut. Pada praktikum ini, konsep tersebut diterapkan pada aplikasi inspeksi QC, di mana data item yang dipilih dikirim ke halaman detail, kemudian status hasil inspeksi diperbarui dan dikirim kembali ke halaman utama. Dengan bantuan state management sederhana menggunakan useState dan useEffect, perubahan status dapat langsung terlihat pada tampilan aplikasi secara dinamis.

D.2 Keterkaitan dengan Teknik Industri

Materi pada modul ini memiliki keterkaitan yang erat dengan bidang Teknik Industri, terutama dalam pengelolaan sistem informasi, pengendalian kualitas, dan digitalisasi proses operasional di perusahaan. Seorang insinyur industri tidak hanya bertugas merancang sistem kerja yang efisien, tetapi juga memanfaatkan teknologi untuk mempermudah proses pengumpulan, pengolahan, dan penyajian data yang dibutuhkan dalam pengambilan keputusan. Oleh karena itu, kemampuan mengembangkan aplikasi sederhana berbasis mobile dapat menjadi salah satu keterampilan pendukung yang bermanfaat.

Salah satu contoh penerapannya adalah pada proses *Quality Control (QC)* di lingkungan manufaktur. Petugas inspeksi dapat menggunakan aplikasi mobile untuk melihat daftar produk yang harus diperiksa, mencatat hasil inspeksi, serta memberikan status lolos atau gagal secara langsung melalui perangkat mobile. Data tersebut kemudian dapat digunakan sebagai dasar evaluasi kualitas produk dan tindakan perbaikan apabila ditemukan ketidaksesuaian terhadap standar yang telah ditetapkan.

Pada praktikum ini, konsep tersebut diterapkan melalui aplikasi inspeksi QC sederhana yang menampilkan daftar item, detail standar kualitas, serta hasil inspeksi. Ketika suatu item dinyatakan gagal, sistem secara otomatis memberikan penanda warna merah pada daftar item sehingga memudahkan pengguna dalam mengidentifikasi produk yang memerlukan tindak lanjut. Penerapan seperti ini mencerminkan bagaimana teknologi informasi dapat mendukung aktivitas pengendalian kualitas dan meningkatkan efektivitas proses kerja di dunia industri.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

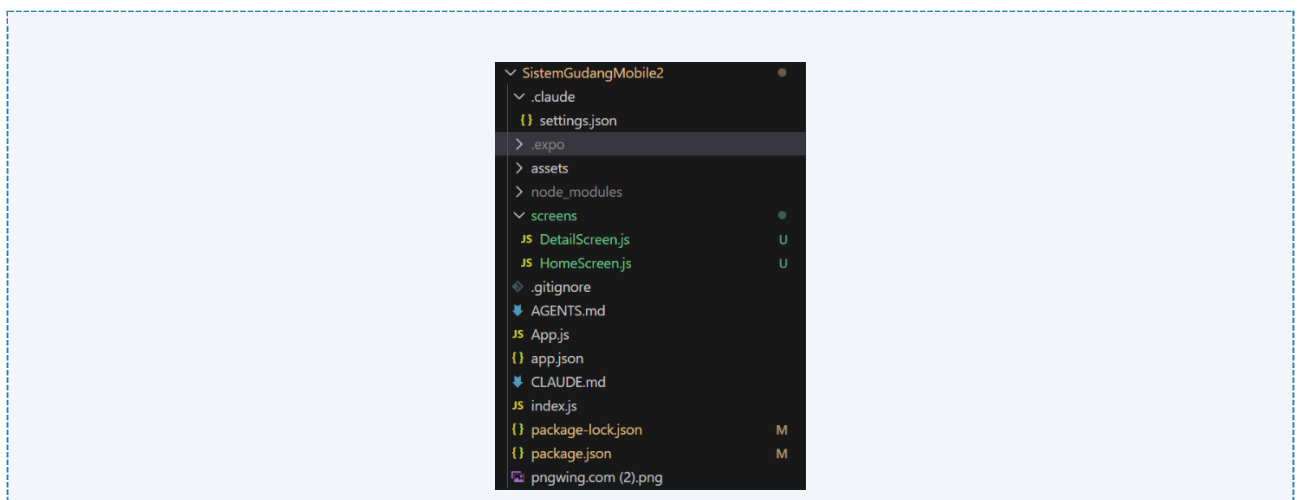
E.1 Langkah 1 – Struktur Folder

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
```

```
SistemGudangMobile/  
├── node_modules/  
├── screens  
│   ├── HomeScreen.js  
│   └── DetailScreen.js  
├── App.js  
└── package.json
```

Instalasi library navigasi (jalankan di terminal project):
Install library navigasi dan dependensinya
npm install @react-navigation/native @react-navigation/native-stack
npx expo install react-native-screens react-native-safe-area-context

Screenshot Hasil:



Gambar E.1 – Kode dari Langkah 1

E.2 Langkah 2 – Membuat HomeScreen (Daftar Inventori)

```
import React from 'react';  
import {  
  View,  
  Text,  
  FlatList,  
  TouchableOpacity,  
  StyleSheet,  
} from 'react-native';  
  
// Data Mock Inventori  
const DATA_INVENTORI = [
```

```
{ id: '1', nama: 'Baut M10', stok: 500, lokasi: 'Rak A-1' },
{ id: '2', nama: 'Oli Mesin 20L', stok: 12, lokasi: 'Rak B-3' },
{ id: '3', nama: 'Packing Kayu', stok: 100, lokasi: 'Gudang Luar' },
{ id: '4', nama: 'Mur Ring 12', stok: 0, lokasi: 'Rak A-2' }, // Stok
Habis
];

function HomeScreen({ navigation }) {
  // Fungsi Render Item untuk FlatList
  const renderItem = ({ item }) => (
    <TouchableOpacity
      style={styles.itemContainer}
      onPress={() =>
        navigation.navigate('Detail', { itemData: item })
      }
    >
    <Text style={styles.itemTitle}>{item.nama}</Text>

    <View style={styles.itemInfo}>
      <Text style={styles.itemSub}>Stok: {item.stok}</Text>
      <Text style={styles.itemSub}>{item.lokasi}</Text>
    </View>
    </TouchableOpacity>
  );

  return (
    <View style={styles.container}>
      <Text style={styles.header}>Daftar Inventori Gudang</Text>

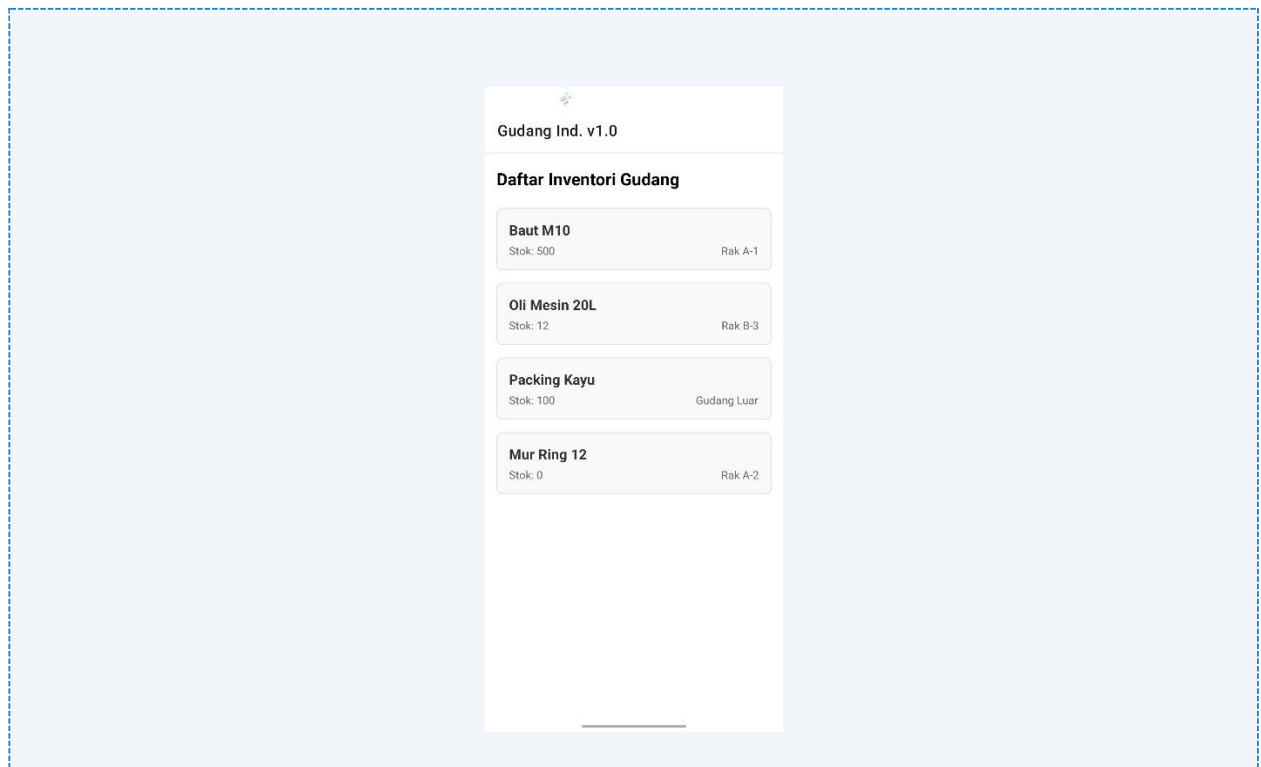
      <FlatList
        data={DATA_INVENTORI}
        renderItem={renderItem}
        keyExtractor={(item) => item.id}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    paddingTop: 20,
  },

  header: {
```

```
        fontSize: 22,  
        fontWeight: 'bold',  
        marginBottom: 15,  
        paddingHorizontal: 15,  
      },  
  
      itemContainer: {  
        backgroundColor: '#f9f9f9',  
        padding: 15,  
        marginVertical: 8,  
        marginHorizontal: 15,  
        borderRadius: 8,  
        borderWidth: 1,  
        borderColor: '#ddd',  
      },  
  
      itemTitle: {  
        fontSize: 18,  
        fontWeight: 'bold',  
        color: '#333',  
      },  
  
      itemInfo: {  
        flexDirection: 'row',  
        justifyContent: 'space-between',  
        marginTop: 5,  
      },  
  
      itemSub: {  
        color: '#666',  
      },  
    },  
  ));  
  
export default HomeScreen;
```

Screenshot Hasil:



Gambar E.2 – Output dari Langkah 2

E.3 Langkah 3 – Membuat DetailScreen

```
import React from 'react';
import { View, Text, StyleSheet, Button } from 'react-native';

function DetailScreen({ route, navigation }) {
  // Menerima data yang dikirim dari HomeScreen
  const { itemData } = route.params;

  return (
    <View style={styles.container}>
      <View style={styles.card}>
        <Text style={styles.label}>Nama Barang:</Text>
        <Text style={styles.value}>{itemData.nama}</Text>

        <Text style={styles.label}>Stok Saat Ini:</Text>
        <Text
          style={[
            styles.value,
            itemData.stok < 20
              ? styles.dangerText
              : styles.successText,
          ]>
          {itemData.stok} Unit
        </Text>
      </View>
    </View>
  );
}
```

```
        </Text>

        <Text style={styles.label}>Lokasi Penyimpanan:</Text>
        <Text style={styles.value}>{itemData.lokasi}</Text>
    </View>

    <Button
        title="Kembali ke Daftar"
        onPress={() => navigation.goBack()}
    />
</View>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    padding: 20,
    backgroundColor: '#f0f0f0',
  },

  card: {
    backgroundColor: 'white',
    padding: 20,
    borderRadius: 10,
    marginBottom: 20,
    elevation: 3,
  },

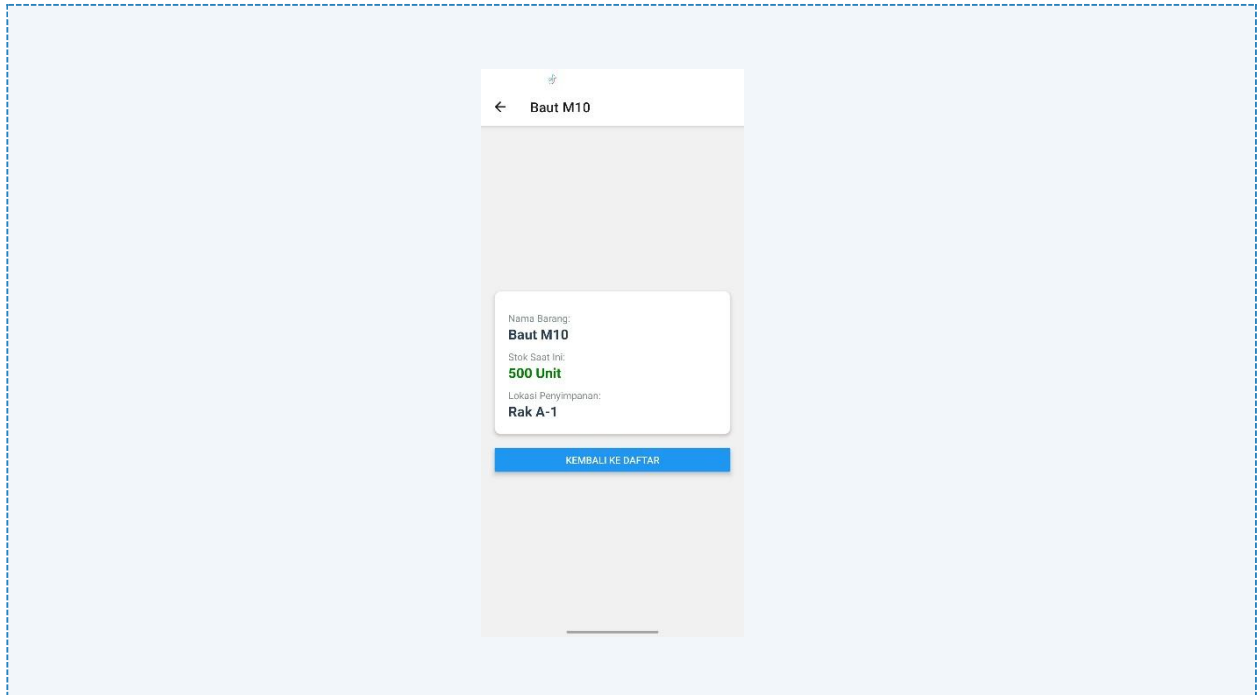
  label: {
    fontSize: 14,
    color: '#7f8c8d',
    marginTop: 10,
  },

  value: {
    fontSize: 20,
    fontWeight: 'bold',
    color: '#2c3e50',
  },

  dangerText: {
    color: 'red',
  },
});
```

```
    successText: {  
      color: 'green',  
    },  
  },  
});  
  
export default DetailScreen;
```

Screenshot Hasil:



Gambar E.3 – Output dari Langkah 3

E.3 Langkah 4 – Menghubungkan Navigation di App.js

```
import React from 'react';  
import { NavigationContainer } from '@react-navigation/native';  
import { createNativeStackNavigator } from '@react-navigation/native-stack';  
  
// Import Screens  
import HomeScreen from './screens/HomeScreen';  
import DetailScreen from './screens/DetailScreen';  
  
// Membuat Stack Navigator  
const Stack = createNativeStackNavigator();  
  
export default function App() {  
  return (  

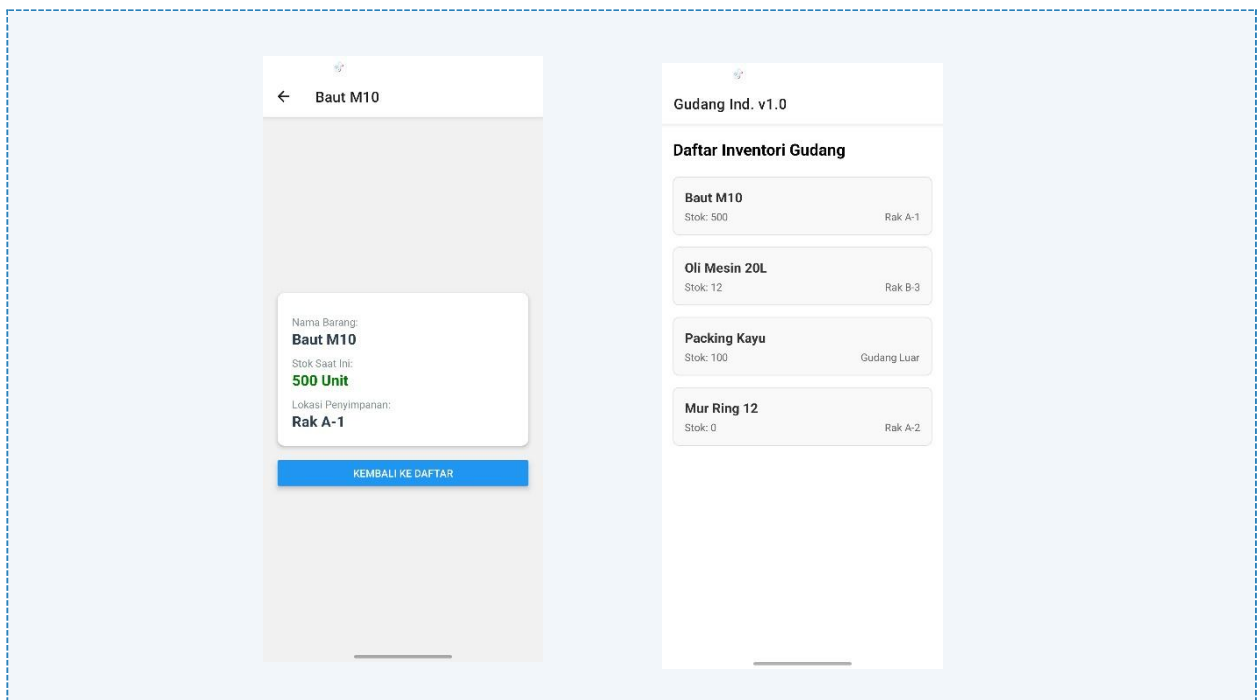
```

```
<NavigationContainer>
  <Stack.Navigator initialRouteName="Home">
    <Stack.Screen
      name="Home"
      component={HomeScreen}
      options={{ title: 'Gudang Ind. v1.0' }}
    />

    <Stack.Screen
      name="Detail"
      component={DetailScreen}
      options={({ route }) => ({
        title: route.params?.itemData.nama || 'Detail',
      })}
    />
  </Stack.Navigator>
</NavigationContainer>

);
}
```

Screenshot Hasil:



Gambar E.4 – Output dari Langkah 4

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Conditional Rendering

Pertanyaan / Instruksi Latihan:

Pada `DetailScreen`, jika stok = 0, munculkan tombol "Request Stok Darurat" (gunakan Button tambahan).

Jawaban / Kode Solusi:

```
import React from 'react';
import { View, Text, StyleSheet, Button } from 'react-native';

function DetailScreen({ route, navigation }) {
  // Menerima data yang dikirim dari HomeScreen
  const { itemData } = route.params;

  return (
    <View style={styles.container}>
      <View style={styles.card}>
        <Text style={styles.label}>Nama Barang:</Text>
        <Text style={styles.value}>{itemData.nama}</Text>

        <Text style={styles.label}>Stok Saat Ini:</Text>
        <Text
          style={[
            styles.value,
            itemData.stok < 20
              ? styles.dangerText
              : styles.successText,
          ]>
          <{itemData.stok}> Unit
        </Text>

        <Text style={styles.label}>Lokasi Penyimpanan:</Text>
        <Text style={styles.value}>{itemData.lokasi}</Text>
      </View>

      {/* Conditional Rendering */}
      {itemData.stok === 0 && (
        <View style={styles.buttonSpacing}>
          <Button
            title="Request Stok Darurat"
            onPress={() =>
              alert('Permintaan stok darurat telah dikirim!')
            }
          />
        </View>
      )}
    </View>
  );
}
```



```
        </View>
      )}

      <Button
        title="Kembali ke Daftar"
        onPress={() => navigation.goBack()}
      />
    </View>
  );
}

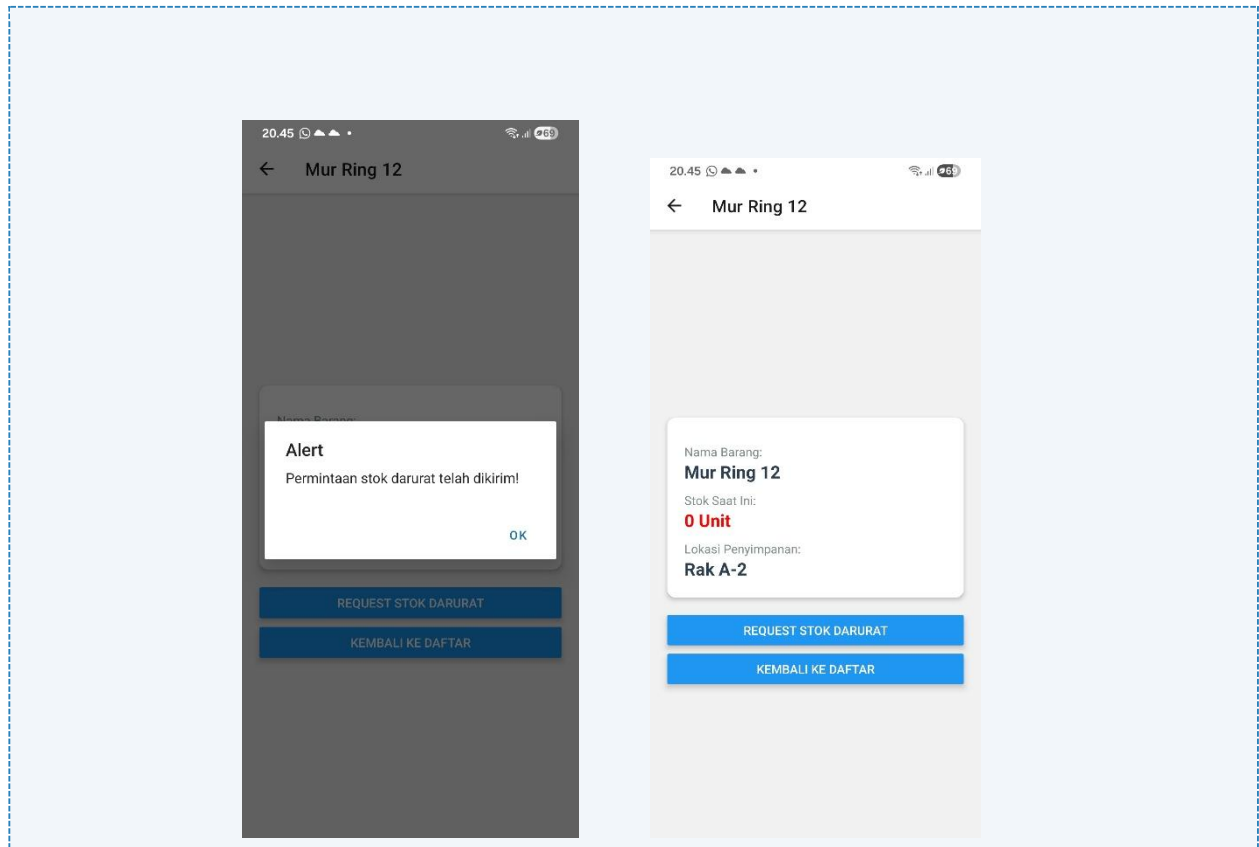
const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    padding: 20,
    backgroundColor: '#f0f0f0',
  },
  card: {
    backgroundColor: 'white',
    padding: 20,
    borderRadius: 10,
    marginBottom: 20,
    elevation: 3,
  },
  label: {
    fontSize: 14,
    color: '#7f8c8d',
    marginTop: 10,
  },
  value: {
    fontSize: 20,
    fontWeight: 'bold',
    color: '#2c3e50',
  },
  dangerText: {
    color: 'red',
  },
  successText: {
    color: 'green',
  },
});
```

```

    buttonSpacing: {
      marginBottom: 10,
    },
  });

export default DetailScreen;

```



Gambar F.1 – Output dari Latihan 1

F.2 Latihan 2 – Passing Data

Pertanyaan / Instruksi Latihan:

Di `HomeScreen`, tambahkan tombol FAB (Floating Action Button) atau Button di bawah untuk "Tambah Barang". Saat ditekan, navigasi ke layar (screen baru bernama `TambahScreen`) yang berisi form kosong.

Jawaban / Kode Solusi:

Membuat File Baru yaitu TambahScreen.js

```

import React from 'react';
import {
  View,

```

```
Text,
TextInput,
StyleSheet,
Button,
} from 'react-native';

function TambahScreen({ navigation }) {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Form Tambah Barang</Text>

      <TextInput
        style={styles.input}
        placeholder="Nama Barang"
      />

      <TextInput
        style={styles.input}
        placeholder="Jumlah Stok"
        keyboardType="numeric"
      />

      <TextInput
        style={styles.input}
        placeholder="Lokasi Penyimpanan"
      />

      <Button
        title="Simpan"
        onPress={() =>
          alert('Data berhasil disimpan!')
        }
      />

      <View style={{ marginTop: 10 }}>
        <Button
          title="Kembali"
          onPress={() => navigation.goBack()}
        />
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
```

```
    flex: 1,
    padding: 20,
    backgroundColor: '#fff',
  },

  title: {
    fontSize: 22,
    fontWeight: 'bold',
    marginBottom: 20,
  },

  input: {
    borderWidth: 1,
    borderColor: '#ccc',
    borderRadius: 8,
    padding: 12,
    marginBottom: 15,
  },
});

export default TambahScreen;
```

Menambahkan tombol "Tambah Barang" di HomeScreen.js

```
//Tambahkan tombol di bagian bawah daftar.//
Import sudah benar:

import {
  View,
  Text,
  FlatList,
  TouchableOpacity,
  StyleSheet,
  Button,
} from 'react-native';

// Kemudian ubah bagian return menjadi:

return (
  <View style={styles.container}>
    <Text style={styles.header}>
      Daftar Inventori Gudang
    </Text>

    <FlatList
      data={DATA_INVENTORI}
      renderItem={renderItem}
      keyExtractor={(item) => item.id}
    />

    <View style={styles.buttonContainer}>
      <Button
```

```
        title="Tambah Barang"
        onPress={() =>
            navigation.navigate('Tambah')
        }
    />
</View>
</View>
);

buttonContainer: {
    margin: 15,
},
```

Modifikasi App.js

Import screen baru:

```
import TambahScreen from './screens/TambahScreen';
```

Tambahkan Stack Screen baru:

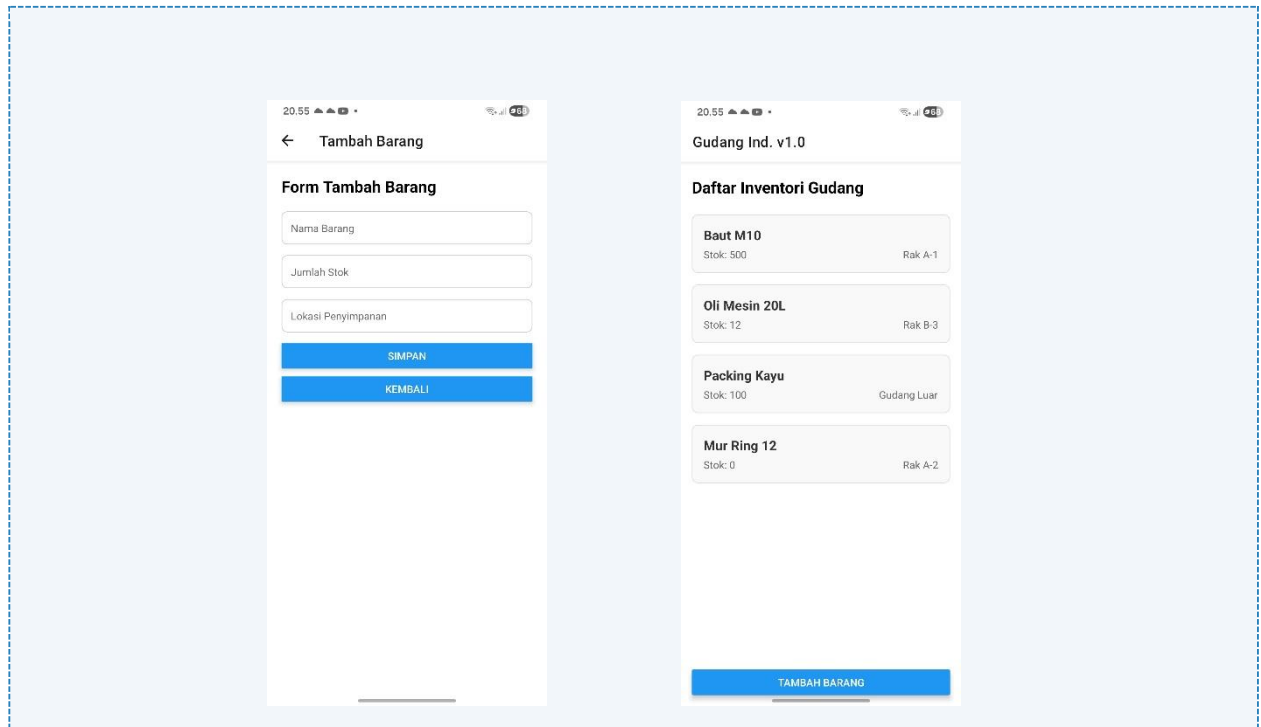
```
<Stack.Screen
  name="Tambah"
  component={TambahScreen}
  options={{ title: 'Tambah Barang' }}
/>
```

Sehingga bagian navigator menjadi:

```
<Stack.Navigator initialRouteName="Home">
  <Stack.Screen
    name="Home"
    component={HomeScreen}
    options={{ title: 'Gudang Ind. v1.0' }}
  />

  <Stack.Screen
    name="Detail"
    component={DetailScreen}
    options={({ route }) => ({
      title: route.params?.itemData.nama || 'Detail',
    })
  />

  <Stack.Screen
    name="Tambah"
    component={TambahScreen}
    options={{ title: 'Tambah Barang' }}
  />
</Stack.Navigator>
```



Gambar F.2 – Output dari Latihan 2

F.3 Tugas Akhir Web

Deskripsi proyek mini:

Buat aplikasi inspeksi sederhana.

- Home: Daftar item yang perlu diinspeksi.
- Detail: Foto item, standar kualitas, dan dropdown status (Lolos/Gagal).
- Simulasi: Saat user memilih "Gagal" di layar detail dan kembali, ubah warna teks di Home menjadi merah (memerlukan sedikit logika state management sederhana).

Penjelasan pendekatan/solusi yang digunakan:

Pada proyek mini ini, aplikasi inspeksi Quality Control (QC) dikembangkan menggunakan React Native dengan menerapkan konsep navigasi antar layar, pengiriman data antar screen, dan pengelolaan state sederhana. Aplikasi terdiri dari dua halaman utama, yaitu HomeScreen yang menampilkan daftar item inspeksi dan DetailScreen yang digunakan untuk melihat informasi detail item serta melakukan proses inspeksi.

Data item yang dipilih pada HomeScreen dikirim ke DetailScreen menggunakan parameter navigasi. Pada halaman detail, pengguna dapat melihat foto komponen, standar kualitas yang harus dipenuhi, serta memilih hasil inspeksi berupa status "Lolos" atau "Gagal". Status yang dipilih kemudian disimpan dan dikirim kembali ke halaman utama melalui parameter navigasi.

Untuk mensimulasikan proses inspeksi secara dinamis, digunakan hook useState dan useEffect untuk mengelola perubahan data. Ketika pengguna memilih status "Gagal", sistem akan memperbarui data item pada HomeScreen dan mengubah warna teks item menjadi merah sebagai indikator bahwa item tersebut tidak memenuhi standar kualitas. Pendekatan ini membuat aplikasi mampu memberikan umpan balik secara langsung kepada pengguna tanpa perlu melakukan refresh atau memuat ulang halaman.

Kode Utama:

HomeScreen.js

```
import React, { useState, useEffect } from 'react';
import {
  View,
  Text,
  FlatList,
  TouchableOpacity,
  StyleSheet,
} from 'react-native';

const DATA_INSPEKSI = [
  {
    id: '1',
    nama: 'Bearing SKF 6205',
    status: 'Belum Dicek',
  },
  {
    id: '2',
    nama: 'V-Belt A-42',
    status: 'Belum Dicek',
  },
  {
    id: '3',
    nama: 'Pompa Air Industri',
    status: 'Belum Dicek',
  },
  {
    id: '4',
    nama: 'Motor Listrik 1 HP',
    status: 'Belum Dicek',
  },
];

function HomeScreen({ navigation, route }) {
  const [items, setItems] = useState(DATA_INSPEKSI);

  useEffect(() => {
    if (route.params?.updatedId) {
```

```

      setItems((prevItems) =>
        prevItems.map((item) =>
          item.id === route.params.updatedId
            ? {
                ...item,
                status: route.params.updatedStatus,
              }
            : item
          )
        );
    }, [route.params]);

const renderItem = ({ item }) => (
  <TouchableOpacity
    style={styles.card}
    onPress={() =>
      navigation.navigate('Detail', {
        itemData: item,
      })
    }
  >
    <Text
      style={[
        styles.namaBarang,
        item.status === 'Gagal'
          ? styles.gagal
          : item.status === 'Lolos'
          ? styles.lolos
          : null,
      ]}
    >
      {item.nama}
    </Text>

    <Text style={styles.status}>
      Status QC : {item.status}
    </Text>
  </TouchableOpacity>
);

return (
  <View style={styles.container}>
    <Text style={styles.header}>
      Sistem Inspeksi Quality Control
    </Text>

    <Text style={styles.subHeader}>
      Monitoring hasil inspeksi komponen gudang
    </Text>

    <FlatList
      data={items}
      renderItem={renderItem}
      keyExtractor={(item) => item.id}
    />
  </View>
);

```



```
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f4f6f8',
    paddingTop: 20,
  },

  header: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#2c3e50',
    marginHorizontal: 15,
  },

  subHeader: {
    fontSize: 14,
    color: '#7f8c8d',
    marginHorizontal: 15,
    marginBottom: 15,
    marginTop: 5,
  },

  card: {
    backgroundColor: '#fff',
    marginHorizontal: 15,
    marginVertical: 8,
    padding: 15,
    borderRadius: 10,
    elevation: 3,
  },

  namaBarang: {
    fontSize: 18,
    fontWeight: 'bold',
  },

  status: {
    marginTop: 5,
    color: '#7f8c8d',
  },

  gagal: {
    color: '#e74c3c',
  },

  lolos: {
    color: '#27ae60',
  },
});

export default HomeScreen;
```

```
import React, { useState } from 'react';
import {
  View,
```

```
Text,
Image,
TouchableOpacity,
StyleSheet,
Alert,
} from 'react-native';

function DetailScreen({ route, navigation }) {
  const { itemData } = route.params;

  const [status, setStatus] = useState(itemData.status);

  const gambarItem = {
    'Bearing SKF 6205': require('../assets/bearing.jpg'),
    'V-Belt A-42': require('../assets/vbelt.jpg'),
    'Pompa Air Industri': require('../assets/pompa.jpg'),
    'Motor Listrik 1 HP': require('../assets/motor.jpg'),
  };

  const standarQC = {
    'Bearing SKF 6205':
      '• Tidak terdapat karat pada permukaan\n• Putaran bearing halus\n• Tidak ada kerusakan fisik',
    'V-Belt A-42':
      '• Tidak terdapat retakan\n• Dimensi sesuai spesifikasi\n• Permukaan masih elastis',
    'Pompa Air Industri':
      '• Tidak terdapat kebocoran\n• Impeller berputar normal\n• Tidak ada suara abnormal',
    'Motor Listrik 1 HP':
      '• Kabel dalam kondisi baik\n• Motor berfungsi normal\n• Tidak terjadi panas berlebih',
  };

  const simpanHasil = () => {
    navigation.navigate({
      name: 'Home',
      params: {
        updatedId: itemData.id,
        updatedStatus: status,
      },
      merge: true,
    });

    Alert.alert(
      'Inspeksi Berhasil',
      `Status inspeksi: ${status}`
    );
  };

  return (
    <View style={styles.container}>
      <View style={styles.card}>
        <Image
          source={gambarItem[itemData.nama]}
          style={styles.image}
        />
      </View>
    </View>
  );
}
```

```

/>

<Text style={styles.label}>
  Nama Komponen
</Text>

<Text style={styles.value}>
  {itemData.nama}
</Text>

<Text style={styles.label}>
  Standar Kualitas
</Text>

<Text style={styles.standar}>
  {standarQC[itemData.nama]}
</Text>
</View>

<Text style={styles.labelStatus}>
  Pilih Hasil Inspeksi
</Text>

<View style={styles.statusContainer}>
  <TouchableOpacity
    style={[
      styles.btnStatus,
      status === 'Lolos' &&
      styles.btnLolos,
    ]}
    onPress={() => setStatus('Lolos')}
  >
    <Text
      style={[
        styles.btnText,
        status === 'Lolos' &&
        styles.textPutih,
      ]}
    >
      LOLLOS
    </Text>
  </TouchableOpacity>

  <TouchableOpacity
    style={[
      styles.btnStatus,
      status === 'Gagal' &&
      styles.btnGagal,
    ]}
    onPress={() => setStatus('Gagal')}
  >
    <Text
      style={[
        styles.btnText,
        status === 'Gagal' &&
        styles.textPutih,
      ]}
    >
      GAGAL

```

```
        </Text>
      </TouchableOpacity>
    </View>

    <TouchableOpacity
      style={styles.btnSimpan}
      onPress={simpanHasil}
    >
      <Text style={styles.btnSimpanText}>
        Simpan Hasil Inspeksi
      </Text>
    </TouchableOpacity>
  </View>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f4f6f8',
    padding: 20,
  },

  card: {
    backgroundColor: '#fff',
    padding: 15,
    borderRadius: 10,
    elevation: 3,
    marginBottom: 20,
  },

  image: {
    width: '100%',
    height: 220,
    borderRadius: 10,
    marginBottom: 15,
  },

  label: {
    fontSize: 13,
    color: '#7f8c8d',
    fontWeight: 'bold',
    marginTop: 10,
  },

  value: {
    fontSize: 20,
    fontWeight: 'bold',
    color: '#2c3e50',
  },

  standar: {
    marginTop: 5,
    lineHeight: 22,
    color: '#34495e',
  },

  labelStatus: {
    fontSize: 16,
```

```
    fontWeight: 'bold',
    marginBottom: 10,
  },

  statusContainer: {
    flexDirection: 'row',
    marginBottom: 20,
  },

  btnStatus: {
    flex: 1,
    padding: 15,
    backgroundColor: '#dfe6e9',
    alignItems: 'center',
    borderRadius: 8,
    marginHorizontal: 5,
  },

  btnLolos: {
    backgroundColor: '#27ae60',
  },

  btnGagal: {
    backgroundColor: '#e74c3c',
  },

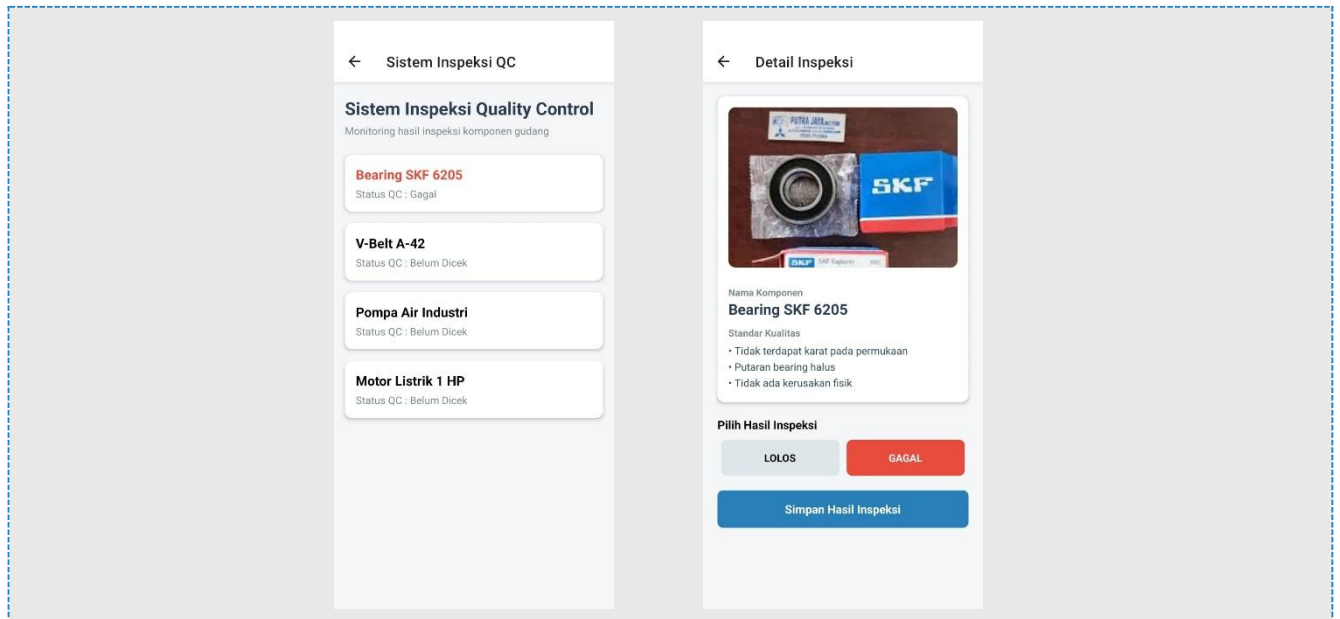
  btnText: {
    fontWeight: 'bold',
  },

  textPutih: {
    color: '#fff',
  },

  btnSimpan: {
    backgroundColor: '#2980b9',
    padding: 15,
    borderRadius: 8,
    alignItems: 'center',
  },

  btnSimpanText: {
    color: '#fff',
    fontWeight: 'bold',
    fontSize: 16,
  },
});

export default DetailScreen;
```



Gambar F.3 – Output dari Tugas Akhir Web

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang diperoleh telah sesuai dengan konsep yang dipelajari pada modul, khususnya mengenai navigasi antar layar menggunakan React Navigation. Pada aplikasi yang dibuat, pengguna dapat berpindah dari halaman HomeScreen yang berisi daftar item inspeksi menuju DetailScreen untuk melihat informasi lebih lengkap mengenai item yang dipilih. Proses perpindahan layar berjalan dengan baik sesuai dengan teori stack navigation yang memungkinkan pengguna membuka halaman baru dan kembali ke halaman sebelumnya dengan mudah.

Konsep passing data antar screen juga berhasil diterapkan pada aplikasi. Data item yang dipilih pada HomeScreen dapat dikirim ke DetailScreen melalui parameter navigasi, kemudian ditampilkan dalam bentuk nama komponen, foto, dan standar kualitas yang harus diperiksa. Hal ini menunjukkan bahwa mekanisme pengiriman data melalui `route.params` dapat digunakan untuk menampilkan informasi yang berbeda sesuai dengan item yang dipilih oleh pengguna.

Selain itu, penggunaan state management sederhana melalui `useState` dan `useEffect` memberikan hasil yang sesuai dengan teori React Native. Ketika pengguna memilih status "Gagal" pada halaman detail, status item akan diperbarui dan ditampilkan kembali pada halaman utama dengan warna teks merah sebagai indikator adanya ketidaksesuaian kualitas. Perubahan tampilan yang terjadi secara otomatis tanpa perlu me-restart aplikasi menunjukkan bahwa state berhasil mengontrol perubahan data dan antarmuka secara dinamis sesuai konsep reaktif yang dimiliki React Native.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Gambar komponen tidak muncul pada halaman detail ketika menggunakan URL dari internet. Beberapa gambar gagal dimuat karena masalah koneksi atau link yang tidak dapat diakses oleh aplikasi.	Menggunakan gambar lokal yang disimpan pada folder assets dan memanggilnya dengan fungsi require(). Cara ini lebih stabil karena tidak bergantung pada koneksi internet dan gambar dapat ditampilkan dengan baik pada semua perangkat.

G.3 Kaitan dengan Penerapan di Industri

Materi yang dipelajari pada modul ini memiliki keterkaitan yang erat dengan kebutuhan dunia industri yang semakin mengandalkan sistem digital untuk mendukung proses operasional. Kemampuan membuat aplikasi mobile dengan fitur navigasi, pengelolaan data, dan pembaruan informasi secara real-time dapat dimanfaatkan untuk membantu proses pengawasan, pencatatan, dan pelaporan di berbagai bidang dalam Teknik Industri. Dengan adanya aplikasi mobile, proses kerja menjadi lebih cepat, akurat, dan mudah dipantau dibandingkan dengan pencatatan manual.

Salah satu contoh penerapannya adalah pada kegiatan Quality Control (QC) di perusahaan manufaktur. Petugas inspeksi dapat menggunakan aplikasi mobile untuk melihat daftar produk yang harus diperiksa, mengakses standar kualitas, serta mencatat hasil inspeksi secara langsung di area produksi. Jika ditemukan produk yang tidak memenuhi standar, sistem dapat memberikan penanda khusus sehingga produk tersebut dapat segera dipisahkan dan ditindaklanjuti oleh bagian terkait. Proses ini membantu mengurangi kesalahan pencatatan dan mempercepat pengambilan keputusan.

Selain pada pengendalian kualitas, konsep yang dipelajari juga dapat diterapkan pada manajemen gudang, inspeksi peralatan, dan kegiatan maintenance. Sebagai contoh, teknisi dapat menggunakan aplikasi mobile untuk memeriksa kondisi mesin, mengisi status pemeriksaan, serta melaporkan kerusakan secara langsung melalui perangkat seluler. Dengan demikian, informasi dapat diterima secara lebih cepat oleh pihak manajemen sehingga tindakan perbaikan dapat segera dilakukan untuk menjaga kelancaran proses produksi.

H. KESIMPULAN

1	Praktikum ini berhasil menerapkan React Navigation untuk melakukan perpindahan antar layar, yaitu dari HomeScreen ke DetailScreen, sehingga pengguna dapat melihat informasi detail dari item yang dipilih.
2	Konsep passing data antar screen berhasil diimplementasikan dengan mengirim data item dari halaman utama ke halaman detail menggunakan parameter navigasi, sehingga informasi yang ditampilkan sesuai dengan item yang dipilih.
3	Penggunaan state management sederhana menggunakan useState dan useEffect memungkinkan perubahan status inspeksi diperbarui secara dinamis tanpa perlu memuat ulang aplikasi.
4	Aplikasi inspeksi QC yang dibuat mampu menampilkan daftar item, foto komponen, standar kualitas, serta hasil inspeksi berupa status lolos atau gagal, sehingga dapat

mensimulasikan proses pengendalian kualitas secara sederhana pada lingkungan industri.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Flanagan, D. (2020). <i>JavaScript: The Definitive Guide: Master the World's Most-Used Programming Language</i> . 7th Edition. Sebastopol: O'Reilly Media.
[3]	React Navigation. (2026). <i>Getting started</i> . React Navigation Documentation. : https://reactnavigation.org/docs/getting-started/

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
 Radhitya Adha Fadhlur Rohman NIM: 23051430005	 Maessa Andrea Vallenia NIM: 23051430044	 Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____